

Zero to Google: DSA, System Design and Test Engineering for QA Engineers

Volume 1 - Data Structures and Algorithms, from zero to expert

Written for Sahil Sharma - SDET preparing for Google SWE-Test / SDET (India). Study it with the web app <https://sahil804-ps.github.io/google-prep/> and Google NotebookLM.

Contents

1. How to Use This Book
2. How Google Coding Interviews Work
3. Python From Zero for DSA
4. A Problem-Solving Framework
5. Foundations 1: Big-O and complexity analysis
6. Foundations 2: Recursion from zero
7. Foundations 3: Sorting algorithms you must know
8. Foundations 4: Prefix sums and how hash maps work
9. Foundations 5: Python toolkit for interviews
10. DSA topic: Arrays & Hashing
11. DSA topic: Two Pointers
12. DSA topic: Sliding Window
13. DSA topic: Stack
14. DSA topic: Binary Search
15. DSA topic: Linked List
16. DSA topic: Trees
17. DSA topic: Tries
18. DSA topic: Heap / Priority Queue
19. DSA topic: Backtracking
20. DSA topic: Graphs
21. DSA topic: Advanced Graphs
22. DSA topic: 1-D Dynamic Programming
23. DSA topic: 2-D Dynamic Programming
24. DSA topic: Greedy
25. DSA topic: Intervals
26. DSA topic: Math & Geometry
27. DSA topic: Bit Manipulation
28. Testing Your Own Code: The QA Superpower
29. Expert Extras: Beyond NeetCode 150
30. The Mock Interview Playbook

How to Use This Book

In this chapter:

- Understand how the three volumes of this book fit together
- Learn how to study with the web app and with Google NotebookLM
- Follow a clear 12-week plan from October 5 to December 27, 2026
- Use spaced repetition so you remember what you learn
- Build habits that stop you from giving up halfway

Time: ~20 minutes | **Level:** Zero

Welcome

You are a test engineer with more than five years of real work. You know Selenium, Playwright, API testing and Python. That is a strong base. What you do not have yet is practice with **DSA** (Data Structures and Algorithms) and **system design**. This book is built to close that gap, step by step, from zero.

Think of this book like a railway journey from your city to a far station. You do not need to see the whole track on day one. You only need to know the next station, and you need to keep the train moving every day.

How the book is organised

The book has three volumes. Each volume is a group of chapters on one big subject.

Volume	Subject	What you get
Volume 1	DSA (Data Structures and Algorithms)	Python basics, a problem-solving method, all core patterns, testing your own code, expert extras, and a mock interview playbook
Volume 2	System Design	How large systems are built: load balancers, caches, databases, queues, and how to talk through a design
Volume 3	Test Engineering	Test strategy, test design techniques, automation architecture, flaky tests, and testing at scale

There is also a short bonus book, **Volume 4: English for technical interviews**. It has 22 grammar lessons with quizzes, an interview phrase bank and speaking prompts. It is not part of the three main subjects, so the counts in this chapter still say "three volumes".

Chapter files are named by volume. For example, `dsa-03-...` is chapter 3 of the DSA volume. Chapters 00 to 03 are the foundation. Chapters 30 to 32 are the finishing chapters: testing your own code, expert

topics, and mock interviews.

The shape of every chapter

Every chapter follows the same shape, so you always know where you are:

1. **In this chapter** - the learning goals.
2. **Body sections** - the teaching, with analogies and short Python examples.
3. **Tester's corner** - how the topic connects to your QA work.
4. **Key takeaways** - the short summary.
5. **Quiz** - ten questions to check yourself.
6. **Answer key** - answers with short explanations.
7. **Flashcards** - question and answer pairs for revision.

A good habit: read the Key takeaways first, then the body, then do the Quiz without looking back.

How to study with the web app

The companion web app is at <https://sahil804-ps.github.io/google-prep/>. It holds lessons, problem lists and progress tracking. Use the book and the app together:

- **The book** explains ideas in depth. Read it when you meet a new topic.
- **The web app** is for daily practice. Open it to pick your problems for the day and to mark them done.
- When the app shows a problem you cannot solve, come back to the chapter for that pattern and read the worked example again.

Some topics, such as Big-O, recursion, sorting, prefix sums and the Python toolkit, are taught as lessons inside the app. This book does not repeat them in full; it points to them.

How to study with NotebookLM

Google NotebookLM is a tool where you upload documents and then ask questions about them. It answers only from your documents, which makes it a good study partner.

Here is a simple way to use it:

1. Create one notebook per volume (for example "DSA Volume").
2. Upload the chapter files for that volume as sources.
3. Generate an **audio overview** of the chapter you are reading this week. Listen during your commute or while walking.
4. Ask NotebookLM to make a **quiz** or **flashcards** from a chapter, and answer them without looking.
5. When something is unclear, ask a direct question, such as "Explain the sliding window pattern with a cricket example."

Audio is great for revision, but it does not replace solving problems by hand. Listening to a cricket commentary does not make you a batsman. You still need to go to the nets.

The 12-week plan

The plan runs from **Monday, October 5, 2026** to **Sunday, December 27, 2026**. That is 12 weeks.

Your daily time budget

Block	Time	What you do
DSA	75 minutes	Read one pattern or solve 2-3 problems
System design	45 minutes	Read one design topic or practise one design question aloud
English	30 minutes	Speak your solutions aloud, record yourself, read one technical article

That is 2.5 hours a day. If a work day is very heavy, do the **minimum day**: 30 minutes of DSA (one problem) and 10 minutes of English. A minimum day still keeps the chain alive.

Sundays: weekly test

Every Sunday is test day. Do this:

- Pick 3 problems from this week's topics that you have not seen before.
- Set a timer for 45 minutes per problem.
- Solve them as if in an interview: talk aloud, write tests, state complexity.
- Score yourself with the rubric in the mock interview chapter (dsa-32).
- Write your mistakes in your mistakes log.

Week by week

Week	Dates	DSA focus	Design focus
1	Oct 5 - Oct 11	Python for DSA, framework, Big-O	What system design is, client-server basics
2	Oct 12 - Oct 18	Arrays, hashing, two pointers	Networking basics, APIs
3	Oct 19 - Oct 25	Sliding window, stack	Databases: SQL vs NoSQL
4	Oct 26 - Nov 1	Binary search, linked lists	Caching
5	Nov 2 - Nov 8	Trees	Load balancing, scaling
6	Nov 9 - Nov 15	Tries, heaps	Queues and async processing
7	Nov 16 - Nov 22	Backtracking	Consistency and replication

Week	Dates	DSA focus	Design focus
8	Nov 23 - Nov 29	Graphs	Design practice: URL shortener, rate limiter
9	Nov 30 - Dec 6	Dynamic programming 1-D	Design practice: chat, feed
10	Dec 7 - Dec 13	Dynamic programming 2-D, greedy, intervals	Test engineering volume: strategy and test design
11	Dec 14 - Dec 20	Expert extras, testing your own code, revision	Testing at scale, flaky tests
12	Dec 21 - Dec 27	Full mock interviews	Full mock design rounds

Week 12 is mock week. Do one full mock every day: a 45-minute coding round, and on alternate days a 45-minute design round. Use the playbook in chapter dsa-32. Ask a friend or colleague to act as interviewer if you can. If not, record yourself on video and review it.

Spaced repetition

Spaced repetition means you review a thing again just before you forget it. Each review pushes the "forget date" further away.

Think of a UPI PIN. The first week after you set it, you might forget it. After using it many times over a few weeks, it stays for years. Repetition at growing gaps is what made it stick.

Use this schedule for every problem you solve:

Review	When	What you do
1	Day 1 (same day)	Solve it
2	Day 3	Re-code from memory, no hints
3	Day 7	Re-code from memory, time yourself
4	Day 14	Explain the approach aloud in 2 minutes
5	Day 30	Re-code once more

A simple spreadsheet is enough. Make columns for problem name, pattern, date solved, and the next review date. Each morning, do the reviews that are due before new problems.

How not to give up

Most people who prepare for big interviews do not fail because the topics are too hard. They stop. Here is how you keep going.

- **Expect to feel slow.** In weeks 1-4 you may take an hour on an "easy" problem. That is normal. Speed comes from patterns, and patterns come from repetition.
- **Never break the chain twice.** Missing one day is fine. Missing two days in a row is how habits die. After a missed day, do at least a minimum day.
- **Measure inputs, not feelings.** Track minutes studied and problems solved. Your mood will go up and down; your tracker will show real progress.
- **Use the 15-20 minute rule.** If you are stuck for 15-20 minutes with no new idea, look at a hint. Struggling for two hours teaches less than learning the pattern and re-coding it later (more in chapter dsa-03).
- **Celebrate small wins.** Your first medium problem solved alone is a big moment. Note it.
- **Remember your edge.** You already think about edge cases and failures every day at work. That skill is exactly what Google interviewers look for when you test your own code.

Tester's corner

- Treat this plan like a test plan: it has scope (topics), schedule (weeks), entry criteria (basics done) and exit criteria (mock scores).
- Your mistakes log is like a bug tracker. Each mistake gets a short title, a root cause, and a fix.
- Sunday tests are your regression suite. They check that old skills still work after you learn new ones.
- Spaced repetition is like running a smoke test on old features at growing intervals.
- Track metrics (problems solved, mock scores) the same way you track pass rates in a test report.

Key takeaways

- The book has three volumes: DSA, System Design and Test Engineering.
- Every chapter has goals, body, Tester's corner, takeaways, quiz, answer key and flashcards.
- Use the web app for daily practice and NotebookLM for audio, quizzes and questions.
- The plan runs October 5 to December 27, 2026: 75 minutes DSA, 45 minutes design, 30 minutes English per day.
- Sundays are weekly tests; week 12 is full mock interviews.
- Review each problem on days 1, 3, 7, 14 and 30.
- On bad days, do a minimum day so the chain never breaks twice.

Quiz

1. How many volumes does this book have, and what are they?
2. What is the daily DSA time in the plan? A) 30 minutes B) 45 minutes C) 75 minutes D) 120 minutes
3. True or false: NotebookLM answers questions using only the documents you upload.
4. What should you do on Sundays during the plan?
5. Which week is dedicated to full mock interviews? A) Week 1 B) Week 6 C) Week 10 D) Week 12
6. On which days do you review a problem in the spaced repetition schedule?
7. True or false: Listening to audio overviews is enough to learn DSA.
8. What is a "minimum day"?

9. You are stuck on a problem for 20 minutes with no new idea. What would you do?
10. Which section in each chapter connects the topic to QA work? A) Key takeaways B) Tester's corner C) Flashcards D) Answer key

Answer key

1. **Three** - Volume 1 DSA, Volume 2 System Design, Volume 3 Test Engineering.
2. **C** - The plan gives 75 minutes a day to DSA, 45 to design and 30 to English.
3. **True** - NotebookLM grounds its answers in your uploaded sources, which is why the chapters must be self-contained.
4. **A weekly test** - Solve 3 unseen problems, 45 minutes each, in interview style, then score yourself and log mistakes.
5. **D** - Week 12 (December 21-27) is mock week.
6. **Days 1, 3, 7, 14 and 30** - Each review comes just before you would forget.
7. **False** - Audio helps revision, but you must solve problems by hand to build the skill.
8. **A short fallback day** - 30 minutes of DSA (one problem) and 10 minutes of English, so the habit chain stays alive on busy days.
9. **Take a hint** - Use the 15-20 minute rule: look at a hint or the approach, then re-code it later from memory.
10. **B** - Tester's corner links each topic to testing and QA work.

Flashcards

- **Q:** What are the three volumes of this book? — **A:** DSA, System Design, and Test Engineering.
- **Q:** What are the plan's start and end dates? — **A:** October 5, 2026 to December 27, 2026 (12 weeks).
- **Q:** What is the daily time split? — **A:** 75 minutes DSA, 45 minutes system design, 30 minutes English.
- **Q:** What happens every Sunday? — **A:** A weekly test of 3 unseen problems in interview style, scored with a rubric.
- **Q:** What is week 12 for? — **A:** Full mock interviews every day, using the mock interview playbook.
- **Q:** What is spaced repetition? — **A:** Reviewing material at growing gaps, just before you would forget it.
- **Q:** What is the review schedule for a problem? — **A:** Days 1, 3, 7, 14 and 30.
- **Q:** What is the rule for missed days? — **A:** Missing one day is fine; never miss two days in a row.
- **Q:** What is the 15-20 minute rule? — **A:** If stuck that long with no new idea, take a hint, then re-code from memory later.
- **Q:** How should you use NotebookLM? — **A:** Upload chapters, generate audio overviews, make quizzes and flashcards, and ask questions.

How Google Coding Interviews Work

In this chapter:

- Understand the stages of Google's hiring process as Google describes it publicly
- Know what each interview round looks like for SWE, SWE-Test, Test Engineer and SDET-style roles
- Learn the five things interviewers evaluate in a coding round
- Choose your language and get ready to code without autocomplete

Time: ~25 minutes | **Level:** Zero

Why this chapter matters

Imagine you are preparing for a cricket match, but you do not know if it is a T20 or a Test match. You would train the wrong way. Knowing the format of the Google interview tells you what to practise and what to ignore.

A quick note on sources. Everything here about Google's process comes from what Google publishes on its careers site ("How we hire", at google.com/about/careers/applications/how-we-hire) and from the Google Tech Dev Guide (techdevguide.withgoogle.com), plus what is commonly and publicly reported by candidates. Google itself says the process **can vary by role, level, team and country**. Your recruiter is the final source of truth for your own interviews. Always ask them.

The role names

Google has used several names for engineering roles that focus on testing and quality. You may see:

- **Software Engineer (SWE)** - builds products and infrastructure.
- **Software Engineer, Test / SWE in Test** - a software engineer who focuses on test infrastructure, tools and quality.
- **Test Engineer (TE)** - focuses on test strategy, test design and quality for a product area, and also writes code.

Outside Google, the term **SDET** (Software Development Engineer in Test) is common for similar work. Exact titles change over time, so read the job description carefully. For every one of these roles, you should expect **coding interviews**. Test-focused roles usually add questions on **test design and test strategy**.

The stages of the process

Google's public "How we hire" page describes a process that goes roughly like this. Think of it as a Swiggy order: you place the order, the restaurant accepts it, the food is cooked, checked, and then delivered. Each step has its own owner.

1. Application and resume review

You apply online, or a recruiter contacts you, or someone refers you. A recruiter reviews your resume against the role. Google's careers site advises you to make your resume clear and to show impact with specifics, for example "reduced test run time from 40 to 12 minutes" instead of "improved tests".

2. Recruiter conversation

A recruiter talks with you, usually by phone or video. This is not a deep technical test. They check your background, your interest in the role, location, level, and timelines. They also explain the next steps. This is your chance to ask:

- Which role and level am I being considered for?
- How many interviews, and what types?
- Which programming languages can I use?
- Will there be a test design or role-related round?

3. Technical screen (phone or virtual)

Google describes short virtual interviews before the main loop. For engineering roles, this is commonly one or two technical interviews of about 45 minutes. You solve one or two coding problems in a shared document or simple online editor while talking with an engineer. Some roles may also use an online assessment first. This varies, so confirm with your recruiter.

4. The interview loop ("onsite", often virtual)

If the screen goes well, you move to the main set of interviews, often called the "onsite" even when it happens over video. Google's public guidance describes interviews that assess these attributes:

- **General cognitive ability** - how you think through hard, open problems. For engineers, this shows up mostly in coding and problem-solving rounds.
- **Role-related knowledge** - skills for this specific job. For a test role, this often means **test design**: how you would test a feature, a system or an API, and how you think about quality and risk.
- **Leadership** - how you have stepped up, influenced others and handled hard situations, even without a manager title.
- **Googlyness** - Google's word for qualities like comfort with ambiguity, a bias to action, caring about the team and users, doing the right thing, and being humble enough to learn.

A loop is commonly about four or five interviews of about 45 minutes each. For test-focused roles, a typical mix (it varies) is two or three coding rounds, one test design or role-related round, and one Googlyness and leadership round.

5. Hiring committee

Google's careers site explains that hiring decisions are not made by one person. Your interviewers write detailed feedback. A **hiring committee**, a group of experienced Googlers, reviews the full packet (feedback, resume, any references) and makes a recommendation. This is meant to reduce the bias of any single interviewer.

Why does this matter for you? Every round is written down. Your words, your code and your test cases are recorded in feedback. Clear communication becomes evidence that the committee can read.

6. Team matching and offer

For many engineering roles, candidates who pass the committee go through **team matching**, where you talk with managers of teams that have open positions. This step is commonly reported for SWE roles, but it may not happen the same way for every role or country. After a match, the offer is reviewed and extended.

What a 45-minute coding round looks like

A coding round usually follows this flow (details vary by interviewer):

Minutes	What happens
0-5	Short introductions
5-10	Interviewer gives the problem; you ask clarifying questions
10-20	You discuss approaches, starting with brute force, then a better one
20-35	You write the code
35-42	You test the code by hand and fix bugs; you state time and space complexity
42-45	Your questions for the interviewer

Sometimes a round has a follow-up question that extends the first problem, such as "What if the input does not fit in memory?" Chapter dsa-32 gives a minute-by-minute playbook with phrases to say.

What interviewers evaluate

Interviewers do not just check "did the code run". Public Google guidance and the Tech Dev Guide stress that they want to see **how you think**. You can group what they look for into five areas.

1. Communication

Can you explain your thinking clearly? Do you ask good questions? Do you listen to hints? Silence is the biggest enemy. If you think quietly for five minutes, the interviewer has nothing to write down.

A good habit: speak like a cricket commentator. "Now I am checking the left side. If it is smaller, I move the pointer right."

2. Problem solving

Do you understand the problem before coding? Can you find a simple solution first and then improve it? Can you break a big problem into small steps? Do you notice patterns, such as "this is a sliding window problem"?

3. Coding

Is the code correct, clean and readable? Do you use good variable names? Do you use the language well, for example Python's `dict` and `set`? Can you write it without heavy help? Google interviewers usually care about logic more than tiny syntax slips, but your code should be close to runnable.

4. Testing

Do you check your own code? Do you think of edge cases such as an empty list, a single element, duplicates or negative numbers? Do you walk through an example line by line? This is your superpower as a test engineer. Chapter dsa-30 is all about it.

5. Complexity analysis

Can you state the **time complexity** (how running time grows with input size) and **space complexity** (how extra memory grows) using Big-O notation? Can you explain why, and say if a better solution is possible? The Big-O lesson in the web app covers the basics.

Choosing your language

Google generally lets you choose a mainstream language for coding interviews, such as Python, Java, C++, JavaScript or Go. Confirm with your recruiter. **Python is a strong choice** for you because:

- You already use it at work.
- It is short. Less typing means more time for thinking and testing.
- It has useful built-in tools: `dict`, `set`, `collections.deque`, `heapq`, and sorting with `key=`.

One warning: pick **one** language and stay with it for all practice. Switching languages in the interview causes small mistakes.

Chapter dsa-02 teaches the Python you need from zero.

Coding without autocomplete

In the interview you will likely code in a shared document or a simple editor. It may have **no autocomplete, no syntax highlighting help, and no way to run the code**. This is a shock for engineers who live in VS Code or PyCharm.

Think of it like driving without Google Maps. You can do it, but only if you have practised the route.

How to prepare:

- **Practise in a plain editor.** Use Notepad, a Google Doc, or a plain text file for at least half of your practice.
- **Know common names by heart:** `len`, `append`, `pop`, `sorted`, `enumerate`, `zip`, `range`, `dict.get`, `set.add`, `heapq.heappush`, `heapq.heappop`, `deque.popleft`.
- **Indent carefully.** In a doc, use 4 spaces and be consistent.
- **Run the code in your head.** Since you cannot press "Run", you must trace it by hand with a small example. This is the dry run skill in chapter dsa-30.

Here is the kind of code you should be able to type cleanly with no help:

```
def two_sum(nums, target):  
    """Return indices of two numbers that add up to target."""  
    seen = {} # value -> index  
    for i, num in enumerate(nums):  
        need = target - num  
        if need in seen: # found the partner  
            return [seen[need], i]  
        seen[num] = i  
    return [] # no pair found  
  
print(two_sum([2, 7, 11, 15], 9)) # Output: [0, 1]
```

What you can do before you apply

- Read the job description and match your resume to it. Show impact with numbers you can explain.
- Read Google's "How we hire" page and the Tech Dev Guide yourself.
- Prepare 6-8 short stories from your work for leadership and Googleyness questions: a hard bug you found, a time you disagreed with a developer, a time you improved a process, a time you failed and learned.
- Practise explaining your solutions in English aloud every day.

Tester's corner

- The test design round is where your 5.5 years of experience shine. Practise describing how you would test a feature end to end: functional, edge, negative, performance, security and usability.
- In coding rounds, testing your own solution is one of the five evaluation areas. Many developers skip it. You should not.
- The hiring committee reads written feedback. Saying test cases aloud ("for an empty list I return 0") gives your interviewer concrete evidence to write.
- Think of the interview like a release gate: each round is a check, and the committee is the final sign-off.
- Prepare stories about quality impact: flaky test reduction, faster pipelines, bugs caught before production.

Key takeaways

- Google's public process: application, recruiter talk, technical screen, interview loop, hiring committee, then team matching and offer. Details vary by role and country.
- The loop assesses general cognitive ability, role-related knowledge, leadership and Googleness.
- Test-focused roles add test design rounds on top of coding rounds.
- Interviewers evaluate communication, problem solving, coding, testing and complexity analysis.
- Python is a good choice; pick one language and practise only in it.
- Practise coding in a plain editor without autocomplete and without running the code.
- Your recruiter is the best source for your exact interview format.

Quiz

1. Who makes the final hiring recommendation at Google, according to its public description? A) Your first interviewer B) The recruiter C) A hiring committee D) The team manager alone
2. Name the four attributes Google says it assesses in interviews.
3. True or false: Google's interview process is exactly the same for every role and country.
4. In a 45-minute coding round, what should you do before writing code?
5. Which of these is NOT one of the five evaluation areas in this chapter? A) Communication B) Testing C) Typing speed D) Complexity analysis
6. What is "role-related knowledge" likely to mean for a Test Engineer role?
7. True or false: Python is generally accepted for Google coding interviews.
8. The shared editor has no autocomplete and you cannot run code. What would you do to check your solution?
9. Why is thinking silently for a long time a problem in an interview?
10. What is team matching? A) A coding round B) Talking with teams that have open roles after passing committee C) A group interview D) A test design round

Answer key

1. **C** - Google describes a hiring committee that reviews interview feedback, so no single interviewer decides.
2. **General cognitive ability, role-related knowledge, leadership, Googleness** - These are the attributes listed on Google's "How we hire" page.
3. **False** - Google says the process can vary by role, level, team and location; confirm with your recruiter.
4. **Clarify and plan** - Ask clarifying questions, confirm examples, and discuss a brute force and a better approach before coding.
5. **C** - Typing speed is not evaluated; communication, problem solving, coding, testing and complexity are.
6. **Test design and quality skills** - For example, designing tests for a feature, system or API and reasoning about risk.

7. **True** - Mainstream languages such as Python are generally accepted, but confirm with your recruiter.
8. **Dry run by hand** - Trace the code line by line with a small example and edge cases, saying the variable values aloud.
9. **No evidence** - The interviewer cannot see your thinking or write feedback about it, and cannot give helpful hints.
10. **B** - Team matching is when you meet managers of teams with open roles after the committee step.

Flashcards

- **Q:** What are the main stages of Google's hiring process? — **A:** Application, recruiter conversation, technical screen, interview loop, hiring committee, team matching and offer.
- **Q:** What four attributes does Google assess? — **A:** General cognitive ability, role-related knowledge, leadership, and Googleyness.
- **Q:** What is the hiring committee? — **A:** A group of experienced Googlers who review all interview feedback and make a recommendation.
- **Q:** How long is a typical coding interview? — **A:** About 45 minutes.
- **Q:** What extra round do test-focused roles usually have? — **A:** A test design or role-related knowledge round.
- **Q:** What five areas do coding interviewers evaluate? — **A:** Communication, problem solving, coding, testing, and complexity analysis.
- **Q:** Why is Python a good interview language? — **A:** It is short, you know it already, and it has strong built-ins like dict, set, deque and heapq.
- **Q:** How should you practise for coding without autocomplete? — **A:** Code in a plain editor or doc and trace the code by hand without running it.
- **Q:** Where should you confirm your exact interview format? — **A:** With your recruiter.
- **Q:** What is Googleyness? — **A:** Google's term for qualities like comfort with ambiguity, bias to action, caring for team and users, and humility.

Python From Zero for DSA

In this chapter:

- Learn the Python basics that DSA problems need, and nothing extra
- Use lists, tuples, dictionaries and sets with confidence
- Write loops, functions and small classes like `ListNode` and `TreeNode`
- Understand `None`, truthiness, scope and list comprehensions
- Recognise and fix the most common Python errors in interviews

Time: ~60 minutes | **Level:** Zero

Why only "DSA Python"

Python is a big language. You do not need all of it for interviews. You need a small toolbox, and you need to use it without thinking, like a cook who knows exactly where the salt and the knife are.

You already write Python for test automation, so some of this will feel easy. Read it anyway. Interview Python is different from framework Python: no libraries like `pytest`, `fixtures` or `Selenium`, just plain data structures and logic. Small gaps here cause big bugs under pressure.

The web app has a separate lesson, "Python toolkit for interviews", that covers `collections`, `heapq` and other power tools. This chapter is the foundation under that lesson.

Every example below is tiny. Type them into a Python shell (`python` in your terminal) and see the output yourself.

Variables

A **variable** is a name that points to a value. Think of it as a label stuck on a box. The label is the name; the box holds the value.

```
age = 30          # name 'age' points to the value 30
name = "Sahil"    # a string
age = age + 1     # now 'age' points to 31
print(age, name)  # Output: 31 Sahil
```

You do not declare a type. Python figures it out from the value. You can also assign many variables at once:

```
a, b = 1, 2      # a = 1, b = 2
a, b = b, a      # swap in one line
print(a, b)      # Output: 2 1
```

The swap trick is used a lot in DSA, for example when reversing an array with two pointers.

The basic types: int, float, str, bool

int and float

An **int** is a whole number. A **float** is a number with a decimal point.

```
x = 7
y = 2
print(x + y)    # 9
print(x - y)    # 5
print(x * y)    # 14
print(x / y)    # 3.5 (true division always gives a float)
print(x // y)   # 3   (floor division: rounds down)
print(x % y)    # 1   (remainder, called "modulo")
print(x ** y)   # 49  (power)
```

Two things matter a lot for DSA:

- Use `//` for integer division, for example the middle index in binary search: `mid = (left + right) // 2`.
- `%` gives the remainder. `n % 2 == 0` checks if `n` is even.

Python ints have no size limit, so you do not get overflow like in Java or C++. For "infinity", use `float('inf')` and `float('-inf')`:

```
best = float('inf')    # bigger than any number
best = min(best, 42)
print(best)            # Output: 42
```

str

A **str** (string) is text. Use single or double quotes.

```
s = "hello"
print(len(s))        # 5 (length)
print(s[0])          # 'h' (first character, index starts at 0)
print(s[-1])         # 'o' (negative index counts from the end)
print(s.upper())     # 'HELLO'
print("ell" in s)    # True
```

Strings are **immutable**. That means you cannot change a character in place.

```
s = "cat"
# s[0] = "b"           # TypeError: 'str' object does not support item assignment
s = "b" + s[1:]        # build a new string instead
print(s)               # Output: bat
```

To build a long string piece by piece, collect pieces in a list and join them at the end. Adding strings in a loop creates a new string every time, which is slow.

```
parts = []
for ch in "abc":
    parts.append(ch.upper())
print("".join(parts)) # Output: ABC
```

Useful string tools: `s.split()`, `" ".join(words)`, `s.isdigit()`, `s.isalpha()`, `s.isalnum()`, `s.lower()`, `ord('a')` gives 97 and `chr(97)` gives 'a'.

```
# Map a lowercase letter to 0..25, common in anagram problems
print(ord('c') - ord('a')) # Output: 2
```

bool

A **bool** is `True` or `False`. Comparisons give bools. Combine them with `and`, `or`, `not`.

```
print(3 > 2)           # True
print(3 == 3 and 2 != 2) # False
print(not False)       # True
print(1 < 5 < 10)      # True (Python allows chained comparisons)
```

Lists

A **list** is an ordered collection that can grow and shrink. It is Python's version of an array. Think of a train: coaches in order, numbered from 0, and you can add coaches at the end.

```
nums = [5, 3, 8]
nums.append(1)           # add to the end -> [5, 3, 8, 1]
print(nums[0])           # 5
print(nums[-1])          # 1
nums[1] = 10             # lists ARE mutable -> [5, 10, 8, 1]
last = nums.pop()        # remove and return last -> 1
print(nums, last)        # [5, 10, 8] 1
print(len(nums))         # 3
```

Common list operations and their cost (Big-O, where n is the list length):

Operation	Example	Cost
Read or write by index	<code>nums[i]</code>	O(1)
Add to end	<code>nums.append(x)</code>	O(1) on average

Operation	Example	Cost
Remove from end	<code>nums.pop()</code>	$O(1)$
Remove from front	<code>nums.pop(0)</code>	$O(n)$ - slow!
Insert at front	<code>nums.insert(0, x)</code>	$O(n)$ - slow!
Search for a value	<code>x in nums</code>	$O(n)$
Sort	<code>nums.sort()</code>	$O(n \log n)$

Because `pop()` from the end is fast, a list works well as a **stack** (last in, first out). For a **queue** (first in, first out), use `collections.deque`, which is covered in the toolkit lesson.

Sorting

```
nums = [4, 1, 3]
nums.sort()                # sorts in place -> [1, 3, 4]
print(sorted([4, 1, 3]))    # returns a NEW sorted list -> [1, 3, 4]
print(sorted([4, 1, 3], reverse=True)) # [4, 3, 1]

pairs = [(1, 'b'), (0, 'z'), (1, 'a')]
pairs.sort(key=lambda p: p[1]) # sort by the second item
print(pairs)                  # [(1, 'a'), (1, 'b'), (0, 'z')]
```

A **lambda** is a tiny function without a name. `lambda p: p[1]` means "given p, return p[1]".

2-D lists (grids)

Many problems use grids: matrices, game boards, maps of islands.

```
rows, cols = 2, 3
grid = [[0] * cols for _ in range(rows)] # correct way
grid[0][1] = 5
print(grid)                             # [[0, 5, 0], [0, 0, 0]]
```

Warning: `[[0] * cols] * rows` looks right but is wrong. It makes every row the **same** list, so changing one row changes all of them. This is a classic interview bug.

```
bad = [[0] * 3] * 2
bad[0][0] = 9
print(bad)                # [[9, 0, 0], [9, 0, 0]] <- both rows changed!
```

Slicing

Slicing takes a part of a list or string. The format is `seq[start:stop:step]`. The `stop` index is **not** included.

```

nums = [10, 20, 30, 40, 50]
print(nums[1:3])      # [20, 30]    (index 1 and 2, not 3)
print(nums[:2])       # [10, 20]   (from the start)
print(nums[3:])       # [40, 50]   (to the end)
print(nums[::-1])     # [50, 40, 30, 20, 10] (reversed)
print(nums[::2])      # [10, 30, 50] (every second item)
print("racecar" == "racecar"[::-1]) # True: palindrome check

```

Important: a slice creates a **copy**. Copying k items costs $O(k)$ time and memory. If you slice inside a loop or a recursive function, your solution can become slower than you think. In interviews, prefer passing indexes (`left`, `right`) instead of slicing, when it matters.

Tuples

A **tuple** is like a list, but it cannot change after creation (it is immutable). Use round brackets.

```

point = (3, 4)
x, y = point          # "unpacking"
print(x, y)           # 3 4
# point[0] = 5        # TypeError: tuples cannot be changed

```

Why use tuples in DSA?

- They can be **dictionary keys** and **set members**, because they are immutable. Lists cannot.
- They are perfect for pairs like `(row, col)` or `(distance, node)`.

```

visited = set()
visited.add((0, 1)) # grid cell (row 0, col 1)
print((0, 1) in visited) # True

```

Dictionaries

A **dictionary** (`dict`) stores **key** -> **value** pairs. Think of a phone contacts list: you look up a name (key) and get a number (value), and you do not need to scan every contact.

```

ages = {"amit": 25, "priya": 30}
ages["ravi"] = 28          # add a new key
ages["amit"] = 26          # update a value
print(ages["priya"])       # 30
print("ravi" in ages)      # True (checks KEYS, O(1) on average)
print(ages.get("zara", 0)) # 0 (default if key is missing)
del ages["ravi"]           # remove a key

```

Looking up, adding and removing a key are **$O(1)$ on average**. This is why dicts appear in so many optimal solutions. The "Prefix sums and how hash maps work" lesson in the web app explains why it is fast.

Looping over a dict

```
ages = {"amit": 25, "priya": 30}
for name in ages:           # Loops over keys
    print(name)
for name, age in ages.items(): # keys and values together
    print(name, age)
print(list(ages.values()))    # [25, 30]
```

Counting with a dict

Counting things is one of the most common DSA tasks.

```
word = "banana"
count = {}
for ch in word:
    count[ch] = count.get(ch, 0) + 1
print(count)           # {'b': 1, 'a': 3, 'n': 2}
```

`collections.Counter(word)` does the same in one line. It is covered in the toolkit lesson, but know the manual way too, because an interviewer may ask you to explain it.

Warning: reading a missing key with square brackets raises an error.

```
d = {}
# print(d["x"])      # KeyError: 'x'
print(d.get("x"))    # None, no error
```

Sets

A **set** is a collection of **unique** values with no order. Think of the list of people who have entered a stadium: you only care whether a person is in or not, and nobody is counted twice.

```
seen = set()           # empty set (note: {} is an empty DICT)
seen.add(3)
seen.add(3)           # duplicate ignored
seen.add(5)
print(seen)           # {3, 5}
print(3 in seen)      # True, O(1) on average
seen.remove(3)        # error if missing; use discard() to be safe
print(len(seen))      # 1
```

Set operations:

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)          # {1, 2, 3, 4} union
print(a & b)          # {2, 3} intersection
print(a - b)          # {1} difference
print(len(set([1, 1, 2])) < 3) # True: the list had a duplicate
```


Rule of thumb: if you are writing `if x in some_list` inside a loop, ask yourself whether `some_list` should be a set. That one change often turns $O(n^2)$ into $O(n)$.

Loops

for loops

A `for` loop goes through each item of a sequence.

```
for fruit in ["apple", "mango"]:
    print(fruit)

for i in range(3):           # 0, 1, 2
    print(i)

for i in range(2, 10, 3):    # start 2, stop before 10, step 3 -> 2, 5, 8
    print(i)

for i in range(5, 0, -1):    # counting down: 5, 4, 3, 2, 1
    print(i)
```

When you need both the index and the value, use `enumerate`:

```
for i, val in enumerate(["a", "b"]):
    print(i, val)           # 0 a, then 1 b
```

To walk two lists together, use `zip`:

```
for x, y in zip([1, 2], ["one", "two"]):
    print(x, y)             # 1 one, then 2 two
```

while loops

A `while` loop runs as long as a condition is true. You use it when you do not know the number of steps in advance, for example in two pointers and binary search.

```
left, right = 0, 4
while left < right:
    print(left, right)      # 0 4, then 1 3
    left += 1
    right -= 1
```

Always make sure something inside the loop moves towards the stop condition. Otherwise you get an **infinite loop**, which looks like your code "hanging".

break and continue

```

for n in [1, 2, 3, 4]:
    if n == 2:
        continue           # skip the rest of this round
    if n == 4:
        break              # leave the loop completely
    print(n)               # prints 1, then 3

```

Functions

A **function** is a named block of code that takes inputs (parameters) and can give back a result with `return`. In interviews you almost always write your solution as a function.

```

def add(a, b):
    """Return the sum of a and b."""
    return a + b

print(add(2, 3))    # Output: 5

```

return

`return` ends the function immediately and sends a value back. If a function has no `return`, it returns `None`.

```

def find_index(nums, target):
    for i, n in enumerate(nums):
        if n == target:
            return i        # stops here as soon as found
    return -1              # only reached if not found

print(find_index([4, 5, 6], 5))    # 1
print(find_index([4, 5, 6], 9))    # -1

```

You can return several values as a tuple:

```

def min_max(nums):
    return min(nums), max(nums)

lo, hi = min_max([3, 9, 1])
print(lo, hi)    # 1 9

```

Default parameters

```

def greet(name, greeting="Hello"):
    return greeting + ", " + name

print(greet("Sahil"))    # Hello, Sahil
print(greet("Sahil", "Hi"))    # Hi, Sahil

```

Warning: never use a mutable default like `def f(x, seen=[])`. The same list is shared between all calls. Use `seen=None` and create the list inside.

Functions inside functions

In interviews, it is common to write a helper function inside the main one, especially for recursion (DFS on trees and graphs). The inner function can read variables of the outer function.

```
def count_paths(n):
    memo = {}
    def helper(k):
        # inner function can see 'memo'
        if k <= 1:
            return 1
        if k not in memo:
            memo[k] = helper(k - 1) + helper(k - 2)
        return memo[k]
    return helper(n)

print(count_paths(5)) # Output: 8
```

Scope

Scope means "where a name can be seen". A variable created inside a function is **local**: it exists only inside that function.

```
def f():
    temp = 10 # local to f
    return temp

f()
# print(temp) # NameError: name 'temp' is not defined
```

The tricky case: an inner function can **read** an outer variable, and can **change the contents** of an outer list or dict. But it cannot **reassign** an outer variable unless you write `nonlocal`.

```
def longest():
    best = 0
    def update(val):
        nonlocal best # without this line: UnboundLocalError
        best = max(best, val)
    for v in [3, 7, 2]:
        update(v)
    return best

print(longest()) # Output: 7
```

This pattern appears in tree problems like "diameter of a binary tree", where a helper updates a running best answer. An alternative is to store the answer in a list, `best = [0]`, and update `best[0]`, which needs no `nonlocal`.

None and truthiness

None

`None` means "no value". It is like an empty seat on a train: the seat exists, but nobody is sitting there. In DSA, `None` marks the end of a linked list or a missing child in a tree.

```
node = None
if node is None:           # use 'is' to compare with None
    print("empty")
```

Always use `is None` or `is not None`, not `== None`.

Truthiness

In an `if` or `while`, Python treats some values as false. These are called **falsy** values:

- `False`, `None`, `0`, `0.0`
- empty containers: `""`, `[]`, `{}`, `set()`, `()`

Everything else is **truthy**.

```
nums = []
if not nums:
    print("list is empty")    # this prints

stack = [1, 2]
while stack:                 # runs while the stack is not empty
    print(stack.pop())        # 2, then 1
```

A trap: `if node:` is False when `node` is `None`, but `if val:` is also False when `val` is `0`. If `0` is a valid value (for example, a tree node with value `0`, or index `0`), use `is not None`.

```
idx = 0
if idx:                # wrong check: 0 is falsy
    print("found")     # does NOT print, but index 0 was valid
if idx is not None:    # correct
    print("found")     # prints
```

Classes for ListNode and TreeNode

A **class** is a blueprint for creating objects. Each object has its own data (attributes). For DSA, you only need classes for simple node types. Think of a class as a form template (like a railway reservation form), and each object as one filled form.

ListNode (linked list)

A linked list is a chain of nodes. Each node holds a value and a pointer to the next node.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val      # the data in this node
        self.next = next    # the next node, or None at the end

# Build 1 -> 2 -> 3
head = ListNode(1, ListNode(2, ListNode(3)))

cur = head
while cur:                  # stop when cur becomes None
    print(cur.val)          # 1, 2, 3
    cur = cur.next
```

`__init__` is the special method that runs when you create an object. `self` means "this object".

TreeNode (binary tree)

A binary tree node has a value and up to two children: `left` and `right`.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

#      1
#     / \
#    2   3
root = TreeNode(1, TreeNode(2), TreeNode(3))

def tree_sum(node):
    if node is None:          # base case: empty tree
        return 0
    return node.val + tree_sum(node.left) + tree_sum(node.right)

print(tree_sum(root))        # Output: 6
```

On sites like LeetCode, these classes are given to you. In a Google interview you may need to write them yourself, so practise typing them from memory.

A class with methods

Some problems ask you to design a class, such as "MinStack" or "LRU Cache".

```
class Counter:
    def __init__(self):
        self.count = 0
    def increment(self):
        self.count += 1
        return self.count

c = Counter()
c.increment()
print(c.increment())    # Output: 2
```

List comprehensions

A **list comprehension** builds a list in one line. It is short and fast, and interviewers like it when it stays readable.

```
squares = [x * x for x in range(5)]           # [0, 1, 4, 9, 16]
evens = [x for x in range(10) if x % 2 == 0]  # [0, 2, 4, 6, 8]
words = ["Hi", "Bye"]
lengths = [len(w) for w in words]             # [2, 3]
```

The same idea works for sets and dicts:

```
unique_lens = {len(w) for w in ["a", "bb", "cc"]} # {1, 2}
index_of = {ch: i for i, ch in enumerate("abc")}  # {'a': 0, 'b': 1, 'c': 2}
```

Do not cram complex logic into one line. If a comprehension needs two `if` conditions and nested loops, write a normal loop. Clear code earns more points than clever code.

Useful built-in functions

Function	Example	Result
<code>len</code>	<code>len([1, 2])</code>	2
<code>min, max</code>	<code>max([3, 8])</code>	8
<code>sum</code>	<code>sum([1, 2, 3])</code>	6
<code>abs</code>	<code>abs(-4)</code>	4
<code>sorted</code>	<code>sorted("cab")</code>	<code>['a', 'b', 'c']</code>
<code>reversed</code>	<code>list(reversed([1, 2]))</code>	<code>[2, 1]</code>
<code>any, all</code>	<code>any([False, True])</code>	True
<code>range</code>	<code>list(range(3))</code>	<code>[0, 1, 2]</code>
<code>divmod</code>	<code>divmod(7, 2)</code>	<code>(3, 1)</code>

Common errors and how to fix them

Under interview pressure, everyone makes small mistakes. Knowing the common ones helps you spot them during your dry run.

Error	Typical cause	Fix
<code>IndexError: list index out of range</code>	Reading <code>nums[i]</code> when <code>i == len(nums)</code> , or reading <code>nums[0]</code> on an empty list	Check loop bounds; handle empty input first
<code>KeyError</code>	Reading <code>d[key]</code> when the key is missing	Use <code>d.get(key, default)</code> or check <code>key in d</code>
<code>TypeError: 'NoneType' object ...</code>	Calling <code>.val</code> or <code>.next</code> on <code>None</code>	Check if node is not <code>None</code> before using it
<code>UnboundLocalError</code>	Reassigning an outer variable inside an inner function	Use <code>nonlocal</code>
<code>RecursionError</code>	Missing or wrong base case; or very deep recursion	Fix the base case; use an iterative version for deep inputs
Infinite loop	<code>while</code> condition never becomes false	Make sure pointers move every round
Wrong answer from grid	<code>[[0] * c] * r</code> shares rows	Use <code>[[0] * c for _ in range(r)]</code>
Off-by-one	<code>range(n)</code> vs <code>range(n + 1)</code> , <code><</code> vs <code><=</code>	Trace with a 1- or 2-element example

A note on recursion depth: Python's default recursion limit is about 1000 calls. For a very deep tree or a long linked list, recursive code may crash. You can mention this in the interview and offer an iterative version with your own stack. Saying this shows maturity.

One more silent bug: modifying a list while looping over it.

```
nums = [1, 2, 2, 3]
# for n in nums:
#     if n == 2: nums.remove(n) # skips items, gives wrong result
nums = [n for n in nums if n != 2] # safe: build a new list
print(nums)                       # Output: [1, 3]
```

Putting it together: a small full solution

Here is a complete interview-style function using many ideas from this chapter: a dict, a loop, `enumerate`, a clear return, and a small test.

```
def first_unique_char(s):
    """Return the index of the first non-repeating character, or -1."""
    count = {}
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
    for i, ch in enumerate(s):
        if count[ch] == 1:
            return i
    return -1

assert first_unique_char("leetcode") == 0
assert first_unique_char("loveleetcode") == 2
assert first_unique_char("aabb") == -1
assert first_unique_char("") == -1 # edge case: empty string
print("all tests passed")
```

Time is $O(n)$ because we walk the string twice. Space is $O(1)$ if the alphabet is fixed (at most 26 lowercase letters), or $O(k)$ for k distinct characters in general.

Tester's corner

- Python's falsy values are a source of real bugs. As a tester, always ask: "What if the value is 0 or an empty string?" when you see `if x:`.
- The shared-row grid bug (`[[[0]*c]*r]`) is a good example of aliasing: two names point to the same object. You have probably seen similar bugs with shared test fixtures.
- `assert` statements are a fast way to test a function in an interview. You already use asserts in `pytest`; the same habit works here.
- Mutable default arguments behave like shared state between tests. Avoid them for the same reason you avoid test order dependencies.
- When an error occurs in your head during a dry run, name it precisely ("this line raises `KeyError` when the key is new"). That is how you would write a good bug report.
- Recursion limits are a scale boundary. Mentioning them is like noting a load limit in a performance test.

Key takeaways

- Use `//` for integer division, `%` for remainder, and `float('inf')` for infinity.
- Lists are fast at the end (`append`, `pop`) and slow at the front; strings and tuples are immutable.
- Dicts and sets give $O(1)$ average lookups; use them to replace slow `in list` checks.
- Slices copy data and cost $O(k)$; pass indexes when it matters.
- Use `is None` checks; remember that 0 and empty containers are falsy.
- Write `ListNode` and `TreeNode` from memory; use inner functions and `nonlocal` for recursive helpers.
- Know the common errors (`IndexError`, `KeyError`, `NoneType`, off-by-one, shared grid rows) and look for them in your dry run.

Quiz

1. What does `7 // 2` return in Python? A) 3.5 B) 3 C) 4 D) 1
2. True or false: you can change a character in a Python string with `s[0] = "x"`.
3. Which operation is slow ($O(n)$) on a Python list? A) `nums.append(x)` B) `nums.pop()` C) `nums.pop(0)` D) `nums[5]`
4. What is wrong with `grid = [[0] * 3] * 3`?
5. What does `nums[::-1]` do?
6. Why can a tuple be a dictionary key but a list cannot?
7. What does `d.get("x", 0)` return if "x" is not in `d`?
8. Your inner helper function does `best = max(best, val)` and crashes with `UnboundLocalError`. What would you do?
9. True or false: `if idx:` is a safe way to check that an index was found.
10. Which of these values is truthy? A) `[]` B) `0` C) `"0"` D) `None`

Answer key

1. **B** - `//` is floor division, so 7 divided by 2 rounds down to 3.
2. **False** - Strings are immutable; you must build a new string.
3. **C** - Removing from the front shifts every other element, which is $O(n)$.
4. **Shared rows** - All three rows are the same list object, so changing one cell changes that column in every row. Use a list comprehension.
5. **Reverses the sequence** - A slice with step -1 returns a reversed copy.
6. **Immutability** - Dict keys must be hashable, which requires the value not to change; tuples are immutable, lists are not.
7. **0** - `get` returns the default value instead of raising `KeyError`.
8. **Add nonlocal best** - This tells Python that `best` belongs to the outer function. Or store it in a one-element list.
9. **False** - Index 0 is falsy, so a found index of 0 would look like "not found". Use `is not None` or compare with -1.
10. **C** - `"0"` is a non-empty string, so it is truthy. The others are falsy.

Flashcards

- **Q:** What is the difference between `/` and `//`? — **A:** `/` gives a float result; `//` gives floor (rounded-down) division.
- **Q:** How do you represent infinity in Python? — **A:** `float('inf')` and `float('-inf')`.
- **Q:** Which list operations are $O(1)$? — **A:** Index access, `append`, and `pop()` from the end.
- **Q:** How do you create a 2-D grid correctly? — **A:** `[[0] * cols for _ in range(rows)]`.
- **Q:** What does a slice cost? — **A:** $O(k)$ time and space, because it copies k elements.
- **Q:** What is the average cost of a dict or set lookup? — **A:** $O(1)$.

- **Q:** How do you count items with a plain dict? — **A:** `count[x] = count.get(x, 0) + 1`.
- **Q:** What are Python's falsy values? — **A:** False, None, 0, 0.0, and empty containers like "", [], {}, set(), ().
- **Q:** How should you compare with None? — **A:** Use `is None` or `is not None`.
- **Q:** What does `nonlocal` do? — **A:** Lets an inner function reassign a variable from the enclosing function.
- **Q:** What fields does a `TreeNode` have? — **A:** `val`, `left`, and `right`.
- **Q:** What is Python's default recursion limit roughly? — **A:** About 1000 calls; use an iterative approach for very deep inputs.

A Problem-Solving Framework

In this chapter:

- Follow a nine-step framework for any coding problem
- Use input constraints to guess the target time complexity
- Practise in a way that builds real skill: the 15-20 minute rule, re-coding from memory, spaced repetition and a mistakes log
- Recognise common patterns from signals in the problem statement

Time: ~45 minutes | **Level:** Beginner

Why you need a framework

When you see a new problem in an interview, your brain may freeze. A framework is a fixed checklist that tells you what to do next, even when you feel stuck. Pilots use checklists before take-off, not because they forget how to fly, but because stress makes people skip steps.

As a tester, you already think in steps: understand the requirement, design the cases, execute, report. Solving a coding problem is very similar. You will use the same framework in every practice session, so in the real interview it runs automatically.

The nine steps

Step	Name	Rough time in a 45-minute round
1	Understand with examples	2-3 min
2	Ask clarifying questions	2-3 min
3	Brute force first	2-3 min
4	Find the pattern and optimise the idea	5-8 min
5	Plan in comments	2-3 min
6	Code	10-15 min
7	Test with cases	5-7 min
8	Analyse complexity	1-2 min
9	Optimise further or discuss follow-ups	remaining time

We will walk through each step using one running example:

Problem: Given a list of integers `nums` and an integer `k`, return the length of the longest contiguous subarray whose sum is at most `k`. All numbers are positive.

Step 1: Understand with examples

Read the problem twice. Then restate it in your own words, aloud. Then make your own small example and solve it **by hand**.

"So I need the longest block of neighbours, without gaps, whose total is at most `k`."

Example: `nums = [2, 1, 3, 1, 1]`, `k = 4`.

- `[2, 1]` sums to 3, length 2.
- `[3, 1]` sums to 4, length 2.
- `[1, 1]` at the end sums to 2, length 2.
- `[3, 1, 1]` sums to 5, too big.
- `[1, 3]` sums to 4, length 2.

So the answer is 2. Solving by hand does two things: it confirms you understand, and it often shows you the pattern.

Step 2: Ask clarifying questions

A clarifying question removes doubt about the input or output. Good questions sound like test design:

- "Can the list be empty? What should I return then?" (Say 0.)
- "Are all numbers positive, or can they be zero or negative?" (This changes the pattern completely.)
- "Can `k` be zero or negative?"
- "How large can the list be?" (This tells you the target complexity; see later.)
- "Do you want the length or the subarray itself?"

Do not ask 15 questions. Ask the 3-5 that change your solution.

Step 3: Brute force first

The **brute force** is the simplest correct solution, even if slow. Say it aloud before optimising:

"The simple way: try every start index, extend to every end index, keep a running sum, and track the best length. That is $O(n^2)$ time and $O(1)$ space."

Why say it? It proves you can solve the problem at all. It gives a fallback. And it gives you something to improve. Do not always code it; usually you just describe it and ask, "Should I code this, or look for a faster approach?"

```
def longest_brute(nums, k):
    best = 0
    for start in range(len(nums)):
        total = 0
        for end in range(start, len(nums)):
            total += nums[end]
            if total > k:
                break          # positives: sum only grows
            best = max(best, end - start + 1)
    return best

print(longest_brute([2, 1, 3, 1, 1], 4))  # Output: 2
```

Step 4: Find the pattern

Now look for repeated work in the brute force. Ask yourself:

- What work am I repeating? (Here: we re-add the same numbers for every start.)
- Is there a known pattern that fits? (Contiguous subarray + all positive numbers = **sliding window**.)
- Would sorting, a hash map, two pointers, a heap, or binary search help?

Because all numbers are positive, when the window sum is too big, moving the left edge right always makes it smaller. That is the signal for a sliding window. Explain this reason aloud. The interviewer wants the "why", not just the name.

Step 5: Plan in comments

Before writing real code, write the plan as short comments. This is like writing test steps before automating them.

```
def longest_at_most_k(nums, k):
    # left = start of window, total = window sum, best = answer
    # for each right index:
    #     add nums[right] to total
    #     while total > k: remove nums[left], move left forward
    #     update best with window length
    # return best
    pass
```

Ask: "Does this plan look good before I code it?" The interviewer can correct you early, which saves time.

Step 6: Code

Now turn each comment into code. Use clear names. Keep talking, but more quietly: "Now I shrink the window while it is too big."

```
def longest_at_most_k(nums, k):
    left, total, best = 0, 0, 0
    for right, num in enumerate(nums):
        total += num # grow window to the right
        while total > k and left <= right:
            total -= nums[left] # shrink from the left
            left += 1
        best = max(best, right - left + 1)
    return best

print(longest_at_most_k([2, 1, 3, 1, 1], 4)) # Output: 2
```

Step 7: Test with cases

Never say "I think it works" without testing. First, **dry run** the given example line by line, saying variable values aloud. Then test edge cases:

- Empty list: `[]` -> loop does not run -> returns 0. Correct.
- Single element bigger than k: `[5]`, `k=4` -> total 5, shrink, `left=1`, length 0. Correct.
- Everything fits: `[1, 1, 1]`, `k=10` -> 3. Correct.

Chapter dsa-30 goes deep on this step. It is your strongest step as a test engineer.

Step 8: Analyse complexity

State time and space, and explain why:

"Time is $O(n)$. Each element enters the window once and leaves at most once, so the inner while loop runs at most n times in total. Space is $O(1)$, just a few variables."

Step 9: Optimise or discuss follow-ups

If time remains, discuss improvements or follow-ups the interviewer raises:

- "What if numbers can be negative?" Then the window trick breaks, because shrinking no longer always reduces the sum. You would need prefix sums with a different technique.
- "What if the data is a stream?" Then you keep only the window state.

Even if you do not code it, talking about trade-offs shows depth.

Reading constraints to guess the target complexity

The **constraints** are the limits on input size, like `1 <= n <= 10^5`. They are a huge clue. A rough rule: a normal computer does around 10^7 to 10^8 simple operations per second. Interview judges often allow about one second. So the input size tells you which complexity will pass.

Input size n	Target complexity	Typical patterns
<code>n <= 10-12</code>	$O(n!)$ or $O(2^n \cdot n)$	Backtracking, permutations

Input size n	Target complexity	Typical patterns
$n \leq 20-25$	$O(2^n)$	Subsets, bitmask DP
$n \leq 500$	$O(n^3)$	3 nested loops, some interval DP
$n \leq 5,000$	$O(n^2)$	2-D DP, nested loops
$n \leq 10^5$ to 10^6	$O(n \log n)$ or $O(n)$	Sorting, heap, binary search, two pointers, sliding window, hash map
n up to 10^9 or more	$O(\log n)$ or $O(1)$	Binary search on the answer, math

These are rough guides, not laws. In a Google interview, constraints are often not given. Then **ask**: "How big can the input be?" If the interviewer says "very large", aim for $O(n)$ or $O(n \log n)$.

Think of it like planning a trip. If you have to travel 5 km, you can walk. If you have to travel 500 km, you need a train. The distance (input size) decides the vehicle (algorithm).

How to practise

Solving many problems is not enough. **How** you practise matters more than how many.

The 15-20 minute rule

Set a timer when you start a problem.

- If you have a clear approach within 15-20 minutes, continue and code it.
- If you are stuck for 15-20 minutes with **no new idea**, stop. Look at a hint, or the approach only.

Why? Struggling for a while builds skill. Struggling for two hours mostly builds frustration. The goal is to learn the pattern, then practise using it.

Write your approach before code

Before you type any code, write 3-5 lines in plain English: the idea, the data structure, and the complexity. If you cannot write the approach, you are not ready to code. This also trains you to explain aloud in interviews.

Watch only the approach in videos

When you use a solution video (NeetCode and others), watch only until the idea is clear. Then **pause** and write the code yourself. Watching someone else type the code feels like learning, but it is like watching a cooking show: you will not be able to cook the dish later.

Re-code from memory

After you understand a solution, close everything and write it again from a blank file. If you get stuck, note exactly where, look once, and start again from blank. This is the single most effective practice habit.

Spaced repetition: days 1, 3, 7, 14, 30

Review each problem on a schedule:

Day	Task
1	Solve or learn it, then re-code from memory
3	Re-code from blank, no hints
7	Re-code with a 20-minute timer
14	Explain the approach aloud in 2 minutes
30	Re-code once more

If you fail a review, reset the schedule for that problem to day 1. A simple spreadsheet works well: problem, pattern, date, next review date, status.

Keep a mistakes log

A **mistakes log** is a list of every error you make while practising, with the root cause. It is your personal bug tracker.

Date	Problem	Mistake	Root cause	Fix rule
Oct 7	Two Sum	Returned values, not indices	Did not re-read output format	Re-read output before coding
Oct 9	Valid Palindrome	Crashed on empty string	Skipped edge cases	Always test empty input
Oct 12	Binary Search	Infinite loop	Used <code>left = mid</code>	Move pointer past mid

Read the log every Sunday before your weekly test. After a few weeks you will see your top 3 repeat mistakes. Fixing those gives the biggest improvement.

Pattern recognition cheat sheet

A **pattern** is a reusable way of solving a family of problems. The signal is a phrase or feature in the problem that points to a pattern. This table is your quick map. Each pattern gets its own chapter in this volume.

Signal in the problem	Likely pattern
"Find if a pair or value exists", fast lookup, counting	Hash map / hash set
Sorted array, find pair or triple with a target sum	Two pointers
Contiguous subarray or substring, "longest" / "shortest" / "at most k"	Sliding window

Signal in the problem	Likely pattern
Sum of a range, many range-sum queries, subarray sum equals k	Prefix sums (+ hash map)
Sorted input, or "minimum value that works" / "maximum that works"	Binary search (on index or on the answer)
Matching brackets, "next greater element", undo	Stack / monotonic stack
"Top k", "k-th largest", merge k sorted lists, running median	Heap (priority queue)
Linked list cycle, middle of list	Fast and slow pointers
Tree traversal, depth, path sums	DFS (recursion) on trees
Level by level, shortest path in an unweighted graph or grid	BFS with a queue
Islands in a grid, connected groups	DFS / BFS or union-find
Tasks with prerequisites, build order	Topological sort
Shortest path with weights	Dijkstra
"All combinations", "all permutations", "all subsets"	Backtracking
"Number of ways", "min cost", "max profit" with choices that overlap	Dynamic programming
Prefix of words, autocomplete	Trie
Overlapping time ranges, meeting rooms	Sort intervals + sweep / heap
Choose the locally best option and it is provably safe	Greedy
Max/min in every window of size k	Monotonic deque

Use the table as a starting point, not a final answer. Many problems combine two patterns, for example "sort + two pointers" or "BFS + hash set".

The framework as a script

Here is a compact script you can memorise. Say these lines in practice until they feel natural:

1. "Let me restate the problem to make sure I understand."
2. "Let me try a small example by hand."
3. "A few questions: can the input be empty? Can there be duplicates or negatives? How large can it be?"
4. "The brute force is ..., which is $O(\dots)$. Let me look for something better."
5. "I notice ..., which suggests a ... pattern, because ..."
6. "Here is my plan in comments. Does this look good?"
7. "Now I will code it." (Code, while narrating key decisions.)
8. "Let me trace through my example, and then test edge cases."
9. "Time is $O(\dots)$ because ..., space is $O(\dots)$ because ..."
10. "If we had more time, I would ... / A follow-up could be ..."

Tester's corner

- Steps 1-2 are requirement analysis. Clarifying questions are the same as questions you ask a product owner about a vague user story.
- The brute force is like a baseline test: simple, known to be correct, used to compare a faster version.
- Planning in comments is like writing test steps before automation. It catches logic errors cheaply.
- Step 7 (testing) is where you can stand out. Use equivalence classes and boundary values, just as you do at work.
- The mistakes log is a root cause analysis habit. Track repeat defects and fix the process, not just the instance.
- Constraint reading is like reading non-functional requirements: the load target decides the architecture.

Key takeaways

- Use nine steps: understand, clarify, brute force, find the pattern, plan, code, test, analyse, optimise.
- Always state a brute force first, then explain why a better pattern applies.
- Input size predicts target complexity: around 10^5 means $O(n \log n)$ or $O(n)$.
- Follow the 15-20 minute rule: when stuck with no new idea, take a hint and learn the pattern.
- Write the approach in English before code, watch only the approach in videos, and re-code from memory.
- Review problems on days 1, 3, 7, 14 and 30, and keep a mistakes log with root causes.
- Use signal-to-pattern mapping as a starting point; many problems combine patterns.

Quiz

1. What is the first step of the framework? A) Code B) Analyse complexity C) Understand with examples D) Optimise
2. Why should you state a brute force solution first?
3. The input size is up to 10^5 . Which target complexity is most reasonable? A) $O(2^n)$ B) $O(n^3)$ C) $O(n^2)$ D) $O(n \log n)$
4. True or false: if you are stuck for 15-20 minutes with no new idea, you should keep trying for at least two more hours.
5. What should you write before writing any code?
6. The problem says "longest substring with at most k distinct characters". Which pattern is most likely?
7. True or false: watching a full video of someone coding the solution is the best way to learn it.
8. You keep getting off-by-one errors in binary search. What would you do about it in your practice system?
9. Which signal points to a heap? A) "Matching brackets" B) "Top k most frequent" C) "All subsets" D) "Islands in a grid"
10. The interviewer gives no constraints. What would you do?

Answer key

1. **C** - Understand the problem by restating it and working a small example by hand.
2. **Baseline and safety** - It proves you can solve it, gives a fallback, and shows the repeated work you can optimise.
3. **D** - With 10^5 elements, $O(n^2)$ is about 10^{10} operations, too slow; $O(n \log n)$ or $O(n)$ fits.
4. **False** - Use the 15-20 minute rule: take a hint, learn the pattern, and re-code from memory later.
5. **The approach in plain English** - 3-5 lines with the idea, data structure and complexity, then a plan in comments.
6. **Sliding window** - "Longest substring" with an "at most k" condition is a classic sliding window signal, usually with a hash map of counts.
7. **False** - Watch only until the idea is clear, then write the code yourself; watching typing gives false confidence.
8. **Log it and add a rule** - Add it to the mistakes log with the root cause and a fix rule, review it weekly, and add binary search problems to the review schedule.
9. **B** - "Top k" problems usually use a heap.
10. **Ask about input size** - Ask how large the input can be; if "very large", aim for $O(n)$ or $O(n \log n)$.

Flashcards

- **Q:** What are the nine steps of the framework? — **A:** Understand, clarify, brute force, find pattern, plan in comments, code, test, analyse complexity, optimise.
- **Q:** Why work an example by hand first? — **A:** It confirms understanding and often reveals the pattern.
- **Q:** What target complexity fits n up to 10^5 ? — **A:** $O(n \log n)$ or $O(n)$.
- **Q:** What target complexity fits n up to about 20? — **A:** $O(2^n)$, such as subsets or bitmask DP.
- **Q:** What is the 15-20 minute rule? — **A:** If stuck that long with no new idea, take a hint or look at the approach only.
- **Q:** What is the most effective practice habit? — **A:** Re-coding a solution from memory on a blank file.
- **Q:** What is the spaced repetition schedule? — **A:** Review on days 1, 3, 7, 14 and 30; reset to day 1 if you fail.
- **Q:** What goes in a mistakes log? — **A:** Date, problem, mistake, root cause, and a fix rule.
- **Q:** What signal suggests sliding window? — **A:** A contiguous subarray or substring with "longest", "shortest" or "at most k".
- **Q:** What signal suggests topological sort? — **A:** Tasks with prerequisites or a build order.
- **Q:** What signal suggests backtracking? — **A:** "Generate all combinations, permutations or subsets."

Foundations 1: Big-O and complexity analysis

What Big-O means

Big-O tells you how the running time (or memory) grows when the input size n grows.

We do not count exact seconds. We count how many basic steps happen, and keep only the biggest term.

Example: $3n^2 + 5n + 10$ steps is $O(n^2)$. Constants and smaller terms are dropped.

Big-O usually means the worst case. Interviewers expect worst case unless they ask for average.

How to count loops

- One loop over n items: $O(n)$.
- Two loops one after another: $O(n) + O(n) = O(n)$.
- A loop inside a loop, both over n : $O(n * n) = O(n^2)$.
- Loops over different inputs: $O(n * m)$, not $O(n^2)$. Keep both letters.
- A loop that halves (or doubles) i each time: $O(\log n)$.

```
def examples(nums):
    n = len(nums)
    total = 0
    for x in nums:           # O(n)
        total += x
    for i in range(n):       # O(n^2): nested
        for j in range(i + 1, n):
            total += nums[i] * nums[j]
    i = n
    while i > 1:              # O(log n): halving
        i //= 2
    return total              # overall O(n^2)
```

The inner loop from $i + 1$ runs $n-1, n-2, \dots, 1$ times. The sum is about $n^2 / 2$, which is still $O(n^2)$.

Why halving gives $\log n$

If you cut n in half each step, after k steps you have $n / 2^k$. You stop when this is 1, so $k = \log_2(n)$.

For $n = 1,000,000$ that is only about 20 steps. Binary search works like this.

Amortised $O(1)$: list append

A Python list has extra hidden space. `append` is $O(1)$ most of the time.

When the space is full, Python makes a bigger array (about $1.125x$ to $2x$) and copies everything: $O(n)$ once.

Because this copy happens rarely, the average cost over many appends is $O(1)$. We call this amortised $O(1)$.

Space complexity

- Count extra memory you create: new lists, dicts, sets.
- Do not count the input itself (unless you copy it).
- Recursion uses the call stack. A recursion of depth d uses $O(d)$ space even with no lists.
- Example: recursive DFS on a linked list of n nodes uses $O(n)$ stack space.

```
def sum_rec(nums, i=0):  
    if i == len(nums):  
        return 0  
    return nums[i] + sum_rec(nums, i + 1)    # O(n) time, O(n) stack space
```

Common complexities ranked (fast to slow)

1. $O(1)$: dict lookup, list index.
2. $O(\log n)$: binary search, heap push/pop.
3. $O(n)$: one scan.
4. $O(n \log n)$: sorting.
5. $O(n^2)$: all pairs.
6. $O(2^n)$: all subsets.
7. $O(n!)$: all permutations.

Input size tells you the target complexity

A computer does roughly 10^7 to 10^8 simple steps per second in Python-ish terms. Use this guide:

- $n \leq 10$ to 12 : $O(n!)$ is OK (permutations).
- $n \leq 20$ to 25 : $O(2^n)$ is OK (subsets, bitmasks).
- $n \leq 500$: $O(n^3)$ is OK.
- $n \leq 5,000$: $O(n^2)$ is OK.
- $n \leq 10^5$ to 10^6 : you need $O(n \log n)$ or $O(n)$.
- n up to 10^9 or more: you need $O(\log n)$ or $O(1)$ (binary search, maths).

Read the constraints first. They often tell you which algorithm the interviewer wants.

How to say complexity in an interview

- Say both time and space: "This is $O(n \log n)$ time because of the sort, and $O(n)$ space for the hashmap."
- Name the variables: " n is the number of words and k is the max word length, so $O(n * k)$."
- Explain where it comes from: "The outer loop runs n times and the inner work is $O(1)$."
- Mention the recursion stack if you use recursion.
- If asked "can you do better?", look at the input size table and think of the next faster class.

Videos

- [Big-O notation for coding interviews](#) - NeetCode (English)
- [Time complexity for coding interviews](#) - Apna College (Hindi)

Quiz

1. What is the Big-O of $4n^2 + 100n + 7$?
 - A) $O(n)$
 - B) $O(n^2)$
 - C) $O(4n^2)$
 - D) $O(100n)$
2. A loop does $i = i // 2$ until i is 1. What is its complexity?
 - A) $O(n)$
 - B) $O(\log n)$
 - C) $O(n^2)$
 - D) $O(1)$
3. If $n \leq 10^5$, which complexity should you aim for?
 - A) $O(2^n)$
 - B) $O(n^2)$
 - C) $O(n \log n)$ or better
 - D) $O(n!)$
4. A recursive function goes n levels deep and creates no lists. What is its space?
 - A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n)$
 - D) $O(n^2)$

Answer key

1. **B** - $O(n^2)$. We drop constants and keep only the biggest term.
2. **B** - $O(\log n)$. Halving n repeatedly takes about $\log_2(n)$ steps.
3. **C** - $O(n \log n)$ or better. n^2 would be 10^{10} steps, which is far too slow.
4. **C** - $O(n)$. Each recursive call uses a stack frame, so depth n means $O(n)$ space.

Foundations 2: Recursion from zero

What recursion is

Recursion is when a function calls itself on a smaller version of the same problem.

Real-life picture: Russian dolls. To find the smallest doll, open one doll and repeat on the doll inside, until there is no doll inside.

Every recursive function needs two parts:

- **Base case:** the smallest input where you return the answer directly. This stops the recursion.
- **Recursive case:** break the problem into a smaller one, call yourself, and use the result.

First example: factorial

```
def fact(n):  
    if n <= 1:           # base case  
        return 1  
    return n * fact(n - 1) # recursive case: smaller n
```

If you forget the base case, the function never stops and Python raises `RecursionError`.

The call stack, drawn in text

Each call waits for the call below it. Python keeps them on a stack.

1. `fact(4)` waits for `fact(3)`
2. `fact(3)` waits for `fact(2)`
3. `fact(2)` waits for `fact(1)`
4. `fact(1)` returns 1 (base case)
5. `fact(2)` returns $2 * 1 = 2$
6. `fact(3)` returns $3 * 2 = 6$
7. `fact(4)` returns $4 * 6 = 24$

The stack grows going down (steps 1 to 4) and shrinks coming back up (steps 5 to 7).

Max stack depth here is 4, so space is $O(n)$.

Recursion tree: Fibonacci

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$, with $\text{fib}(0) = 0$ and $\text{fib}(1) = 1$.

```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

Each call makes two calls. The tree for fib(5):

- fib(5) calls fib(4) and fib(3)
- fib(4) calls fib(3) and fib(2)
- fib(3) calls fib(2) and fib(1) (and this happens twice!)
- fib(2) is computed 3 times, fib(1) 5 times

The tree has about 2^n nodes, so time is $O(2^n)$. Very slow for $n = 50$.

Memoisation: remember answers

The tree repeats the same calls. Save each answer in a dict the first time, and reuse it.

```
def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n < 2:
        return n
    if n in memo:
        return memo[n]
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

from functools import lru_cache
@lru_cache(None)          # same idea, one line
def fib_cache(n):
    return n if n < 2 else fib_cache(n - 1) + fib_cache(n - 2)
```

Now each n is computed once: $O(n)$ time and $O(n)$ space. This is the start of dynamic programming.

How to write a recursive function (3 questions)

1. What is the smallest input I can answer directly? (base case)
2. If I trust my function for a smaller input, how do I build the answer? (leap of faith)
3. Does every call move closer to the base case?

Tree problems are perfect for this: $\text{depth}(\text{node}) = 1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$, base case node is None.

```
def max_depth(node):
    if not node:
        return 0
    return 1 + max(max_depth(node.left), max_depth(node.right))
```


Recursion vs iteration

- Any recursion can be written with a loop and your own stack.
- Recursion is shorter for trees, graphs, backtracking and divide-and-conquer.
- Loops avoid stack overflow and are a bit faster in Python.
- Simple linear recursion (like factorial) is usually better as a loop.

```
def fact_loop(n):  
    res = 1  
    for i in range(2, n + 1):  
        res *= i  
    return res
```

Python recursion limit

Python stops at about 1000 nested calls by default (RecursionError).
A DFS on a 10^4 or 10^5 node linked list or a big grid can hit this limit.

```
import sys  
sys.setrecursionlimit(10**6)  # raise it, but very deep stacks can still crash
```

In interviews, say: "If the depth can be large, I would switch to an iterative version with a stack."

Videos

- [A better way to understand recursion](#) - Alex Hyett (English)
- [Easiest way to learn recursion](#) - Engineering Digest (Hindi)

Quiz

1. What happens if a recursive function has no base case?
 - A) It returns None
 - B) It runs forever until RecursionError
 - C) It becomes a loop
 - D) It returns 0
2. What is the time complexity of plain recursive fib(n)?
 - A) $O(n)$
 - B) $O(\log n)$
 - C) $O(2^n)$
 - D) $O(n^2)$
3. What does memoisation do?
 - A) Sorts the input
 - B) Saves answers to sub-problems so they are computed once

- C) Removes the base case
 - D) Makes recursion use no memory
4. What is the default recursion limit in Python, roughly?
- A) 100
 - B) 1000
 - C) 100000
 - D) No limit

Answer key

1. **B** - It runs forever until RecursionError. Nothing stops the calls, so the stack grows until Python stops it.
2. **C** - $O(2^n)$. Each call makes two more calls, so the tree has about 2^n nodes.
3. **B** - Saves answers to sub-problems so they are computed once. Storing results turns repeated work into a quick lookup.
4. **B** - 1000. CPython defaults to about 1000 nested calls.

Foundations 3: Sorting algorithms you must know

Why sorting matters

In interviews you almost always call Python `sorted()`. But you must know how merge sort and quick sort work, their complexity, and when sorting first makes a problem easy.

Bubble sort and insertion sort (idea only)

- **Bubble sort:** swap neighbours that are in the wrong order, again and again. The biggest value "bubbles" to the end. $O(n^2)$.
- **Insertion sort:** like sorting playing cards in your hand. Take the next card and slide it left into its place. $O(n^2)$, but $O(n)$ if the list is almost sorted.

```
def insertion_sort(a):
    for i in range(1, len(a)):
        cur, j = a[i], i - 1
        while j >= 0 and a[j] > cur:
            a[j + 1] = a[j]      # shift right
            j -= 1
        a[j + 1] = cur
```

Merge sort

Divide the list into two halves, sort each half (recursion), then merge two sorted lists.

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
    res, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:    # <= keeps it stable
            res.append(left[i]); i += 1
        else:
            res.append(right[j]); j += 1
    res.extend(left[i:]); res.extend(right[j:])
    return res
```

Dry run on [5, 2, 4, 1]:

1. Split into [5, 2] and [4, 1].

2. [5, 2] splits to [5] and [2]; merge gives [2, 5].
 3. [4, 1] splits to [4] and [1]; merge gives [1, 4].
 4. Merge [2, 5] and [1, 4]: compare 2 vs 1, take 1. Compare 2 vs 4, take 2. Compare 5 vs 4, take 4. Add 5.
 5. Result: [1, 2, 4, 5].
- Time $O(n \log n)$ always: $\log n$ levels, $O(n)$ merge work per level. Space $O(n)$.

Quick sort

Pick a pivot. Put smaller values on the left and bigger on the right (partition). Then sort both sides.

```
def quick_sort(a, lo=0, hi=None):
    if hi is None:
        hi = len(a) - 1
    if lo >= hi:
        return
    pivot, p = a[hi], lo          # Lomuto partition
    for i in range(lo, hi):
        if a[i] < pivot:
            a[i], a[p] = a[p], a[i]
            p += 1
    a[p], a[hi] = a[hi], a[p]     # pivot to its final place
    quick_sort(a, lo, p - 1)
    quick_sort(a, p + 1, hi)
```

Partition dry run on [3, 7, 1, 5, 4] with pivot = 4, p = 0:

1. i=0: 3 < 4, swap a[0] with a[p], p = 1. List [3, 7, 1, 5, 4].
 2. i=1: 7 is not < 4, skip.
 3. i=2: 1 < 4, swap a[2] and a[p], p = 2. List [3, 1, 7, 5, 4].
 4. i=3: 5 is not < 4, skip.
 5. Swap pivot into a[2]: [3, 1, 4, 5, 7]. Left of 4 is smaller, right is bigger.
- Average $O(n \log n)$, worst $O(n^2)$ when the pivot is always the smallest or biggest (sorted input). A random pivot avoids this. Space $O(\log n)$ stack on average, in place.

Stability

A sort is **stable** if equal items keep their original order.

- Merge sort and Python sorted() (Timsort) are stable.
 - Quick sort and heap sort are usually not stable.
- Stability matters when you sort by one key, then by another.

Python sorted(), key= and lambda

```

nums = [5, 2, 9]
sorted(nums)           # new list [2, 5, 9]
nums.sort(reverse=True) # in place [9, 5, 2]
words = ["bb", "a", "ccc"]
sorted(words, key=len)  # ["a", "bb", "ccc"]
people = [("amit", 30), ("ravi", 25), ("neha", 30)]
sorted(people, key=lambda p: (-p[1], p[0])) # age desc, then name asc
intervals = [[3, 4], [1, 2]]
intervals.sort(key=lambda x: x[0])          # by start

```

Tuple keys compare item by item. Use a minus sign to reverse a number inside a tuple key.

Counting sort and bucket sort (idea)

- **Counting sort:** if values are small integers (like 0..100), count each value, then write them out. $O(n + k)$.
- **Bucket sort:** put items into buckets by a value, like frequency. Used in Top K Frequent Elements for $O(n)$.

```

def top_k(nums, k):
    from collections import Counter
    buckets = [[] for _ in range(len(nums) + 1)]
    for x, c in Counter(nums).items():
        buckets[c].append(x)          # index = frequency
    res = []
    for c in range(len(buckets) - 1, 0, -1):
        res.extend(buckets[c])
    return res[:k]

```

When sorting first helps

- Two pointers on pairs/triplets (3Sum).
 - Intervals: sort by start or end, then merge.
 - Grouping duplicates together (skip duplicates in backtracking).
 - Greedy choices (smallest first, earliest end first).
 - Binary search needs sorted data.
- Sorting costs $O(n \log n)$, so it is fine when $n \leq 10^5$.

Videos

- [How Merge Sort works](#) - Gate Smashers (Hindi)
- [How Quick Sort works](#) - Gate Smashers (Hindi)

Quiz

1. What is the worst-case time of quick sort?

- A) $O(n \log n)$
 - B) $O(n^2)$
 - C) $O(n)$
 - D) $O(\log n)$
2. Which of these is stable?
- A) Quick sort
 - B) Heap sort
 - C) Merge sort
 - D) Selection sort
3. How do you sort people by age descending, then name ascending?
- A) `key=lambda p: (p[1], p[0])`
 - B) `key=lambda p: (-p[1], p[0])`
 - C) `reverse=True` only
 - D) `key=len`
4. What is the time complexity of merge sort?
- A) $O(n)$ best, $O(n^2)$ worst
 - B) $O(n \log n)$ always
 - C) $O(\log n)$
 - D) $O(n^2)$ always

Answer key

1. **B** - $O(n^2)$. A bad pivot every time makes one side empty, giving n levels of $O(n)$ work.
2. **C** - Merge sort. Merge sort takes from the left half first on ties, so equal items keep their order.
3. **B** - `key=lambda p: (-p[1], p[0])`. The minus sign reverses age, and tuples compare the name next.
4. **B** - $O(n \log n)$ always. It always splits in half ($\log n$ levels) and merges in $O(n)$ per level.

Foundations 4: Prefix sums and how hash maps work

Prefix sum array

A prefix sum array stores the running total. $\text{prefix}[i] = \text{sum of the first } i \text{ numbers}$.

Real-life picture: a bank passbook. The balance column lets you find how much you spent between two dates by subtracting two balances.

```
nums = [3, 1, 4, 1, 5]
prefix = [0] * (len(nums) + 1)
for i, x in enumerate(nums):
    prefix[i + 1] = prefix[i] + x
# prefix = [0, 3, 4, 8, 9, 14]
```

We add a 0 at the start so the sum of an empty prefix is 0. This removes special cases.

Range sum query

Sum of $\text{nums}[l..r]$ (both inclusive) = $\text{prefix}[r + 1] - \text{prefix}[l]$.

```
def range_sum(prefix, l, r):
    return prefix[r + 1] - prefix[l]
# range_sum(prefix, 1, 3) = prefix[4] - prefix[1] = 9 - 3 = 6 (1 + 4 + 1)
```

Build once in $O(n)$. Then each query is $O(1)$ instead of $O(n)$.

Subarray sum equals k (prefix + hashmap)

Count subarrays whose sum is k . A subarray $(j, i]$ has sum $\text{prefix}_i - \text{prefix}_j$.

We want $\text{prefix}_i - \text{prefix}_j = k$, so $\text{prefix}_j = \text{prefix}_i - k$. Keep a count of every prefix seen so far.

```
def subarraySum(nums, k):
    count, cur = 0, 0
    seen = {0: 1}  # empty prefix seen once
    for x in nums:
        cur += x
        count += seen.get(cur - k, 0)
        seen[cur] = seen.get(cur, 0) + 1
    return count
```

Dry run: $\text{nums} = [1, 2, 3]$, $k = 3$.

1. $x=1$: $cur=1$. Need $1-3=-2$, not seen. $seen = \{0:1, 1:1\}$. $count=0$.
2. $x=2$: $cur=3$. Need 0, seen once. $count=1$ ($[1,2]$). $seen = \{0:1, 1:1, 3:1\}$.
3. $x=3$: $cur=6$. Need 3, seen once. $count=2$ ($[3]$). $seen$ adds 6.
4. Answer: 2. Time $O(n)$, space $O(n)$.

This works with negative numbers too, where sliding window fails.

How a hash table works

A Python dict or set is a hash table. It gives $O(1)$ average lookup, insert and delete.

1. **Hashing**: Python calls `hash(key)` to turn the key into a big integer.
2. **Buckets**: the table is an array of slots. $index = hash(key) \bmod table_size$ picks a slot.
3. **Store**: the key and value go into that slot.
4. **Lookup**: compute the same hash, go to the same slot, compare keys with `==`.

```
hash("apple")    # some big integer (changes between runs for strings)
hash(42)         # 42
table_size = 8
slot = hash(42) % table_size    # 2
```

Collisions

Two different keys can land in the same slot. This is a collision.

- **Chaining**: each slot holds a small list of (key, value) pairs. Java HashMap uses this.
 - **Open addressing**: if the slot is taken, probe another slot. Python dict uses this.
- Either way, the table must compare keys to find the right one.

Load factor and resizing

- Load factor = number of items / number of slots.
- When it gets too high (Python: about 2/3 full), the table grows to a bigger array and re-inserts all keys.
- Resizing is $O(n)$, but rare, so insert is amortised $O(1)$ (same idea as list append).

Why $O(1)$ average but $O(n)$ worst

- With a good hash function, keys spread evenly, so each slot has very few keys: $O(1)$ average.
- In the worst case, all keys collide into one slot, and lookup checks every key: $O(n)$.
- In interviews, say " $O(1)$ average" for dict and set operations.

Hashable keys in Python

A key must be hashable: its hash must never change. That means immutable types.

- OK: int, float, str, bool, tuple (of hashable items), frozenset.
- Not OK: list, dict, set. They can change, so Python refuses them as keys.

```
groups = {}
key = tuple(sorted("eat"))          # ("a", "e", "t"): hashable
groups.setdefault(key, []).append("eat")
counts = [0] * 26                  # Group Anagrams with letter counts
groups2 = {tuple(counts): []}      # List -> tuple to use as key
# {[1, 2]: 0} -> TypeError: unhashable type: list
```

Videos

- [Prefix Sum in 4 minutes](#) - AlgoMasterIO (English)
- [What is Prefix Sum](#) - Newton School (Hindi)

Quiz

1. With prefix = [0, 3, 4, 8, 9, 14], what is the sum of nums[1..3]?
 - A) 8
 - B) 6
 - C) 5
 - D) 9
2. In Subarray Sum Equals K, why do we start with seen = {0: 1}?
 - A) To avoid a KeyError
 - B) So subarrays starting at index 0 are counted
 - C) To make it faster
 - D) It is not needed
3. What is the worst-case lookup time in a hash table?
 - A) O(1)
 - B) O(log n)
 - C) O(n)
 - D) O(n²)
4. Which of these can be a Python dict key?
 - A) [1, 2]
 - B) {1, 2}
 - C) (1, 2)
 - D) {"a": 1}

Answer key

1. **B** - 6. prefix[4] - prefix[1] = 9 - 3 = 6.

2. **B** - So subarrays starting at index 0 are counted. If the running sum itself equals k , we need prefix 0 to have been seen once.
3. **C** - $O(n)$. If every key collides into one slot, we must check them all.
4. **C** - (1, 2). Tuples are immutable and hashable; lists, sets and dicts are not.

Foundations 5: Python toolkit for interviews

List, dict and set: operations and costs

- list: index `a[i]` $O(1)$, append $O(1)$ amortised, pop() from end $O(1)$.
- list: insert(0, x) and pop(0) are $O(n)$ because all items shift. Use deque instead.
- list: `x in a` is $O(n)$. Slicing `a[i:j]` is $O(j - i)$ and makes a copy.
- dict and set: get, add, delete, `x in s` are $O(1)$ average.
- sorted(a) and a.sort() are $O(n \log n)$.

```
a = [1, 2, 3]
a.append(4); a.pop()           # O(1)
d = {"x": 1}
d.get("y", 0)                  # 0, no KeyError
s = set([1, 2]); s.add(3)
print(2 in s)                  # True, O(1)
```

collections: Counter, defaultdict, deque

```
from collections import Counter, defaultdict, deque
c = Counter("banana")          # {"a": 3, "n": 2, "b": 1}
c.most_common(1)               # [("a", 3)]
Counter("abc") == Counter("cab") # True: anagram check

g = defaultdict(list)          # missing key starts as []
g["u"].append("v")

q = deque([1, 2])
q.append(3); q.appendleft(0)    # O(1) both ends
q.popleft()                    # O(1): use for BFS
```

heapq: min-heap

heapq works on a normal list and keeps the smallest item at index 0.

```
import heapq
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1) # O(log n)
heapq.heappop(h)                           # 1, O(log n)
h[0]                                        # peek smallest, O(1)
heapq.heapify(a)                           # list to heap in O(n)
heapq.heappush(h, -7)                       # max-heap trick: push negatives
heapq.heappush(h, (2, "task"))              # tuples sort by first item
heapq.nlargest(2, [4, 1, 9])                # [9, 4]
```

bisect: binary search on a sorted list

```
import bisect
a = [1, 3, 3, 7]
bisect.bisect_left(a, 3)      # 1: first index where 3 could go
bisect.bisect_right(a, 3)    # 3: after the last 3
bisect.insort(a, 5)          # insert keeping order (O(n) shift)
```

Sorting with keys

```
words = ["kiwi", "fig", "apple"]
words.sort(key=len)          # by length
pairs = [(1, "b"), (1, "a"), (0, "z")]
sorted(pairs, key=lambda p: (p[0], p[1])) # tuple key
sorted(pairs, key=lambda p: -p[0])       # descending number
```

Building strings: use join

Strings are immutable. `s += ch` in a loop can copy the string each time ($O(n^2)$ worst case).

```
parts = []
for ch in "abc":
    parts.append(ch.upper())
res = "".join(parts)      # "ABC" in O(n)
res = ",".join(["1", "2"]) # "1,2"
```

enumerate and zip

```
for i, x in enumerate(["a", "b"]): # index and value
    print(i, x)
for x, y in zip([1, 2], [3, 4]):    # walk two lists together
    print(x + y)
grid = [[1, 2], [3, 4]]
cols = list(zip(*grid))             # transpose: [(1, 3), (2, 4)]
```

Infinity and integer division

```
best = float("inf")      # bigger than any number
7 // 2                   # 3 (floor division)
-7 // 2                  # -4 (floors toward minus infinity!)
int(-7 / 2)              # -3 (truncates toward zero)
7 % 3                   # 1
divmod(7, 3)            # (2, 1)
```

Python ints never overflow. But some problems (Reverse Integer) ask you to act like 32-bit.

Common gotchas

- **Mutable default argument:** `def f(a=[])` shares the same list across calls. Use `a=None` and create the list inside.
- **2-D list trap:** `[[0] * n] * m` makes `m` references to ONE row. Use `[[0] * n for _ in range(m)]`.
- **Copy vs reference:** `b = a` does not copy. Use `a[:]` or `list(a)`; for nested lists use `copy.deepcopy`.
- **Changing a list while looping over it** skips items. Loop over a copy or build a new list.
- **Recursion limit** is about 1000. Use an iterative version for deep inputs.

```
def bad(x, acc=[]):
    acc.append(x); return acc
bad(1); print(bad(2))           # [1, 2] surprise!

grid = [[0] * 3] * 2
grid[0][0] = 9
print(grid)                     # [[9, 0, 0], [9, 0, 0]] both rows changed
grid = [[0] * 3 for _ in range(2)] # correct
```

Videos

- [Python for coding interviews](#) - NeetCode (English)

Quiz

1. Which structure gives $O(1)$ pop from the front for BFS?
 - A) list
 - B) `collections.deque`
 - C) set
 - D) tuple
2. How do you make a max-heap with `heapq`?
 - A) `heapq.maxheap()`
 - B) Push negative values
 - C) Use `reverse=True`
 - D) Sort the list first
3. What is `-7 // 2` in Python?
 - A) -3
 - B) -4
 - C) -3.5
 - D) 3
4. Why is `def f(a=[])` dangerous?
 - A) Lists cannot be default values
 - B) The same list is shared across all calls

- C) It is slow
- D) It raises an error

Answer key

1. **B** - `collections.deque.deque.popleft()` is $O(1)$, while `list.pop(0)` is $O(n)$.
2. **B** - Push negative values. `heapq` is only a min-heap, so negating values flips the order.
3. **B** - -4. Floor division rounds down toward minus infinity.
4. **B** - The same list is shared across all calls. Default values are created once, so changes stay between calls.

DSA topic: Arrays & Hashing

What it is

An array is a row of boxes. Each box has an index (0, 1, 2, ...). Reading a box by index is very fast.

A hash map (Python `dict`) and a hash set (Python `set`) let you check "have I seen this before?" in $O(1)$ time on average.

Real-life analogy: a hash map is like the contacts app on your phone. You type a name and you get the number at once. You do not scroll through every contact.

Most array problems become easy when you trade a little extra memory (a dict or set) for a big speed gain ($O(n^2)$ becomes $O(n)$).

How to recognise it

- The problem asks "does a duplicate exist?" or "have we seen this value?"
- You need to count things: frequency of letters, numbers, words.
- You need to find a pair that adds up to a target (and the array is NOT sorted).
- You need to group items that are "the same" in some way (anagrams, same key).
- The brute force uses two nested loops and you want to remove one loop.

Pattern 1: Seen set / seen map

Use when you walk the array once and ask "did I already see X?" or "did I already see the partner of X?"

```
def has_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:           # O(1) average lookup
            return True
        seen.add(x)
    return False
```

Pattern 2: Frequency count

Use when you need how many times each value appears. Compare two counters, or pick the most frequent items.

```

from collections import Counter

def is_anagram(s, t):
    if len(s) != len(t):
        return False
    count = {}
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
    for ch in t:
        count[ch] = count.get(ch, 0) - 1
        if count[ch] < 0: # t has an extra ch
            return False
    return True
# Short version: return Counter(s) == Counter(t)

```

Pattern 3: Group by key

Use when items belong to the same group if they share a "signature". Build the signature, use it as the dict key.

```

from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        key = [0] * 26 # Letter counts as the signature
        for ch in w:
            key[ord(ch) - ord("a")] += 1
        groups[tuple(key)].append(w) # list is not hashable, tuple is
    return list(groups.values())

```

Worked example: Two Sum

Given an array `nums` and a `target`, return the indexes of two numbers that add up to `target`. Exactly one answer exists.

Input: `nums = [2, 7, 11, 15]`, `target = 9`. Idea: for each number, the partner we need is `target - x`. Keep a map value -> index.

1. `i = 0`, `x = 2`, need = 7. Is 7 in the map `{}`? No. Save 2 -> 0. Map = `{2: 0}`.
2. `i = 1`, `x = 7`, need = 2. Is 2 in the map `{2: 0}`? Yes, at index 0.
3. Return `[0, 1]`. We looked at each number only once.

```

def two_sum(nums, target):
    index_of = {} # value -> index
    for i, x in enumerate(nums):
        need = target - x
        if need in index_of:
            return [index_of[need], i]
        index_of[x] = i # save AFTER the check
    return []

```


Complexity cheat sheet

- Array read/write by index -> $O(1)$
- Array search for a value -> $O(n)$
- Insert/delete in the middle of an array -> $O(n)$
- dict/set insert, lookup, delete -> $O(1)$ average, $O(n)$ worst case
- Sorting an array -> $O(n \log n)$ time
- Using a dict/set of n items -> $O(n)$ extra space

Common mistakes

- Saving the current number in the map BEFORE checking, so a number pairs with itself (Two Sum with target = $2 * x$).
- Using a `list` as a dict key. Lists are not hashable. Convert to `tuple` first.
- Using `x in some_list` inside a loop. That is $O(n)$ per check, so the total becomes $O(n^2)$. Use a set.
- Forgetting the empty input or the "all items are the same" case.
- Sorting when the problem asks for original indexes. Sorting loses the indexes.

What to say in the interview

- "The brute force is two nested loops, which is $O(n^2)$. I can do better with a hash map."
- "I will store each value and its index in a dictionary, so I can check for the partner in $O(1)$."
- "This gives $O(n)$ time and $O(n)$ extra space. I am trading memory for speed."
- "Let me quickly test it with a small example and an edge case like duplicates."

Practice order

- Contains Duplicate first: the simplest "seen set" problem.
- Valid Anagram, then Two Sum: frequency count and seen map.
- Group Anagrams: learn the "signature as key" idea.
- Top K Frequent Elements and Product of Array Except Self: counting plus bucket sort, and prefix/suffix products.
- Longest Consecutive Sequence last: a clever set trick (only start counting when $x - 1$ is not in the set).

Cheat sheet

Idea

A hash map / set gives $O(1)$ average lookup. Most "find pair / count / group / duplicate" problems become one pass with a map.

Use it when

- You need "have I seen this before?" -> set
- You need counts -> `collections.Counter` / dict
- You need to group items by a key (sorted string, tuple of counts) -> dict of lists
- Prefix sums: running total + map of earlier totals

Template

```
seen = {}
for i, x in enumerate(nums):
    if target - x in seen:
        return [seen[target - x], i]
    seen[x] = i
```

Complexity

Time $O(n)$, extra space $O(n)$. Sorting first instead costs $O(n \log n)$ but $O(1)$ extra space - say this trade-off out loud.

Common mistakes

- Checking the map after inserting the current element (pairs with itself)
- Using a list as a dict key (use tuple)

Visualise it

- <https://visualgo.net/en/hashtable>

Videos

- [Hash Table in Python](#) - codebasics (English)
- [Hashing, maps, collisions](#) - take U forward (Hindi + English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Contains Duplicate (Blind 75)	Easy	open	watch
2	Valid Anagram (Blind 75)	Easy	open	watch

#	Problem	Level	LeetCode	Solution video
3	Two Sum (Blind 75)	Easy	open	watch
4	Group Anagrams (Blind 75)	Medium	open	watch
5	Top K Frequent Elements (Blind 75)	Medium	open	watch
6	Product of Array Except Self (Blind 75)	Medium	open	watch
7	Valid Sudoku	Medium	open	watch
8	Encode and Decode Strings (Blind 75)	Medium	open	watch
9	Longest Consecutive Sequence (Blind 75)	Medium	open	watch

Problem hints

Try each problem on your own first. Open one hint at a time. Only go to the next hint when you are stuck for 10 or more minutes.

Contains Duplicate (Easy)

Restate: Given a list of numbers, return True if any number appears two or more times.

Hint 1: You need to answer "have I seen this number before?" very fast. Which Python structure answers that in $O(1)$?

Hint 2: A set remembers every number you have already passed. The first time a number is already in the set, you have found a duplicate and can stop early.

Hint 3:

- Create an empty set.
- Walk through the numbers one by one.
- If the number is already in the set, return True.
- Otherwise add it to the set.
- After the loop, return False.

Complexity: $O(n)$ time, $O(n)$ space. (Sorting first gives $O(n \log n)$ time and $O(1)$ extra space, a valid trade-off to mention.)

Edge cases to test:

- `[1]` (one item, answer False)
- `[1, 1]` (smallest duplicate, answer True)
- `[1, 2, 3, 4]` (all unique, answer False)
- `[-1, 0, -1]` (negative numbers)
- A very large list where the duplicate is the last element

Valid Anagram (Easy)

Restate: Given two strings `s` and `t`, return True if `t` uses exactly the same letters as `s`, the same number of times, in any order.

Hint 1: This is a counting problem. Think "frequency count" with a dict or a fixed array of 26.

Hint 2: Two strings are anagrams exactly when every letter has the same count in both. If the lengths differ, you can return False at once.

Hint 3:

- If `len(s) != len(t)`, return False.
- Count each letter of `s` (add 1).
- For each letter of `t`, subtract 1 from its count.
- If any count goes below zero, return False.
- Otherwise return True.

Complexity: $O(n)$ time, $O(1)$ space for 26 lowercase letters ($O(k)$ for k distinct characters in general).

Edge cases to test:

- `s = "a", t = "a"` (single letter, True)
- `s = "ab", t = "a"` (different lengths, False)
- `s = "aacc", t = "ccac"` (same letters, wrong counts, False)
- `s = "rat", t = "car"` (same length, different letters, False)
- Unicode input such as `"é"` (follow-up question: a dict works, a 26-array does not)

Two Sum (Easy)

Restate: Given a list of numbers and a target, return the two indexes whose values add up to the target (exactly one answer exists, and you cannot use the same element twice).

Hint 1: For each number `x`, the partner you need is `target - x`. Use a hash map to find the partner fast.

Hint 2: Store `value -> index` for numbers you have already seen. Check for the partner **before** you store the current number, so you never pair a number with itself.

Hint 3:

- Create an empty dict `seen`.
- For each index `i` and value `x`:
- Compute `need = target - x`.
- If `need` is in `seen`, return `[seen[need], i]`.
- Otherwise save `seen[x] = i`.

Complexity: $O(n)$ time, $O(n)$ space.

Edge cases to test:

- `nums = [3, 3], target = 6` (two equal values)
- `nums = [3, 2, 4], target = 6` (must not return `[0, 0]`)
- `nums = [-3, 4, 3, 90], target = 0` (negative numbers)
- `nums = [0, 4, 3, 0], target = 0` (zeros)
- Answer pair at the very end of a long list

Group Anagrams (Medium)

Restate: Given a list of words, put the words that are anagrams of each other into the same group.

Hint 1: Use a hash map where the key is a "signature" that is the same for all anagrams.

Hint 2: Two good signatures: the sorted word ("eat" becomes "aet"), or a tuple of 26 letter counts. The count tuple avoids the sort, so it is faster for long words.

Hint 3:

- Create a dict that maps signature -> list of words.
- For each word, build its signature.
- Append the word to the list for that signature.
- Return all the lists (the dict values).

Complexity: $O(n * k \log k)$ with sorted keys, or $O(n * k)$ with count keys, where n = number of words and k = max word length. $O(n * k)$ space.

Edge cases to test:

- `[""]` (one empty string, answer `[[""]]`)
- `["a"]` (one word)
- `["ab", "ba", "abc"]` (similar but different lengths)
- `["aab", "abb"]` (same letters, different counts, must be separate groups)
- All words identical, such as `["x", "x", "x"]`

Top K Frequent Elements (Medium)

Restate: Given a list of numbers and `k`, return the `k` numbers that appear most often.

Hint 1: First count frequencies with a hash map. Then you need the top `k` by count.

Hint 2: A frequency can never be more than `n`. So make "buckets": a list of `n + 1` lists, where `bucket[f]` holds every number that appears exactly `f` times. Reading buckets from high to low gives the answer in $O(n)$. (A heap of size `k`, $O(n \log k)$, is also fine.)

Hint 3:

- Count each number with a dict.

- Create `buckets = [[] for _ in range(n + 1)]`.
- For each number and count, append the number to `buckets[count]`.
- Walk the buckets from index `n` down to `1`, collecting numbers.
- Stop as soon as you have `k` numbers.

Complexity: $O(n)$ time and $O(n)$ space with bucket sort.

Edge cases to test:

- `nums = [1]`, `k = 1`
- `nums = [1, 1, 1, 2, 2, 3]`, `k = 2` (answer `[1, 2]`)
- `nums = [4, 4, -1, -1, -1]`, `k = 1` (negative numbers)
- `k` equal to the number of distinct values (return all of them)
- All numbers with the same frequency (any `k` of them is valid; ask the interviewer)

Product of Array Except Self (Medium)

Restate: Given a list of numbers, return a new list where each position holds the product of all the other numbers, without using division.

Hint 1: Think "prefix and suffix". Every answer is (product of everything to the left) times (product of everything to the right).

Hint 2: You can fill the answer array with left products in one pass from the start, then multiply in the right products in a second pass from the end, using a single running variable. That gives $O(1)$ extra space (the output array does not count).

Hint 3:

- Create `res` of length `n` filled with `1`.
- Pass 1, left to right: keep `left = 1`; set `res[i] = left`, then `left *= nums[i]`.
- Pass 2, right to left: keep `right = 1`; set `res[i] *= right`, then `right *= nums[i]`.
- Return `res`.

Complexity: $O(n)$ time, $O(1)$ extra space (besides the output).

Edge cases to test:

- `[1, 2]` (smallest valid input, answer `[2, 1]`)
- `[1, 2, 3, 4]` (answer `[24, 12, 8, 6]`)
- `[0, 4, 5]` (one zero: only the zero position is non-zero)
- `[0, 4, 0]` (two zeros: every answer is 0)
- `[-1, 1, 0, -3, 3]` (negatives with a zero)

Valid Sudoku (Medium)

Restate: Given a partly filled 9x9 Sudoku board, check that no row, no column and no 3x3 box contains the same digit twice (empty cells are "."); you do not need to solve it).

Hint 1: Use hash sets to remember which digits you have already seen in each row, each column and each box.

Hint 2: The box number for cell (r, c) is $(r // 3, c // 3)$. With that key, one pass over the 81 cells checks all three rules at the same time.

Hint 3:

- Create 9 sets for rows, 9 for columns, and 9 for boxes (a dict keyed by $(r // 3, c // 3)$ works well).
- Loop over every cell; skip ".".
- If the digit is already in its row set, column set or box set, return False.
- Otherwise add the digit to all three sets.
- After the loop, return True.

Complexity: $O(81) = O(1)$ time and space for a fixed 9x9 board ($O(n^2)$ for an $n \times n$ board).

Edge cases to test:

- A completely empty board (all ".", answer True)
- Duplicate in a row only
- Duplicate in a column only
- Duplicate inside one 3x3 box but in different rows and columns
- A valid-looking board that is not solvable (still True; the task only checks the rules)

Encode and Decode Strings (Medium)

Restate: Design two functions: one turns a list of strings into a single string, and the other turns that single string back into the exact same list.

Hint 1: Any separator character could also appear inside a word. You need a format that tells the decoder where each word ends without guessing.

Hint 2: Put the length in front of every word with a marker: "4#neet4#code". The decoder reads digits until #, then takes exactly that many characters. Because it jumps by length, a # inside a word is harmless.

Hint 3:

- Encode: for each word, append `str(len(word)) + "#" + word`.
- Decode: set `i = 0`.
- Move `j` forward from `i` until `s[j] == "#"`; the length is `int(s[i:j])`.
- The word is `s[j + 1 : j + 1 + length]`; add it to the result.
- Set `i = j + 1 + length` and repeat until the end.

Complexity: $O(\text{total characters})$ time and space for both functions.

Edge cases to test:

- `[]` (empty list)
- `[""]` (one empty string; must not decode to `[]`)
- `["", ""]` (two empty strings)
- `["a#b", "4#x"]` (words containing `#` and digits)
- A word with 10 or more characters (length has two digits)

Longest Consecutive Sequence (Medium)

Restate: Given an unsorted list of numbers, return the length of the longest run of consecutive integers (like 1, 2, 3, 4), in $O(n)$ time.

Hint 1: Sorting gives $O(n \log n)$. To reach $O(n)$, put all numbers in a set so you can ask "is $x + 1$ here?" in $O(1)$.

Hint 2: Only start counting from a number that begins a run, which means $x - 1$ is NOT in the set. Then each number is visited at most twice in total, so the whole thing stays $O(n)$.

Hint 3:

- Put all numbers into a set.
- For each number x in the set:
- If $x - 1$ is in the set, skip it (it is not a start).
- Otherwise count up: while $x + \text{length}$ is in the set, increase `length`.
- Keep the best length seen.

Complexity: $O(n)$ time, $O(n)$ space.

Edge cases to test:

- `[]` (answer 0)
- `[7]` (answer 1)
- `[1, 2, 0, 1]` (duplicates, answer 3)
- `[100, 4, 200, 1, 3, 2]` (answer 4)
- `[-2, -1, 0, 1]` (run crosses zero, answer 4)

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Sort an Array	Medium	open	watch
2	Sort Colors	Medium	open	watch
3	Encode and Decode TinyURL	Medium	open	watch

#	Problem	Level	LeetCode	Solution video
4	Brick Wall	Medium	open	watch
5	Best Time to Buy And Sell Stock II	Medium	open	watch
6	Subarray Sum Equals K	Medium	open	watch

Quiz

- What is the average time to check if a value is in a Python set?
 - A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n)$
 - D) $O(n \log n)$
- In Two Sum with an unsorted array, which approach gives $O(n)$ time?
 - A) Two nested loops
 - B) A hash map of value $\rightarrow$ index
 - C) Binary search for each element
- Why can you not use a Python list as a dict key?
 - A) Lists are too slow
 - B) Lists are mutable, so they are not hashable
 - C) Lists cannot hold numbers
- Which pattern fits "Group strings that are anagrams of each other"?
 - A) Sliding window
 - B) Group by key (signature as dict key)
 - C) Two pointers
 - D) Binary search
- You call `x in my_list` inside a loop over n items. What is the total time?
 - A) $O(n)$
 - B) $O(n \log n)$
 - C) $O(n^2)$
- Average lookup time in a Python dict?
 - A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n)$
- Best key to group anagrams?
 - A) The word itself
 - B) Sorted letters or a 26-count tuple
 - C) Word length
- Two Sum in one pass needs:
 - A) Sorting
 - B) A map of value $\rightarrow$ index
 - C) Two nested loops

Answer key

1. **A** - $O(1)$. A set uses hashing, so membership checks are $O(1)$ on average.
2. **B** - A hash map of value $\rightarrow$ index. One pass with a map lets you find the partner target - x in $O(1)$.
3. **B** - Lists are mutable, so they are not hashable. Dict keys must be hashable; convert the list to a tuple.
4. **B** - Group by key (signature as dict key). All anagrams share the same sorted string or letter count, which works as a dict key.
5. **C** - $O(n^2)$. Each list membership check is $O(n)$, done n times.
6. **A** - $O(1)$.
7. **B** - Sorted letters or a 26-count tuple.
8. **B** - A map of value $\rightarrow$ index.

More quiz

1. In Longest Consecutive Sequence, why do you only start counting when `x - 1` is not in the set?
 - A. To handle negative numbers
 - B. So that each run is counted only once, which keeps the total time $O(n)$
 - C. Because sets are unordered
 - D. To save memory
2. In Encode and Decode Strings, why is `", ".join(words)` a bad encoding?
 - A. It is too slow
 - B. It uses too much memory
 - C. A word may itself contain a comma, so the decoder cannot tell where words end
 - D. Python strings cannot hold commas
3. Which pattern fits "Product of Array Except Self" without division?
 - A. Hash map lookup
 - B. Prefix products and suffix products
 - C. Sliding window
 - D. Binary search
4. In Top K Frequent Elements with bucket sort, how many buckets do you need for a list of n numbers?
 - A. k
 - B. 26
 - C. $n + 1$ (frequencies go from 0 to n)
 - D. $\log n$
5. In Valid Sudoku, which key gives the 3x3 box for cell (r, c) ?
 - A. `(r % 3, c % 3)`

- B. `r * 9 + c`
- C. `(r // 3, c // 3)`
- D. `r + c`

More quiz: answer key

1. **B** - Only the smallest number of a run starts the inner count, so every number is touched a constant number of times. Without this check the same run is recounted from every member, which can become $O(n^2)$.
2. **C** - A separator character can appear inside the data. A length prefix like `"3#abc"` tells the decoder exactly how many characters to read, so any character is safe.
3. **B** - Each answer is the product of everything to the left times everything to the right. Two passes with running products give $O(n)$ time and $O(1)$ extra space.
4. **C** - A number can appear between 1 and n times, so you index buckets by frequency from 0 to n . That is $n + 1$ buckets.
5. **C** - Integer division by 3 maps rows 0-2 to 0, rows 3-5 to 1, rows 6-8 to 2, and the same for columns. The pair names one of the 9 boxes.

Flashcards

- **Q:** What is the brute force for Contains Duplicate, and why is it slow? — **A:** Compare every pair with two loops; it is $O(n^2)$ because each number is compared with every other number.
- **Q:** In Two Sum, why check for the partner before storing the current number? — **A:** So a number is never paired with itself, for example `[3, 2, 4]` with target 6 must not return `[0, 0]`.
- **Q:** Two signature choices for Group Anagrams? — **A:** The sorted word ($O(k \log k)$ per word) or a tuple of 26 letter counts ($O(k)$ per word).
- **Q:** Why must the count signature be a tuple and not a list? — **A:** Dict keys must be hashable, and a list is mutable, so it is not hashable.
- **Q:** Time of bucket-sort Top K Frequent? — **A:** $O(n)$, because frequencies are bounded by n and you read each bucket once.
- **Q:** Product of Array Except Self: answer at index i in one formula? — **A:** $(\text{product of nums}[0..i-1]) \times (\text{product of nums}[i+1..n-1])$.
- **Q:** What does the length-prefix format look like for `["hi", ""]`? — **A:** `"2#hi0#"`.
- **Q:** How many sets does a one-pass Valid Sudoku need? — **A:** 27: nine for rows, nine for columns and nine for 3x3 boxes.
- **Q:** What test input breaks a Valid Anagram solution that only compares sets of letters? — **A:** `"aacc"` and `"ccac"`: same letters, different counts.
- **Q:** Longest Consecutive Sequence: how do you know x starts a run? — **A:** When $x - 1$ is not in the set.

DSA topic: Two Pointers

What it is

Two pointers means using two index variables that move through the data, often from both ends or at different speeds.

Real-life analogy: two people searching a long bookshelf, one starts from the left end and one from the right end. They walk toward each other and meet in the middle.

It usually turns an $O(n^2)$ double loop into one $O(n)$ pass, and it needs only $O(1)$ extra space.

How to recognise it

- The array is **sorted**, or you are allowed to sort it.
- You need a pair (or triplet) with a given sum.
- You need to check a palindrome or compare both ends of a string.
- You need to remove duplicates or move items in place with $O(1)$ extra space.
- The problem talks about "container", "area", or "water" between two walls.

Pattern 1: Opposite ends (left and right)

Use on a sorted array or a string when you compare the two ends and move one side based on the result.

```
def pair_with_sum(nums, target):    # nums is sorted
    l, r = 0, len(nums) - 1
    while l < r:
        s = nums[l] + nums[r]
        if s == target:
            return [l, r]
        elif s < target:
            l += 1    # need a bigger sum
        else:
            r -= 1    # need a smaller sum
    return []
```

Pattern 2: Slow and fast (read/write pointers)

Use to change an array in place. The fast pointer reads every item; the slow pointer marks where to write the next kept item.

```
def remove_duplicates(nums):    # nums is sorted
    write = 1
    for read in range(1, len(nums)):
        if nums[read] != nums[read - 1]:
            nums[write] = nums[read]
            write += 1
    return write    # Length of the unique part
```

Pattern 3: Fix one, two-pointer the rest

Use for triplets (3Sum). Sort, fix index *i*, then run Pattern 1 on the part to the right of *i*.

```
def three_sum(nums):
    nums.sort()
    res = []
    for i in range(len(nums)):
        if i > 0 and nums[i] == nums[i - 1]:
            continue    # skip duplicate first values
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                l += 1
                while l < r and nums[l] == nums[l - 1]:
                    l += 1    # skip duplicate second values
    return res
```

Worked example: Valid Palindrome

Given a string, return True if it reads the same forward and backward, after ignoring non-letters/digits and case.

Input: *s* = "A b,a". Clean view: "aba".

1. *l* = 0 ("A"), *r* = 4 ("a"). Both are letters. "a" == "a". Move: *l* = 1, *r* = 3.
2. *l* = 1 (" ") is not alphanumeric. Skip: *l* = 2 ("b").
3. *r* = 3 (",") is not alphanumeric. Skip: *r* = 2 ("b").
4. Now *l* = 2, *r* = 2, so *l* < *r* is False. Loop ends. Return True.

```
def is_palindrome(s):
    l, r = 0, len(s) - 1
    while l < r:
        if not s[l].isalnum():
            l += 1
        elif not s[r].isalnum():
            r -= 1
        else:
            if s[l].lower() != s[r].lower():
                return False
            l += 1
            r -= 1
    return True
```

Complexity cheat sheet

- One pass with two pointers -> $O(n)$ time, $O(1)$ space
- Sorting first -> adds $O(n \log n)$ time
- 3Sum (sort + loop + two pointers) -> $O(n^2)$ time, $O(1)$ extra space (ignoring output)
- Container With Most Water -> $O(n)$ time, $O(1)$ space

Common mistakes

- Using two pointers on an unsorted array for a pair sum. It only works when sorted.
- Writing `while l <= r` when the two pointers must point to different items. Use `l < r`.
- Forgetting to skip duplicates in 3Sum, so the answer has repeated triplets.
- Moving the wrong pointer. In Container With Most Water, always move the shorter wall.
- Sorting when the problem needs the original indexes (use a hash map instead).

What to say in the interview

- "I notice the array is sorted, so I will use two pointers, one at each end."
- "If the sum is too small, I move the left pointer right. If it is too big, I move the right pointer left."
- "Each step removes one candidate, so the whole loop is $O(n)$ time and $O(1)$ space."
- "For three numbers, I will sort first, fix one number, and use two pointers for the other two."

Practice order

- Valid Palindrome first: the cleanest opposite-ends example.
- Two Sum II (sorted input): the sum-and-move rule.
- 3Sum: adds sorting and duplicate skipping.
- Container With Most Water: learn why you move the shorter side.
- Trapping Rain Water last: two pointers with left-max and right-max, a hard classic.

Cheat sheet

Idea

Two indexes move toward each other (or same direction) so you check pairs in $O(n)$ instead of $O(n^2)$. Usually needs a sorted array or a symmetric check (palindrome).

Use it when

- Sorted array + pair/triplet with a target sum
- Palindrome checks
- Container / trapping water (move the smaller side)

Template

```
l, r = 0, len(a) - 1
while l < r:
    s = a[l] + a[r]
    if s == target: return [l, r]
    if s < target: l += 1
    else: r -= 1
```

Complexity

$O(n)$ time, $O(1)$ space (plus $O(n \log n)$ if you must sort).

Common mistakes

- Forgetting to skip duplicates in 3Sum
- `l <= r` vs `l < r` - decide whether the same element can be used twice

Visualise it

- <https://visualgo.net/en/sorting>

Videos

- [Two Pointers in 7 minutes](#) - AlgoMasterIO (English)
- [Two Pointer technique](#) - Gate Smashers (Hindi)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Valid Palindrome (Blind 75)	Easy	open	watch
2	Two Sum II Input Array Is Sorted	Medium	open	watch
3	3Sum (Blind 75)	Medium	open	watch
4	Container With Most Water (Blind 75)	Medium	open	watch
5	Trapping Rain Water	Hard	open	watch

Problem hints

Try each problem on your own first. Open one hint at a time. Only go to the next hint when you are stuck for 10 or more minutes.

Valid Palindrome (Easy)

Restate: Return True if a string reads the same forwards and backwards after you ignore case and remove every character that is not a letter or a digit.

Hint 1: Use two pointers, one at the start and one at the end, moving towards each other.

Hint 2: You do not need to build a cleaned copy. Just skip non-alphanumeric characters with the pointers and compare the lowercase forms.

Hint 3:

- Set `l = 0` and `r = len(s) - 1`.
- While `l < r`: move `l` right while `s[l]` is not alphanumeric (and `l < r`).
- Move `r` left while `s[r]` is not alphanumeric (and `l < r`).
- If `s[l].lower() != s[r].lower()`, return False.
- Move both pointers inward; after the loop return True.

Complexity: O(n) time, O(1) space.

Edge cases to test:

- `""` (empty string, True)
- `" "` (only a space, True after cleaning)
- `".,"` (only punctuation, True)
- `"0P"` (digit and letter, False; a common bug if you lowercase digits wrongly or skip digits)
- `"A man, a plan, a canal: Panama"` (True)

Two Sum II Input Array Is Sorted (Medium)

Restate: Given a sorted list and a target, return the 1-based positions of the two numbers that add up to the target, using only O(1) extra space.

Hint 1: The list is sorted. That is the signal for two pointers at opposite ends instead of a hash map.

Hint 2: If the sum is too small, the only way to grow it is to move the left pointer right. If the sum is too big, move the right pointer left. You never skip the real answer.

Hint 3:

- Set $l = 0$, $r = n - 1$.
- Compute $total = numbers[l] + numbers[r]$.
- If $total == target$, return $[l + 1, r + 1]$.
- If $total < target$, do $l += 1$; else do $r -= 1$.
- Repeat while $l < r$.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test:

- $[2, 7, 11, 15]$, $target = 9$ (answer $[1, 2]$, note 1-based)
- $[-1, 0]$, $target = -1$ (negatives)
- $[1, 1, 3]$, $target = 2$ (duplicate values)
- $[1, 2, 3, 4, 9]$, $target = 13$ (answer is the last two elements, $[4, 5]$)
- Two-element list

3Sum (Medium)

Restate: Find all unique groups of three numbers in the list that add up to zero (no repeated triplets in the output).

Hint 1: Sort the list first. Then fix one number and solve "Two Sum II" on the rest with two pointers.

Hint 2: The hard part is avoiding duplicate triplets. After sorting, equal values sit next to each other, so you skip a fixed number if it equals the one before it, and after finding a triplet you move l past all equal values.

Hint 3:

- Sort `nums`.
- For each index i : if $i > 0$ and $nums[i] == nums[i - 1]$, skip. (You can also stop when $nums[i] > 0$.)
- Set $l = i + 1$, $r = n - 1$; compare $nums[i] + nums[l] + nums[r]$ with 0.
- Too small: $l += 1$. Too big: $r -= 1$.
- Equal: save the triplet, move l right, and keep moving l while $nums[l] == nums[l - 1]$ and $l < r$.

Complexity: $O(n^2)$ time, $O(1)$ extra space besides the output (sorting may use $O(n)$ depending on the language).

Edge cases to test:

- `[0, 0, 0]` (answer `[[0, 0, 0]]`)
- `[0, 0, 0, 0]` (still only one triplet)
- `[0, 1, 1]` (no answer, `[]`)
- `[-1, 0, 1, 2, -1, -4]` (answer `[-1, -1, 2], [-1, 0, 1]`)
- `[-2, 0, 0, 2, 2]` (duplicates on the right side)

Container With Most Water (Medium)

Restate: Given heights of vertical lines, pick two lines that, with the x-axis, hold the most water, and return that area.

Hint 1: Start with the widest container: one pointer at each end.

Hint 2: Area = `min(height[l], height[r]) * (r - l)`. Moving the taller line inward can never help, because the width shrinks and the shorter line still limits the height. So always move the shorter line.

Hint 3:

- Set `l = 0`, `r = n - 1`, `best = 0`.
- Compute the area and update `best`.
- If `height[l] < height[r]`, do `l += 1`; else do `r -= 1`.
- Repeat while `l < r`.
- Return `best`.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test:

- `[1, 1]` (answer 1)
- `[1, 8, 6, 2, 5, 4, 8, 3, 7]` (answer 49)
- `[4, 3, 2, 1, 4]` (best uses both ends, answer 16)
- `[0, 0, 0]` (answer 0)
- `[1, 2, 1]` (answer 2, uses the two outer lines)

Trapping Rain Water (Hard)

Restate: Given bar heights, compute how many units of rain water get trapped between the bars.

Hint 1: Water above bar `i` = `min(max height on its left, max height on its right) - height[i]`, if that is positive.

Hint 2: You can avoid the two prefix-max arrays. Keep two pointers and two running maxima. Whichever side has the smaller running max is the side whose water you can safely compute now, because the other side is guaranteed to be at least as tall.

Hint 3:

- Set `l = 0`, `r = n - 1`, `left_max = height[l]`, `right_max = height[r]`, `water = 0`.

- While $l < r$: if `left_max < right_max`, move `l` right, update `left_max = max(left_max, height[l])`, add `left_max - height[l]`.
- Else move `r` left, update `right_max = max(right_max, height[r])`, add `right_max - height[r]`.
- Return `water`.

Complexity: $O(n)$ time, $O(1)$ space (the prefix/suffix array method is $O(n)$ space and also accepted).

Edge cases to test:

- `[]` or a single bar (answer 0)
- `[1, 2, 3, 4]` (always rising, answer 0)
- `[4, 3, 2, 1]` (always falling, answer 0)
- `[2, 0, 2]` (answer 2)
- `[0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]` (answer 6)

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Remove Duplicates From Sorted Array II	Medium	open	watch
2	4Sum	Medium	open	watch
3	Number of Subsequences That Satisfy The Given Sum Condition	Medium	open	watch
4	Rotate Array	Medium	open	watch
5	Array With Elements Not Equal to Average of Neighbors	Medium	open	watch
6	Boats to Save People	Medium	open	watch

Quiz

1. When does the opposite-ends two pointer approach for pair sum work?
 - A) Always
 - B) Only when the array is sorted
 - C) Only when the array has no negatives
2. What is the time complexity of 3Sum with sorting plus two pointers?
 - A) $O(n)$
 - B) $O(n \log n)$
 - C) $O(n^2)$
 - D) $O(n^3)$
3. In Container With Most Water, which pointer do you move?
 - A) The taller wall
 - B) The shorter wall
 - C) Both always

- D) A random one
- 4. Which pattern fits "remove duplicates from a sorted array in place"?
 - A) Slow/fast read-write pointers
 - B) Hash map grouping
 - C) Binary search
 - D) Heap
- 5. Extra space used by Valid Palindrome with two pointers?
 - A) $O(1)$
 - B) $O(n)$
 - C) $O(\log n)$
- 6. Two pointers on a pair-sum problem usually needs the array to be:
 - A) Sorted
 - B) Reversed
 - C) Unique
- 7. 3Sum overall time complexity:
 - A) $O(n)$
 - B) $O(n^2)$
 - C) $O(n^3)$
- 8. In Container With Most Water, which pointer moves?
 - A) The taller side
 - B) The shorter side
 - C) Both always

Answer key

1. **B** - Only when the array is sorted. Sorted order tells you which pointer to move to make the sum bigger or smaller.
2. **C** - $O(n^2)$. For each fixed i , the two-pointer scan is $O(n)$, so total is $O(n^2)$.
3. **B** - The shorter wall. The area is limited by the shorter wall, so only moving it can give a bigger area.
4. **A** - Slow/fast read-write pointers. A read pointer scans and a write pointer keeps unique values in place with $O(1)$ space.
5. **A** - $O(1)$. You only keep two index variables and do not build a cleaned copy.
6. **A** - Sorted.
7. **B** - $O(n^2)$.
8. **B** - The shorter side.

More quiz

1. In 3Sum, why do you sort the array first?
 - A. Sorting is required for any Python list
 - B. So you can use two pointers for the remaining pair and skip duplicates easily
 - C. To reduce the time to $O(n)$
 - D. To find the median

2. In Trapping Rain Water with two pointers, when `left_max < right_max`, why is it safe to compute water at the left pointer?
- A. Because the left side is always taller
 - B. Because the water at the left pointer is limited by `left_max`, since the right side has a bar at least as tall
 - C. Because `right_max` is wrong
 - D. It is not safe; you need both arrays
3. Which pattern fits "check if a string is a palindrome after removing at most one character"?
- A. Hash map counting
 - B. Two pointers from both ends; on the first mismatch, try skipping the left char or the right char
 - C. Binary search
 - D. Heap
4. What does Two Sum II return for `numbers = [1, 3, 4, 5]`, `target = 8`?
- A. `[1, 4]`
 - B. `[2, 4]`
 - C. `[3, 4]`
 - D. `[2, 3]`
5. Container With Most Water: heights `[1, 2, 4, 3]`. What is the maximum area?
- A. 3
 - B. 4
 - C. 6
 - D. 8

More quiz: answer key

- B** - Sorting lets you fix one number and move two pointers inward based on whether the sum is too small or too large. Equal values become neighbours, so skipping duplicates is a simple comparison with the previous value.
- B** - The right side already has a bar of height `right_max`, which is taller than `left_max`. So the water at the left pointer cannot be more than `left_max` minus its height, and it cannot be less either.
- B** - This is "Valid Palindrome II". Move inward until the first mismatch, then check if `s[l+1..r]` or `s[l..r-1]` is a palindrome.
- B** - The values at 1-based positions are 1, 3, 4, 5. The pair `3 + 5 = 8` sits at positions 2 and 4, so the answer is `[2, 4]`.
- B** - Index 1 (height 2) and index 3 (height 3) give `min(2, 3) * 2 = 4`. Every other pair gives 3 or less, for example index 2 and 3 give `min(4, 3) * 1 = 3`.

Flashcards

- **Q:** What signal in a pair-sum problem tells you to use two pointers instead of a hash map? — **A:** The array is sorted, or you are allowed to sort it.
- **Q:** Two Sum II: the sum is bigger than the target. Which pointer moves? — **A:** The right pointer moves left, to make the sum smaller.
- **Q:** How do you skip duplicate fixed values in 3Sum? — **A:** If `i > 0` and `nums[i] == nums[i - 1]`, continue to the next i.
- **Q:** 3Sum: early stop condition after sorting? — **A:** If `nums[i] > 0`, no triplet starting here can sum to zero, so stop.
- **Q:** Container With Most Water area formula? — **A:** `min(height[l], height[r]) * (r - l)`.
- **Q:** Trapping Rain Water: water above bar i? — **A:** `min(max_left, max_right) - height[i]`, or 0 if that is negative.
- **Q:** Why does a strictly increasing height list trap no water? — **A:** Every bar has no taller bar on its left, so the left max equals its own height.
- **Q:** Space of the prefix-max solution vs the two-pointer solution for Trapping Rain Water? — **A:** Prefix-max uses $O(n)$ for two arrays; two pointers uses $O(1)$.
- **Q:** Valid Palindrome test that catches "letters only" bugs? — **A:** `"0P"`, which must return False because digits count as characters.

DSA topic: Sliding Window

What it is

A sliding window is a part of the array or string between a left index `l` and a right index `r`. You grow the window by moving `r`, and shrink it by moving `l`.

Real-life analogy: looking at a long train through a small window. As the train moves, one coach enters on the right and one coach leaves on the left. You do not re-count the whole train each time.

The key trick: when the window moves, you update your data (sum, counts) by adding the new item and removing the old item, instead of recomputing everything.

How to recognise it

- The problem asks about a **contiguous** subarray or substring.
- Words like "longest", "shortest", "maximum sum", "at most K", "without repeating".
- A fixed size is given: "every window of size k".
- Brute force checks every subarray ($O(n^2)$ or worse).
- You can say if a window is "valid" or "invalid" using counts or a sum.

Pattern 1: Fixed-size window

Use when the window size `k` is given. Add the new right item, remove the item that falls off the left.

```
def max_sum_k(nums, k):
    window = sum(nums[:k])
    best = window
    for r in range(k, len(nums)):
        window += nums[r] - nums[r - k] # add new, drop old
        best = max(best, window)
    return best
```

Pattern 2: Variable-size window (expand, then shrink)

Use for "longest/shortest valid window". Move `r` every step. While the window is invalid, move `l`.

```
def longest_valid(s):
    count = {}
    l = 0
    best = 0
    for r in range(len(s)):
        count[s[r]] = count.get(s[r], 0) + 1    # expand
        while window_is_invalid(count):        # your rule here
            count[s[l]] -= 1                    # shrink
            l += 1
        best = max(best, r - l + 1)
    return best
```

Pattern 3: Window with "replace up to k" rule

Used in Longest Repeating Character Replacement. The window is valid if `window_length - most_frequent_count <= k`.

```
def character_replacement(s, k):
    count = {}
    l = 0
    max_f = 0
    best = 0
    for r in range(len(s)):
        count[s[r]] = count.get(s[r], 0) + 1
        max_f = max(max_f, count[s[r]])
        while (r - l + 1) - max_f > k:
            count[s[l]] -= 1
            l += 1
        best = max(best, r - l + 1)
    return best
```

Worked example: Longest Substring Without Repeating Characters

Given a string, find the length of the longest substring with no repeated characters.

Input: `s = "abca"`. We keep a set of characters in the window.

1. `r = 0`, "a". Set = {a}. `l = 0`. Length 1. best = 1.
2. `r = 1`, "b". Set = {a, b}. Length 2. best = 2.
3. `r = 2`, "c". Set = {a, b, c}. Length 3. best = 3.
4. `r = 3`, "a". "a" is already in the set. Remove `s[0] = "a"`, `l = 1`. Now add "a". Set = {b, c, a}. Length 3. best = 3.
5. Loop ends. Return 3.


```
def length_of_longest_substring(s):
    seen = set()
    l = 0
    best = 0
    for r in range(len(s)):
        while s[r] in seen:      # duplicate: shrink from the left
            seen.remove(s[l])
            l += 1
        seen.add(s[r])
        best = max(best, r - l + 1)
    return best
```

Complexity cheat sheet

- Fixed or variable window -> $O(n)$ time (each index enters and leaves once)
- Count map over letters -> $O(1)$ space if alphabet is fixed (26), else $O(k)$
- Minimum Window Substring -> $O(n + m)$ time
- Sliding Window Maximum with a deque -> $O(n)$ time, $O(k)$ space

Common mistakes

- Using sliding window when the subarray does not need to be contiguous (that is often DP).
- Using it for "subarray sum equals k" with negative numbers. Shrinking does not work there; use prefix sums + hash map.
- Off-by-one in window length. The length is `r - l + 1`.
- Using `if` instead of `while` when shrinking. Sometimes you must shrink many steps.
- Forgetting to decrease the count (or remove from the set) when `l` moves.

What to say in the interview

- "The problem asks for a contiguous substring, so I will use a sliding window."
- "I move the right pointer to grow the window, and when the window breaks the rule, I move the left pointer to shrink it."
- "Each character is added once and removed once, so the time is $O(n)$."
- "I keep a count map for the window, which is $O(1)$ space for a fixed alphabet."

Practice order

- Best Time to Buy and Sell Stock first: a simple window of "lowest price so far".
- Longest Substring Without Repeating Characters: the classic expand/shrink.
- Longest Repeating Character Replacement: learn the `length - max_freq <= k` rule.
- Permutation in String: fixed-size window with count matching.
- Minimum Window Substring and Sliding Window Maximum last: both are hard and need careful bookkeeping.

Cheat sheet

Idea

Keep a window $[l, r]$ over a contiguous subarray/substring. Expand r every step; shrink l while the window is invalid. Each index enters and leaves once $\rightarrow O(n)$.

Use it when

- "Longest / shortest substring or subarray such that ..."
- Fixed size k windows (max average, anagram in string)

Template

```
l = 0
count = {}
best = 0
for r, ch in enumerate(s):
    count[ch] = count.get(ch, 0) + 1
    while window_invalid(count):
        count[s[l]] -= 1
        l += 1
    best = max(best, r - l + 1)
```

Complexity

$O(n)$ time, $O(k)$ space for the window map.

Common mistakes

- Using `if` instead of `while` when shrinking
- Off-by-one in window length: `r - l + 1`

Visualise it

- <https://visualgo.net/en/list>

Videos

- [Sliding Window in 7 minutes](#) - AlgoMasterIO (English)
- [Sliding Window and 2 Pointers templates](#) - take U forward (Hindi + English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Best Time to Buy And Sell Stock (Blind 75)	Easy	open	watch
2	Longest Substring Without Repeating Characters (Blind 75)	Medium	open	watch
3	Longest Repeating Character Replacement (Blind 75)	Medium	open	watch
4	Permutation In String	Medium	open	watch
5	Minimum Window Substring (Blind 75)	Hard	open	watch
6	Sliding Window Maximum	Hard	open	watch

Problem hints

Try each problem on your own first. Open one hint at a time. Only go to the next hint when you are stuck for 10 or more minutes.

Best Time to Buy And Sell Stock (Easy)

Restate: Given daily stock prices, choose one day to buy and a later day to sell to get the maximum profit, or return 0 if no profit is possible.

Hint 1: Walk the prices once from left to right. Think of the buy day as the left edge and today as the right edge of a window.

Hint 2: For each day, the best buy so far is simply the lowest price seen before today. So you only need to remember one number: the minimum price so far.

Hint 3:

- Set `min_price = prices[0]`, `best = 0`.
- For each later price `p`:
- Update `best = max(best, p - min_price)`.
- Update `min_price = min(min_price, p)`.
- Return `best`.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test:

- `[5]` (one day, answer 0)
- `[7, 6, 4, 3, 1]` (always falling, answer 0)
- `[1, 2]` (answer 1)
- `[7, 1, 5, 3, 6, 4]` (answer 5, buy at 1 and sell at 6)
- `[2, 4, 1]` (the minimum comes after the best sell day; answer 2, not 0)

Longest Substring Without Repeating Characters (Medium)

Restate: Return the length of the longest contiguous part of a string in which no character appears twice.

Hint 1: Variable-size sliding window. Grow the right edge; shrink the left edge when the window becomes invalid.

Hint 2: Store the last index where each character was seen. When `s[r]` was seen inside the current window, jump `l` straight to one past that old index, instead of removing characters one by one.

Hint 3:

- Set `l = 0`, `best = 0`, `last = {}`.
- For each `r` and character `ch`:
- If `ch` is in `last` and `last[ch] >= l`, set `l = last[ch] + 1`.
- Set `last[ch] = r`.
- Update `best = max(best, r - l + 1)`.

Complexity: $O(n)$ time, $O(\min(n, \text{alphabet size}))$ space.

Edge cases to test:

- `" "` (answer 0)
- `" "` (a single space, answer 1)
- `"bbbbbb"` (answer 1)
- `"abba"` (answer 2; catches the bug where `l` moves backwards if you forget `last[ch] >= l`)
- `"pwwkew"` (answer 3, `"wke"`)

Longest Repeating Character Replacement (Medium)

Restate: You may change up to `k` characters of an uppercase string; return the length of the longest substring that can be made of one repeated letter.

Hint 1: Variable window with a count of each letter inside the window.

Hint 2: A window is fine when `window_length - count_of_most_common_letter <= k`, because you replace all the other letters. A known trick: you never need to decrease `max_freq` when shrinking, because the answer only grows when a bigger `max_freq` appears.

Hint 3:

- Keep `count` (dict), `l = 0`, `max_freq = 0`, `best = 0`.
- For each `r`: add `s[r]` to `count`; `max_freq = max(max_freq, count[s[r]])`.
- While `(r - l + 1) - max_freq > k`: remove `s[l]` from `count`, `l += 1`.
- Update `best = max(best, r - l + 1)`.

Complexity: $O(n)$ time, $O(26) = O(1)$ space.

Edge cases to test:

- `s = "A", k = 0` (answer 1)
- `s = "ABAB", k = 2` (answer 4)
- `s = "AABABBA", k = 1` (answer 4)
- `s = "ABCD", k = 0` (answer 1)
- `s = "AAAA", k = 10` (k bigger than the string; answer 4, not 14)

Permutation In String (Medium)

Restate: Return True if some contiguous part of `s2` is a rearrangement (permutation) of `s1`.

Hint 1: Fixed-size sliding window. The window size is `len(s1)`.

Hint 2: A window is a permutation of `s1` when its letter counts equal the counts of `s1`. Slide the window by adding one letter on the right and removing one on the left, and compare two 26-length count arrays (or track how many of the 26 letters currently match).

Hint 3:

- If `len(s1) > len(s2)`, return False.
- Build counts for `s1` and for the first `len(s1)` letters of `s2`.
- If they are equal, return True.
- Slide: add `s2[r]`, remove `s2[r - len(s1)]`, compare again.
- If no window matches, return False.

Complexity: $O(26 * n) = O(n)$ time with array comparison ($O(n)$ with a "matches" counter), $O(1)$ space.

Edge cases to test:

- `s1 = "ab", s2 = "eidbaooo"` (True)
- `s1 = "ab", s2 = "eidboao"` (False)
- `s1 = "abc", s2 = "ab"` (`s1` longer, False)
- `s1 = "a", s2 = "a"` (True)
- `s1 = "aab", s2 = "abbb"` (same letters but wrong counts in every window, False)

Minimum Window Substring (Hard)

Restate: Return the shortest contiguous part of `s` that contains every character of `t` (including repeats), or "" if none exists.

Hint 1: Variable sliding window: expand right until the window is valid, then shrink left as much as possible while it stays valid.

Hint 2: Keep `need` = counts of `t`, and a number `have` = how many distinct characters currently meet their required count. The window is valid when `have == len(need)`. This check is $O(1)$, so you do not compare whole dicts each step.

Hint 3:

- Build `need` from `t`; set `window = {}`, `have = 0`, `l = 0`.
- For each `r`: add `s[r]` to `window`; if it is in `need` and `window[s[r]] == need[s[r]]`, do `have += 1`.
- While `have == len(need)`: record the window if it is the shortest so far.
- Then remove `s[l]`; if it is in `need` and `window[s[l]] < need[s[l]]`, do `have -= 1`; `l += 1`.
- Return the best window, or `""`.

Complexity: $O(\text{len}(s) + \text{len}(t))$ time, $O(\text{alphabet})$ space.

Edge cases to test:

- `s = "a"`, `t = "a"` (answer `"a"`)
- `s = "a"`, `t = "aa"` (answer `""`, not enough copies)
- `s = "ADOBECODEBANC"`, `t = "ABC"` (answer `"BANC"`)
- `s = "ab"`, `t = "b"` (answer `"b"`)
- `s = "aA"`, `t = "a"` (case sensitive, answer `"a"`)

Sliding Window Maximum (Hard)

Restate: For every window of size `k` that slides over the array, return the maximum value inside the window.

Hint 1: Use a deque (double-ended queue) that holds indexes.

Hint 2: Keep the deque in decreasing order of values (a monotonic deque). A smaller value that sits before a bigger new value can never be a maximum again, so pop it from the back. The front is always the current maximum; pop it from the front when it falls out of the window.

Hint 3:

- For each index `r`:
- While the deque is not empty and `nums[deque[-1]] <= nums[r]`, pop from the back.
- Append `r`.
- If `deque[0] <= r - k`, pop from the front (it left the window).
- If `r >= k - 1`, append `nums[deque[0]]` to the result.

Complexity: $O(n)$ time (each index enters and leaves the deque once), $O(k)$ space.

Edge cases to test:

- `nums = [1]`, `k = 1` (answer `[1]`)
- `nums = [1, -1]`, `k = 1` (answer `[1, -1]`)
- `nums = [9, 8, 7, 6]`, `k = 2` (decreasing, answer `[9, 8, 7]`)
- `nums = [1, 3, -1, -3, 5, 3, 6, 7]`, `k = 3` (answer `[3, 3, 5, 5, 6, 7]`)
- `k` equal to the array length (one answer, the global maximum)

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Number of Sub Arrays of Size K and Avg Greater than or Equal to Threshold	Medium	open	watch
2	Frequency of The Most Frequent Element	Medium	open	watch
3	Fruits into Basket	Medium	open	watch
4	Maximum Number of Vowels in a Substring of Given Length	Medium	open	watch
5	Minimum Number of Flips to Make The Binary String Alternating	Medium	open	watch
6	Minimum Size Subarray Sum	Medium	open	watch

Quiz

- Which keyword most strongly suggests a sliding window?
 - A) Sorted
 - B) Contiguous subarray or substring
 - C) Tree
 - D) Shortest path
- What is the length of the window from index l to index r (inclusive)?
 - A) $r - l$
 - B) $r - l + 1$
 - C) $r + l$
 - D) $r - l - 1$
- Time complexity of a variable sliding window over n items?
 - A) $O(n)$
 - B) $O(n \log n)$
 - C) $O(n^2)$
- "Subarray sum equals k" where numbers can be negative. Best pattern?
 - A) Sliding window
 - B) Prefix sum + hash map
 - C) Two pointers on sorted array
- In Longest Repeating Character Replacement, when is the window valid?
 - A) When all letters are unique
 - B) When $\text{length} - \text{max_freq} \leq k$
 - C) When $\text{length} \leq k$
- Why is a variable sliding window $O(n)$?
 - A) Each index is added and removed at most once
 - B) It uses binary search
 - C) It sorts first
- Window length for [l, r] inclusive is:

- A) $r - l$
- B) $r - l + 1$
- C) $r + l$

8. Shrinking the window should use:

- A) if
- B) while
- C) for

Answer key

1. **B** - Contiguous subarray or substring. A window is always a contiguous range of indexes.
2. **B** - $r - l + 1$. Both ends are included, so you add 1.
3. **A** - $O(n)$. Each index enters the window once and leaves once.
4. **B** - Prefix sum + hash map. Negative numbers break the shrink rule, so prefix sums with a map are used.
5. **B** - When $\text{length} - \text{max_freq} \leq k$. You can replace the non-majority letters, and there must be at most k of them.
6. **A** - Each index is added and removed at most once.
7. **B** - $r - l + 1$.
8. **B** - while.

More quiz

1. In Sliding Window Maximum, what does the deque store, and in what order?
 - A. Values, in increasing order
 - B. Indexes, with their values in decreasing order from front to back
 - C. All k values of the window, unsorted
 - D. Only the maximum
2. In Minimum Window Substring, what does the variable `have` count?
 - A. The window length
 - B. The total number of characters in t
 - C. The number of distinct characters whose required count is fully met in the window
 - D. The number of times the window moved
3. Which pattern fits "find all start indexes of anagrams of p in s "?
 - A. Fixed-size sliding window with letter counts
 - B. Binary search
 - C. Monotonic stack
 - D. Two heaps
4. For Best Time to Buy and Sell Stock, prices `[3, 8, 1, 4]`. What is the answer?

- A. 3
- B. 5
- C. 7
- D. 0

5. In Longest Substring Without Repeating Characters, why must you check `last[ch] >= l` before moving `l`?

- A. To avoid an index error
- B. To make the code faster
- C. An old position outside the window would move `l` backwards, as in `"abba"`
- D. It is not needed

More quiz: answer key

1. **B** - The deque holds indexes so you can tell when the front has left the window. Values are kept decreasing, so the front is always the maximum.
2. **C** - `have` goes up by one when a character reaches its needed count and goes down when it drops below. The window is valid when `have` equals the number of distinct characters in `t`.
3. **A** - This is "Find All Anagrams in a String". The window size is `len(p)`, and you compare letter counts as the window slides, exactly like Permutation in String.
4. **B** - Buy at 3 and sell at 8 for a profit of 5. Buying at 1 later only allows selling at 4 for a profit of 3.
5. **C** - In `"abba"`, at the last `"a"` the left edge is already at index 2. The old `"a"` at index 0 is outside the window, so jumping to index 1 would wrongly move `l` backwards.

Flashcards

- **Q:** Best Time to Buy and Sell Stock: what single value do you track besides the best profit? — **A:** The minimum price seen so far.
- **Q:** Longest Substring Without Repeating: what map speeds up the shrink step? — **A:** A dict of character -> last index seen, so `l` can jump directly.
- **Q:** Character Replacement validity rule? — **A:** `window_length - max_freq <= k`.
- **Q:** Why is it fine to never decrease `max_freq` in Character Replacement? — **A:** The best answer only grows when a larger `max_freq` appears, so a stale `max_freq` never produces a wrong larger answer.
- **Q:** Permutation in String: window size? — **A:** Fixed at `len(s1)`.
- **Q:** Minimum Window Substring: when do you shrink the left edge? — **A:** While the window is valid (`have == len(need)`), recording the shortest window each time.
- **Q:** Sliding Window Maximum: when do you pop from the front of the deque? — **A:** When the front index is `<= r - k`, meaning it has left the window.
- **Q:** Sliding Window Maximum: why pop smaller values from the back? — **A:** A smaller value before a bigger newer value can never be a window maximum again.
- **Q:** Minimum Window Substring test that checks repeat counts? — **A:** `s = "a", t = "aa"` must return `""`.

DSA topic: Stack

What it is

A stack is a "last in, first out" (LIFO) list. You add on top (push) and remove from the top (pop).

Real-life analogy: a pile of plates in a wedding buffet. You put a new plate on top, and you take the top plate first.

In Python, a normal `list` is a stack: `append()` is push, `pop()` is pop, and `stack[-1]` is peek. All are $O(1)$.

How to recognise it

- Matching pairs: brackets, tags, "open and close".
- "Next greater element", "next warmer day", "previous smaller value".
- You must undo or go back to the most recent thing (undo, backspace, path like "..").
- Evaluate an expression (Reverse Polish Notation, calculator).
- You need the minimum or maximum while pushing and popping (Min Stack).

Pattern 1: Matching pairs

Push openers. When you see a closer, the top of the stack must be its matching opener.

```
def is_valid(s):
    pairs = {"(": ")", "[": "]", "{": "}"
    stack = []
    for ch in s:
        if ch in pairs:
            # closing bracket
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
        else:
            stack.append(ch)
    return not stack
```

opening bracket
nothing left open

Pattern 2: Monotonic stack

Use for "next greater" or "next smaller" questions. Keep the stack in decreasing (or increasing) order. When a new item breaks the order, pop and answer for the popped items.

```
def next_greater(nums):
    res = [-1] * len(nums)
    stack = [] # holds indexes, values decreasing
    for i, x in enumerate(nums):
        while stack and nums[stack[-1]] < x:
            j = stack.pop()
            res[j] = x # x is the next greater for j
        stack.append(i)
    return res
```

Pattern 3: Expression evaluation

Push numbers. When you see an operator, pop two numbers, compute, and push the result.

```
def eval_rpn(tokens):
    stack = []
    for t in tokens:
        if t in "+-*/*":
            b, a = stack.pop(), stack.pop() # order matters
            if t == "+": stack.append(a + b)
            elif t == "-": stack.append(a - b)
            elif t == "*": stack.append(a * b)
            else: stack.append(int(a / b)) # truncate toward zero
        else:
            stack.append(int(t))
    return stack[0]
```

Worked example: Daily Temperatures

Given daily temperatures, for each day return how many days you wait for a warmer day (0 if never).
Input: temps = [73, 74, 72, 76]. Stack holds indexes of days still waiting. res = [0, 0, 0, 0].

1. i = 0, 73. Stack empty. Push 0. Stack = [0].
2. i = 1, 74. 74 > temps[0] = 73. Pop 0, res[0] = 1 - 0 = 1. Push 1. Stack = [1].
3. i = 2, 72. 72 > 74? No. Push 2. Stack = [1, 2].
4. i = 3, 76. 76 > 72: pop 2, res[2] = 1. 76 > 74: pop 1, res[1] = 2. Push 3. Stack = [3].
5. End. Day 3 never gets warmer, stays 0. Result = [1, 2, 1, 0].

```
def daily_temperatures(temps):
    res = [0] * len(temps)
    stack = [] # indexes, temps decreasing
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop()
            res[j] = i - j
        stack.append(i)
    return res
```

Complexity cheat sheet

- push / pop / peek -> $O(1)$
- Valid Parentheses -> $O(n)$ time, $O(n)$ space
- Monotonic stack over n items -> $O(n)$ time (each index pushed and popped once), $O(n)$ space
- Min Stack (store pairs of value and current min) -> $O(1)$ for every operation

Common mistakes

- Calling `stack[-1]` or `stack.pop()` on an empty stack. Always check `if stack` first.
- Forgetting to check that the stack is empty at the end (input like "(" is not valid).
- Popping operands in the wrong order for "-" and "/". The first pop is the right operand.
- Using `a // b` for division in RPN. Python floors negative numbers; use `int(a / b)`.
- Storing values when you need distances. Store indexes in the monotonic stack.

What to say in the interview

- "Brackets must close in reverse order of opening, so a stack is a natural fit."
- "For next warmer day, I will use a monotonic decreasing stack of indexes."
- "When I find a warmer day, I pop all colder days and fill in their answers."
- "Each index is pushed and popped at most once, so the total time is $O(n)$."

Practice order

- Valid Parentheses first: the basic push/pop idea.
- Min Stack: learn to store extra info with each item.
- Evaluate Reverse Polish Notation: stack as a calculator.
- Daily Temperatures and Car Fleet: the monotonic stack idea.
- Largest Rectangle in Histogram last: hard, but the best monotonic stack practice.

Cheat sheet

Idea

Last in, first out. Great for matching pairs and for "next greater/smaller element" (monotonic stack).

Use it when

- Brackets / nested structure
- Next greater element, daily temperatures, largest rectangle
- Evaluating expressions (RPN)

Monotonic stack template

```
res = [0] * len(t)
stack = [] # indexes, temps decreasing
for i, x in enumerate(t):
    while stack and t[stack[-1]] < x:
        j = stack.pop()
        res[j] = i - j
    stack.append(i)
```

Complexity

O(n) - every index pushed and popped once.

Common mistakes

- Popping from an empty stack
- Storing values when you need indexes (distances)

Visualise it

- <https://visualgo.net/en/list>

Videos

- [Monotonic Stack in 6 minutes](#) - AlgoMasterIO (English)
- [Next Greater Element](#) - take U forward (Hindi + English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Valid Parentheses (Blind 75)	Easy	open	watch
2	Min Stack	Medium	open	watch
3	Evaluate Reverse Polish Notation	Medium	open	watch
4	Generate Parentheses	Medium	open	watch
5	Daily Temperatures	Medium	open	watch
6	Car Fleet	Medium	open	watch
7	Largest Rectangle In Histogram	Hard	open	watch

Problem hints

Try each problem on your own first. Open one hint at a time. Only go to the next hint when you are stuck for 10 or more minutes.

Valid Parentheses (Easy)

Restate: Given a string of only `()[]{}` , return `True` if every bracket is closed by the same type of bracket in the correct order.

Hint 1: Use a stack. The most recent unclosed opening bracket must be the first one to close.

Hint 2: Keep a small dict that maps each closing bracket to its opening partner: `{")": "(", "]" : "[", "}" : "{"}`. On a closing bracket, the top of the stack must be its partner.

Hint 3:

- Create an empty stack.
- For an opening bracket, push it.
- For a closing bracket: if the stack is empty or the top is not its partner, return `False`; else pop.
- After the loop, return `True` only if the stack is empty.

Complexity: $O(n)$ time, $O(n)$ space.

Edge cases to test:

- `"]"` (closing with an empty stack, `False`)
- `"( ("` (left-over openers, `False`)
- `"([)]"` (crossed pairs, `False`)
- `"{[]}"` (nested, `True`)
- `"()[{}]"` (in sequence, `True`)

Min Stack (Medium)

Restate: Design a stack that supports `push`, `pop`, `top` and `getMin` (the smallest value in the stack), all in $O(1)$ time.

Hint 1: Use a second stack (or store pairs) to remember the minimum.

Hint 2: When you push `x`, also push `min(x, current_min)` on a "min stack". Both stacks always have the same height, so popping both keeps the minimum correct after any pop.

Hint 3:

- `push(x)` : push `x` on the main stack; push `min(x, min_stack[-1])` (or `x` if empty) on the min stack.
- `pop()` : pop from both stacks.
- `top()` : return `stack[-1]`.
- `getMin()` : return `min_stack[-1]`.

Complexity: $O(1)$ time for every operation, $O(n)$ space.

Edge cases to test:

- push 0, push 1, push 0, getMin, pop, getMin (duplicates of the min; answers 0 then 0)
- push -2, push 0, push -3, getMin, pop, top, getMin (answers -3, 0, -2)
- One element: push 5, getMin, top (both 5)
- Negative and very large values (for example -2^{31} and $2^{31} - 1$)
- Strictly decreasing pushes followed by pops one at a time

Evaluate Reverse Polish Notation (Medium)

Restate: Evaluate an arithmetic expression written in postfix order (operators come after their two numbers), where division truncates toward zero.

Hint 1: Use a stack of numbers.

Hint 2: When you see an operator, pop two numbers. The first pop is the **right** operand and the second pop is the **left** operand. Order matters for `-` and `/`. In Python, use `int(a / b)` for truncation toward zero, because `//` rounds toward negative infinity.

Hint 3:

- For each token: if it is a number, push `int(token)`.
- If it is an operator: `b = pop()`, `a = pop()`.
- Compute `a op b` (with `int(a / b)` for division) and push the result.
- At the end, return the only value left on the stack.

Complexity: $O(n)$ time, $O(n)$ space.

Edge cases to test:

- `["42"]` (single number)
- `["4", "13", "5", "/", "+"]` (answer 6)
- `["6", "-132", "/"]` (negative division, answer 0, not -1)
- `["3", "4", "-"]` (operand order, answer -1)
- `["-3", "2", "*"]` (negative number token; do not treat `"-3"` as an operator)

Generate Parentheses (Medium)

Restate: Given `n`, return every string of `n` pairs of parentheses that is well formed.

Hint 1: This is backtracking (build the string one character at a time and undo choices). A stack or list holds the current partial string.

Hint 2: You may add `"("` while `open_count < n`. You may add `)"` only while `close_count < open_count`. Following these two rules, every finished string of length `2n` is valid, so you never need to check validity at the end.

Hint 3:

- Write a helper `build(open_count, close_count)`.
- If both counts equal `n`, save the current string and return.
- If `open_count < n`: add `"("`, recurse, then remove it.
- If `close_count < open_count`: add `)"`, recurse, then remove it.

Complexity: The number of results is the n -th Catalan number, about $4^n / (n * \sqrt{n})$; time is that times n for building strings. Space $O(n)$ for the recursion, plus the output.

Edge cases to test:

- `n = 1` (answer `["()"]`)
- `n = 2` (answer `["()", "()()"]`)
- `n = 3` (5 strings)
- `n = 4` (14 strings; a quick count check)
- Check that no result contains `")("` at the start or is unbalanced

Daily Temperatures (Medium)

Restate: For each day, return how many days you must wait for a warmer temperature, or 0 if it never gets warmer.

Hint 1: Monotonic stack of indexes.

Hint 2: Keep the stack in decreasing temperature order. When today is warmer than the day on top, today is that day's answer, so pop it and record the gap. One pass is enough.

Hint 3:

- Create `res = [0] * n` and an empty stack.
- For each index `i`:
- While the stack is not empty and `temps[i] > temps[stack[-1]]`: pop `j`, set `res[j] = i - j`.
- Push `i`.
- Return `res`.

Complexity: $O(n)$ time, $O(n)$ space.

Edge cases to test:

- `[30]` (answer `[0]`)
- `[30, 40, 50, 60]` (answer `[1, 1, 1, 0]`)
- `[60, 50, 40]` (answer `[0, 0, 0]`)
- `[70, 70, 71]` (equal is not warmer, answer `[2, 1, 0]`)
- `[73, 74, 75, 71, 69, 72, 76, 73]` (answer `[1, 1, 4, 2, 1, 1, 0, 0]`)

Car Fleet (Medium)

Restate: Cars drive toward the same target on a one-lane road; a faster car that catches a slower one must slow down and they move together as a fleet. Return how many fleets reach the target.

Hint 1: Sort the cars by position, closest to the target first. Then use a stack of arrival times.

Hint 2: Each car's arrival time alone is $(\text{target} - \text{position}) / \text{speed}$. Going from the front car backwards, a car whose time is less than or equal to the fleet ahead of it catches up and joins that fleet. A car with a bigger time starts a new fleet.

Hint 3:

- Pair each position with its speed and sort by position in descending order.
- For each car, compute $\text{time} = (\text{target} - \text{pos}) / \text{speed}$.
- If the stack is empty or $\text{time} > \text{stack}[-1]$, push time (new fleet).
- Otherwise do nothing (it joins the fleet ahead).
- Return the stack size.

Complexity: $O(n \log n)$ time for sorting, $O(n)$ space.

Edge cases to test:

- One car (answer 1)
- $\text{target} = 10$, $\text{position} = [6, 8]$, $\text{speed} = [4, 2]$ (both arrive at time 1; they meet exactly at the target, answer 1)
- $\text{target} = 100$, $\text{position} = [0, 2, 4]$, $\text{speed} = [4, 2, 1]$ (answer 1)
- $\text{target} = 10$, $\text{position} = [3]$, $\text{speed} = [3]$ (answer 1)
- $\text{target} = 12$, $\text{position} = [10, 8, 0, 5, 3]$, $\text{speed} = [2, 4, 1, 1, 3]$ (answer 3)

Largest Rectangle In Histogram (Hard)

Restate: Given bar heights of width 1, return the area of the largest rectangle that fits inside the histogram.

Hint 1: Monotonic increasing stack. Store pairs of $(\text{start_index}, \text{height})$.

Hint 2: When a shorter bar arrives, every taller bar on the stack cannot extend further right, so pop it and compute its area: $\text{height} * (i - \text{start})$. The new shorter bar can extend left to the start index of the last bar you popped, so push it with that start.

Hint 3:

- For each index i and height h : set $\text{start} = i$.
- While the stack top height is greater than h : pop $(\text{idx}, \text{height})$, update $\text{best} = \max(\text{best}, \text{height} * (i - \text{idx}))$, set $\text{start} = \text{idx}$.
- Push (start, h) .
- After the loop, for each remaining $(\text{idx}, \text{height})$, update best with $\text{height} * (n - \text{idx})$.

Complexity: $O(n)$ time, $O(n)$ space.

Edge cases to test:

- [5] (answer 5)
- [2, 1, 5, 6, 2, 3] (answer 10)
- [2, 4] (answer 4)
- [1, 1, 1, 1] (equal heights, answer 4)
- [0, 0] and strictly increasing [1, 2, 3, 4, 5] (answers 0 and 9; the second tests the after-loop cleanup)

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Removing Stars From a String	Medium	open	watch
2	Validate Stack Sequences	Medium	open	watch
3	Asteroid Collision	Medium	open	watch
4	Online Stock Span	Medium	open	watch
5	Simplify Path	Medium	open	watch
6	Decode String	Medium	open	watch

Quiz

1. What order does a stack follow?
 - A) First in, first out
 - B) Last in, first out
 - C) Sorted order
2. Which pattern fits "for each day, find the next warmer day"?
 - A) Monotonic stack
 - B) Binary search
 - C) Sliding window
 - D) Trie
3. Time complexity of a monotonic stack pass over n items?
 - A) $O(n)$
 - B) $O(n \log n)$
 - C) $O(n^2)$
4. In RPN, tokens ["6", "2", "-"]. What is popped first?
 - A) 6
 - B) 2
 - C) -
5. How do you get $O(1)$ getMin in Min Stack?
 - A) Sort the stack on each push

- B) Store the current minimum with each pushed item
 - C) Scan the stack when asked
6. Monotonic stack time complexity over n items:
- A) $O(n)$
 - B) $O(n \log n)$
 - C) $O(n^2)$
7. Valid Parentheses: on a closing bracket you should:
- A) Push it
 - B) Check the top matches, then pop
 - C) Ignore it
8. Min Stack gets min in $O(1)$ by:
- A) Sorting
 - B) Keeping a second stack of current minimums
 - C) Scanning

Answer key

1. **B** - Last in, first out. The most recently pushed item is popped first.
2. **A** - Monotonic stack. A decreasing stack of indexes finds the next greater element in $O(n)$.
3. **A** - $O(n)$. Each index is pushed once and popped at most once.
4. **B** - 2. The last pushed number (2) is the right operand, so the result is $6 - 2$.
5. **B** - Store the current minimum with each pushed item. Each item remembers the min at that time, so the top always has the current min.
6. **A** - $O(n)$.
7. **B** - Check the top matches, then pop.
8. **B** - Keeping a second stack of current minimums.

More quiz

1. In Evaluate Reverse Polish Notation, why should you avoid `a // b` in Python for division?
 - A. It is slower
 - B. It rounds toward negative infinity, but the problem wants truncation toward zero
 - C. It returns a float
 - D. It fails for positive numbers
2. In Car Fleet, why do you process cars from the closest to the target first?
 - A. Closer cars are always faster
 - B. A car can only be blocked by cars ahead of it, so the cars ahead must be known first
 - C. To make sorting stable
 - D. Because the stack must be in increasing order of position
3. Which pattern fits "for each element, find the previous smaller element"?

- A. Monotonic stack
- B. Hash set
- C. Binary search on answer
- D. Trie

4. In Generate Parentheses, when may you add a closing bracket?

- A. Any time
- B. Only when the open count equals n
- C. Only when the close count is less than the open count
- D. Only at the end

5. Largest Rectangle in Histogram with heights `[3, 1, 3]`. What is the answer?

- A. 1
- B. 3
- C. 6
- D. 9

More quiz: answer key

- B** - For example `-7 // 2` is `-4` in Python, but truncation toward zero gives `-3`. Use `int(a / b)`.
- B** - The cars ahead decide whether a car gets blocked. Starting from the front lets you compare each car only with the fleet directly ahead of it.
- A** - A monotonic increasing stack keeps candidates; pop while the top is not smaller, and the remaining top is the previous smaller element.
- C** - A closing bracket is only safe when there is an unmatched opening bracket, which means `close < open`.
- B** - Each tall bar alone gives 3. The full width of 3 bars has height 1, giving 3. No rectangle beats 3.

Flashcards

- **Q:** Valid Parentheses: what two checks happen at the end or on a close bracket? — **A:** On a close, the stack must be non-empty with the matching opener on top; at the end, the stack must be empty.
- **Q:** Min Stack: what is pushed onto the min stack with each value? — **A:** `min(x, current_min)`, so both stacks stay the same height.
- **Q:** RPN: which pop is the right operand? — **A:** The first pop is the right operand; the second pop is the left operand.
- **Q:** RPN test case for truncation? — **A:** `["6", "-132", "/"]` must give 0, not -1.
- **Q:** How many valid strings does Generate Parentheses return for `n = 3`? — **A:** 5 (the 3rd Catalan number).
- **Q:** Daily Temperatures: what does the stack hold? — **A:** Indexes of days still waiting for a warmer day, with temperatures in decreasing order.

- **Q:** Car Fleet: formula for a car's arrival time? — **A:** $(\text{target} - \text{position}) / \text{speed}$.
- **Q:** Car Fleet: when does a car start a new fleet? — **A:** When its arrival time is strictly greater than the fleet time on top of the stack.
- **Q:** Largest Rectangle: area of a popped bar at index i ? — **A:** $\text{height} * (i - \text{start_index})$.
- **Q:** Largest Rectangle: why process the stack after the loop? — **A:** Bars still on the stack extend to the end, so their area is $\text{height} * (n - \text{start_index})$.

DSA topic: Binary Search

What it is

Binary search finds a target in a sorted range by checking the middle and throwing away half of the range each step.

Real-life analogy: finding a word in a paper dictionary. You open the middle. If your word comes later, you ignore the left half. You repeat until you find it.

Each step halves the range, so it takes $O(\log n)$ steps. For 1 billion items, that is only about 30 steps.

Binary search also works on an **answer range**, not just an array: "what is the smallest speed that works?"

How to recognise it

- The input is **sorted** (or rotated sorted).
- The problem asks for $O(\log n)$ time.
- "Find the minimum X such that condition is true" or "maximum X such that ...".
- The condition is monotonic: once it becomes true, it stays true for bigger values.
- A 2D matrix where rows and columns are sorted.

Pattern 1: Classic search for a target

Use when you need the exact index of a value in a sorted array.

```
def search(nums, target):
    l, r = 0, len(nums) - 1
    while l <= r:
        m = (l + r) // 2
        if nums[m] == target:
            return m
        elif nums[m] < target:
            l = m + 1      # target is on the right
        else:
            r = m - 1      # target is on the left
    return -1
```

Pattern 2: Binary search on the answer

Use when you can write `ok(x)` that says "x is good enough", and `ok` is false, false, ..., true, true. Find the first true.

```
def min_good_value(lo, hi):
    while lo < hi:
        m = (lo + hi) // 2
        if ok(m):           # m works, maybe smaller also works
            hi = m
        else:               # m does not work, go bigger
            lo = m + 1
    return lo               # first value where ok is True
```

Pattern 3: Rotated sorted array

One half around `m` is always sorted. Check which half is sorted, then check if the target is inside that half.

```
def search_rotated(nums, target):
    l, r = 0, len(nums) - 1
    while l <= r:
        m = (l + r) // 2
        if nums[m] == target:
            return m
        if nums[l] <= nums[m]:           # Left half sorted
            if nums[l] <= target < nums[m]:
                r = m - 1
            else:
                l = m + 1
        else:                           # right half sorted
            if nums[m] < target <= nums[r]:
                l = m + 1
            else:
                r = m - 1
    return -1
```

Worked example: Koko Eating Bananas

Piles of bananas, `h` hours. Koko eats at speed `k` per hour (one pile per hour max). Find the smallest `k` to finish in `h` hours.

Input: piles = [3, 6, 7], h = 6. Speed range is 1 to max(piles) = 7. Hours at speed `k` = sum of `ceil(p / k)`.

1. lo = 1, hi = 7. m = 4. Hours = 1 + 2 + 2 = 5 <= 6. Works. hi = 4.
2. lo = 1, hi = 4. m = 2. Hours = 2 + 3 + 4 = 9 > 6. Too slow. lo = 3.
3. lo = 3, hi = 4. m = 3. Hours = 1 + 2 + 3 = 6 <= 6. Works. hi = 3.
4. lo = 3, hi = 3. Loop stops. Answer = 3.


```
import math

def min_eating_speed(piles, h):
    lo, hi = 1, max(piles)
    while lo < hi:
        k = (lo + hi) // 2
        hours = sum(math.ceil(p / k) for p in piles)
        if hours <= h:
            hi = k          # k works, try smaller
        else:
            lo = k + 1      # too slow, go faster
    return lo
```

Complexity cheat sheet

- Search in a sorted array -> $O(\log n)$ time, $O(1)$ space
- Search a sorted $m \times n$ matrix (treat as 1D) -> $O(\log(m * n))$
- Binary search on answer -> $O(\log(\text{range}) * \text{cost of ok}())$
- Koko Eating Bananas -> $O(n \log \max(\text{piles}))$
- Median of Two Sorted Arrays -> $O(\log(\min(m, n)))$

Common mistakes

- Infinite loop: using `lo = m` with `m = (lo + hi) // 2`. When `lo` and `hi` are next to each other, `m` equals `lo` forever.
- Mixing up templates: `while l <= r` goes with `r = m - 1`; `while lo < hi` goes with `hi = m`.
- Wrong search range for "binary search on answer" (for example starting Koko at 0 causes divide by zero).
- Forgetting the rotated case where `nums[l] == nums[m]` (use `<=` in the sorted-left check).
- Using binary search on unsorted data without a monotonic condition.

What to say in the interview

- "The array is sorted, so I can use binary search and get $O(\log n)$ time."
- "Here the answer itself is in a range from 1 to max, and if a speed works, any faster speed also works. So I will binary search on the answer."
- "I will use the template with `lo < hi` and `hi = m`, which returns the first value that works."
- "Let me check the edge case where `lo` and `hi` are next to each other, to make sure there is no infinite loop."

Practice order

- Binary Search first: get the basic template right without bugs.
- Search a 2D Matrix: treat the matrix as one long sorted array.

- Koko Eating Bananas: the most important "search on answer" problem.
- Find Minimum in Rotated Sorted Array, then Search in Rotated Sorted Array.
- Time Based Key-Value Store, then Median of Two Sorted Arrays last (hard).

Cheat sheet

Idea

Cut the search space in half each step -> $O(\log n)$. Works on sorted arrays AND on any monotonic yes/no condition ("binary search on the answer").

Use it when

- Sorted / rotated sorted array
- "Minimum value such that condition holds" (Koko bananas, ship capacity)

Template

```
lo, hi = 0, len(a) - 1
while lo <= hi:
    mid = (lo + hi) // 2
    if a[mid] == target: return mid
    if a[mid] < target: lo = mid + 1
    else: hi = mid - 1
return -1
```

Answer-space template

```
lo, hi = min_ans, max_ans
while lo < hi:
    mid = (lo + hi) // 2
    if ok(mid): hi = mid
    else: lo = mid + 1
return lo
```

Common mistakes

- Infinite loop when `lo = mid` with `lo < hi`
- Wrong boundaries in rotated arrays: first find which half is sorted

Visualise it

- <https://visualgo.net/en/bst>

Videos

- [What is Binary Search](#) - Jenny's Lectures (English)
- [Binary Search theory + code](#) - Apna College (Hindi)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Binary Search	Easy	open	watch
2	Search a 2D Matrix	Medium	open	watch
3	Koko Eating Bananas	Medium	open	watch
4	Find Minimum In Rotated Sorted Array (Blind 75)	Medium	open	watch
5	Search In Rotated Sorted Array (Blind 75)	Medium	open	watch
6	Time Based Key Value Store	Medium	open	watch
7	Median of Two Sorted Arrays	Hard	open	watch

Problem hints

Try each problem on your own first. Open one hint at a time. Only go to the next hint when you are stuck for 10 or more minutes.

Binary Search (Easy)

Restate: Given a sorted list of distinct numbers and a target, return the index of the target, or -1 if it is not there, in $O(\log n)$ time.

Hint 1: Keep a search range `[lo, hi]` and look at the middle element each time.

Hint 2: If the middle is smaller than the target, the target can only be on the right, so throw away the left half including the middle. If it is bigger, throw away the right half. Use `while lo <= hi` when both ends are inclusive.

Hint 3:

- Set `lo = 0`, `hi = n - 1`.
- While `lo <= hi`: `mid = (lo + hi) // 2`.
- If `nums[mid] == target`, return `mid`.
- If `nums[mid] < target`, set `lo = mid + 1`; else set `hi = mid - 1`.
- Return -1.

Complexity: $O(\log n)$ time, $O(1)$ space.

Edge cases to test:

- `nums = [5], target = 5` (answer 0)
- `nums = [5], target = 3` (answer -1)
- Target at the first index and target at the last index
- Target smaller than every element and bigger than every element
- `nums = [-1, 0, 3, 5, 9, 12], target = 2` (missing value in the middle, answer -1)

Search a 2D Matrix (Medium)

Restate: Each row of the matrix is sorted and each row's first number is bigger than the previous row's last number; return True if the target is in the matrix, in $O(\log(m * n))$ time.

Hint 1: The whole matrix behaves like one long sorted list.

Hint 2: Do binary search over indexes `0` to `m * n - 1`. Turn index `i` into a cell with `row = i // n` and `col = i % n`, where `n` is the number of columns.

Hint 3:

- Set `lo = 0, hi = m * n - 1`.
- While `lo <= hi`: `mid = (lo + hi) // 2`.
- Read `value = matrix[mid // n][mid % n]`.
- Compare with the target and move `lo` or `hi` as in normal binary search.
- Return False if the loop ends.

Complexity: $O(\log(m * n))$ time, $O(1)$ space.

Edge cases to test:

- `[[1]]` with target 1 and target 2
- A single row: `[[1, 3, 5, 7]]`
- A single column: `[[1], [3], [5]]`
- Target equal to the last element of a row (row boundary)
- Target smaller than `matrix[0][0]` or bigger than the last element

Koko Eating Bananas (Medium)

Restate: Koko eats from one pile per hour at speed `k` bananas per hour (she finishes a pile and waits if it has fewer than `k`); return the smallest `k` that lets her finish all piles within `h` hours.

Hint 1: Binary search on the answer (the speed), not on the array.

Hint 2: If speed `k` works, every faster speed also works. So the "works" check is monotonic. The speed range is 1 to `max(piles)`, and hours needed at speed `k` is the sum of `ceil(pile / k)`.

Hint 3:

- Set `lo = 1, hi = max(piles)`.
- While `lo < hi`: `mid = (lo + hi) // 2`.

- Compute `hours = sum(ceil(p / mid) for p in piles)` (use `(p + mid - 1) // mid` to stay in integers).
- If `hours <= h`, set `hi = mid` (it works, try slower); else set `lo = mid + 1`.
- Return `lo`.

Complexity: $O(n \log M)$ time, where $M = \max \text{pile}$; $O(1)$ space.

Edge cases to test:

- `piles = [3, 6, 7, 11]`, `h = 8` (answer 4)
- `piles = [30, 11, 23, 4, 20]`, `h = 5` (`h` equals number of piles, answer 30 = max pile)
- `piles = [30, 11, 23, 4, 20]`, `h = 6` (answer 23)
- `piles = [1]`, `h = 1` (answer 1)
- `piles = [1000000000]`, `h = 2` (very large pile; answer 500000000)

Find Minimum In Rotated Sorted Array (Medium)

Restate: A sorted list of distinct numbers was rotated at an unknown point (for example `[3, 4, 5, 1, 2]`); return the smallest number in $O(\log n)$ time.

Hint 1: Binary search, comparing the middle with the right end.

Hint 2: If `nums[mid] > nums[hi]`, the drop (the minimum) is to the right of `mid`. Otherwise the minimum is at `mid` or to its left. Comparing with `hi` avoids the special case of "no rotation".

Hint 3:

- Set `lo = 0`, `hi = n - 1`.
- While `lo < hi`: `mid = (lo + hi) // 2`.
- If `nums[mid] > nums[hi]`, set `lo = mid + 1`.
- Else set `hi = mid`.
- Return `nums[lo]`.

Complexity: $O(\log n)$ time, $O(1)$ space.

Edge cases to test:

- `[1]` (answer 1)
- `[1, 2, 3, 4]` (not rotated, answer 1)
- `[2, 1]` (answer 1)
- `[4, 5, 6, 7, 0, 1, 2]` (answer 0)
- `[2, 3, 4, 5, 1]` (minimum at the last index)

Search In Rotated Sorted Array (Medium)

Restate: Given a rotated sorted list of distinct numbers and a target, return the index of the target or -1, in $O(\log n)$ time.

Hint 1: Binary search. At every step, at least one half (left of mid or right of mid) is fully sorted.

Hint 2: Find which half is sorted by comparing `nums[lo]` with `nums[mid]`. Then check if the target lies inside that sorted half's range. If yes, search there; if no, search the other half.

Hint 3:

- While `lo <= hi`: `mid = (lo + hi) // 2`; if `nums[mid] == target`, return `mid`.
- If `nums[lo] <= nums[mid]` (left half sorted):
- If `nums[lo] <= target < nums[mid]`, set `hi = mid - 1`; else `lo = mid + 1`.
- Else (right half sorted): if `nums[mid] < target <= nums[hi]`, set `lo = mid + 1`; else `hi = mid - 1`.
- Return -1.

Complexity: $O(\log n)$ time, $O(1)$ space.

Edge cases to test:

- `nums = [1]`, `target = 0` (answer -1)
- `nums = [3, 1]`, `target = 1` (answer 1; tests the `<=` in `nums[lo] <= nums[mid]`)
- `nums = [4, 5, 6, 7, 0, 1, 2]`, `target = 0` (answer 4)
- `nums = [4, 5, 6, 7, 0, 1, 2]`, `target = 3` (answer -1)
- An array that is not rotated at all

Time Based Key Value Store (Medium)

Restate: Design a store with `set(key, value, timestamp)` and `get(key, timestamp)`, where `get` returns the value set at the largest timestamp less than or equal to the given one, or `""`.

Hint 1: Use a dict from key to a list of `(timestamp, value)` pairs.

Hint 2: The problem says timestamps for `set` arrive in increasing order, so each list is already sorted. `get` becomes "binary search for the last timestamp `<= t`".

Hint 3:

- `set`: append `(timestamp, value)` to `store[key]`.
- `get`: if the key is missing, return `""`.
- Binary search the list: if `list[mid].timestamp <= t`, remember its value and go right; else go left.
- Return the remembered value, or `""` if none was found.

Complexity: `set` $O(1)$, `get` $O(\log n)$ where n = number of entries for that key; $O(\text{total entries})$ space.

Edge cases to test:

- `get` for a key that was never set (answer `""`)
- `get` with a timestamp smaller than the first `set` for that key (answer `""`)

- `get` with exactly a stored timestamp
- `get` with a timestamp between two stored timestamps (returns the older value)
- `get` with a timestamp larger than all stored ones (returns the newest value)

Median of Two Sorted Arrays (Hard)

Restate: Given two sorted lists, return the median of all their numbers combined, in $O(\log(\min(m, n)))$ time.

Hint 1: Do not merge. Binary search on how many elements to take from the **smaller** list into the "left half".

Hint 2: If you take i elements from A, you must take $j = (m + n + 1) // 2 - i$ from B so the left half has the right size. The split is correct when $A[i-1] \leq B[j]$ and $B[j-1] \leq A[i]$ (treat out-of-range as minus or plus infinity).

Hint 3:

- Make A the smaller list. Search i in the range $[0, m]$.
- For each i , compute j , and the four border values A_{left} , A_{right} , B_{left} , B_{right} .
- If $A_{left} > B_{right}$, move i left; if $B_{left} > A_{right}$, move i right.
- When correct: if the total is odd, the median is $\max(A_{left}, B_{left})$.
- If even, it is $(\max(A_{left}, B_{left}) + \min(A_{right}, B_{right})) / 2$.

Complexity: $O(\log(\min(m, n)))$ time, $O(1)$ space.

Edge cases to test:

- $[1, 3]$ and $[2]$ (answer 2.0)
- $[1, 2]$ and $[3, 4]$ (answer 2.5)
- $[]$ and $[1]$ (one list empty, answer 1.0)
- $[1, 1]$ and $[1, 1]$ (all equal, answer 1.0)
- $[1, 2, 3]$ and $[100, 200]$ (all of A is smaller than B, answer 3.0)

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Single Element in a Sorted Array	Medium	open	watch
2	Capacity to Ship Packages	Medium	open	watch
3	Find Peak Element	Medium	open	watch
4	Successful Pairs of Spells and Potions	Medium	open	watch
5	Minimize the Maximum Difference of Pairs	Medium	open	watch
6	Search In Rotated Sorted Array II	Medium	open	watch

Quiz

1. What is the time complexity of binary search on n sorted items?
 - A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n)$
 - D) $O(n \log n)$
2. What condition must be true to binary search on the answer?
 - A) The input must be sorted
 - B) The $ok(x)$ check must be monotonic (false...false, true...true)
 - C) The answer must be an index
3. Which pattern fits Koko Eating Bananas?
 - A) Sliding window
 - B) Binary search on the answer (speed)
 - C) Heap
 - D) Two pointers
4. With `while lo < hi and m = (lo + hi) // 2`, which update can cause an infinite loop?
 - A) `hi = m`
 - B) `lo = m`
 - C) `lo = m + 1`
5. In a rotated sorted array, what is always true around the middle index?
 - A) Both halves are sorted
 - B) At least one half is sorted
 - C) Neither half is sorted
6. Binary search time complexity:
 - A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n)$
7. Koko Eating Bananas searches over:
 - A) The piles array
 - B) Possible eating speeds
 - C) Hours
8. In a rotated sorted array, first decide:
 - A) Which half is sorted
 - B) The max element
 - C) The length

Answer key

1. **B** - $O(\log n)$. Each step cuts the range in half.
2. **B** - The $ok(x)$ check must be monotonic (false...false, true...true). Monotonic means you can safely throw away half of the range.
3. **B** - Binary search on the answer (speed). If speed k works, every faster speed also works, so search the speed range.

4. **B** - $lo = m$. When $hi = lo + 1$, m equals lo , so $lo = m$ never moves.
5. **B** - At least one half is sorted. The rotation point is in only one half, so the other half is sorted.
6. **B** - $O(\log n)$.
7. **B** - Possible eating speeds.
8. **A** - Which half is sorted.

More quiz

1. In Search a 2D Matrix with n columns, which cell does flat index `i` map to?
 - A. `(i % n, i // n)`
 - B. `(i // n, i % n)`
 - C. `(i // m, i % m)`
 - D. `(i, i)`
2. In Find Minimum in Rotated Sorted Array, why compare `nums[mid]` with `nums[hi]` and not `nums[lo]`?
 - A. It is faster
 - B. It handles the non-rotated case cleanly: when `nums[mid] <= nums[hi]`, the minimum is at mid or left of it
 - C. `nums[lo]` can be out of range
 - D. There is no reason
3. Which pattern fits "find the smallest ship capacity to deliver all packages within D days"?
 - A. Sliding window
 - B. Binary search on the answer (capacity) with a "can ship in D days" check
 - C. Heap
 - D. Monotonic stack
4. In Koko Eating Bananas, piles `[4, 9]` and speed 3. How many hours does she need?
 - A. 4
 - B. 5
 - C. 13
 - D. 3
5. In Median of Two Sorted Arrays, why do you binary search on the smaller array?
 - A. The smaller array is always sorted
 - B. It keeps `j` in a valid range and gives $O(\log(\min(m, n)))$ time
 - C. The larger array may be empty
 - D. It avoids using infinity

More quiz: answer key

1. **B** - Each row holds n cells, so the row is $i // n$ and the column is $i \% n$.
2. **B** - Comparing with the right end tells you directly which side holds the drop. If the middle is bigger than the right end, the drop is on the right; otherwise it is at mid or to the left, including the non-rotated case.
3. **B** - This is "Capacity To Ship Packages Within D Days". If a capacity works, any bigger capacity also works, so the check is monotonic. Search capacities from $\max(\text{weights})$ to $\text{sum}(\text{weights})$.
4. **B** - $\text{ceil}(4 / 3) = 2$ and $\text{ceil}(9 / 3) = 3$, so $2 + 3 = 5$ hours.
5. **B** - With i chosen from the smaller array, $j = \text{half} - i$ never becomes negative or larger than the other array. It also makes the search range as small as possible.

Flashcards

- **Q:** Why write $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) // 2$ in languages like Java? — **A:** To avoid integer overflow when $\text{lo} + \text{hi}$ is bigger than the max int; in Python it is not needed.
- **Q:** Search a 2D Matrix: total search range? — **A:** Indexes 0 to $m * n - 1$, treating the matrix as one sorted list.
- **Q:** Koko Eating Bananas: search range for the speed? — **A:** From 1 to $\max(\text{piles})$.
- **Q:** Integer formula for $\text{ceil}(p / k)$? — **A:** $(p + k - 1) // k$.
- **Q:** Rotated array minimum: what does $\text{nums}[\text{mid}] > \text{nums}[\text{hi}]$ tell you? — **A:** The minimum is strictly to the right of mid, so set $\text{lo} = \text{mid} + 1$.
- **Q:** Search in Rotated Sorted Array: how do you know the left half is sorted? — **A:** $\text{nums}[\text{lo}] \leq \text{nums}[\text{mid}]$.
- **Q:** Time Based Key Value Store: why no sorting step? — **A:** The problem guarantees that set timestamps are strictly increasing, so each list is already sorted.
- **Q:** Time Based Key Value Store: what does get binary search for? — **A:** The last entry with timestamp less than or equal to the query.
- **Q:** Median of Two Sorted Arrays: when is a partition correct? — **A:** When $A_{\text{left}} \leq B_{\text{right}}$ and $B_{\text{left}} \leq A_{\text{right}}$.
- **Q:** Median of Two Sorted Arrays: median when the total length is odd? — **A:** $\max(A_{\text{left}}, B_{\text{left}})$, the largest value of the left half.

DSA topic: Linked List

What it is

A linked list is a chain of nodes. Each node has a value and a `next` pointer to the next node. The last node points to `None`.

Real-life analogy: a treasure hunt. Each clue tells you where the next clue is. You cannot jump to clue 5 directly; you must follow clues 1, 2, 3, 4 first.

There is no index access. Reading the k -th node is $O(k)$. But inserting or removing a node is $O(1)$ if you already have the node before it.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

How to recognise it

- The input is given as `head` of a linked list.
- Reverse a list, or reverse part of it.
- Detect a cycle, or find the middle node.
- Merge two or more sorted lists.
- "Remove the n -th node from the end" in one pass.

Pattern 1: Reverse in place

Use three pointers: `prev`, `curr`, and a saved `nxt`. Flip each arrow to point backwards.

```
def reverse_list(head):
    prev, curr = None, head
    while curr:
        nxt = curr.next      # save the rest of the list
        curr.next = prev     # flip the arrow
        prev = curr          # move prev forward
        curr = nxt           # move curr forward
    return prev              # new head
```

Pattern 2: Fast and slow pointers

Slow moves 1 step, fast moves 2 steps. Use it to find the middle, or to detect a cycle (they meet if there is a loop).

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False
# When the loop ends without a cycle, slow is at the middle.
```

Pattern 3: Dummy head node

Use a fake first node when the real head may change (merging, deleting). At the end, return `dummy.next`.

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n + 1): # fast goes n + 1 steps ahead
        fast = fast.next
    while fast:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next # skip the target node
    return dummy.next
```

Worked example: Merge Two Sorted Lists

Given two sorted linked lists, merge them into one sorted list.

Input: a = 1 -> 3, b = 2 -> 4. Start: dummy -> (nothing), tail = dummy.

1. Compare 1 and 2. 1 is smaller. tail.next = 1. tail = 1. a = 3.
2. Compare 3 and 2. 2 is smaller. tail.next = 2. tail = 2. b = 4.
3. Compare 3 and 4. 3 is smaller. tail.next = 3. tail = 3. a = None.
4. a is empty. Attach the rest of b: tail.next = 4.
5. Return dummy.next: 1 -> 2 -> 3 -> 4.

```
def merge_two_lists(a, b):
    dummy = ListNode()
    tail = dummy
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b # attach whatever is left
    return dummy.next
```

Complexity cheat sheet

- Access k-th node -> $O(k)$
- Insert/delete after a known node -> $O(1)$
- Reverse a list -> $O(n)$ time, $O(1)$ space
- Cycle detection (Floyd) -> $O(n)$ time, $O(1)$ space
- Merge K sorted lists with a heap -> $O(N \log k)$

Common mistakes

- Losing the rest of the list: changing `curr.next` before saving it in `nxt`.
- Not checking `fast` and `fast.next` before `fast.next.next`, which crashes on `None`.
- Forgetting the dummy node, then writing many special cases for the head.
- Returning `head` after reversing. The new head is `prev`.
- Using `==` on nodes when you mean "same node". Use `is`.

What to say in the interview

- "The head might change, so I will use a dummy node to keep the code simple."
- "To reverse, I keep `prev` and `curr`, and save next before I flip each pointer."
- "For the cycle, I will use slow and fast pointers. If there is a loop, fast will catch slow."
- "This is $O(n)$ time and $O(1)$ extra space because I only change pointers."

Practice order

- Reverse Linked List first: you will reuse this in many problems.
- Merge Two Sorted Lists: learn the dummy node.
- Linked List Cycle, then Reorder List (middle + reverse + merge combined).
- Remove Nth Node From End, Copy List with Random Pointer, Add Two Numbers.
- LRU Cache, Merge K Sorted Lists, and Reverse Nodes in k-Group last: these are the hard ones.

Cheat sheet

Idea

Nodes with next pointers. Most problems are pointer rewiring; draw boxes and arrows before coding.

Tricks

- Dummy head node to avoid special cases
- Fast/slow pointers: middle, cycle detection (Floyd)
- Reverse in place with `prev/cur/next`

Reverse template

```
prev, cur = None, head
while cur:
    nxt = cur.next
    cur.next = prev
    prev, cur = cur, nxt
return prev
```

Complexity

Usually $O(n)$ time, $O(1)$ space.

Common mistakes

- Losing the rest of the list (save `next` first)
- Not handling empty list / single node

Visualise it

- <https://visualgo.net/en/list>

Videos

- [Introduction to Linked List](#) - Neso Academy (English)
- [Introduction to Linked List](#) - Shradha Khapra (Hindi)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Reverse Linked List (Blind 75)	Easy	open	watch
2	Merge Two Sorted Lists (Blind 75)	Easy	open	watch
3	Linked List Cycle (Blind 75)	Easy	open	watch
4	Reorder List (Blind 75)	Medium	open	watch
5	Remove Nth Node From End of List (Blind 75)	Medium	open	watch
6	Copy List With Random Pointer	Medium	open	watch
7	Add Two Numbers	Medium	open	watch
8	Find The Duplicate Number	Medium	open	watch
9	LRU Cache	Medium	open	watch
10	Merge K Sorted Lists (Blind 75)	Hard	open	watch
11	Reverse Nodes In K Group	Hard	open	watch

Problem hints

Try each problem on your own first. Open one hint at a time. Only go to the next hint when you are stuck for 10 or more minutes.

Reverse Linked List (Easy)

Restate: Reverse a singly linked list and return the new head.

Hint 1: Walk the list once with three references: `prev`, `curr` and `nxt`.

Hint 2: At every node, save the next node first, then point the current node backwards to `prev`. If you change `curr.next` before saving it, you lose the rest of the list.

Hint 3:

- Set `prev = None`, `curr = head`.
- While `curr`: save `nxt = curr.next`.
- Set `curr.next = prev`.
- Move forward: `prev = curr`, `curr = nxt`.
- Return `prev`.

Complexity: $O(n)$ time, $O(1)$ space (the recursive version uses $O(n)$ stack space).

Edge cases to test:

- Empty list `[]` (return `None`)
- One node `[1]`
- Two nodes `[1, 2]` (answer `[2, 1]`)
- `[1, 2, 3, 4, 5]`
- A list with duplicate values `[1, 1, 2]` (values do not matter, only links)

Merge Two Sorted Lists (Easy)

Restate: Merge two sorted linked lists into one sorted linked list by relinking their nodes.

Hint 1: Use a dummy head node and a `tail` pointer.

Hint 2: Always attach the smaller of the two current nodes to `tail`. When one list runs out, attach the whole rest of the other list in one step.

Hint 3:

- Create `dummy` and set `tail = dummy`.
- While both lists have nodes: attach the smaller node to `tail.next`, move that list forward, move `tail` forward.
- Attach whichever list is not empty: `tail.next = a or b`.

- Return `dummy.next`.

Complexity: $O(m + n)$ time, $O(1)$ space.

Edge cases to test:

- Both empty (answer empty)
- One empty, one `[0]`
- `[1, 2, 4]` and `[1, 3, 4]` (equal values, answer `[1, 1, 2, 3, 4, 4]`)
- `[5]` and `[1, 2, 3]` (all of one list is smaller)
- Negative values `[-3, 0]` and `[-2]`

Linked List Cycle (Easy)

Restate: Return True if following `next` pointers from the head ever reaches a node you have already visited.

Hint 1: Fast and slow pointers (Floyd's cycle detection), or a hash set of visited nodes.

Hint 2: The fast pointer moves 2 steps and the slow pointer moves 1 step. If there is a loop, the fast one gains one step per move and must catch the slow one. If there is no loop, fast reaches `None`.

Hint 3:

- Set `slow = fast = head`.
- While `fast` and `fast.next`: `slow = slow.next`, `fast = fast.next.next`.
- If `slow is fast`, return True.
- After the loop, return False.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test:

- Empty list
- One node with no cycle
- One node pointing to itself (True)
- Two nodes where the tail points to the head (True)
- A long list where the tail points to a middle node (True)

Reorder List (Medium)

Restate: Rearrange the list `L0 -> L1 -> ... -> Ln` into `L0 -> Ln -> L1 -> Ln-1 -> ...` in place.

Hint 1: Break it into three smaller problems you already know: find the middle, reverse a list, merge two lists.

Hint 2: Find the middle with slow and fast pointers, cut the list there, reverse the second half, then weave the two halves together one node from each.

Hint 3:

- Find the middle with slow/fast pointers.
- Set `second = slow.next` and cut with `slow.next = None`.
- Reverse `second`.
- Merge alternately: take one node from the first half, then one from the reversed second half, until the second half is empty.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test:

- One node `[1]`
- Two nodes `[1, 2]` (unchanged)
- `[1, 2, 3, 4]` (answer `[1, 4, 2, 3]`)
- `[1, 2, 3, 4, 5]` (answer `[1, 5, 2, 4, 3]`)
- Check that the last node's `next` is `None` (no accidental cycle)

Remove Nth Node From End of List (Medium)

Restate: Delete the n -th node counting from the end of the list and return the head.

Hint 1: Two pointers with a fixed gap, plus a dummy node before the head.

Hint 2: If `fast` is n steps ahead of `slow`, then when `fast` reaches the last node, `slow` sits just before the node to delete. The dummy node handles the case where the head itself is removed.

Hint 3:

- Create `dummy` with `dummy.next = head`; set `slow = fast = dummy`.
- Move `fast` forward n steps.
- Move both forward until `fast.next` is `None`.
- Remove with `slow.next = slow.next.next`.
- Return `dummy.next`.

Complexity: $O(\text{length})$ time in one pass, $O(1)$ space.

Edge cases to test:

- `[1]`, $n = 1$ (answer empty)
- `[1, 2]`, $n = 2$ (remove the head, answer `[2]`)
- `[1, 2]`, $n = 1$ (remove the tail, answer `[1]`)
- `[1, 2, 3, 4, 5]`, $n = 2$ (answer `[1, 2, 3, 5]`)
- n equal to the list length on a long list

Copy List With Random Pointer (Medium)

Restate: Each node has a `next` pointer and a `random` pointer (to any node or `None`); build a deep copy where all pointers point to the new nodes.

Hint 1: Use a hash map from each old node to its new copy.

Hint 2: Do two passes. In pass 1, create every copy node (values only) and store `old -> new`. In pass 2, set `new.next = map[old.next]` and `new.random = map[old.random]`. Map `None` to `None` so you do not need special checks.

Hint 3:

- Create `copy = {None: None}`.
- Pass 1: for each old node, `copy[old] = Node(old.val)`.
- Pass 2: for each old node, set `copy[old].next = copy[old.next]` and `copy[old].random = copy[old.random]`.
- Return `copy[head]`.

Complexity: $O(n)$ time, $O(n)$ space. (An interleaving trick gives $O(1)$ extra space; mention it as a follow-up.)

Edge cases to test:

- Empty list
- One node whose random points to itself
- All random pointers are `None`
- Two nodes with random pointers crossing ($1 \rightarrow 2$, $2 \rightarrow 1$)
- Check the copy shares no node with the original (identity check, not just values)

Add Two Numbers (Medium)

Restate: Two numbers are stored as linked lists with digits in reverse order (ones digit first); return their sum as a linked list in the same format.

Hint 1: Do school addition digit by digit, with a carry. Use a dummy head for the result.

Hint 2: Keep looping while either list has nodes **or** the carry is not zero. That last condition creates the extra digit for cases like $5 + 5 = 10$.

Hint 3:

- Set `carry = 0`, `dummy`, `tail = dummy`.
- While `l1` or `l2` or `carry`: take `v1`, `v2` (0 if the list ended).
- `total = v1 + v2 + carry`; `carry = total // 10`.
- Append a node with `total % 10`; move the pointers forward.
- Return `dummy.next`.

Complexity: $O(\max(m, n))$ time, $O(\max(m, n))$ space for the output.

Edge cases to test:

- `[0]` and `[0]` (answer `[0]`)
- `[5]` and `[5]` (answer `[0, 1]`, extra carry digit)
- `[9, 9, 9, 9]` and `[9, 9]` (different lengths with long carry, answer `[8, 9, 0, 0, 1]`)
- `[2, 4, 3]` and `[5, 6, 4]` (answer `[7, 0, 8]`)
- `[1]` and `[9, 9]` (answer `[0, 0, 1]`)

Find The Duplicate Number (Medium)

Restate: An array of $n + 1$ integers holds values from 1 to n , with exactly one value repeated (possibly many times); find it without changing the array and with $O(1)$ extra space.

Hint 1: This is a linked list problem in disguise. Treat each index as a node and `nums[i]` as its next pointer.

Hint 2: Because two indexes point to the same value, this "list" has a cycle, and the duplicate is the start of the cycle. Use Floyd's algorithm: first find a meeting point, then start a second pointer from the beginning; where they meet is the cycle start.

Hint 3:

- Set `slow = fast = 0`.
- Loop: `slow = nums[slow]`, `fast = nums[nums[fast]]`, until `slow == fast`.
- Set `slow2 = 0`.
- Loop: `slow = nums[slow]`, `slow2 = nums[slow2]`, until they are equal.
- Return that value.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test:

- `[1, 1]` (smallest case, answer 1)
- `[1, 3, 4, 2, 2]` (answer 2)
- `[3, 1, 3, 4, 2]` (answer 3)
- `[2, 2, 2, 2, 2]` (repeated many times, answer 2)
- Duplicate equal to n , for example `[1, 4, 4, 2, 3]` (answer 4)

LRU Cache (Medium)

Restate: Design a cache with a fixed capacity where `get` and `put` run in $O(1)$, and when it is full, `put` removes the least recently used key.

Hint 1: Combine a hash map with a doubly linked list.

Hint 2: The map gives `key -> node` in $O(1)$. The doubly linked list keeps usage order: most recent near one end, least recent near the other. Dummy `head` and `tail` nodes remove all the "empty list" special cases. Every `get` or `put` moves the node to the "most recent" end.

Hint 3:

- Write two helpers: `remove(node)` and `add_to_front(node)`.
- `get(key)` : if missing, return -1; else remove the node, add it to the front, return its value.
- `put(key, value)` : if the key exists, remove its old node.
- Create a new node, add it to the front, store it in the map.
- If the size is over capacity, remove the node just before `tail` and delete its key from the map.

Complexity: $O(1)$ time per operation, $O(\text{capacity})$ space.

Edge cases to test:

- Capacity 1: `put(1,1)`, `put(2,2)`, `get(1)` (answer -1)
- `put(1,1)`, `put(2,2)`, `get(1)`, `put(3,3)`, `get(2)` (2 was evicted, answer -1; 1 was kept because get made it recent)
- `put` on an existing key updates the value and makes it most recent
- `put` on an existing key when the cache is full must NOT evict anything
- `get` on a key that was never added (answer -1)

Merge K Sorted Lists (Hard)

Restate: Merge k sorted linked lists into one sorted linked list.

Hint 1: Use a min-heap of the current head of each list, or merge the lists in pairs.

Hint 2: With a heap, the smallest of all k fronts is always on top. Pop it, attach it, and push that node's next. Push tuples `(val, index, node)`, because Python cannot compare two `ListNode` objects when values tie.

Hint 3:

- Push `(head.val, i, head)` for every non-empty list.
- Create `dummy` and `tail`.
- While the heap is not empty: pop the smallest, attach it to `tail`, move `tail`.
- If the popped node has a `next`, push `(next.val, i, next)`.
- Return `dummy.next`.

Complexity: $O(N \log k)$ time, where N = total nodes; $O(k)$ space for the heap. (Pairwise merging is also $O(N \log k)$.)

Edge cases to test:

- `[]` (no lists)
- `[[] ]` (one empty list)
- `[[] , [1] ]`
- `[[1, 4, 5], [1, 3, 4], [2, 6] ]` (answer `[1, 1, 2, 3, 4, 4, 5, 6]`)
- Many lists with equal values (tests the tie-breaker)

Reverse Nodes In K Group (Hard)

Restate: Reverse the nodes of the list k at a time; if fewer than k nodes remain at the end, leave them as they are.

Hint 1: Dummy node, plus a helper that finds the k-th node ahead.

Hint 2: Before reversing a group, check that k nodes exist. Then reverse exactly those k nodes and reconnect: the node before the group must point to the new group head, and the old group head (now the group tail) must point to the next group.

Hint 3:

- Create `dummy` before head; set `group_prev = dummy`.
- Find `kth`, the k-th node after `group_prev`; if it does not exist, stop.
- Save `group_next = kth.next`.
- Reverse the nodes from `group_prev.next` up to `kth`, with `prev` starting as `group_next`.
- Connect: `old_first = group_prev.next`; `group_prev.next = kth`; `group_prev = old_first`. Repeat.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test:

- `k = 1` (list unchanged)
- `[1, 2, 3, 4, 5]`, `k = 2` (answer `[2, 1, 4, 3, 5]`)
- `[1, 2, 3, 4, 5]`, `k = 3` (answer `[3, 2, 1, 4, 5]`)
- `k` equal to the list length (whole list reversed)
- `[1, 2, 3, 4, 5, 6]`, `k = 3` (exact multiple, answer `[3, 2, 1, 6, 5, 4]`)

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Maximum Twin Sum Of A Linked List	Medium	open	watch
2	Swapping Nodes in a Linked List	Medium	open	watch
3	Design Linked List	Medium	open	watch
4	Design Browser History	Medium	open	watch
5	Swap Nodes In Pairs	Medium	open	watch
6	Sort List	Medium	open	watch

Quiz

1. Time to access the k-th node of a linked list?
 - A) $O(1)$
 - B) $O(\log k)$
 - C) $O(k)$
2. Which pattern detects a cycle with $O(1)$ space?
 - A) Hash set of visited nodes
 - B) Fast and slow pointers
 - C) Sorting the list
3. After reversing a list with prev/curr, what is the new head?
 - A) head
 - B) curr
 - C) prev
4. Why use a dummy head node?
 - A) It makes the list faster
 - B) It removes special cases when the real head changes
 - C) It saves memory
5. Which pattern fits "find the middle node in one pass"?
 - A) Fast and slow pointers
 - B) Binary search
 - C) Stack
6. Cycle detection with $O(1)$ space uses:
 - A) A hash set
 - B) Fast and slow pointers
 - C) Recursion
7. A dummy head node helps because:
 - A) It is faster
 - B) It removes special cases for the first node
 - C) It saves memory
8. Reversing a list in place takes extra space:
 - A) $O(1)$
 - B) $O(n)$
 - C) $O(\log n)$

Answer key

1. **C** - $O(k)$. You must walk from the head one node at a time.
2. **B** - Fast and slow pointers. Fast moves 2 and slow moves 1; they meet only if there is a loop.
3. **C** - prev. When curr becomes None, prev points to the old last node.
4. **B** - It removes special cases when the real head changes. You always have a node before the head, so insert/delete code is the same everywhere.
5. **A** - Fast and slow pointers. When fast reaches the end, slow has moved half the distance.
6. **B** - Fast and slow pointers.
7. **B** - It removes special cases for the first node.
8. **A** - $O(1)$.

More quiz

1. In LRU Cache, why is the linked list doubly linked and not singly linked?
 - A. To use less memory
 - B. To remove a node from the middle in $O(1)$, you need its previous node, which a singly linked list does not give you
 - C. To make get faster than $O(1)$
 - D. Python requires it
2. Find the Duplicate Number is solved with Floyd's algorithm. What plays the role of the `next` pointer?
 - A. `i + 1`
 - B. `nums[i]`, the value at index `i`
 - C. `nums[i] - 1`
 - D. The sorted order
3. Which pattern fits "check if a linked list is a palindrome in $O(1)$ space"?
 - A. Copy the values into a list
 - B. Find the middle with slow/fast, reverse the second half, compare the halves
 - C. Use a min-heap
 - D. Binary search
4. In Add Two Numbers, `[9, 9] + [1]` gives what list?
 - A. `[0, 0, 1]`
 - B. `[1, 0, 0]`
 - C. `[0, 1]`
 - D. `[10, 9]`
5. In Remove Nth Node From End, why start both pointers at a dummy node?
 - A. To make the loop shorter
 - B. So that removing the head works with the same code as removing any other node
 - C. To avoid recursion
 - D. Because `n` can be zero

More quiz: answer key

1. **B** - To unlink a node in $O(1)$, you set `prev.next = node.next` and `next.prev = node.prev`. Only a doubly linked node knows its previous node directly.
2. **B** - Index `i` "points to" index `nums[i]`. Two indexes point to the duplicate value, so the path enters a cycle whose start is the duplicate.

3. **B** - Reversing the second half in place lets you compare from both ends without extra memory. A good tester also restores the list afterwards if the caller expects it unchanged.
4. **A** - $99 + 1 = 100$, which in reverse digit order is `[0, 0, 1]`. The loop must continue while a carry is left.
5. **B** - When n equals the list length, the node to remove is the head. With a dummy, `slow` stops at the dummy and `slow.next = slow.next.next` removes the head with no special case.

Flashcards

- **Q:** Reverse Linked List: what must you save before changing `curr.next`? — **A:** The next node (`nxt = curr.next`), or the rest of the list is lost.
- **Q:** Merge Two Sorted Lists: how do you finish when one list runs out? — **A:** Attach the remaining list in one step: `tail.next = a` or `b`.
- **Q:** Reorder List: which three known sub-problems does it combine? — **A:** Find the middle, reverse the second half, merge the two halves alternately.
- **Q:** Remove Nth from End: how far ahead does `fast` start? — **A:** n steps ahead of `slow`, both starting from the dummy node.
- **Q:** Copy List with Random Pointer: why put `None: None` in the map? — **A:** So `copy[old.next]` and `copy[old.random]` work even when the pointer is `None`.
- **Q:** Add Two Numbers: loop condition? — **A:** While `l1` or `l2` or `carry` is non-zero.
- **Q:** Find the Duplicate Number: second phase of Floyd's algorithm? — **A:** Start a new pointer at 0; move it and the slow pointer one step at a time; they meet at the duplicate.
- **Q:** LRU Cache: which two data structures? — **A:** A hash map (key to node) and a doubly linked list (usage order) with dummy head and tail.
- **Q:** Merge K Sorted Lists: why add an index to the heap tuple? — **A:** On equal values, Python would compare `ListNode` objects and raise `TypeError`; the index breaks ties.
- **Q:** Reverse Nodes in k-Group: what happens to a last group with fewer than k nodes? — **A:** It is left in its original order.

DSA topic: Trees

What it is

A binary tree is made of nodes. Each node has a value, a `left` child and a `right` child. The top node is the root.

A **Binary Search Tree (BST)** has a rule: everything on the left is smaller, everything on the right is bigger. Real-life analogy: a family tree, or the folder structure on your laptop. Each folder can contain sub-folders. Almost every tree problem is solved with recursion: solve for the left subtree, solve for the right subtree, then combine.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

How to recognise it

- Input is `root` of a binary tree or BST.
- Words like depth, height, diameter, path, ancestor, level, subtree.
- "Level by level" or "right side view" means BFS.
- "Sorted order" or "k-th smallest" in a BST means in-order traversal.
- "Validate", "same tree", "balanced" means recursion that returns info upward.

Pattern 1: DFS recursion (return a value up)

Ask: "if I know the answer for left and right children, how do I get the answer for this node?"

```
def max_depth(root):
    if not root:                # base case: empty tree
        return 0
    left = max_depth(root.left)
    right = max_depth(root.right)
    return 1 + max(left, right) # combine
```

Pattern 2: BFS level order (queue)

Use when you need levels: level averages, right side view, minimum depth.

```

from collections import deque

def level_order(root):
    if not root:
        return []
    res, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):      # process one full level
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res

```

Pattern 3: DFS with bounds or a global answer

Pass allowed limits down (Validate BST), or update an outside variable while returning a different value (Diameter, Max Path Sum).

```

def is_valid_bst(root):
    def dfs(node, low, high):
        if not node:
            return True
        if not (low < node.val < high):
            return False
        return (dfs(node.left, low, node.val) and
                dfs(node.right, node.val, high))
    return dfs(root, float("-inf"), float("inf"))

```

Worked example: Diameter of Binary Tree

The diameter is the number of edges on the longest path between any two nodes. The path may not pass through the root.

Input tree: root 1, children 2 and 3; node 2 has children 4 and 5. Each call returns height; we update `best = max(best, left + right)`.

1. Node 4: left = 0, right = 0. best = 0. Return height 1.
2. Node 5: same. Return height 1.
3. Node 2: left = 1, right = 1. best = max(0, 2) = 2. Return 1 + 1 = 2.
4. Node 3: left = 0, right = 0. Return 1.
5. Node 1: left = 2, right = 1. best = max(2, 3) = 3. Return 3.
6. Answer = best = 3 (path 4 -> 2 -> 1 -> 3).

```
def diameter_of_binary_tree(root):
    best = 0
    def height(node):
        nonlocal best
        if not node:
            return 0
        left = height(node.left)
        right = height(node.right)
        best = max(best, left + right)    # path through this node
        return 1 + max(left, right)
    height(root)
    return best
```

Complexity cheat sheet

- DFS or BFS visiting every node -> $O(n)$ time
- DFS recursion stack -> $O(h)$ space (h = height; $O(\log n)$ balanced, $O(n)$ skewed)
- BFS queue -> $O(w)$ space (w = widest level, up to $n/2$)
- BST search/insert -> $O(h)$: $O(\log n)$ if balanced, $O(n)$ worst case
- In-order traversal of BST -> values come out sorted

Common mistakes

- Forgetting the base case `if not node`, which causes an error on `None`.
- Validating a BST by only comparing a node with its direct children. You must pass min/max bounds down.
- Mixing "what the function returns" with "the final answer" (Diameter, Max Path Sum need a separate variable).
- Forgetting `nonlocal` when changing an outer variable inside a nested function.
- Deep recursion on a skewed tree can hit Python recursion limit; mention an iterative version.

What to say in the interview

- "I will solve this with DFS recursion. The base case is an empty node, which returns 0."
- "For each node, I get the answer from the left and right subtrees and combine them."
- "The function returns the height, but I keep the best diameter in a separate variable."
- "Time is $O(n)$ because I visit each node once, and space is $O(h)$ for the recursion stack."

Practice order

- Invert Binary Tree, Maximum Depth, Same Tree first: basic recursion.
- Diameter of Binary Tree and Balanced Binary Tree: return one thing, track another.
- Binary Tree Level Order Traversal and Right Side View: BFS.
- Validate BST, Kth Smallest in BST, Lowest Common Ancestor of BST: use the BST rule.

- Construct Tree from Preorder and Inorder, Max Path Sum, Serialize/Deserialize last (harder).

Cheat sheet

Idea

Almost every tree problem is recursion: solve for left, solve for right, combine. Decide what each call returns.

Traversals

- DFS: preorder (node, L, R), inorder (L, node, R -> sorted for BST), postorder (L, R, node)
- BFS with a queue: level order, right side view

DFS template

```
def dfs(node):  
    if not node:  
        return 0  
    left = dfs(node.left)  
    right = dfs(node.right)  
    return 1 + max(left, right)
```

BFS template

```
q = deque([root])  
while q:  
    for _ in range(len(q)):  
        node = q.popleft()  
        # process  
        if node.left: q.append(node.left)  
        if node.right: q.append(node.right)
```

Complexity

$O(n)$ time, $O(h)$ recursion space (h = height, worst $O(n)$).

Visualise it

- <https://visualgo.net/en/bst>

Videos

- [Binary Tree traversals \(BFS, DFS\)](#) - take U forward (Hindi + English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Invert Binary Tree (Blind 75)	Easy	open	watch
2	Maximum Depth of Binary Tree (Blind 75)	Easy	open	watch
3	Diameter of Binary Tree	Easy	open	watch
4	Balanced Binary Tree	Easy	open	watch
5	Same Tree (Blind 75)	Easy	open	watch
6	Subtree of Another Tree (Blind 75)	Easy	open	watch
7	Lowest Common Ancestor of a Binary Search Tree (Blind 75)	Medium	open	watch
8	Binary Tree Level Order Traversal (Blind 75)	Medium	open	watch
9	Binary Tree Right Side View	Medium	open	watch
10	Count Good Nodes In Binary Tree	Medium	open	watch
11	Validate Binary Search Tree (Blind 75)	Medium	open	watch
12	Kth Smallest Element In a Bst (Blind 75)	Medium	open	watch
13	Construct Binary Tree From Preorder And Inorder Traversal (Blind 75)	Medium	open	watch
14	Binary Tree Maximum Path Sum (Blind 75)	Hard	open	watch
15	Serialize And Deserialize Binary Tree (Blind 75)	Hard	open	watch

Problem hints

Try each problem on your own first. Open one hint at a time. Only go to the next hint when you are stuck for 10 or more minutes.

Invert Binary Tree (Easy)

Restate: Swap the left and right child of every node (make a mirror image of the tree) and return the root.

Hint 1: DFS recursion (or BFS with a queue). Every node needs the same small action.

Hint 2: At each node, swap its two children, then invert each child subtree. The base case is an empty node, which you return as it is.

Hint 3:

- If `root` is None, return None.
- Swap `root.left` and `root.right`.

- Invert `root.left` recursively.
- Invert `root.right` recursively.
- Return `root`.

Complexity: $O(n)$ time, $O(h)$ space for recursion, where h is the tree height.

Edge cases to test:

- Empty tree (return None)
- One node
- `[2, 1, 3]` (answer `[2, 3, 1]`)
- A left-skewed tree (becomes right-skewed)
- `[4, 2, 7, 1, 3, 6, 9]` (answer `[4, 7, 2, 9, 6, 3, 1]`)

Maximum Depth of Binary Tree (Easy)

Restate: Return the number of nodes on the longest path from the root down to a leaf.

Hint 1: DFS that returns a value up to the parent.

Hint 2: The depth of a node is 1 plus the larger depth of its two children. An empty tree has depth 0.

Hint 3:

- If `root` is None, return 0.
- Get the depth of the left subtree.
- Get the depth of the right subtree.
- Return `1 + max(left, right)`.

Complexity: $O(n)$ time, $O(h)$ space. (BFS counting levels also works.)

Edge cases to test:

- Empty tree (answer 0)
- One node (answer 1)
- `[1, null, 2]` (answer 2)
- `[3, 9, 20, null, null, 15, 7]` (answer 3)
- A skewed tree of 10,000 nodes (recursion depth: Python's default limit is about 1000, so an iterative version may be needed)

Diameter of Binary Tree (Easy)

Restate: Return the number of edges on the longest path between any two nodes (the path may skip the root).

Hint 1: DFS that returns height, while updating a global "best" answer.

Hint 2: The longest path through a node uses its left height plus its right height. Every node is a possible "top" of the path, so check this sum at every node, but return only `1 + max(left, right)` to the

parent.

Hint 3:

- Keep `best = 0` outside the helper.
- Helper `height(node)` : if None, return 0.
- Compute `left = height(node.left)`, `right = height(node.right)`.
- Update `best = max(best, left + right)`.
- Return `1 + max(left, right)`. The answer is `best`.

Complexity: $O(n)$ time, $O(h)$ space.

Edge cases to test:

- One node (answer 0)
- `[1, 2]` (answer 1)
- `[1, 2, 3, 4, 5]` (answer 3)
- A tree where the longest path does not pass through the root (a deep left subtree with two long branches)
- A skewed tree of 5 nodes (answer 4)

Balanced Binary Tree (Easy)

Restate: Return True if, for every node, the heights of its left and right subtrees differ by at most 1.

Hint 1: DFS that returns height, bottom-up.

Hint 2: Checking height separately at each node gives $O(n^2)$. Instead, let the helper return the height, or a special value like -1 meaning "already unbalanced", so each node is visited once.

Hint 3:

- Helper `check(node)` : if None, return 0.
- Get `left = check(node.left)`; if -1, return -1.
- Get `right = check(node.right)`; if -1, return -1.
- If `abs(left - right) > 1`, return -1.
- Return `1 + max(left, right)`. The tree is balanced if the result is not -1.

Complexity: $O(n)$ time, $O(h)$ space.

Edge cases to test:

- Empty tree (True)
- `[3, 9, 20, null, null, 15, 7]` (True)
- `[1, 2, 2, 3, 3, null, null, 4, 4]` (False)
- A root with balanced height difference, but an unbalanced deeper node (False; tests that every node is checked)
- `[1, null, 2, null, 3]` (skewed, False)

Same Tree (Easy)

Restate: Return True if two binary trees have the same shape and the same values in every position.

Hint 1: DFS on both trees at the same time.

Hint 2: Two trees are the same when both roots are None, or both exist with equal values and both pairs of children are the same.

Hint 3:

- If both nodes are None, return True.
- If only one is None, or the values differ, return False.
- Return `same(p.left, q.left)` and `same(p.right, q.right)`.

Complexity: $O(n)$ time, $O(h)$ space.

Edge cases to test:

- Both empty (True)
- One empty, one not (False)
- `[1, 2]` and `[1, null, 2]` (same values, different shape, False)
- `[1, 2, 1]` and `[1, 1, 2]` (False)
- Two identical large trees (True)

Subtree of Another Tree (Easy)

Restate: Return True if `subRoot` appears inside `root` as a full subtree (a node plus all of its descendants).

Hint 1: Reuse "Same Tree" as a helper.

Hint 2: Visit every node in `root`. At each node, ask "is the tree starting here the same as `subRoot`?" If any answer is yes, return True.

Hint 3:

- If `subRoot` is None, return True.
- If `root` is None, return False.
- If `same(root, subRoot)`, return True.
- Return `isSubtree(root.left, subRoot)` or `isSubtree(root.right, subRoot)`.

Complexity: $O(m * n)$ time in the worst case, $O(h)$ space. (Serializing both trees and using string matching can reach $O(m + n)$.)

Edge cases to test:

- `root = [3, 4, 5, 1, 2]`, `subRoot = [4, 1, 2]` (True)
- `root = [3, 4, 5, 1, 2, null, null, null, null, 0]`, `subRoot = [4, 1, 2]` (False, the node 2 has an extra child)

- `subRoot` equal to the whole `root` (True)
- `root = [1, 1]`, `subRoot = [1]` (True, matches the leaf)
- Single node trees with different values (False)

Lowest Common Ancestor of a Binary Search Tree (Medium)

Restate: In a binary search tree, return the lowest node that has both given nodes `p` and `q` in its subtree (a node counts as its own descendant).

Hint 1: Use the BST property: left values are smaller, right values are bigger.

Hint 2: Start at the root. If both `p` and `q` are smaller, the answer is on the left. If both are bigger, it is on the right. Otherwise, they split here (or one of them is this node), so this node is the answer.

Hint 3:

- Set `node = root`.
- While True: if `p.val < node.val` and `q.val < node.val`, go left.
- Else if `p.val > node.val` and `q.val > node.val`, go right.
- Else return `node`.

Complexity: $O(h)$ time, $O(1)$ space for the loop version.

Edge cases to test:

- `p` is the root (answer is the root)
- `p` is an ancestor of `q` (answer is `p`)
- `p` and `q` in different subtrees of the root (answer is the root)
- `p` and `q` both deep in the left subtree
- A two-node tree `[2, 1]` with `p = 2`, `q = 1` (answer 2)

Binary Tree Level Order Traversal (Medium)

Restate: Return the node values level by level, from top to bottom and left to right within each level.

Hint 1: BFS with a queue (`collections.deque`).

Hint 2: At the start of each level, the queue holds exactly the nodes of that level. Record the queue length, pop exactly that many nodes, and push their children for the next level.

Hint 3:

- If `root` is None, return `[]`.
- Put `root` in the queue.
- While the queue is not empty: `size = len(queue)`; create `level = []`.
- Pop `size` nodes, add each value to `level`, push their non-null children.
- Append `level` to the result.

Complexity: $O(n)$ time, $O(w)$ space, where w is the maximum width of the tree (up to about $n / 2$).

Edge cases to test:

- Empty tree (answer `[]`)
- One node (answer `[[1]]`)
- `[3, 9, 20, null, null, 15, 7]` (answer `[[3], [9, 20], [15, 7]]`)
- A skewed tree (one value per level)
- A full tree with 7 nodes (levels of size 1, 2, 4)

Binary Tree Right Side View (Medium)

Restate: Imagine you stand on the right side of the tree; return the values you can see, from top to bottom.

Hint 1: BFS level by level (or DFS that visits the right child first).

Hint 2: The visible node of each level is the last node of that level in left-to-right order. It is not always in the right subtree: if the right side is shorter, a left node can be visible.

Hint 3:

- Do level order BFS as usual.
- For each level, after popping all `size` nodes, append the value of the last node popped.
- Or with DFS (right child first): pass the depth; if `depth == len(result)`, this is the first node seen at that depth, so append it.

Complexity: $O(n)$ time, $O(w)$ space for BFS or $O(h)$ for DFS.

Edge cases to test:

- Empty tree (answer `[]`)
- `[1, 2, 3, null, 5, null, 4]` (answer `[1, 3, 4]`)
- `[1, 2, 3, 4]` (left side deeper, answer `[1, 3, 4]`)
- Only left children `[1, 2, null, 3]` (answer `[1, 2, 3]`)
- One node

Count Good Nodes In Binary Tree (Medium)

Restate: A node is "good" if no node on the path from the root to it has a bigger value; return how many good nodes the tree has.

Hint 1: DFS that passes information **down**: the maximum value seen on the path so far.

Hint 2: At each node, compare its value with the path maximum. If it is greater than or equal, it is good. Then pass `max(path_max, node.val)` to both children.

Hint 3:

- Helper `dfs(node, path_max)` : if None, return 0.
- `good = 1` if `node.val >= path_max` else 0.
- `new_max = max(path_max, node.val)`.
- Return `good + dfs(node.left, new_max) + dfs(node.right, new_max)`.
- Start with `dfs(root, root.val)`.

Complexity: $O(n)$ time, $O(h)$ space.

Edge cases to test:

- One node (answer 1; the root is always good)
- `[3, 1, 4, 3, null, 1, 5]` (answer 4)
- `[3, 3, null, 4, 2]` (equal values count as good, answer 3)
- All values equal (every node is good)
- Negative values, for example `[-1, -2, 0]` (answer 2)

Validate Binary Search Tree (Medium)

Restate: Return True if the tree is a valid BST: every node in a left subtree is strictly smaller than the node, and every node in a right subtree is strictly bigger.

Hint 1: DFS that passes allowed bounds (`low`, `high`) down to each node.

Hint 2: Going left, the current value becomes the new upper bound. Going right, it becomes the new lower bound. Each node must lie strictly between its bounds. (Another way: an in-order traversal must produce strictly increasing values.)

Hint 3:

- Helper `valid(node, low, high)` : if None, return True.
- If not `low < node.val < high`, return False.
- Return `valid(node.left, low, node.val)` and `valid(node.right, node.val, high)`.
- Start with `valid(root, -inf, inf)`.

Complexity: $O(n)$ time, $O(h)$ space.

Edge cases to test:

- `[2, 1, 3]` (True)
- `[5, 1, 4, null, null, 3, 6]` (False)
- `[5, 4, 6, null, null, 3, 7]` (False: 3 is in the right subtree of 5; only bounds catch this)
- `[2, 2, 2]` (duplicates, False because the rule is strict)
- `[2147483647]` (the max int value; do not use the max int as your starting bound)

Kth Smallest Element In a Bst (Medium)

Restate: Return the k-th smallest value (1-based) in a binary search tree.

Hint 1: In-order traversal of a BST visits values in sorted order.

Hint 2: Do an iterative in-order traversal with a stack and stop as soon as you have visited k nodes. You do not need to collect all values.

Hint 3:

- Set `stack = []`, `node = root`, `count = 0`.
- While `node` or `stack`: go left as far as possible, pushing every node.
- Pop a node; `count += 1`; if `count == k`, return its value.
- Move to `node = popped.right`.

Complexity: $O(h + k)$ time, $O(h)$ space.

Edge cases to test:

- One node, $k = 1$
- `[3, 1, 4, null, 2]`, $k = 1$ (answer 1)
- `[5, 3, 6, 2, 4, null, null, 1]`, $k = 3$ (answer 3)
- k equal to the number of nodes (the largest value)
- A right-skewed tree (in-order equals the path order)

Construct Binary Tree From Preorder And Inorder Traversal (Medium)

Restate: Given the pre-order and in-order traversal lists of a tree with unique values, rebuild the tree.

Hint 1: Recursion. Pre-order tells you the root; in-order tells you which values are on the left and which are on the right.

Hint 2: The first pre-order value is the root. Find it in the in-order list: everything left of it is the left subtree, everything right is the right subtree. Store `value -> index` of the in-order list in a dict so the lookup is $O(1)$, and use index ranges instead of slicing.

Hint 3:

- Build `pos = {value: index}` from `inorder`; keep a pointer `pre_i = 0` into `preorder`.
- Helper `build(lo, hi)` builds the tree for `inorder[lo..hi]`; if `lo > hi`, return `None`.
- Root value = `preorder[pre_i]`; `pre_i += 1`; `mid = pos[root value]`.
- Build the left child with `build(lo, mid - 1)` **first**, then the right with `build(mid + 1, hi)`.
- Return the root.

Complexity: $O(n)$ time with the dict and index ranges, $O(n)$ space. (Slicing lists makes it $O(n^2)$.)

Edge cases to test:

- One value `[-1]` and `[-1]`
- `preorder = [3, 9, 20, 15, 7]`, `inorder = [9, 3, 15, 20, 7]`
- A left-skewed tree: `preorder = [3, 2, 1]`, `inorder = [1, 2, 3]`

- A right-skewed tree: `preorder = [1, 2, 3]`, `inorder = [1, 2, 3]`
- Verify by computing the pre-order and in-order of your result and comparing

Binary Tree Maximum Path Sum (Hard)

Restate: A path is any chain of connected nodes (each node used at most once, not required to pass the root); return the largest possible sum of node values on a path.

Hint 1: DFS that returns a value up and updates a global best, like Diameter.

Hint 2: Each node returns to its parent the best "one-arm" sum going down: `node.val + max(left_gain, right_gain)`. Negative gains are dropped (use `max(gain, 0)`). The best path that turns at this node is `node.val + left_gain + right_gain`; compare that with the global best.

Hint 3:

- Keep `best = -inf`.
- Helper `gain(node)`: if None, return 0.
- `left = max(gain(node.left), 0)`, `right = max(gain(node.right), 0)`.
- Update `best = max(best, node.val + left + right)`.
- Return `node.val + max(left, right)`. The answer is `best`.

Complexity: $O(n)$ time, $O(h)$ space.

Edge cases to test:

- One node `[-3]` (answer -3, not 0; start `best` at minus infinity)
- `[1, 2, 3]` (answer 6)
- `[-10, 9, 20, null, null, 15, 7]` (answer 42)
- All negative `[-2, -1]` (answer -1)
- `[2, -1]` (answer 2; dropping the negative child)

Serialize And Deserialize Binary Tree (Hard)

Restate: Design two functions: one turns a binary tree into a string, the other turns that string back into the exact same tree.

Hint 1: Pre-order DFS with a marker for empty children (for example `"N"`), separated by commas.

Hint 2: Writing the null markers makes the string unambiguous: the deserializer reads values in the same pre-order, and a `"N"` tells it "this child is empty, go back up". It works with a single pointer into the list of tokens.

Hint 3:

- Serialize: DFS; for None, write `"N"`; else write the value, then serialize left, then right; join with commas.
- Deserialize: split the string by commas; keep an index `i`.

- Helper `build()`: read token `i`, `i += 1`; if it is "N", return None.
- Otherwise create the node, set `node.left = build()`, then `node.right = build()`, return it.

Complexity: $O(n)$ time and $O(n)$ space for both functions.

Edge cases to test:

- Empty tree (string like "N", and it must come back as None)
- One node
- Negative and multi-digit values like `[-100, 25]` (do not parse digit by digit)
- A skewed tree of 1,000 nodes (recursion depth)
- Round trip check: `deserialize(serialize(t))` has the same structure as `t`

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Insert into a Binary Search Tree	Medium	open	watch
2	Delete Node in a BST	Medium	open	watch
3	Minimum Time to Collect All Apples in a Tree	Medium	open	watch
4	Binary Tree Zigzag Level Order Traversal	Medium	open	watch
5	Construct Quad Tree	Medium	open	watch
6	Find Duplicate Subtrees	Medium	open	watch

Quiz

- Which traversal gives the values of a BST in sorted order?
 - A) Pre-order
 - B) In-order
 - C) Post-order
 - D) Level-order
- Which pattern fits "Binary Tree Right Side View"?
 - A) BFS level by level
 - B) Binary search
 - C) Two pointers
 - D) Sliding window
- Space used by DFS recursion on a tree of height h ?
 - A) $O(1)$
 - B) $O(h)$
 - C) $O(n^2)$
- Why is checking only `node.left.val < node.val < node.right.val` not enough to validate a BST?
 - A) It is too slow

- B) A deeper node can break the rule with an ancestor
 - C) It fails on empty trees
5. BST search time when the tree is completely skewed (like a linked list)?
- A) $O(\log n)$
 - B) $O(n)$
 - C) $O(1)$
6. Inorder traversal of a BST gives values:
- A) Random order
 - B) Sorted order
 - C) Reverse insertion order
7. Level order traversal uses:
- A) A stack
 - B) A queue
 - C) A heap
8. Recursion space for a balanced tree of n nodes:
- A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n^2)$

Answer key

1. **B** - In-order. In-order visits left, node, right, which matches the BST order.
2. **A** - BFS level by level. You take the last node of each level, which BFS gives directly.
3. **B** - $O(h)$. The call stack holds one frame per level of the current path.
4. **B** - A deeper node can break the rule with an ancestor. Every node in the left subtree must be smaller than the ancestor, so pass bounds down.
5. **B** - $O(n)$. Height becomes n , and search is $O(\text{height})$.
6. **B** - Sorted order.
7. **B** - A queue.
8. **B** - $O(\log n)$.

More quiz

1. In Binary Tree Maximum Path Sum, why do you use `max(gain, 0)` for each child?
 - A. To avoid recursion errors
 - B. A negative branch only lowers the sum, so the path should skip it
 - C. To keep values positive for the heap
 - D. The problem says all values are positive
2. In Construct Binary Tree from Preorder and Inorder, why must you build the left subtree before the right one?
 - A. Python requires it

- B. Pre-order lists the whole left subtree before the right subtree, so the pre-order pointer reaches left values first
- C. In-order requires it
- D. It does not matter

3. Which pattern fits "return the average value of the nodes on each level"?

- A. BFS level order, summing each level
- B. In-order DFS
- C. Binary search
- D. Two heaps

4. For Count Good Nodes, tree [2, 1, 3] (root 2, left 1, right 3). How many good nodes?

- A. 1
- B. 2
- C. 3
- D. 0

5. Why does Serialize and Deserialize write a marker for null children?

- A. To make the string longer
- B. Without markers, different trees can produce the same string, so the shape cannot be rebuilt
- C. Null markers make it faster
- D. JSON requires it

More quiz: answer key

1. **B** - Adding a negative gain can only make the path worse. Taking 0 means "do not extend into that child".
2. **B** - Pre-order is root, then the full left subtree, then the full right subtree. A single shared pointer moves through the list in that exact order, so you must recurse left first.
3. **A** - This is "Average of Levels in Binary Tree". BFS gives each level as a group, so you add up the values and divide by the level size.
4. **B** - The root 2 is good. Node 1 is smaller than 2, so it is not good. Node 3 is at least 2, so it is good. That makes 2.
5. **B** - For example, the pre-order [1, 2] without markers could mean 2 is the left child or the right child. Markers fix the exact shape.

Flashcards

- **Q:** Invert Binary Tree: action at each node? — **A:** Swap its left and right children, then invert both subtrees.

- **Q:** Diameter of Binary Tree: what does the helper return, and what does it update? — **A:** It returns height; it updates the best answer with `left_height + right_height`.
- **Q:** Balanced Binary Tree: how do you avoid $O(n^2)$? — **A:** Return height and -1 for "unbalanced" from one bottom-up DFS.
- **Q:** Subtree of Another Tree: which helper do you reuse? — **A:** Same Tree, called at every node of the main tree.
- **Q:** LCA in a BST: when is the current node the answer? — **A:** When p and q are not both smaller and not both bigger than it, meaning they split here or one equals it.
- **Q:** Level order BFS: how do you know where a level ends? — **A:** Record `len(queue)` at the start of the level and pop exactly that many nodes.
- **Q:** Validate BST: which test input catches the "only check children" bug? — **A:** `[5, 4, 6, null, null, 3, 7]`, where 3 is in the right subtree of 5.
- **Q:** Kth Smallest in a BST: which traversal, and when do you stop? — **A:** In-order traversal; stop when you have visited k nodes.
- **Q:** Max Path Sum: what do you return to the parent? — **A:** `node.val + max(left_gain, right_gain)`, a path with one arm only.
- **Q:** Max Path Sum test for initial value? — **A:** A single node `[-3]` must return -3, so `best` must start at minus infinity.

DSA topic: Tries

What it is

A trie (say "try") is a tree for storing words letter by letter. Each node has children (one per next letter) and a flag that says "a word ends here".

Real-life analogy: the search box on Google or your phone keyboard. You type "goo" and it already knows words that start with "goo". Words with the same start share the same path.

Insert and search take $O(L)$ time, where L is the word length. This does not depend on how many words are stored.

How to recognise it

- The problem mentions **prefix**: "starts with", autocomplete, search suggestions.
- Many words must be searched at once (Word Search II on a grid).
- Wildcard search, like "." matching any letter.
- You must design a dictionary class with insert / search / startsWith.
- Longest common prefix of many strings.

Pattern 1: Basic trie (insert, search, startsWith)

Use a dict for children so you can handle any character.

```
class TrieNode:
    def __init__(self):
        self.children = {}    # char -> TrieNode
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True
```

Pattern 2: Walk the trie (search vs prefix)

Both follow the letters. The only difference: `search` needs `is_end` to be True at the end; `starts_with` does not.

```
def _walk(self, s):
    node = self.root
    for ch in s:
        if ch not in node.children:
            return None
        node = node.children[ch]
    return node

def search(self, word):
    node = self._walk(word)
    return node is not None and node.is_end

def starts_with(self, prefix):
    return self._walk(prefix) is not None
```

Pattern 3: Trie + DFS (wildcard or grid)

When a letter can be "any letter" or you walk a grid, use DFS that moves through the trie at the same time.

```
def search_with_dot(node, word, i):
    if i == len(word):
        return node.is_end
    ch = word[i]
    if ch == ".":
        # try every child
        return any(search_with_dot(child, word, i + 1)
                   for child in node.children.values())
    if ch not in node.children:
        return False
    return search_with_dot(node.children[ch], word, i + 1)
```

Worked example: Implement Trie (Prefix Tree)

Build a class with `insert(word)`, `search(word)`, and `startsWith(prefix)`.

Operations: `insert("app")`, `insert("apple")`, `search("ap")`, `startsWith("ap")`, `search("app")`.

1. `insert("app")`: create nodes `a -> p -> p`. Mark the last `p` with `is_end = True`.
2. `insert("apple")`: `a`, `p`, `p` already exist, reuse them. Create `l -> e`. Mark `e` with `is_end = True`.
3. `search("ap")`: walk `a -> p`. Node exists, but `is_end` is False. Return False.
4. `startsWith("ap")`: walk `a -> p`. Node exists. Return True.
5. `search("app")`: walk `a -> p -> p`. `is_end` is True (from step 1). Return True.

```

class Trie:
    def __init__(self):
        self.root = {} # nested dicts as nodes

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.setdefault(ch, {})
        node["#"] = True # end-of-word marker

    def search(self, word):
        node = self._walk(word)
        return node is not None and "#" in node

    def startsWith(self, prefix):
        return self._walk(prefix) is not None

    def _walk(self, s):
        node = self.root
        for ch in s:
            if ch not in node:
                return None
            node = node[ch]
        return node

```

Complexity cheat sheet

- insert / search / startsWith -> $O(L)$ time, L = length of the word
- Space for all words -> $O(\text{total characters})$ in the worst case
- Wildcard search with "." -> up to $O(26^d * L)$ in the worst case (d = number of dots)
- Word Search II -> roughly $O(\text{rows} * \text{cols} * 4^L)$, much faster in practice thanks to pruning

Common mistakes

- Forgetting the `is_end` flag. Then "app" looks like a word only because "apple" was inserted.
- Returning True in `search` just because the path exists. That is `startsWith`, not `search`.
- Using a fixed array of 26 when input may have uppercase letters or other characters.
- In Word Search II, not removing found words, so the same word is added many times.
- Not pruning: keep trie branches that can never match, which makes the DFS slow.

What to say in the interview

- "The problem is about prefixes, so a trie is a good fit."
- "Each node has a dictionary of children and a flag that marks the end of a word."
- "Insert and search are $O(L)$, where L is the word length, no matter how many words we store."
- "For the grid, I will put all words in a trie and run DFS, so I search all words in one pass."

Practice order

- Implement Trie (Prefix Tree) first: learn the node structure.
- Design Add and Search Words Data Structure: add DFS for the "." wildcard.
- Word Search II last: trie plus grid backtracking, with pruning. A common hard question.

Cheat sheet

Idea

A tree of characters. Each path from root spells a prefix. Insert/search a word in $O(\text{length})$.

Use it when

- Prefix search, autocomplete
- Many words searched in a grid (Word Search II)

Template

```
class Node:
    def __init__(self):
        self.kids = {}
        self.end = False

def insert(root, word):
    cur = root
    for ch in word:
        cur = cur.kids.setdefault(ch, Node())
    cur.end = True
```

Complexity

Insert/search $O(L)$. Space $O(\text{total characters})$.

Visualise it

- <https://visualgo.net/en/suffixtree>

Videos

- [Tries in 5 minutes](#) - HackerRank (English)
- [Implement a Trie](#) - take U forward (Hindi + English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Implement Trie Prefix Tree (Blind 75)	Medium	open	watch
2	Design Add And Search Words Data Structure (Blind 75)	Medium	open	watch
3	Word Search II (Blind 75)	Hard	open	watch

Problem hints

Try each problem on your own first. Open one hint at a time. Only go to the next hint when you are stuck for 10 or more minutes.

Implement Trie Prefix Tree (Medium)

Restate: Build a Trie class with `insert(word)`, `search(word)` (is this exact word stored?) and `startsWith(prefix)` (is any stored word starting with this prefix?).

Hint 1: Each node holds a map of children (one per next letter) and a flag saying "a word ends here".

Hint 2: All three operations walk the same path from the root, one letter per step. The only difference: `search` needs the end flag on the last node, while `startsWith` only needs the path to exist. Write one shared "walk" helper.

Hint 3:

- Node: `children = {}`, `is_end = False`.
- `insert`: walk letter by letter, creating missing children; set `is_end = True` on the last node.
- `walk(s)`: follow letters; return `None` as soon as a letter is missing, else the last node.
- `search`: `node = walk(word)`; return `node is not None and node.is_end`.
- `startsWith`: return `walk(prefix) is not None`.

Complexity: $O(L)$ time per operation, L = length of the word; $O(\text{total characters inserted})$ space.

Edge cases to test:

- `search("app")` after only `insert("apple")` (False) and `startsWith("app")` (True)
- `insert("app")` after `insert("apple")`, then `search("app")` (True)
- Insert the same word twice, then search it (True, no crash)
- `search` on an empty trie (False)
- A prefix longer than any stored word, like `startsWith("applesauce")` (False)

Design Add And Search Words Data Structure (Medium)

Restate: Build a class with `addWord(word)` and `search(word)`, where the search pattern may contain `.` that matches any single letter.

Hint 1: A trie, plus DFS for the dots.

Hint 2: Normal letters follow one child, exactly like a trie. At a `.`, you do not know which letter it is, so try every child and return True if any branch succeeds. Pass the current index and current node into the DFS.

Hint 3:

- `addWord`: same as trie insert.
- `dfs(i, node)`: if `i == len(word)`, return `node.is_end`.
- If `word[i] == "."`: for each child, if `dfs(i + 1, child)` is True, return True; then return False.
- Else: if `word[i]` is not a child, return False; otherwise return `dfs(i + 1, node.children[word[i]])`.
- `search` returns `dfs(0, root)`.

Complexity: `addWord` $O(L)$. `search` $O(L)$ with no dots; worst case $O(26^d * L)$ where d is the number of dots (bounded by the number of stored nodes). Space $O(\text{total characters})$.

Edge cases to test:

- `addWord("bad")`, then `search(".ad")` (True) and `search("b..")` (True)
- `search("...")` when only "bad" is stored (True) and `search("....")` (False, length matters)
- `search("ba")` after adding "bad" (False, no end flag)
- `search(".")` on an empty structure (False)
- A pattern that is all dots over many stored words (performance check)

Word Search II (Hard)

Restate: Given a grid of letters and a list of words, return every word that can be traced in the grid by moving up, down, left or right, without using the same cell twice in one word.

Hint 1: Put all the words into a trie. Then run a DFS (backtracking) from every cell, walking the trie as you walk the grid.

Hint 2: Searching each word separately repeats a lot of work. With a trie, one DFS from a cell checks all words with that prefix at once, and stops immediately when the current path is not a prefix of any word. Store the full word at its end node so you can add it to the result directly, then clear it so it is not added twice.

Hint 3:

- Insert every word into a trie; at each end node store `node.word = word`.
- For each cell, call `dfs(r, c, root)`.
- In `dfs`: stop if out of bounds, already visited, or the letter is not a child of the current node.
- Move to the child; if `child.word` is set, add it to the result and set `child.word = None`.
- Mark the cell visited, DFS into the 4 neighbours, then unmark it (backtrack). Optional speed-up: remove trie nodes that have no children left.

Complexity: About $O(\text{rows} * \text{cols} * 4 * 3^{(L-1)})$ in the worst case, where L is the longest word length; much faster in practice because of pruning. Space $O(\text{total characters of the words})$ for the trie.

Edge cases to test:

- A 1x1 grid `[["a"]]` with words `["a"]` (answer `["a"]`)
- Grid `[["a", "a"]]` with words `["aaa"]` (answer `[]`; a cell cannot be used twice)
- The same word appearing in two places in the grid (must be returned once)
- Words that share a prefix, like `["oath", "oat"]` (both found)
- Duplicate words in the input list (return each word once)

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Extra Characters in a String	Medium	open	watch

Quiz

1. What is the time to search a word of length L in a trie?
 - A) $O(1)$
 - B) $O(L)$
 - C) $O(N)$ where N is number of words
 - D) $O(N * L)$
2. What is the difference between search and startsWith in a trie?
 - A) search needs the end-of-word flag at the last node
 - B) startsWith is slower
 - C) There is no difference
3. Which data structure fits an autocomplete feature best?
 - A) Heap
 - B) Trie
 - C) Stack
 - D) Queue
4. Which pattern fits "search words where . matches any letter"?
 - A) Trie + DFS trying every child at a dot
 - B) Binary search
 - C) Sliding window
5. You insert "apple" only. Without an is_end flag, which bug happens?
 - A) search("apple") returns False
 - B) search("app") returns True
 - C) insert crashes
6. Searching a word of length L in a trie costs:
 - A) $O(L)$
 - B) $O(n)$
 - C) $O(\log n)$
7. The end flag marks:
 - A) Leaf nodes

- B) That a full word ends here
 - C) The root
8. Tries are best for:
- A) Prefix queries
 - B) Sorting numbers
 - C) Shortest paths

Answer key

1. **B** - $O(L)$. You follow one node per letter.
2. **A** - search needs the end-of-word flag at the last node. A prefix only needs the path to exist; a full word also needs `is_end = True`.
3. **B** - Trie. A trie groups words by shared prefix, so suggestions are under one node.
4. **A** - Trie + DFS trying every child at a dot. At a dot you must branch into all children, which is a DFS.
5. **B** - `search("app")` returns `True`. The path a-p-p exists, so without the flag you cannot tell "app" was never inserted.
6. **A** - $O(L)$.
7. **B** - That a full word ends here.
8. **A** - Prefix queries.

More quiz

1. In Word Search II, why do you set `node.word = None` after finding a word?
 - A. To save memory
 - B. So the same word is not added to the result again when it is found by another path
 - C. Because the trie is read-only
 - D. To stop the DFS completely
2. In Add and Search Words, what does `search("b..")` need at the very end?
 - A. Only that the path exists
 - B. That the last node has `is_end = True`
 - C. That the node has no children
 - D. Nothing
3. Which pattern fits "replace each word in a sentence with its shortest root word from a dictionary"?
 - A. Trie: walk each word and stop at the first node with the end flag
 - B. Sliding window
 - C. Heap
 - D. Monotonic stack
4. You insert "car", "cart" and "care" into an empty trie. How many nodes are there, not counting the root?

- A. 5
- B. 6
- C. 11
- D. 4

5. Why is a trie better than a hash set for Word Search II?

- A. A hash set cannot store strings
- B. A trie can tell you during the DFS that no word starts with the current path, so you can stop early
- C. A trie uses less memory in all cases
- D. A hash set lookup is $O(n)$

More quiz: answer key

1. **B** - A word can be traced in more than one place in the grid. Clearing the stored word after the first find makes the result unique without an extra set.
2. **B** - `search` is a full-word match, so the last node must be the end of a word. If only "badge" is stored, the path b-a-d exists, but "b.." must return False.
3. **A** - This is "Replace Words". For each word, walk the trie and return the first prefix that is marked as a word end.
4. **A** - The letters c, a, r are shared by all three words (3 nodes). "cart" adds one node for t, and "care" adds one node for e. That is $3 + 1 + 1 = 5$.
5. **B** - With a set, you could only check full words or store every prefix separately. The trie gives prefix checks for free as you walk, which prunes dead paths.

Flashcards

- **Q:** What two things does a trie node store? — **A:** A map of children (letter to node) and an end-of-word flag.
- **Q:** Difference between `search` and `startsWith`? — **A:** `search` needs the end flag on the last node; `startsWith` only needs the path to exist.
- **Q:** Time to insert a word of length L ? — **A:** $O(L)$, no matter how many words are stored.
- **Q:** Add and Search Words: what happens at a `.` character? — **A:** Try every child with DFS and return True if any branch matches.
- **Q:** Why does `search("....")` fail when only "bad" is stored? — **A:** The pattern has 4 characters, but the path ends after 3 letters.
- **Q:** Word Search II: why not run Word Search I for each word? — **A:** It repeats the grid DFS for every word; a trie shares one DFS across all words with the same prefix.
- **Q:** Word Search II: how is a cell marked as used? — **A:** Temporarily change it (for example to `"#"`) or add it to a visited set, then restore it after the DFS returns.
- **Q:** Word Search II: what test catches reuse of a cell? — **A:** Grid `[["a", "a"]]` with word "aaa" must return nothing.

- **Q:** Optional pruning in Word Search II? — **A:** Remove a trie node once it has no children and no word, so later DFS calls skip it.

DSA topic: Heap / Priority Queue

What it is

A heap is a tree stored in an array that always gives you the smallest item (min-heap) in $O(1)$, and lets you add or remove items in $O(\log n)$.

Real-life analogy: a hospital emergency room. The patient with the most serious condition is treated first, not the one who came first.

Python has `heapq`, which is a **min-heap** on a normal list. For a max-heap, push negative numbers.

```
import heapq
h = []
heapq.heappush(h, 5)      # add,  $O(\log n)$ 
smallest = h[0]           # peek,  $O(1)$ 
heapq.heappop(h)          # remove smallest,  $O(\log n)$ 
heapq.heapify(nums)       # build heap in place,  $O(n)$ 
```

How to recognise it

- "K largest", "K smallest", "K most frequent", "K closest".
- You keep needing the current minimum or maximum while data changes.
- Merge K sorted lists or streams.
- Running median of a data stream.
- Scheduling: pick the next task with the highest priority or earliest time.

Pattern 1: Top K with a heap of size k

For the k largest items, keep a **min-heap** of size k. The smallest of the top k sits at the top, so it is easy to throw out.

```
import heapq

def k_largest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)    # remove the smallest
    return h                  # h[0] is the k-th largest
```

Pattern 2: Max-heap with negatives

Python only has a min-heap. Push `-x` to get the biggest first. Use tuples to attach data: `(-priority, item)`.

```
import heapq

def last_stone_weight(stones):
    h = [-s for s in stones]
    heapq.heapify(h)
    while len(h) > 1:
        a = -heapq.heappop(h) # biggest
        b = -heapq.heappop(h) # second biggest
        if a != b:
            heapq.heappush(h, -(a - b))
    return -h[0] if h else 0
```

Pattern 3: Two heaps (median)

Keep the smaller half in a max-heap and the bigger half in a min-heap. Balance their sizes. The median is at the tops.

```
import heapq

class MedianFinder:
    def __init__(self):
        self.small = [] # max-heap (store negatives)
        self.large = [] # min-heap

    def addNum(self, num):
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self):
        if len(self.small) > len(self.large):
            return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2
```

Worked example: Kth Largest Element in an Array

Given an array and k, return the k-th largest element (no full sort needed).

Input: nums = [3, 2, 1, 5, 6, 4], k = 2. Keep a min-heap with at most 2 items.

1. Push 3. Heap = [3].
2. Push 2. Heap = [2, 3].
3. Push 1. Heap = [1, 2, 3]. Size 3 > 2, pop 1. Heap = [2, 3].
4. Push 5. Heap = [2, 3, 5]. Pop 2. Heap = [3, 5].
5. Push 6. Heap = [3, 5, 6]. Pop 3. Heap = [5, 6].
6. Push 4. Heap = [4, 5, 6] (4 at top). Pop 4. Heap = [5, 6].
7. Top is 5. The 2nd largest is 5.

```
import heapq

def find_kth_largest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)
    return h[0]
```

Complexity cheat sheet

- peek min -> $O(1)$
- push / pop -> $O(\log n)$
- heapify a list -> $O(n)$
- Top K with a size-k heap -> $O(n \log k)$ time, $O(k)$ space
- Merge K sorted lists -> $O(N \log k)$, N = total nodes
- Find Median from Data Stream -> $O(\log n)$ add, $O(1)$ find

Common mistakes

- Forgetting that `heapq` is a min-heap. Use negatives for max-heap.
- Using a max-heap of all n items for top k . A min-heap of size k is faster: $O(n \log k)$.
- Pushing tuples where the first values tie and the second values cannot be compared (like `ListNode`). Add an index as a tie-breaker: `(val, i, node)`.
- Thinking `h` is sorted. Only `h[0]` is guaranteed to be the smallest.
- Forgetting to flip the sign back when you pop from a negative max-heap.

What to say in the interview

- "I only need the top k items, so I will keep a min-heap of size k instead of sorting everything."
- "When the heap grows bigger than k , I pop the smallest. At the end the top of the heap is the answer."
- "This is $O(n \log k)$ time and $O(k)$ space, which is better than $O(n \log n)$ when k is small."
- "Python `heapq` is a min-heap, so for a max-heap I will push negative values."

Practice order

- Kth Largest Element in a Stream and Last Stone Weight first: basic heap operations.
- K Closest Points to Origin and Kth Largest Element in an Array: the size- k heap pattern.
- Task Scheduler: max-heap plus a cooldown queue.
- Design Twitter: merge K lists with a heap.
- Find Median from Data Stream last: the two-heaps idea, a very common hard question.

Cheat sheet

Idea

Get min (or max) in $O(1)$, push/pop in $O(\log n)$. Python `heapq` is a min-heap; push negatives for max-heap.

Use it when

- Top K / Kth largest -> min-heap of size k
- Merge K sorted lists
- Scheduling, running median (two heaps)

Template

```
import heapq
h = []
for x in nums:
    heapq.heappush(h, x)
    if len(h) > k:
        heapq.heappop(h)
return h[0] # kth Largest
```

Complexity

Top-K with heap: $O(n \log k)$.

Visualise it

- <https://visualgo.net/en/heap>

Videos

- [Heaps visually explained](#) - ByteQuest (English)
- [Introduction to Heap](#) - Gate Smashers (Hindi)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Kth Largest Element In a Stream	Easy	open	watch
2	Last Stone Weight	Easy	open	watch
3	K Closest Points to Origin	Medium	open	watch

#	Problem	Level	LeetCode	Solution video
4	Kth Largest Element In An Array	Medium	open	watch
5	Task Scheduler	Medium	open	watch
6	Design Twitter	Medium	open	watch
7	Find Median From Data Stream (Blind 75)	Hard	open	watch

Problem hints

Try each problem on your own first. Open one hint at a time. Only go to the next hint when you are stuck for 10 or more minutes.

Kth Largest Element In a Stream (Easy)

Restate: Design a class that receives numbers one at a time and, after each `add`, returns the k-th largest number seen so far.

Hint 1: Keep a min-heap that holds only the k largest numbers.

Hint 2: The top of that min-heap (the smallest of the top k) is exactly the k-th largest. When the heap grows past k, pop the top, because that number can never be the k-th largest again.

Hint 3:

- In the constructor, push all starting numbers, then pop until the size is k.
- In `add(val)`: push `val`.
- If the size is more than k, pop once.
- Return `heap[0]`.

Complexity: $O(\log k)$ per `add`, $O(n \log k)$ to build (or $O(n)$ with `heapify` plus pops); $O(k)$ space.

Edge cases to test:

- Starting list empty with $k = 1$, then `add(-3)` (answer -3)
- Starting list shorter than k (the problem promises the k-th exists after the add)
- $k = 3$, `nums = [4, 5, 8, 2]`, then `add(3)` gives 4, `add(5)` gives 5, `add(10)` gives 5
- Adding many equal values
- Adding a value smaller than the current k-th largest (answer does not change)

Last Stone Weight (Easy)

Restate: Repeatedly smash the two heaviest stones; if they differ, the smaller weight is subtracted from the larger and that stone stays; return the last stone's weight or 0.

Hint 1: You keep needing the two biggest values, so use a max-heap.

Hint 2: Python's `heapq` is a min-heap. Store negative weights to turn it into a max-heap, and flip the sign when you pop.

Hint 3:

- Turn every weight `w` into `-w` and `heapify`.
- While more than one stone: pop the biggest `y` and the next biggest `x`.
- If `y != x`, push `-(y - x)`.
- Return the remaining weight, or 0 if the heap is empty.

Complexity: $O(n \log n)$ time, $O(n)$ space.

Edge cases to test:

- `[1]` (answer 1)
- `[2, 2]` (both destroyed, answer 0)
- `[2, 7, 4, 1, 8, 1]` (answer 1)
- `[10, 4, 2, 10]` (answer 2)
- All stones equal and an even count (answer 0)

K Closest Points to Origin (Medium)

Restate: Given points on a 2D plane, return the `k` points closest to `(0, 0)`, in any order.

Hint 1: "K smallest by distance" means a heap. Compare squared distances; you never need the square root.

Hint 2: Keep a max-heap of size `k` (push `(-distance, x, y)`). When it grows past `k`, pop the farthest. At the end it holds the `k` closest. (Heapifying all points and popping `k` times is also fine.)

Hint 3:

- For each point, compute `d = x * x + y * y`.
- Push `(-d, x, y)` into the heap.
- If the size is more than `k`, pop (removes the farthest so far).
- Return the points left in the heap.

Complexity: $O(n \log k)$ time, $O(k)$ space. (Quickselect gives $O(n)$ average.)

Edge cases to test:

- `points = [[1, 3], [-2, 2]]`, `k = 1` (answer `[[-2, 2]]`)
- `k` equal to the number of points (return all)
- Points with equal distance like `[1, 0]` and `[0, 1]` (either is valid; ask the interviewer)
- The point `[0, 0]` itself (distance 0)
- Large coordinates like `[10000, -10000]` (squared value is $2 * 10^8$, fine in Python)

Kth Largest Element In An Array (Medium)

Restate: Return the k-th largest number in an unsorted array (counting duplicates, so it is the k-th position in sorted descending order).

Hint 1: A min-heap of size k, as in the stream problem. Ask if sorting is allowed: sorting is $O(n \log n)$.

Hint 2: The interview follow-up is quickselect: partition the array around a pivot like quicksort, but recurse only into the side that contains the target index $n - k$. Average $O(n)$, worst $O(n^2)$; a random pivot makes the worst case very unlikely.

Hint 3:

- Heap way: push each number; if the size is more than k, pop; return `heap[0]`.
- Quickselect way: target index $t = n - k$ in ascending order.
- Partition around a pivot so smaller values are on the left.
- If the pivot lands at t , return it; if it lands left of t , search the right part; else search the left part.

Complexity: Heap: $O(n \log k)$ time, $O(k)$ space. Quickselect: $O(n)$ average time, $O(1)$ extra space for the in-place version.

Edge cases to test:

- `[1]`, $k = 1$ (answer 1)
- `[3, 2, 1, 5, 6, 4]`, $k = 2$ (answer 5)
- `[3, 2, 3, 1, 2, 4, 5, 5, 6]`, $k = 4$ (duplicates count, answer 4)
- All values equal, like `[7, 7, 7]`, $k = 2$ (answer 7; a naive quickselect can become very slow here)
- Already sorted input (worst case for a quickselect that always picks the last element as pivot)

Task Scheduler (Medium)

Restate: Given tasks (letters) and a cooldown n , where the same task must wait at least n time units before running again, return the minimum time units needed (idle slots allowed).

Hint 1: Greedy: always run the task with the most remaining copies. That needs a max-heap of counts, plus a queue for tasks that are cooling down.

Hint 2: Each time unit, pop the biggest count, run it once, and if copies remain, put it into a queue with the time when it becomes available ($time + n$). When the front of the queue is ready, push it back into the heap. There is also a math formula: $\max(\text{len}(\text{tasks}), (\text{max_freq} - 1) * (n + 1) + \text{count_of_tasks_with_max_freq})$.

Hint 3:

- Count tasks; push `-count` for each into a heap. `time = 0`, `queue = deque()`.
- While the heap or the queue is not empty: `time += 1`.
- If the heap is not empty: pop, add 1 to the negative count (one copy done); if still non-zero, append `(count, time + n)` to the queue.
- If the queue front's ready time equals `time`, pop it from the queue and push its count back into the heap.

- Return `time`.

Complexity: $O(\text{total tasks} * \log 26) = O(\text{total tasks})$ time, $O(26) = O(1)$ space. The formula is $O(\text{total tasks})$.

Edge cases to test:

- `tasks = ["A", "A", "A", "B", "B", "B"]`, `n = 2` (answer 8)
- Same tasks with `n = 0` (no cooldown, answer 6)
- `tasks = ["A", "A", "A", "B", "B", "B"]`, `n = 50` (answer 104, mostly idle)
- `tasks = ["A", "B", "C", "D"]`, `n = 3` (no idle needed, answer 4)
- `tasks = ["A", "A", "A", "B", "C", "D", "E", "F"]`, `n = 2` (many different tasks fill the gaps, answer 8)

Design Twitter (Medium)

Restate: Design a simple Twitter with `postTweet(userId, tweetId)`, `follow`, `unfollow`, and `getNewsFeed(userId)` that returns the 10 most recent tweet ids from the user and the people they follow.

Hint 1: Store each user's tweets as a list of `(time, tweetId)` and each user's followees as a set. For the feed, merge several lists with a heap.

Hint 2: Use a global counter as the timestamp. Every user's list is already in time order, so the newest tweet is at the end. Push the newest tweet of each followee (and the user) into a max-heap, then pop up to 10 times; after each pop, push the next older tweet from the same user. This is the "merge k sorted lists" idea.

Hint 3:

- `postTweet`: append `(time, tweetId)` to `tweets[userId]`; increase `time`.
- `follow` / `unfollow`: add to or discard from `following[followerId]` (ignore a user following themselves, or always include the user in the feed).
- `getNewsFeed`: for the user and each followee with tweets, push `(-time, tweetId, user, index)` of their last tweet.
- Pop up to 10 times; after each pop, if that user has an older tweet (`index - 1 >= 0`), push it.
- Return the popped tweet ids in order.

Complexity: `postTweet`, `follow`, `unfollow` $O(1)$. `getNewsFeed` $O(f + 10 \log f)$, where f is the number of followees. Space $O(\text{users} + \text{tweets} + \text{follow edges})$.

Edge cases to test:

- Feed for a user with no tweets and no followees (answer `[]`)
- A user with more than 10 tweets (only the 10 newest, newest first)
- Unfollow, then check that the old followee's tweets are gone from the feed
- Unfollow someone you never followed (no crash)
- A user follows themselves (their tweets must not appear twice)

Find Median From Data Stream (Hard)

Restate: Design a class with `addNum(num)` and `findMedian()`, where the median is the middle value of all numbers added so far (or the average of the two middle values).

Hint 1: Two heaps: a max-heap for the smaller half and a min-heap for the larger half.

Hint 2: Keep two rules after every add: every number in the small half is less than or equal to every number in the large half, and the small half has the same size or one more. Then the median is always at the tops of the heaps.

Hint 3:

- `addNum`: push into the small max-heap (as a negative number).
- Move the biggest of the small half into the large min-heap (this fixes the order rule).
- If the large half is now bigger than the small half, move its smallest back.
- `findMedian`: if the small half is bigger, return its top; else return the average of both tops.

Complexity: `addNum` $O(\log n)$, `findMedian` $O(1)$; $O(n)$ space.

Edge cases to test:

- Add 1, then `findMedian` (answer 1.0)
- Add 1 and 2, then `findMedian` (answer 1.5)
- Add 1, 2, 3 (answer 2.0)
- Numbers added in decreasing order like 5, 4, 3, 2, 1 (answer 3.0)
- Negative numbers and duplicates, like -1, -1, -2 (answer -1.0)

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Maximum Subsequence Score	Medium	open	watch
2	Single Threaded Cpu	Medium	open	watch
3	Seat Reservation Manager	Medium	open	watch
4	Process Tasks Using Servers	Medium	open	watch
5	Find The Kth Largest Integer In The Array	Medium	open	watch
6	Reorganize String	Medium	open	watch

Quiz

1. What kind of heap does Python `heapq` give you?
 - A) Max-heap

- B) Min-heap
 - C) Sorted list
2. To find the k largest items efficiently, what do you keep?
- A) A max-heap of all n items
 - B) A min-heap of size k
 - C) A sorted array of size n
3. What is the time to build a heap from a list with heapify?
- A) $O(n)$
 - B) $O(n \log n)$
 - C) $O(\log n)$
4. Which pattern fits "find the median from a data stream"?
- A) Two heaps
 - B) Sliding window
 - C) Trie
 - D) Binary search on answer
5. You push (value, node) tuples and two values are equal. What can go wrong?
- A) Nothing
 - B) Python tries to compare the nodes and may raise `TypeError`
 - C) The heap becomes a max-heap
6. Python `heapq` is a:
- A) Max-heap
 - B) Min-heap
 - C) Sorted list
7. Kth largest element with a heap of size k costs:
- A) $O(n \log k)$
 - B) $O(n^2)$
 - C) $O(k)$
8. Find Median from Data Stream uses:
- A) One heap
 - B) Two heaps
 - C) A trie

Answer key

1. **B** - Min-heap. `heapq` always keeps the smallest item at index 0.
2. **B** - A min-heap of size k. The min-heap top is the smallest of the top k, so you can drop it quickly; total $O(n \log k)$.
3. **A** - $O(n)$. Bottom-up heapify runs in linear time.
4. **A** - Two heaps. A max-heap for the low half and a min-heap for the high half keep the middle at the tops.
5. **B** - Python tries to compare the nodes and may raise `TypeError`. On a tie, Python compares the next element; add an index as a tie-breaker.
6. **B** - Min-heap.
7. **A** - $O(n \log k)$.

8. **B** - Two heaps.

More quiz

1. In Task Scheduler, tasks ["A", "A", "B"] with $n = 2$. What is the minimum time?
 - A. 3
 - B. 4
 - C. 5
 - D. 6
2. In K Closest Points to Origin, why can you skip the square root?
 - A. Python has no square root
 - B. Squaring keeps the order of non-negative distances the same, so comparisons give the same result
 - C. The square root is always an integer
 - D. It changes the answer, but only slightly
3. Which pattern fits "merge the newest tweets from many users' timelines"?
 - A. Sliding window
 - B. Heap that holds the current newest item from each list (merge k sorted lists)
 - C. Trie
 - D. Binary search on the answer
4. What is the average time of quickselect for Kth Largest Element in an Array?
 - A. $O(\log n)$
 - B. $O(n)$
 - C. $O(n \log n)$
 - D. $O(n^2)$
5. In Find Median from Data Stream, after adding 5, 15 and 1, what are the heap tops (small max-heap, large min-heap)?
 - A. small top 5, large top 15
 - B. small top 1, large top 5
 - C. small top 15, large top 1
 - D. small top 5, large top 1

More quiz: answer key

1. **B** - A has 2 copies and must wait 2 units between them: A, B, idle, A. That is 4 units. The formula gives $(2 - 1) * (2 + 1) + 1 = 4$.

2. **B** - For non-negative numbers, $a < b$ exactly when $a^2 < b^2$. Skipping the root also avoids floating-point rounding problems.
3. **B** - Each user's tweets are already in time order. A heap of the newest unread tweet from each list gives the next newest tweet overall in $O(\log k)$.
4. **B** - Each partition step on average throws away about half of the remaining part, so the total work is about $n + n/2 + n/4 + \dots = O(n)$. The worst case is $O(n^2)$.
5. **A** - The small half holds $\{1, 5\}$ with top 5, and the large half holds $\{15\}$ with top 15. The small half has one more item, so the median is 5.

Flashcards

- **Q:** Kth Largest in a Stream: which heap and what size? — **A:** A min-heap holding at most k numbers; its top is the answer.
- **Q:** Last Stone Weight: how do you get a max-heap from `heapq`? — **A:** Push negative weights and flip the sign when you pop.
- **Q:** K Closest Points: what distance value do you compare? — **A:** The squared distance $x * x + y * y$.
- **Q:** K Closest Points: heap type for $O(n \log k)$? — **A:** A max-heap of size k by distance, so the farthest point is popped when the size goes over k .
- **Q:** Kth Largest in an Array: target index for quickselect in ascending order? — **A:** $n - k$.
- **Q:** Task Scheduler formula? — **A:** $\max(\text{len}(\text{tasks}), (\text{max_freq} - 1) * (n + 1) + \text{count_with_max_freq})$.
- **Q:** Task Scheduler: why put cooling tasks in a queue? — **A:** They cannot run until $\text{time} + n$, so they wait outside the heap until they are ready.
- **Q:** Design Twitter: why use a global counter as the timestamp? — **A:** It gives a strict order between all tweets from all users, so the heap can compare them.
- **Q:** Find Median: which half may hold one extra number? — **A:** The small half (max-heap), so with an odd count the median is its top.
- **Q:** Find Median: time of `addNum` and `findMedian`? — **A:** $O(\log n)$ for `addNum` and $O(1)$ for `findMedian`.

DSA topic: Backtracking

What it is

Backtracking means: try a choice, go deeper, and if it does not work, undo the choice and try the next one. Think of a maze. You walk down a path. If you hit a wall, you walk back to the last turn and try another path.

In code, backtracking is recursion plus an "undo" step. It explores a tree of choices.

It is used when the problem asks for ALL answers (all subsets, all permutations, all valid boards).

How to recognise it

- The problem says "return all", "list all", "generate all" combinations, subsets or permutations.
- Input size is small ($n \leq 15$ or so), because the answer count is exponential.
- You build an answer step by step, one choice at a time.
- There is a rule that can stop a path early (sum too big, queen attacked, word does not match).
- Grid word search, N-Queens, Sudoku, palindrome partitioning.

Pattern 1: Subsets (include / exclude)

Use when each element can be taken or skipped. Every node of the tree is an answer.

```
def subsets(nums):
    res, path = [], []
    def bt(start):
        res.append(path[:])          # every path is a valid subset
        for i in range(start, len(nums)):
            path.append(nums[i])      # choose
            bt(i + 1)                 # explore
            path.pop()               # undo
    bt(0)
    return res
```

For duplicates (Subsets II): sort first, and skip `nums[i] == nums[i-1]` when `i > start`.

Pattern 2: Permutations (used set)

Use when order matters and every element must appear once.


```
def permute(nums):
    res, path, used = [], [], [False] * len(nums)
    def bt():
        if len(path) == len(nums):
            res.append(path[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True; path.append(nums[i])
            bt()
            used[i] = False; path.pop()
    bt()
    return res
```

Pattern 3: Grid search with visited marking

Use for Word Search. Mark the cell, explore 4 neighbours, then unmark.

```
def exist(board, word):
    R, C = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word):
            return True
        if r < 0 or c < 0 or r >= R or c >= C or board[r][c] != word[i]:
            return False
        tmp, board[r][c] = board[r][c], "#" # mark visited
        found = (dfs(r+1, c, i+1) or dfs(r-1, c, i+1) or
                dfs(r, c+1, i+1) or dfs(r, c-1, i+1))
        board[r][c] = tmp # undo
        return found
    return any(dfs(r, c, 0) for r in range(R) for c in range(C))
```

Worked example: Combination Sum

Given distinct candidates and a target, return all combinations that sum to target. A number can be reused.

Input: candidates = [2, 3, 6, 7], target = 7.

1. Start bt(start=0, path=[], total=0).
2. Pick 2: path=[2], total=2. Pick 2 again: path=[2,2], total=4.
3. Pick 2 again: path=[2,2,2], total=6. Pick 2: total=8 > 7, stop. Pop.
4. At [2,2,2] try 3: total=9 > 7, stop. Try 6, 7: too big. Pop back to [2,2].
5. At [2,2] try 3: path=[2,2,3], total=7. Save [2,2,3]. Pop.
6. Continue: [2,3] gives 5, then 5+3=8 is too big. Paths starting [3], [3,3] and [6] never hit exactly 7.
7. Pick 7: path=[7], total=7. Save [7].
8. Answer: [[2,2,3], [7]].

```
def combinationSum(candidates, target):
    res, path = [], []
    def bt(start, total):
        if total == target:
            res.append(path[:])
            return
        if total > target:
            return
        for i in range(start, len(candidates)):
            path.append(candidates[i])
            bt(i, total + candidates[i]) # i, not i+1: reuse allowed
            path.pop()
    bt(0, 0)
    return res
```

Complexity cheat sheet

- Subsets: $O(n * 2^n)$ time, $O(n)$ extra space for the recursion.
- Permutations: $O(n * n!)$ time.
- Combination Sum: about $O(2^t)$ where $t = \text{target} / \text{smallest number}$.
- Word Search: $O(R * C * 4^L)$ where L is word length.
- N-Queens: about $O(n!)$ time.

Common mistakes

- Appending `path` instead of `path[:]`. Later pops change the saved list.
- Forgetting the undo step (`pop`, `unmark`, `used[i] = False`).
- Using `i+1` when reuse is allowed, or `i` when reuse is not allowed.
- Not sorting before skipping duplicates.
- No pruning, so the code is too slow (for example, keep going when `total > target`).

What to say in the interview

"We need all valid combinations, so I will use backtracking."

"At each step I choose one element, recurse, then undo the choice."

"I will prune early when the running sum is bigger than the target."

"The time is exponential, which is fine because the input is small."

Practice order

- Subsets first: the simplest template, every node is an answer.
- Combination Sum and Permutations next: learn reuse vs used-set.
- Subsets II and Combination Sum II: learn the sort + skip duplicates trick.
- Word Search and Palindrome Partitioning: backtracking with a validity check.
- N-Queens last: several constraint sets (columns, diagonals) at once.

Cheat sheet

Idea

Build a solution step by step; at each step choose, recurse, then undo the choice. It is DFS over a decision tree.

Use it when

- "All subsets / permutations / combinations"
- Constraint puzzles (N-Queens, word search)

Template

```
res, path = [], []
def bt(i):
    if i == len(nums):
        res.append(path[:])
        return
    path.append(nums[i]); bt(i + 1)  # choose
    path.pop(); bt(i + 1)          # skip
bt(0)
```

Complexity

Exponential: subsets $O(2^n)$, permutations $O(n!)$. Say it clearly.

Common mistakes

- Appending `path` instead of a copy `path[:]`
- Forgetting to undo (pop) the choice

Visualise it

- <https://visualgo.net/en/recursion>

Videos

- [Backtracking template](#) - Bitflip (English)
- [Introduction to Backtracking](#) - Abdul Bari (English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Subsets	Medium	open	watch
2	Combination Sum (Blind 75)	Medium	open	watch
3	Permutations	Medium	open	watch
4	Subsets II	Medium	open	watch
5	Combination Sum II	Medium	open	watch
6	Word Search (Blind 75)	Medium	open	watch
7	Palindrome Partitioning	Medium	open	watch
8	Letter Combinations of a Phone Number	Medium	open	watch
9	N Queens	Hard	open	watch

Problem hints

Subsets (Medium)

Restate: Given a list of unique numbers, return every possible subset (the power set), including the empty one.

Hint 1: Use backtracking. Think of each number as a light switch: ON (take it) or OFF (skip it).

Hint 2: Every node in the recursion tree is already a valid answer, not only the leaves. So you save the current path at the start of every call.

Hint 3:

- Keep a `path` list and a `start` index.
- On entering a call, save a copy of `path`.
- Loop `i` from `start` to the end: add `nums[i]`, recurse with `i + 1`, then remove it.
- Moving forward only (`i + 1`) stops you from making `[2, 1]` after `[1, 2]`.

Complexity: $O(n \cdot 2^n)$ time (2^n subsets, each copied in $O(n)$), $O(n)$ extra space for recursion.

Edge cases to test: `[]` gives `[[]]`; `[0]` gives `[[], [0]]`; `[1, 2, 3]` gives 8 subsets; negative numbers `[-1, 5]`; check the output has no duplicate subsets.

Combination Sum (Medium)

Restate: Given unique positive numbers and a target, return all combinations that add up to the target, where each number can be used many times.

Hint 1: Backtracking with a running total.

Hint 2: To allow reuse but avoid duplicate orderings, the next call starts at the same index `i`, not `i + 1`.

Hint 3:

- Recurse with `(start, total)`.
- If `total == target`, save a copy of the path.

- If `total > target`, stop (prune).
- Loop `i` from `start`: add `candidates[i]`, recurse with `(i, total + candidates[i])`, then pop.
- Optional: sort first, and break the loop as soon as a number is too big.

Complexity: Exponential, roughly $O(2^{(\text{target} / \text{min})})$ time; $O(\text{target} / \text{min})$ recursion depth.

Edge cases to test: `[2]`, 1 gives `[]`; `[1]`, 3 gives `[[1,1,1]]`; `[2,3,6,7]`, 7 gives `[[2,2,3], [7]]`; target equal to one candidate; a large candidate bigger than the target.

Permutations (Medium)

Restate: Given a list of unique numbers, return every possible ordering of them.

Hint 1: Backtracking with a `used` array (or set).

Hint 2: Unlike subsets, order matters, so every level loops over ALL numbers and skips only the ones already used in this path.

Hint 3:

- If `len(path) == len(nums)`, save a copy and return.
- Loop over every index `i`.
- Skip `i` if `used[i]` is true.
- Mark used, append, recurse, then unmark and pop.

Complexity: $O(n * n!)$ time, $O(n)$ extra space.

Edge cases to test: `[1]` gives `[[1]]`; `[0, 1]` gives 2 orders; `[1, 2, 3]` gives 6; negative numbers `[-1, 0, 1]`; check every output has length `n`.

Subsets II (Medium)

Restate: Same as Subsets, but the input can have repeated numbers, and the output must not contain duplicate subsets.

Hint 1: Sort the list first, then use the normal subsets backtracking.

Hint 2: At one level of the tree, take only the first copy of each value. Skip `nums[i]` when `i > start` and `nums[i] == nums[i - 1]`.

Hint 3:

- Sort `nums`.
- Save a copy of `path` on entering each call.
- Loop `i` from `start`; skip duplicates using the rule above.
- Append, recurse with `i + 1`, pop.

Complexity: $O(n * 2^n)$ time, $O(n)$ extra space.

Edge cases to test: `[0]`; `[2, 2]` gives `[], [2], [2,2]`; `[1, 2, 2]` gives 6 subsets; all equal `[5,5,5]` gives 4 subsets; unsorted input `[4, 4, 1, 4]`.

Combination Sum II (Medium)

Restate: Given numbers that may repeat and a target, return all unique combinations that sum to the target, where each number (each position) is used at most once.

Hint 1: It mixes Combination Sum (target + prune) with Subsets II (sort + skip duplicates).

Hint 2: Each element is used once, so recurse with $i + 1$. Skip `candidates[i] == candidates[i - 1]` when $i > \text{start}$ to avoid duplicate combinations.

Hint 3:

- Sort the list.
- If `total == target`, save; if `total > target`, return.
- Loop i from `start`, skip duplicates at this level.
- If `candidates[i]` makes the total too big, `break` (the list is sorted).
- Append, recurse with $(i + 1, \text{total} + \text{candidates}[i])$, pop.

Complexity: $O(2^n)$ time in the worst case, $O(n)$ extra space.

Edge cases to test: `[10,1,2,7,6,1,5]`, 8 gives `[[1,1,6],[1,2,5],[1,7],[2,6]]`; `[1,1,1]`, 2 gives `[[1,1]]` only once; `[2]`, 1 gives `[]`; target equal to the sum of all numbers.

Word Search (Medium)

Restate: Given a grid of letters and a word, say whether the word can be formed by walking through neighbouring cells (up, down, left, right) without using a cell twice.

Hint 1: DFS from every cell, with backtracking on a "visited" mark.

Hint 2: Mark a cell as used while it is on the current path, and unmark it when you come back. A cell can be used again in a different path.

Hint 3:

- For each cell, start `dfs(r, c, i = 0)`.
- Fail if out of bounds, already used, or letter does not match `word[i]`.
- Succeed if $i == \text{len}(\text{word}) - 1$ after a match (or $i == \text{len}(\text{word})$ at entry).
- Mark the cell, try 4 neighbours with $i + 1$, then unmark.
- Pruning idea: if the grid does not have enough of some letter, return False early.

Complexity: $O(R * C * 4^L)$ time where L is the word length (really 3^L after the first step); $O(L)$ recursion space.

Edge cases to test: 1x1 grid `[[ "a" ]]`, word "a"; word longer than $R * C$; word that needs to reuse a cell, like "ABA" in `[[ "A", "B" ]]` (should be False); word going around a corner; same letter many times `[[ "a", "a" ], [ "a", "a" ]]` with "aaaaa" (False).

Palindrome Partitioning (Medium)

Restate: Split a string into pieces so that every piece is a palindrome, and return all such splits.

Hint 1: Backtracking where each choice is "where does the next piece end?".

Hint 2: From position `start`, try every end `j`. Only recurse if `s[start..j]` is a palindrome.

Hint 3:

- If `start == len(s)`, save a copy of the path.
- Loop j from `start` to `len(s) - 1`.
- If `s[start:j+1]` is a palindrome, append it and recurse with $j + 1$, then pop.

- Optional speed-up: precompute a 2-D table `isPal[i][j]`.

Complexity: $O(n * 2^n)$ time, $O(n)$ recursion space (plus $O(n^2)$ if you precompute the table).

Edge cases to test: "a" gives `[["a"]]`; "aab" gives `[["a", "a", "b"], ["aa", "b"]]`; "aaa" gives 4 splits; "abc" gives only single letters; empty string (if allowed) gives `[[]]`.

Letter Combinations of a Phone Number (Medium)

Restate: Given a string of digits 2-9, return all letter strings they could spell on an old phone keypad.

Hint 1: Backtracking with a digit-to-letters map (2 = "abc", ..., 7 = "pqrs", 9 = "wxyz").

Hint 2: The depth of the tree equals the number of digits. At depth `i`, you try every letter of `digits[i]`.

Hint 3:

- If `digits` is empty, return `[]` (not `[""]`).
- Recurse with index `i` and the current string.
- If `i == len(digits)`, save the string.
- For each letter of `map[digits[i]]`, recurse with `i + 1`.

Complexity: $O(4^n * n)$ time, $O(n)$ recursion space.

Edge cases to test: "" gives `[]`; "2" gives `["a", "b", "c"]`; "23" gives 9 strings; "79" gives 16 strings (both keys have 4 letters); repeated digit "22".

N Queens (Hard)

Restate: Place `n` queens on an `n x n` chessboard so that no two attack each other, and return all such boards.

Hint 1: Backtracking, one row at a time. Each row gets exactly one queen.

Hint 2: Keep three sets: used columns, used "r - c" diagonals, and used "r + c" anti-diagonals. A cell is safe only if it is in none of them. This makes the check $O(1)$.

Hint 3:

- Recurse on `row`.
- If `row == n`, convert the board to strings and save.
- For each `col`, skip if `col`, `row - col` or `row + col` is already used.
- Place the queen, add to all three sets, recurse with `row + 1`.
- Remove the queen and remove from the three sets.

Complexity: About $O(n!)$ time, $O(n)$ extra space (plus the board).

Edge cases to test: `n = 1` gives 1 board; `n = 2` and `n = 3` give 0 boards; `n = 4` gives 2 boards; `n = 8` gives 92 boards.

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Combinations	Medium	open	watch
2	Permutations II	Medium	open	watch

#	Problem	Level	LeetCode	Solution video
3	Restore IP Addresses	Medium	open	watch
4	Matchsticks to Square	Medium	open	watch
5	Splitting a String Into Descending Consecutive Values	Medium	open	watch
6	Find Unique Binary String	Medium	open	watch

Quiz

- Why do we append `path[:]` and not `path` to the result?
 - A) It is faster
 - B) `path` keeps changing after we save it, so we need a copy
 - C) Python does not allow appending lists
 - D) To sort the path
- In Combination Sum (reuse allowed), what start index do we pass to the next call?
 - A) $i + 1$
 - B) i
 - C) 0
 - D) $\text{start} + 1$
- How do we skip duplicate subsets when `nums` has repeated values?
 - A) Use a set of tuples only
 - B) Sort, then skip `nums[i] == nums[i-1]` when $i > \text{start}$
 - C) Reverse the list
 - D) It is not possible
- What is the time complexity of generating all permutations of n items?
 - A) $O(n^2)$
 - B) $O(2^n)$
 - C) $O(n * n!)$
 - D) $O(n \log n)$
- In Word Search, why do we set the cell to "#" before exploring?
 - A) To mark it visited so the same cell is not used twice in one path
 - B) To save memory
 - C) To end the search
 - D) To speed up the `any()` call
- Number of subsets of n items:
 - A) n^2
 - B) 2^n
 - C) $n!$
- Why append `path[:]` instead of `path`?
 - A) Faster
 - B) `path` keeps changing later
 - C) Style
- Backtracking is a form of:

- A) BFS
- B) DFS
- C) Greedy

Answer key

1. **B** - path keeps changing after we save it, so we need a copy. Without a copy, all saved answers point to the same list, which ends up empty.
2. **B** - i. Passing i lets us pick the same number again but never go back to smaller indexes.
3. **B** - Sort, then skip `nums[i] == nums[i-1]` when `i > start`. Sorting puts equal values together so we take the first copy at each level only.
4. **C** - $O(n * n!)$. There are $n!$ permutations and copying each one costs $O(n)$.
5. **A** - To mark it visited so the same cell is not used twice in one path. A letter cell can be used only once per word, and we restore it after exploring.
6. **B** - 2^n .
7. **B** - path keeps changing later.
8. **B** - DFS.

More quiz

1. In Subsets, where do you save the current path?
 - A. Only when `path` has length `n`
 - B. At the start of every recursive call
 - C. Only when the loop ends
 - D. Only at the root
2. Which pattern fits "split a string so every piece is in a dictionary, return all splits"?
 - A. Sliding window
 - B. Backtracking that tries every end position for the next piece
 - C. Binary search
 - D. Heap
3. In N Queens, which value is the same for every cell on one "" diagonal (top-left to bottom-right)?
 - A. `row + col`
 - B. `row * col`
 - C. `row - col`
 - D. `col`
4. What should Letter Combinations return for the input "" ?
 - A. `[""]`
 - B. `[]`
 - C. `None`
 - D. An error
5. In Combination Sum II, which detail makes each element usable only once?
 - A. Sorting the list
 - B. Recursing with `i + 1`

- C. Using a set for the result
- D. Pruning when the total is too big

More quiz: answer key

1. **B** - Every node in the subsets tree is a valid subset, so you save a copy each time you enter a call.
2. **B** - You need all splits, so you try each possible next piece and backtrack. This is the same shape as Palindrome Partitioning.
3. **C** - Moving down-right adds 1 to both row and col, so `row - col` stays the same. `row + col` is constant on the other diagonal.
4. **B** - There are no digits, so there are no combinations. Returning `[""]` is a common bug.
5. **B** - Passing `i + 1` moves past the current position. Sorting and skipping only remove duplicate combinations.

Flashcards

- **Q:** What three parts does every backtracking step have? — **A:** Choose (add to path), explore (recurse), un-choose (undo the change).
- **Q:** Subsets vs permutations: what changes in the loop? — **A:** Subsets loop from `start` forward; permutations loop over all indexes and skip used ones.
- **Q:** Reuse allowed vs not allowed: which index goes to the next call? — **A:** Reuse allowed: `i`. Not allowed: `i + 1`.
- **Q:** What is the rule to skip duplicates after sorting? — **A:** Skip `nums[i]` when `i > start` and `nums[i] == nums[i - 1]`.
- **Q:** How does N Queens check a cell in $O(1)$? — **A:** Three sets: columns, `row - col` diagonals, `row + col` anti-diagonals.
- **Q:** Why must Word Search unmark a cell after exploring? — **A:** The cell may be needed by a different path that starts elsewhere or turns differently.
- **Q:** How many permutations does `[1, 2, 3, 4]` have? — **A:** $4! = 24$.
- **Q:** How many subsets does a list of 5 unique numbers have? — **A:** $2^5 = 32$, including the empty subset.
- **Q:** How many valid boards are there for 4 Queens? — **A:** 2.
- **Q:** What is a good tester-style input for Combination Sum II? — **A:** Repeated values like `[1, 1, 1]` with target 2, to check the answer `[1, 1]` appears only once.

DSA topic: Graphs

What it is

A graph is a set of nodes connected by edges. Cities and roads is a good picture.

A grid is also a graph: each cell is a node, and it connects to its 4 neighbours.

Most graph problems ask: can I reach X, how many groups, what is the shortest path, or what order for tasks. Main tools: DFS, BFS, topological sort (Kahn), and Union-Find.

How to recognise it

- Words like "connected", "islands", "network", "path", "neighbours", "dependencies", "prerequisites".
- A 2-D grid of "1"/"0" or characters where you move up, down, left, right.
- "Minimum steps" in an unweighted graph means BFS.
- "Order of courses / tasks" or "detect a cycle in a directed graph" means topological sort.
- "Are these two in the same group?" or "redundant connection" means Union-Find.

Pattern 1: DFS / BFS on a grid and on an adjacency list

Use DFS to count or fill regions. Use BFS for shortest steps in unweighted graphs.

```
from collections import deque, defaultdict
def num_islands(grid):
    R, C, count = len(grid), len(grid[0]), 0
    def dfs(r, c):
        if r < 0 or c < 0 or r >= R or c >= C or grid[r][c] != "1":
            return
        grid[r][c] = "0"  # mark visited
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            dfs(r + dr, c + dc)
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "1":
                dfs(r, c); count += 1
    return count
def bfs(n, edges, start):
    adj = defaultdict(list)
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    dist, q = {start: 0}, deque([start])
    while q:
        node = q.popleft()
        for nei in adj[node]:
            if nei not in dist:
                dist[nei] = dist[node] + 1; q.append(nei)
    return dist
```

Multi-source BFS (Rotting Oranges, Walls and Gates): put ALL sources in the queue at the start.

Pattern 2: Topological sort (Kahn algorithm)

Use for "prerequisites" and directed cycle detection. Start from nodes with indegree 0.

```
def topo(n, prereqs):
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:          # b must come before a
        adj[b].append(a); indeg[a] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        node = q.popleft(); order.append(node)
        for nei in adj[node]:
            indeg[nei] -= 1
            if indeg[nei] == 0:
                q.append(nei)
    return order if len(order) == n else []    # [] means cycle
```

Pattern 3: Union-Find (Disjoint Set Union)

Use to merge groups and check "same group?" fast. Good for counting components and finding a redundant edge.

```
parent = list(range(n)); rank = [0] * n
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]    # path compression
        x = parent[x]
    return x
def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb: return False            # already same group: cycle
    if rank[ra] < rank[rb]: ra, rb = rb, ra
    parent[rb] = ra
    if rank[ra] == rank[rb]: rank[ra] += 1
    return True
```

Worked example: Course Schedule

n courses, prerequisites [a, b] mean take b before a. Can you finish all courses?

Input: n = 4, prereqs = [[1,0],[2,0],[3,1],[3,2]].

1. Build edges: 0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3. indeg = [0, 1, 1, 2].
2. Queue starts with nodes of indeg 0: q = [0].
3. Pop 0, order = [0]. indeg[1] = 0, indeg[2] = 0. q = [1, 2].
4. Pop 1, order = [0,1]. indeg[3] = 1. q = [2].
5. Pop 2, order = [0,1,2]. indeg[3] = 0. q = [3].

6. Pop 3, order = [0,1,2,3]. q is empty.

7. len(order) == 4 == n, so return True. If there was a cycle, some node would never reach indeg 0.

```
def canFinish(n, prereqs):
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a); indeg[a] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    done = 0
    while q:
        node = q.popleft(); done += 1
        for nei in adj[node]:
            indeg[nei] -= 1
            if indeg[nei] == 0:
                q.append(nei)
    return done == n
```

Complexity cheat sheet

- DFS / BFS on adjacency list: $O(V + E)$ time, $O(V)$ space.
- DFS / BFS on a grid: $O(R * C)$ time and space.
- Kahn topological sort: $O(V + E)$.
- Union-Find with path compression + rank: almost $O(1)$ per operation (inverse Ackermann).

Common mistakes

- Marking visited when you POP in BFS instead of when you PUSH. Nodes get added many times.
- Forgetting bounds checks in grid DFS.
- Building edges in the wrong direction for prerequisites.
- Deep recursion on a big grid hits the Python recursion limit; use BFS or an explicit stack.
- In Union-Find, comparing x and y instead of find(x) and find(y).

What to say in the interview

"I will model this as a graph: cells or items are nodes, and the relation is an edge."

"I need the minimum steps and edges have no weight, so I will use BFS."

"For ordering with dependencies I will use Kahn algorithm; if not all nodes are processed, there is a cycle."

"The time is $O(V + E)$ because each node and edge is visited once."

Practice order

- Number of Islands and Max Area of Island: basic grid DFS.
- Clone Graph: adjacency list traversal with a hashmap of copies.
- Rotting Oranges and Walls and Gates: multi-source BFS.
- Course Schedule I and II: topological sort.

- Redundant Connection and Number of Connected Components: Union-Find.

Cheat sheet

Idea

Nodes + edges. Build an adjacency list, then DFS/BFS with a visited set. Grids are graphs too (4 neighbours).

Use it when

- Islands / connected components -> DFS/BFS
- Shortest path, unweighted -> BFS
- Dependencies / order -> topological sort (Kahn's BFS with indegrees)
- Union-Find for "are these connected?"

Grid DFS template

```
def dfs(r, c):
    if r < 0 or c < 0 or r >= R or c >= C or grid[r][c] != "1":
        return
    grid[r][c] = "0"
    for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
        dfs(r + dr, c + dc)
```

Complexity

O(V + E). Grid: O(rows * cols).

Visualise it

- <https://visualgo.net/en/dfsdfs>

Videos

- [BFS and DFS](#) - Abdul Bari (English)
- [Breadth-First Search](#) - take U forward (Hindi + English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Number of Islands (Blind 75)	Medium	open	watch

#	Problem	Level	LeetCode	Solution video
2	Clone Graph (Blind 75)	Medium	open	watch
3	Max Area of Island	Medium	open	watch
4	Pacific Atlantic Water Flow (Blind 75)	Medium	open	watch
5	Surrounded Regions	Medium	open	watch
6	Rotting Oranges	Medium	open	watch
7	Walls And Gates	Medium	open	watch
8	Course Schedule (Blind 75)	Medium	open	watch
9	Course Schedule II	Medium	open	watch
10	Redundant Connection	Medium	open	watch
11	Number of Connected Components In An Undirected Graph (Blind 75)	Medium	open	watch
12	Graph Valid Tree (Blind 75)	Medium	open	watch
13	Word Ladder	Hard	open	watch

Problem hints

Number of Islands (Medium)

Restate: Given a grid of "1" (land) and "0" (water), count how many separate islands there are, where land connects up, down, left and right.

Hint 1: Grid DFS or BFS. Each island is one connected group of cells.

Hint 2: Every time you find an unvisited "1", you have found a new island. Flood-fill it (mark all its cells visited) so you never count it again.

Hint 3:

- Loop over every cell.
- If the cell is "1" and not visited, add 1 to the count.
- Run DFS/BFS from it and mark every connected "1" as visited (for example, set it to "0").
- Diagonal cells do NOT connect.

Complexity: $O(R * C)$ time, $O(R * C)$ space in the worst case (recursion or queue).

Edge cases to test: all water gives 0; all land gives 1; a checkerboard like `[[ "1", "0" ], [ "0", "1" ]]` gives 2 (diagonals do not join); a single cell `[[ "1" ]]`; one long snake-shaped island in a big grid (tests recursion depth).

Clone Graph (Medium)

Restate: Given one node of a connected undirected graph, return a deep copy of the whole graph.

Hint 1: DFS or BFS plus a hash map from old node to new node.

Hint 2: The map does two jobs: it is the "visited" set, and it lets you reuse a copy that already exists, so cycles do not cause infinite loops.

Hint 3:

- If the input node is null, return null.
- `clone(node)` : if `node` is in the map, return its copy.
- Otherwise create a copy with the same value, store it in the map FIRST.
- Then for each neighbour, append `clone(neighbour)` to the copy's neighbour list.

Complexity: $O(V + E)$ time, $O(V)$ space.

Edge cases to test: null input; a single node with no neighbours; two nodes pointing to each other (a cycle); a 4-node square `1-2-3-4-1`; check that the copy shares NO objects with the original.

Max Area of Island (Medium)

Restate: In a grid of 1s and 0s, return the size (number of cells) of the biggest island, or 0 if there is none.

Hint 1: Same flood fill as Number of Islands.

Hint 2: Make the DFS return the number of cells it filled: `1 + dfs(up) + dfs(down) + dfs(left) + dfs(right)`.

Hint 3:

- Loop over every cell.
- If it is 1, call DFS, which marks cells and returns the area.
- Keep the maximum area seen.
- Return 0 for out-of-bounds, water, or visited cells.

Complexity: $O(R * C)$ time and space.

Edge cases to test: all zeros gives 0; all ones gives $R * C$; one cell `[[1]]` gives 1; two islands of different sizes; diagonal-only touching cells (they are separate islands).

Pacific Atlantic Water Flow (Medium)

Restate: Water flows from a cell to a neighbour with equal or lower height. Return all cells from which water can reach both the Pacific (top and left edges) and the Atlantic (bottom and right edges).

Hint 1: Reverse the direction. Start from the oceans and walk "uphill".

Hint 2: Do one search from all Pacific border cells and one from all Atlantic border cells. A cell is an answer if both searches reach it. Running a search from every cell instead would be too slow.

Hint 3:

- Make two visited sets: `pac` and `atl`.
- DFS/BFS from every top-row and left-column cell into `pac`.
- DFS/BFS from every bottom-row and right-column cell into `atl`.
- Move to a neighbour only if its height is $\geq$ the current height.
- Return cells that are in both sets.

Complexity: $O(R * C)$ time and space.

Edge cases to test: a 1x1 grid (the cell touches both oceans); a single row `[[1,2,3]]` (every cell touches both); a flat grid of all equal heights (every cell is an answer); a strict "valley" in the middle; a grid where heights increase only toward one corner.

Surrounded Regions (Medium)

Restate: In a board of "X" and "O", change every "O" region that is fully surrounded by "X" into "X". Regions touching the border stay.

Hint 1: Again, think in reverse: find the "O" cells that are SAFE.

Hint 2: Only "O" cells connected to the border survive. Mark them first from the border, then flip everything else.

Hint 3:

- DFS/BFS from every "O" on the border, marking connected "O" cells as a temporary letter like "T".
- Scan the whole board.
- Change remaining "O" to "X" (they are surrounded).
- Change "T" back to "O".

Complexity: $O(R * C)$ time and space.

Edge cases to test: an empty board or 1x1 board; all "O" (nothing flips); an "O" in the middle surrounded by "X" (flips); an "O" region that touches the border only through a long path (stays); a board with 1 or 2 rows (every cell is on the border).

Rotting Oranges (Medium)

Restate: Each minute, a rotten orange (2) rots its fresh neighbours (1). Return the minutes until no fresh orange is left, or -1 if that is impossible.

Hint 1: Multi-source BFS. All rotten oranges start in the queue together.

Hint 2: Each BFS "level" is one minute. Count the fresh oranges first, so at the end you know if some were never reached.

Hint 3:

- Push all 2s into the queue; count all 1s.
- If fresh count is 0, return 0.
- Process the queue level by level; for each fresh neighbour, make it 2, decrease fresh, push it.
- Add 1 minute after each level that rotted at least one orange.
- Return minutes if fresh is 0, else -1.

Complexity: $O(R * C)$ time and space.

Edge cases to test: no oranges at all `[[0]]` gives 0; only fresh `[[1]]` gives -1; only rotten `[[2]]` gives 0; a fresh orange walled off by 0s gives -1; two rotten oranges at opposite ends (tests that they spread at the same time).

Walls And Gates (Medium)

Restate: In a grid with walls (-1), gates (0) and empty rooms (a very large number, INF), fill each room with its distance to the nearest gate, leaving unreachable rooms as INF.

Hint 1: Multi-source BFS from all gates at once.

Hint 2: Because BFS from all gates spreads one step at a time, the first time a room is reached is from its nearest gate. You never need to update it again.

Hint 3:

- Push every gate into the queue.

- Pop a cell; for each neighbour that is an INF room, set it to current distance + 1 and push it.
- Skip walls, gates and rooms already filled.
- Change the grid in place.

Complexity: $O(R * C)$ time and space.

Edge cases to test: no gates (all rooms stay INF); no rooms; a room completely surrounded by walls (stays INF); two gates with a room exactly in the middle; an empty grid `[]`.

Course Schedule (Medium)

Restate: Given n courses and pairs `[a, b]` meaning "take b before a ", say whether you can finish all courses.

Hint 1: Model it as a directed graph. The question is "is there a cycle?".

Hint 2: You can use topological sort (Kahn: process nodes with indegree 0) or DFS with three colours: unvisited, visiting (on the current path), done. Meeting a "visiting" node means a cycle.

Hint 3: (DFS version; Kahn's algorithm also works)

- Build an adjacency list `b -> a`.
- `state[node]`: 0 = new, 1 = visiting, 2 = done.
- DFS: if state is 1 return False (cycle); if 2 return True.
- Set state 1, DFS all neighbours, then set state 2.
- Run DFS from every node; if any returns False, answer False.

Complexity: $O(V + E)$ time and space.

Edge cases to test: no prerequisites (True); a self-loop `[[0,0]]` (False); a 2-cycle `[[1,0],[0,1]]` (False); a disconnected graph with a cycle only in one part (False); a long chain `0 -> 1 -> ... -> n-1` (True).

Course Schedule II (Medium)

Restate: Same as Course Schedule, but return one valid order to take all the courses, or an empty list if it is impossible.

Hint 1: Topological sort.

Hint 2: With Kahn's algorithm, the order in which nodes leave the queue is a valid order. With DFS, add a node to the list when it is DONE, then reverse the list at the end.

Hint 3:

- Build `b -> a` edges and indegrees.
- Start a queue with all indegree-0 courses.
- Pop, append to the order, decrease neighbours' indegree, push new zeros.
- If the order has n courses, return it; else return `[]`.

Complexity: $O(V + E)$ time and space.

Edge cases to test: $n = 1$, no prerequisites gives `[0]`; a cycle gives `[]`; many valid orders (your test must check "is the order valid", not one exact list); duplicate prerequisite pairs; isolated courses with no edges must still appear.

Redundant Connection (Medium)

Restate: A tree with n nodes got one extra edge. Return the edge that can be removed so it becomes a tree again (if several, return the last one in the input).

Hint 1: Union-Find.

Hint 2: Add edges one by one. The first edge whose two ends already have the same root closes a cycle. Because you scan in input order, that edge is the last edge of the cycle in the input, which is what the problem wants.

Hint 3:

- Make `parent[i] = i` for nodes $1..n$.
- For each edge (a, b) : find both roots.
- If roots are equal, return this edge.
- Otherwise union them.

Complexity: $O(n * \alpha(n))$ time, which is nearly $O(n)$; $O(n)$ space.

Edge cases to test: smallest case `[[1,2],[1,3],[2,3]]` gives `[2,3]`; a long cycle where the extra edge is in the middle of the input; nodes labelled from 1 (off-by-one bugs if the array has size n); a cycle not involving node 1.

Number of Connected Components In An Undirected Graph (Medium)

Restate: Given n nodes and a list of undirected edges, return how many connected groups there are.

Hint 1: Union-Find, or DFS from every unvisited node.

Hint 2: With Union-Find, start with n groups. Every successful union (two different roots) reduces the count by 1.

Hint 3:

- `count = n`, `parent[i] = i`.
- For each edge, find both roots.
- If they differ, union them and do `count -= 1`.
- Return `count`.

Complexity: $O(E * \alpha(n))$ time, $O(n)$ space.

Edge cases to test: no edges gives n ; `n = 1` gives 1; all nodes in one chain gives 1; a duplicate edge `[[0,1],[0,1]]` must not reduce the count twice; an edge list that contains a cycle.

Graph Valid Tree (Medium)

Restate: Given n nodes and undirected edges, say whether they form a valid tree (connected and with no cycle).

Hint 1: Union-Find or DFS.

Hint 2: A tree with n nodes has exactly $n - 1$ edges. If the edge count is not $n - 1$, return False immediately. If it is $n - 1$ and there is no cycle, the graph is also connected.

Hint 3:

- If `len(edges) != n - 1`, return False.
- Union every edge; if two ends already share a root, return False (cycle).
- Otherwise return True.

Complexity: $O(n * \alpha(n))$ time, $O(n)$ space.

Edge cases to test: $n = 1$, no edges (True); $n = 2$, no edges (False, not connected); a triangle $n = 3$, $[[0,1],[1,2],[2,0]]$ (False); $n - 1$ edges but with a cycle plus an isolated node, like $n = 4$, $[[0,1],[1,2],[2,0]]$ (False); a star shape (True).

Word Ladder (Hard)

Restate: Change `beginWord` into `endWord` one letter at a time, where every middle word must be in the word list. Return the number of words in the shortest such chain, or 0 if impossible.

Hint 1: BFS on an implicit graph. Each word is a node; words differing by one letter are neighbours.

Hint 2: Do not compare every pair of words. Use patterns: "hot" belongs to buckets "ot", "ht", "ho*". Words in the same bucket are neighbours.

Hint 3:

- If `endWord` is not in the list, return 0.
- Build a map from pattern to words.
- BFS from `beginWord` with level = 1.
- For each word, for each of its patterns, visit unvisited words in that bucket.
- Return the level when you pop `endWord`.

Complexity: $O(N * L^2)$ time where N is the number of words and L the word length (building L patterns of length L for each word); $O(N * L^2)$ space for the buckets.

Edge cases to test: `endWord` missing from the list gives 0; `beginWord` one letter away from `endWord` gives 2; no path gives 0; `beginWord` is not in the list (that is allowed); a list with many words sharing patterns (performance).

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Count Sub Islands	Medium	open	watch
2	Reorder Routes to Make All Paths Lead to The City Zero	Medium	open	watch
3	Snakes And Ladders	Medium	open	watch
4	Open The Lock	Medium	open	watch
5	Find Eventual Safe States	Medium	open	watch
6	Course Schedule IV	Medium	open	watch

Quiz

1. Which traversal gives the shortest path in an unweighted graph?
 - A) DFS
 - B) BFS
 - C) Topological sort

- D) Union-Find
2. In Kahn algorithm, how do we know the graph has a cycle?
 - A) The queue is never empty
 - B) Fewer than n nodes end up in the order
 - C) Some node has indegree 0
 - D) The graph has more edges than nodes
 3. When should a node be marked visited in BFS?
 - A) When it is pushed into the queue
 - B) When it is popped
 - C) At the end
 - D) Never
 4. What does union(a , b) returning False mean in Redundant Connection?
 - A) a and b are not nodes
 - B) a and b are already connected, so this edge makes a cycle
 - C) The graph is empty
 - D) The ranks are equal
 5. What is the time complexity of DFS over a grid with R rows and C columns?
 - A) $O(R + C)$
 - B) $O(R * C)$
 - C) $O((R * C)^2)$
 - D) $O(\log(R * C))$
 6. Shortest path in an unweighted graph:
 - A) DFS
 - B) BFS
 - C) Dijkstra only
 7. Course Schedule (cycle in dependencies) is solved with:
 - A) Topological sort
 - B) Binary search
 - C) Two pointers
 8. DFS/BFS complexity on a graph:
 - A) $O(V + E)$
 - B) $O(V * E)$
 - C) $O(\log V)$

Answer key

1. **B** - BFS. BFS visits nodes level by level, so the first time we reach a node is with the fewest edges.
2. **B** - Fewer than n nodes end up in the order. Nodes in a cycle never reach indegree 0, so they are never processed.
3. **A** - When it is pushed into the queue. Marking on push stops the same node from being added to the queue many times.
4. **B** - a and b are already connected, so this edge makes a cycle. If both have the same root, the new edge closes a cycle and is redundant.
5. **B** - $O(R * C)$. Each cell is visited a constant number of times.

6. **B** - BFS.
7. **A** - Topological sort.
8. **A** - $O(V + E)$.

More quiz

1. Which pattern fits "find all cells that can reach BOTH the top edge and the bottom edge"?
 - A. One DFS from every cell
 - B. Two reverse searches, one from each edge, then intersect
 - C. Binary search on heights
 - D. Sort the cells
2. A graph has n nodes and exactly $n - 1$ edges, and it has no cycle. What else must be true?
 - A. It has two components
 - B. It is connected, so it is a tree
 - C. It is a directed graph
 - D. Nothing else can be said
3. In Clone Graph, why do you store the new copy in the map BEFORE cloning the neighbours?
 - A. To save memory
 - B. So a cycle that comes back to this node finds the copy and stops
 - C. To keep neighbours sorted
 - D. It does not matter
4. In DFS cycle detection for a directed graph, what does reaching a node in the "visiting" state mean?
 - A. The node is a leaf
 - B. There is a cycle
 - C. The graph is disconnected
 - D. The node is done
5. In Word Ladder, why are patterns like "h*t" useful?
 - A. They sort the words
 - B. They find one-letter neighbours without comparing every pair of words
 - C. They remove duplicates
 - D. They make DFS possible

More quiz: answer key

1. **B** - Searching backwards from each edge visits each cell a small number of times. One search per cell would be $O((R \cdot C)^2)$.
2. **B** - An acyclic graph with n nodes and $n - 1$ edges is always connected. That is why Graph Valid Tree can check the edge count first.
3. **B** - If a neighbour leads back to the current node, the map already has its copy, so the recursion stops instead of looping forever.
4. **B** - The node is still on the current path, so you came back to it through a loop.
5. **B** - Words that share a pattern differ in exactly one position, so buckets give neighbours directly.

Flashcards

- **Q:** What is a "reverse search" trick in grid problems? — **A:** Start from the target (border, ocean, gate) and search outward, instead of searching from every cell.
- **Q:** How do you find safe "O" cells in Surrounded Regions? — **A:** Flood-fill from every border "O"; anything not reached is surrounded.
- **Q:** What does each BFS level mean in Rotting Oranges? — **A:** One minute of time.
- **Q:** When does Rotting Oranges return -1? — **A:** When some fresh orange is still left after the BFS ends.
- **Q:** What are the three DFS states for directed cycle detection? — **A:** New, visiting (on current path), done.
- **Q:** How do you get a topological order from DFS? — **A:** Add each node when it finishes, then reverse the list.
- **Q:** Quick check for Graph Valid Tree? — **A:** Edge count must be exactly $n - 1$, then make sure there is no cycle.
- **Q:** In Number of Connected Components, when does the count go down? — **A:** Only when a union joins two different roots.
- **Q:** Why is Word Ladder BFS and not DFS? — **A:** It asks for the shortest chain, and edges have no weight.
- **Q:** What is a good tester input for Course Schedule II? — **A:** A graph with many valid orders; check the output is A valid order, not one fixed list.

DSA topic: Advanced Graphs

What it is

Advanced graph problems have weights on edges: cost, time, distance or price.

Think of Google Maps. Roads have different lengths, so fewest roads is not the same as shortest distance.

The main algorithms are Dijkstra (shortest path, no negative weights), Prim and Kruskal (minimum spanning tree), and Bellman-Ford (shortest path with at most k edges, or negative weights).

How to recognise it

- Edges come as [u, v, weight] or [from, to, price].
- "Minimum time for signal to reach all nodes", "cheapest path", "minimum effort" means Dijkstra.
- "Connect all points with minimum total cost" means minimum spanning tree (Prim or Kruskal).
- "At most k stops" means Bellman-Ford with k+1 rounds (or BFS by levels).
- Grid where moving has a cost (Swim in Rising Water) is Dijkstra on a grid.

Pattern 1: Dijkstra with a min-heap

Always expand the closest node not finished yet. Works only with non-negative weights.

```
import heapq
from collections import defaultdict
def dijkstra(n, edges, src):
    adj = defaultdict(list)
    for u, v, w in edges:
        adj[u].append((v, w))
    dist = {}
    heap = [(0, src)]  # (distance, node)
    while heap:
        d, node = heapq.heappop(heap)
        if node in dist:
            continue  # already finalised
        dist[node] = d
        for nei, w in adj[node]:
            if nei not in dist:
                heapq.heappush(heap, (d + w, nei))
    return dist
```

Pattern 2: Minimum spanning tree (Prim and Kruskal)

Prim: grow the tree from one node, always add the cheapest edge leaving the tree (min-heap).

Kruskal: sort all edges by weight, add an edge if it joins two different groups (Union-Find).


```

def prim(points):                                # Min Cost to Connect ALL Points
    n = len(points)
    seen, total, heap = set(), 0, [(0, 0)]
    while len(seen) < n:
        cost, i = heapq.heappop(heap)
        if i in seen:
            continue
        seen.add(i); total += cost
        for j in range(n):
            if j not in seen:
                d = abs(points[i][0]-points[j][0]) + abs(points[i][1]-points[j][1])
                heapq.heappush(heap, (d, j))
    return total

def kruskal(n, edges):                            # edges: (w, u, v)
    total = 0
    for w, u, v in sorted(edges):
        if union(u, v):                          # union from Union-Find
            total += w
    return total

```

Pattern 3: Bellman-Ford (at most k stops)

Relax every edge $k+1$ times. Copy the array each round so one round uses at most one more edge.

```

def cheapest(n, flights, src, dst, k):
    prices = [float("inf")] * n
    prices[src] = 0
    for _ in range(k + 1):
        tmp = prices[:]                        # copy: use only last round values
        for u, v, p in flights:
            if prices[u] + p < tmp[v]:
                tmp[v] = prices[u] + p
        prices = tmp
    return -1 if prices[dst] == float("inf") else prices[dst]

```

Worked example: Network Delay Time

times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2. How long until all nodes get the signal?

1. adj: 2 -> (1,1), (3,1). 3 -> (4,1). heap = [(0, 2)], dist = {}.
2. Pop (0, 2). dist = {2:0}. Push (1, 1) and (1, 3).
3. Pop (1, 1). dist = {2:0, 1:1}. Node 1 has no edges.
4. Pop (1, 3). dist = {2:0, 1:1, 3:1}. Push (2, 4).
5. Pop (2, 4). dist = {2:0, 1:1, 3:1, 4:2}. Heap empty.
6. All 4 nodes reached. Answer = max(dist.values()) = 2.

```
def networkDelayTime(times, n, k):
    adj = defaultdict(list)
    for u, v, w in times:
        adj[u].append((v, w))
    dist, heap = {}, [(0, k)]
    while heap:
        d, node = heapq.heappop(heap)
        if node in dist:
            continue
        dist[node] = d
        for nei, w in adj[node]:
            if nei not in dist:
                heapq.heappush(heap, (d + w, nei))
    return max(dist.values()) if len(dist) == n else -1
```

Complexity cheat sheet

- Dijkstra with heap: $O(E \log V)$ time, $O(V + E)$ space.
- Prim with heap: $O(E \log V)$. On a full graph of n points, $O(n^2 \log n)$.
- Kruskal: $O(E \log E)$ for sorting, plus near $O(1)$ per union.
- Bellman-Ford with k rounds: $O(k * E)$.

Common mistakes

- Using Dijkstra with negative edge weights. It gives wrong answers.
- Not skipping nodes already finalised, so the same node is processed many times.
- In Bellman-Ford with k stops, updating the same array in place. This lets one round use many edges.
- Putting (node, dist) in the heap instead of (dist, node). The heap sorts by the first item.
- Forgetting to return -1 when some node is unreachable.

What to say in the interview

"The edges have non-negative weights, so I will use Dijkstra with a min-heap."

"I pop the closest node, finalise its distance, and push its neighbours."

"Because there is a limit of k stops, plain Dijkstra does not fit; I will use Bellman-Ford for $k+1$ rounds."

"Dijkstra runs in $O(E \log V)$."

Practice order

- Network Delay Time: the cleanest Dijkstra.
- Min Cost to Connect All Points: Prim with a heap.
- Swim in Rising Water: Dijkstra on a grid using max instead of sum.
- Cheapest Flights Within K Stops: Bellman-Ford with copied arrays.
- Reconstruct Itinerary and Alien Dictionary last: they are harder graph modelling problems.

Cheat sheet

Idea

Weighted graphs. Dijkstra = BFS with a min-heap on distance (no negative edges). MST = Prim (heap) or Kruskal (sort edges + Union-Find).

Dijkstra template

```
dist = {src: 0}
h = [(0, src)]
while h:
    d, u = heapq.heappop(h)
    if d > dist.get(u, inf): continue
    for v, w in adj[u]:
        if d + w < dist.get(v, inf):
            dist[v] = d + w
            heapq.heappush(h, (d + w, v))
```

Complexity

Dijkstra $O(E \log V)$. Bellman-Ford $O(V \cdot E)$ handles negative edges / k stops.

Also see

- MST animation: <https://visualgo.net/en/mst>
- Union-Find animation: <https://visualgo.net/en/ufds>

Visualise it

- <https://visualgo.net/en/sssp>

Videos

- [How Dijkstra's algorithm works](#) - Spanning Tree (English)
- [Dijkstra's algorithm](#) - Abdul Bari (English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Min Cost to Connect All Points	Medium	open	watch
2	Network Delay Time	Medium	open	watch
3	Cheapest Flights Within K Stops	Medium	open	watch

#	Problem	Level	LeetCode	Solution video
4	Reconstruct Itinerary	Hard	open	watch
5	Swim In Rising Water	Hard	open	watch
6	Alien Dictionary (Blind 75)	Hard	open	watch

Problem hints

Min Cost to Connect All Points (Medium)

Restate: Given points on a 2-D plane, connect all of them with the minimum total cost, where the cost of an edge is the Manhattan distance $|x1 - x2| + |y1 - y2|$.

Hint 1: This is a minimum spanning tree (MST). Use Prim or Kruskal.

Hint 2: Every pair of points can be connected, so the graph is dense (about $n^2 / 2$ edges). Prim with a simple array (no heap) runs in $O(n^2)$ and never builds the edge list.

Hint 3: (array-based Prim)

- Keep `dist[i]` = cheapest known cost to connect point `i` to the tree; start with `dist[0] = 0`, others infinity.
- Repeat `n` times: pick the unvisited point with the smallest `dist`.
- Add that `dist` to the total and mark the point visited.
- Update `dist[j]` for every unvisited `j` with the Manhattan distance to the new point, if smaller.

Complexity: $O(n^2)$ time, $O(n)$ space. (Kruskal: $O(n^2 \log n)$ time, $O(n^2)$ space for edges.)

Edge cases to test: one point gives 0; two points gives their distance; points on a straight line; duplicate points (cost 0 edge); negative coordinates like `[[-1000000, 0], [1000000, 0]]`.

Network Delay Time (Medium)

Restate: Given directed weighted edges (travel times) and a starting node `k`, return the time for a signal to reach all `n` nodes, or -1 if some node cannot be reached.

Hint 1: Single-source shortest path with non-negative weights means Dijkstra.

Hint 2: The answer is the LARGEST of the shortest distances, because the signal reaches all nodes only when the slowest one gets it.

Hint 3:

- Build an adjacency list `u -> (v, w)`.
- Min-heap starts with `(0, k)`.
- Pop the smallest; skip if already finalised; otherwise record its distance.
- Push `(d + w, v)` for each neighbour not yet finalised.
- If fewer than `n` nodes are finalised, return -1; else return the max distance.

Complexity: $O(E \log V)$ time, $O(V + E)$ space.

Edge cases to test: `n = 1` gives 0; a node with no incoming edge gives -1; two paths to a node where the longer-hop path is cheaper; edges pointing away from `k` only (directed, so reverse edges do not count); parallel edges with different weights.

Cheapest Flights Within K Stops (Medium)

Restate: Given flights with prices, find the cheapest price from `src` to `dst` using at most `k` stops (so at most `k + 1` flights), or -1 if none.

Hint 1: Bellman-Ford limited to `k + 1` rounds, or BFS level by level.

Hint 2: Plain Dijkstra can fail, because the cheapest path to a middle city might use too many stops. Each Bellman-Ford round adds at most one flight, so `k + 1` rounds respect the limit.

Hint 3:

- `prices[src] = 0`, all others infinity.
- Repeat `k + 1` times: copy `prices` to `temp`.
- For every flight `(u, v, p)`: if `prices[u]` is finite, set `temp[v] = min(temp[v], prices[u] + p)`.
- Set `prices = temp`.
- Return `prices[dst]` or -1 if still infinity.

Complexity: $O(k * E)$ time, $O(n)$ space.

Edge cases to test: `k = 0` (only direct flights); a cheap 2-stop path vs a costly 1-stop path with `k = 1` (must pick the costly one); `src == dst` gives 0; no route gives -1; a cycle in the flights.

Reconstruct Itinerary (Hard)

Restate: Given airline tickets `[from, to]`, use every ticket exactly once starting from "JFK", and return the itinerary that is smallest in alphabetical (lexical) order.

Hint 1: This is an Eulerian path (use every edge once). Use Hierholzer's algorithm with DFS.

Hint 2: Greedy "always take the smallest next airport" can get stuck in a dead end. Hierholzer fixes this: add an airport to the answer only AFTER all its outgoing tickets are used, then reverse the answer at the end. Dead ends naturally end up at the back.

Hint 3:

- Build `from -> list of destinations`, sorted (or use a min-heap per airport).
- DFS from "JFK": while the airport has unused tickets, take the smallest one, remove it, DFS into it.
- After the loop, append the airport to `route`.
- Return `route` reversed.

Complexity: $O(E \log E)$ time for sorting, $O(E)$ space.

Edge cases to test: one ticket `[["JFK", "ATL"]]`; the dead-end case `[["JFK", "KUL"], ["JFK", "NRT"], ["NRT", "JFK"]]` gives JFK, NRT, JFK, KUL; duplicate tickets (same from and to twice); a cycle back to JFK; check the output length is tickets + 1.

Swim In Rising Water (Hard)

Restate: In an $n \times n$ grid of heights, at time `t` you can swim through any cell with height $\leq t$. Return the least time to go from the top-left to the bottom-right.

Hint 1: Dijkstra on a grid, or binary search on `t` plus BFS.

Hint 2: The "cost" of a path is the MAXIMUM height on it, not the sum. Dijkstra still works if the priority of

a cell is $\max(\text{current cost}, \text{cell height})$.

Hint 3:

- Min-heap starts with $(\text{grid}[0][0], 0, 0)$; keep a visited set.
- Pop the cell with the smallest cost; if it is the target, return the cost.
- Skip it if visited; mark visited.
- For each in-bounds neighbour, push $(\max(\text{cost}, \text{grid}[\text{nr}][\text{nc}]), \text{nr}, \text{nc})$.

Complexity: $O(n^2 \log n)$ time, $O(n^2)$ space.

Edge cases to test: $[[0]]$ gives 0; a 2x2 grid $[[0, 2], [1, 3]]$ gives 3 (the target height itself counts); a high start cell; a spiral-shaped low path; the target being the highest cell.

Alien Dictionary (Hard)

Restate: Given words sorted in an unknown alien alphabet, return a valid order of the letters, or "" if the order is impossible.

Hint 1: Build a directed graph of letters, then topological sort.

Hint 2: Compare only ADJACENT words. The first position where they differ gives one edge $a \rightarrow b$.

Letters after that position tell you nothing. If a longer word comes before its own prefix (like "abc" before "ab"), the input is invalid.

Hint 3:

- Add every letter from every word as a node (even letters with no edges).
- For each adjacent pair, find the first different letter and add an edge.
- If no difference and the first word is longer, return "".
- Run topological sort (Kahn or DFS).
- If the order does not include all letters, there is a cycle: return "".

Complexity: $O(C)$ time where C is the total number of characters; $O(1)$ extra space for letters (at most 26 nodes, 26^2 edges).

Edge cases to test: one word $["z"]$ gives "z"; $["abc", "ab"]$ gives ""; a cycle $["z", "x", "z"]$ gives ""; $["wrt", "wrf", "er", "ett", "rftt"]$ gives "wertf"; duplicate adjacent words.

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Path with Minimum Effort	Medium	open	watch
2	Path with Maximum Probability	Medium	open	watch
3	Number of Good Paths	Hard	open	watch
4	Remove Max Number of Edges to Keep Graph Fully Traversable	Hard	open	watch
5	Find Critical and Pseudo Critical Edges in Minimum Spanning Tree	Hard	open	watch

Quiz

1. Why can Dijkstra fail with negative edge weights?
 - A) The heap cannot store negatives
 - B) A node finalised early could later get a smaller distance
 - C) It becomes $O(n^3)$
 - D) It cannot find any path
2. What does the heap store in Dijkstra?
 - A) (node, distance)
 - B) (distance, node)
 - C) Only nodes
 - D) Only edges
3. In Cheapest Flights Within K Stops, why do we copy prices each round?
 - A) To save memory
 - B) So one round extends paths by only one edge
 - C) Because lists are immutable
 - D) To sort the prices
4. Kruskal algorithm needs which helper structure?
 - A) Stack
 - B) Trie
 - C) Union-Find
 - D) Deque
5. What is the time complexity of Dijkstra with a binary heap?
 - A) $O(V + E)$
 - B) $O(E \log V)$
 - C) $O(V^3)$
 - D) $O(k * E)$
6. Dijkstra does NOT work with:
 - A) Negative edge weights
 - B) Directed graphs
 - C) Cycles
7. Dijkstra with a binary heap costs:
 - A) $O(E \log V)$
 - B) $O(V^3)$
 - C) $O(E)$
8. Kruskal's MST needs:
 - A) A trie
 - B) Union-Find
 - C) A stack

Answer key

1. **B** - A node finalised early could later get a smaller distance. Dijkstra assumes a popped distance is final, which negative edges can break.
2. **B** - (distance, node). heapq orders by the first element, so distance must come first.

3. **B** - So one round extends paths by only one edge. Without the copy, a round could chain several edges and break the stop limit.
4. **C** - Union-Find. Union-Find tells us if an edge would join two nodes already in the same tree.
5. **B** - $O(E \log V)$. Each edge can push to the heap, and each heap operation costs $O(\log V)$.
6. **A** - Negative edge weights.
7. **A** - $O(E \log V)$.
8. **B** - Union-Find.

More quiz

1. Which algorithm fits "minimum total wire to connect every house"?
 - A. Dijkstra
 - B. Minimum spanning tree (Prim or Kruskal)
 - C. Topological sort
 - D. BFS
2. In Swim In Rising Water, what is the cost of a path?
 - A. Sum of heights
 - B. Number of cells
 - C. Maximum height on the path
 - D. Minimum height on the path
3. Why is plain Dijkstra risky for Cheapest Flights Within K Stops?
 - A. Prices can be negative
 - B. The cheapest path to a middle city may use too many stops, and Dijkstra keeps only that one
 - C. Dijkstra cannot use a heap
 - D. The graph is undirected
4. In Alien Dictionary, which pair of words is INVALID input?
 - A. ["ab", "abc"]
 - B. ["abc", "ab"]
 - C. ["a", "b"]
 - D. ["ba", "bc"]
5. In Reconstruct Itinerary, when do you add an airport to the route?
 - A. When you first visit it
 - B. After all of its outgoing tickets are used
 - C. Only if it is "JFK"
 - D. When the heap is empty

More quiz: answer key

1. **B** - Connecting all nodes with minimum total edge weight is exactly a minimum spanning tree.
2. **C** - You must wait until the water covers the highest cell on your path, so the cost is the maximum.
3. **B** - Dijkstra keeps only the cheapest way to each city, but that way may already use all your stops. Bellman-Ford with $k + 1$ rounds tracks the stop limit.

4. **B** - A word must come after its own prefix. "abc" before "ab" breaks every possible alphabet.
5. **B** - This is post-order adding (Hierholzer). It makes dead ends go to the end of the reversed route.

Flashcards

- **Q:** Why is array-based Prim good for Min Cost to Connect All Points? — **A:** The graph is complete, so $O(n^2)$ Prim beats building and sorting all n^2 edges.
- **Q:** What is the Manhattan distance between (1, 2) and (4, 6)? — **A:** $|1 - 4| + |2 - 6| = 3 + 4 = 7$.
- **Q:** What is the final answer in Network Delay Time? — **A:** The maximum shortest distance, or -1 if any node is unreachable.
- **Q:** How many Bellman-Ford rounds for at most k stops? — **A:** $k + 1$ rounds, because k stops means $k + 1$ flights.
- **Q:** What is Hierholzer's algorithm used for? — **A:** Finding a path that uses every edge exactly once (Eulerian path).
- **Q:** What heap priority does Swim In Rising Water use? — **A:** $\max(\text{cost so far}, \text{height of the next cell})$.
- **Q:** Which words do you compare in Alien Dictionary? — **A:** Only adjacent words, at their first different letter.
- **Q:** How do you detect an impossible alien order? — **A:** A prefix problem (longer word first) or a cycle in the letter graph.
- **Q:** Which nodes must Alien Dictionary include even with no edges? — **A:** Every letter that appears in any word.
- **Q:** A good tester input for Reconstruct Itinerary? — **A:** A dead-end case where picking the smallest airport first gets stuck, like JFK->KUL, JFK->NRT, NRT->JFK.

DSA topic: 1-D Dynamic Programming

What it is

Dynamic programming (DP) means: solve small sub-problems once, save the answers, and build the big answer from them.

Think of climbing stairs. To know the ways to reach step 10, you only need the ways to reach step 9 and step 8.

1-D DP means the state is one number, like "index i " or "amount a ". The table is a single list.

DP = recursion + memory. If plain recursion repeats the same calls, DP makes it fast.

How to recognise it

- "Count the number of ways", "minimum cost", "maximum profit", "is it possible".
- The answer at position i depends on answers at smaller positions ($i-1$, $i-2$, ...).
- A greedy choice does not always work; you must compare options.
- Brute force recursion has overlapping calls (the same i computed many times).
- Classic names: Climbing Stairs, House Robber, Coin Change, Word Break, Longest Increasing Subsequence.

Pattern 1: The 4-step method

Use these 4 steps for every DP problem:

1. State: what does $dp[i]$ mean? Write it in one sentence.
 2. Transition: how is $dp[i]$ built from smaller states?
 3. Base case: the smallest states you know directly.
 4. Answer: which state is the final answer ($dp[n]$, $\max(dp)$, ...)?
- Then go: memoised recursion -> bottom-up table -> space optimisation.

```

from functools import lru_cache
def climb_memo(n):
    @lru_cache(None)
    def f(i):
        # ways to reach step i
        if i <= 1: return 1 # base case
        return f(i - 1) + f(i - 2)
    return f(n)

def climb_table(n):
    dp = [1] * (n + 1)
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

def climb_O1(n):
    a, b = 1, 1 # dp[i-2], dp[i-1]
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

```

Pattern 2: Unbounded choice (Coin Change style)

Use when you pick from a set of items again and again to reach a total.

```

def coinChange(coins, amount):
    dp = [float("inf")] * (amount + 1) # dp[a] = min coins for a
    dp[0] = 0
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != float("inf") else -1

```

Pattern 3: Look back over all earlier j (LIS style)

Use when $dp[i]$ can come from any earlier index $j < i$.

```

def lengthOfLIS(nums):
    dp = [1] * len(nums) # dp[i] = LIS ending at i
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

```

Worked example: House Robber

Houses in a row have money. You cannot rob two neighbours. Find the maximum money.

Input: `nums = [2, 7, 9, 3, 1]`.

State: $dp[i]$ = best money using houses $0..i$. Transition: $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$.

1. $dp[0] = 2$.
2. $dp[1] = \max(2, 7) = 7$.
3. $dp[2] = \max(dp[1]=7, dp[0]+9=11) = 11$.
4. $dp[3] = \max(dp[2]=11, dp[1]+3=10) = 11$.
5. $dp[4] = \max(dp[3]=11, dp[2]+1=12) = 12$.
6. Answer: $dp[4] = 12$ (rob houses 2, 9, 1).

We only use the last two values, so we keep two variables.

```
def rob(nums):
    prev2, prev1 = 0, 0          # dp[i-2], dp[i-1]
    for x in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + x)
    return prev1
```

Complexity cheat sheet

- Climbing Stairs, House Robber: $O(n)$ time, $O(1)$ space after optimisation.
- Coin Change: $O(\text{amount} * \text{len}(\text{coins}))$ time, $O(\text{amount})$ space.
- Word Break: $O(n^2)$ or $O(n * \text{maxWordLen})$ time.
- LIS: $O(n^2)$ simple DP, $O(n \log n)$ with patience sorting and bisect.
- Plain recursion without memo is usually $O(2^n)$.

Common mistakes

- Not writing the state meaning clearly, so the transition becomes wrong.
- Wrong base case (for example $dp[0] = 1$ vs 0 in counting vs min problems).
- Off-by-one: using a list of size n when you need $n + 1$.
- Returning $dp[-1]$ when the answer is $\max(dp)$ (like LIS).
- Using `float("inf")` and forgetting to convert it to -1 at the end.

What to say in the interview

"Brute force recursion repeats the same sub-problems, so I will use dynamic programming."

"My state is $dp[i]$, which means the best answer using the first i items."

"The transition is: take or skip, and I keep the better one."

"This is $O(n)$ time and I can reduce space to $O(1)$ by keeping only the last two values."

Practice order

- Climbing Stairs and Min Cost Climbing Stairs: learn the 4 steps.
- House Robber and House Robber II: take or skip choices.
- Coin Change and Decode Ways: counting and minimum with many choices.
- Word Break and Longest Increasing Subsequence: look back over many earlier states.

- Longest Palindromic Substring and Partition Equal Subset Sum: harder state design.

Cheat sheet

Idea

Overlapping subproblems. Write the recursion first, add memo, then convert to a bottom-up array, then to two variables if possible.

4 steps (say them in the interview)

1. State: $dp[i]$ means ...
2. Transition: $dp[i] = f(dp[i-1], dp[i-2], \dots)$
3. Base cases
4. Answer location

Template (House Robber)

```
prev2, prev1 = 0, 0
for x in nums:
    prev2, prev1 = prev1, max(prev1, prev2 + x)
return prev1
```

Complexity

Usually $O(n)$ time; $O(1)$ space after optimisation.

Visualise it

- <https://visualgo.net/en/recursion>

Videos

- [A beginner's guide to DP](#) - Matt Guest (English)
- [DP intro: memoization, tabulation](#) - take U forward (Hindi + English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Climbing Stairs (Blind 75)	Easy	open	watch
2	Min Cost Climbing Stairs	Easy	open	watch

#	Problem	Level	LeetCode	Solution video
3	House Robber (Blind 75)	Medium	open	watch
4	House Robber II (Blind 75)	Medium	open	watch
5	Longest Palindromic Substring (Blind 75)	Medium	open	watch
6	Palindromic Substrings (Blind 75)	Medium	open	watch
7	Decode Ways (Blind 75)	Medium	open	watch
8	Coin Change (Blind 75)	Medium	open	watch
9	Maximum Product Subarray (Blind 75)	Medium	open	watch
10	Word Break (Blind 75)	Medium	open	watch
11	Longest Increasing Subsequence (Blind 75)	Medium	open	watch
12	Partition Equal Subset Sum	Medium	open	watch

Problem hints

Climbing Stairs (Easy)

Restate: You climb a staircase of n steps, taking 1 or 2 steps at a time. Count the different ways to reach the top.

Hint 1: 1-D DP. It is the Fibonacci pattern.

Hint 2: Your last move to step i was either a 1-step from $i - 1$ or a 2-step from $i - 2$. So the ways add up.

State: $dp[i]$ = number of ways to reach step i .

Transition: $dp[i] = dp[i - 1] + dp[i - 2]$, with $dp[1] = 1$, $dp[2] = 2$.

Hint 3:

- Handle $n \leq 2$ directly (return n).
- Keep two variables for the last two values.
- Loop from 3 to n , computing the new value as their sum.
- Shift the two variables forward.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: $n = 1$ gives 1; $n = 2$ gives 2; $n = 3$ gives 3; $n = 5$ gives 8; $n = 45$ (largest usual input, checks you do not use slow plain recursion).

Min Cost Climbing Stairs (Easy)

Restate: Each step has a cost you pay when you stand on it. You may start at step 0 or 1 and climb 1 or 2 steps at a time. Return the minimum cost to reach the top (just past the last step).

Hint 1: 1-D DP, like Climbing Stairs, but with `min` instead of `+`.

Hint 2: The "top" is position n , one past the last index. Starting at step 0 or 1 is free.

State: $dp[i]$ = minimum cost to arrive at position i (before paying $cost[i]$).

Transition: $dp[i] = \min(dp[i - 1] + cost[i - 1], dp[i - 2] + cost[i - 2])$, with $dp[0] =$

$dp[1] = 0$. Answer is $dp[n]$.

Hint 3:

- Set $dp[0] = dp[1] = 0$.
- Loop i from 2 to n using the transition.
- Return $dp[n]$.
- You can keep only two variables.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: $[10, 15, 20]$ gives 15; $[1, 100, 1, 1, 1, 100, 1, 1, 100, 1]$ gives 6; two steps $[5, 3]$ gives 3; all zeros gives 0; all equal costs.

House Robber (Medium)

Restate: Houses in a row hold money. You cannot rob two neighbouring houses. Return the maximum money you can rob.

Hint 1: 1-D DP with a "take or skip" choice.

Hint 2: At house i , either skip it (keep the best up to $i - 1$) or rob it (its money plus the best up to $i - 2$).

State: $dp[i]$ = best money from houses $0..i$.

Transition: $dp[i] = \max(dp[i - 1], dp[i - 2] + \text{nums}[i])$.

Hint 3:

- Keep prev2 (best up to $i - 2$) and prev1 (best up to $i - 1$), both 0 at the start.
- For each house, $\text{cur} = \max(\text{prev1}, \text{prev2} + \text{money})$.
- Shift: $\text{prev2} = \text{prev1}$, $\text{prev1} = \text{cur}$.
- Return prev1 .

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: one house $[5]$ gives 5; $[2, 1, 1, 2]$ gives 4 (rob the first and last, skipping two in the middle); $[2, 7, 9, 3, 1]$ gives 12; all zeros; two houses $[1, 2]$ gives 2.

House Robber II (Medium)

Restate: Same as House Robber, but the houses are in a circle, so the first and last houses are neighbours.

Hint 1: Reuse House Robber twice.

Hint 2: You cannot rob both the first and the last house. So the answer is the better of: rob houses $0..n-2$, or rob houses $1..n-1$.

State: Same as House Robber, on each of the two ranges.

Transition: $dp[i] = \max(dp[i - 1], dp[i - 2] + \text{nums}[i])$ inside each range.

Hint 3:

- If there is only one house, return its value.
- Run House Robber on $\text{nums}[:-1]$.
- Run House Robber on $\text{nums}[1:]$.
- Return the larger result.

Complexity: $O(n)$ time, $O(1)$ space (if you pass index ranges instead of slices).

Edge cases to test: [5] gives 5 (both slices would be empty without the special case); [2, 3, 2] gives 3; [1, 2, 3, 1] gives 4; [1, 2] gives 2; [200, 3, 140, 20, 10] gives 340.

Longest Palindromic Substring (Medium)

Restate: Return the longest substring of s that reads the same forwards and backwards.

Hint 1: You can use a 2-D DP table, but "expand around the centre" is simpler and uses $O(1)$ space.

Hint 2: Every palindrome has a centre: one character (odd length) or the gap between two characters (even length). There are only $2n - 1$ centres.

State: (DP version) $dp[i][j]$ = True if $s[i..j]$ is a palindrome.

Transition: $dp[i][j] = (s[i] == s[j]) \text{ and } (j - i < 2 \text{ or } dp[i + 1][j - 1])$.

Hint 3: (expand-around-centre version)

- For each index i , expand from (i, i) and from $(i, i + 1)$.
- While both ends are in bounds and the characters match, move left down and right up.
- After expanding, the palindrome is $s[\text{left} + 1 : \text{right}]$.
- Keep the longest one.

Complexity: $O(n^2)$ time, $O(1)$ extra space.

Edge cases to test: one character "a"; "babad" gives "bab" or "aba" (accept either); "cbbd" gives "bb" (even length); the whole string is a palindrome "racecar"; all different letters "abcd" gives one letter.

Palindromic Substrings (Medium)

Restate: Count how many substrings of s are palindromes (the same substring at different positions counts separately).

Hint 1: Same expand-around-centre idea.

Hint 2: Each successful expansion step finds exactly one new palindrome. So count every step where the two ends match.

State: (DP version) $dp[i][j]$ = True if $s[i..j]$ is a palindrome; the answer is the number of True cells.

Transition: $dp[i][j] = (s[i] == s[j]) \text{ and } (j - i < 2 \text{ or } dp[i + 1][j - 1])$.

Hint 3:

- For each centre ($2n - 1$ of them, odd and even), expand outward.
- Add 1 to the count every time the ends match.
- Stop when they do not match or go out of bounds.
- Return the count.

Complexity: $O(n^2)$ time, $O(1)$ extra space.

Edge cases to test: "a" gives 1; "abc" gives 3; "aaa" gives 6; "abba" gives 6; a long string of one letter (worst case speed).

Decode Ways (Medium)

Restate: Letters map to numbers ($A = 1 \dots Z = 26$). Given a digit string, count how many ways it can be decoded back into letters.

Hint 1: 1-D DP over prefixes, very similar to Climbing Stairs.

Hint 2: The last letter uses either 1 digit (valid if it is not "0") or 2 digits (valid if they form 10 to 26). "0" can never stand alone.

State: $dp[i]$ = number of ways to decode the first i characters.

Transition: $dp[i] = (dp[i - 1] \text{ if } s[i - 1] \neq '0') + (dp[i - 2] \text{ if } 10 \leq \text{int}(s[i - 2 : i]) \leq 26)$, with $dp[0] = 1$.

Hint 3:

- Set $dp[0] = 1$ (empty prefix has one way).
- For i from 1 to n , add the one-digit case.
- For i from 2 to n , add the two-digit case.
- Return $dp[n]$; you can keep only two variables.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: "12" gives 2; "226" gives 3; "06" gives 0 (leading zero); "10" gives 1; "100" gives 0; "27" gives 1 (27 is not a letter).

Coin Change (Medium)

Restate: Given coin values (unlimited supply) and an amount, return the fewest coins that make the amount, or -1 if it is impossible.

Hint 1: 1-D DP over amounts (unbounded knapsack, minimum version).

Hint 2: Greedy (always take the biggest coin) is WRONG here. For coins [1, 3, 4] and amount 6, greedy gives 4+1+1 (3 coins), but 3+3 is 2 coins.

State: $dp[a]$ = fewest coins to make amount a .

Transition: $dp[a] = \min(dp[a - c] + 1)$ for every coin $c \leq a$, with $dp[0] = 0$.

Hint 3:

- Make dp of size amount + 1 filled with infinity; $dp[0] = 0$.
- For a from 1 to amount, for each coin, apply the transition.
- At the end, return $dp[\text{amount}]$, or -1 if it is still infinity.

Complexity: $O(\text{amount} * \text{number of coins})$ time, $O(\text{amount})$ space.

Edge cases to test: amount 0 gives 0; [2], 3 gives -1; [1], 0 gives 0; [1, 3, 4], 6 gives 2 (beats greedy); a coin larger than the amount [5], 3 gives -1.

Maximum Product Subarray (Medium)

Restate: Find the contiguous subarray with the largest product and return that product.

Hint 1: Like Kadane's algorithm, but track TWO values.

Hint 2: A negative number turns the smallest (most negative) product into the largest. So keep both the max and the min product of a subarray ending here.

State: $hi[i]$, $lo[i]$ = largest and smallest product of a subarray ending at i .

Transition: $hi[i] = \max(x, x * hi[i - 1], x * lo[i - 1])$, $lo[i] = \min(x, x * hi[i - 1], x * lo[i - 1])$, where $x = \text{nums}[i]$.

Hint 3:

- Start $hi = lo = \text{best} = \text{nums}[0]$.

- For each next x , compute new hi and lo from the OLD values (use temporaries).
- Update $best = \max(best, hi)$.
- Return $best$.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: $[2, 3, -2, 4]$ gives 6; $[-2, 0, -1]$ gives 0; $[-2, 3, -4]$ gives 24 (two negatives); a single negative $[-3]$ gives -3; zeros splitting the array $[0, 2, 0, 3]$.

Word Break (Medium)

Restate: Given a string and a dictionary of words, say whether the string can be split into a sequence of dictionary words (words can be reused).

Hint 1: 1-D DP over prefixes, with the dictionary in a set.

Hint 2: A prefix of length i is breakable if some shorter breakable prefix of length j is followed by a dictionary word $s[j:i]$.

State: $dp[i] = \text{True}$ if $s[:i]$ can be split into words.

Transition: $dp[i] = \text{any}(dp[j] \text{ and } s[j:i] \text{ in words for } j < i)$, with $dp[0] = \text{True}$.

Hint 3:

- Put the words in a set; note the max word length.
- $dp[0] = \text{True}$.
- For each i from 1 to n , check j from $i - 1$ down to $i - \text{maxLen}$.
- Stop early once $dp[i]$ becomes True.
- Return $dp[n]$.

Complexity: $O(n * L)$ checks where L is the max word length (each slice costs $O(L)$, so $O(n * L^2)$ in total); $O(n)$ space.

Edge cases to test: "leetcode", ["leet", "code"] gives True; "applepenapple", ["apple", "pen"] gives True (reuse); "catsanddog", ["cats", "dog", "sand", "and", "cat"] gives False; a single word equal to s ; "aaaaaab", ["a", "aa", "aaa"] gives False (slow for naive recursion).

Longest Increasing Subsequence (Medium)

Restate: Return the length of the longest strictly increasing subsequence (elements do not need to be next to each other).

Hint 1: The basic DP is $O(n^2)$. A faster $O(n \log n)$ method uses binary search.

Hint 2: Keep a list $tails$, where $tails[k]$ is the smallest possible last value of an increasing subsequence of length $k + 1$. This list is always sorted, so you can binary search it.

State: ($O(n^2)$ version) $dp[i] = \text{length of the LIS that ends at index } i$.

Transition: $dp[i] = 1 + \max(dp[j] \text{ for } j < i \text{ if } \text{nums}[j] < \text{nums}[i])$, or 1 if no such j .

Hint 3: ($O(n \log n)$ version)

- Start with empty $tails$.
- For each x , find the first position in $tails$ with value $\geq x$ (bisect_left).
- If there is none, append x ; otherwise replace that value with x .

- The answer is `len(tails)`. (Note: `tails` itself is not always a real subsequence.)

Complexity: $O(n \log n)$ time, $O(n)$ space.

Edge cases to test: `[10,9,2,5,3,7,101,18]` gives 4; `[0,1,0,3,2,3]` gives 4; `[7,7,7,7]` gives 1 (strictly increasing); one element gives 1; already sorted `[1,2,3,4]` gives 4.

Partition Equal Subset Sum (Medium)

Restate: Say whether the list can be split into two groups with equal sums.

Hint 1: 0/1 knapsack (subset sum).

Hint 2: If the total is odd, the answer is False. Otherwise, you only need ONE subset that sums to `total / 2`; the rest automatically forms the other half.

State: `dp[s]` = True if some subset of the numbers seen so far sums to `s`.

Transition: For each number `x`, for `s` from target DOWN to `x`: `dp[s] = dp[s] or dp[s - x]`.

Hint 3:

- If the total is odd, return False.
- `target = total // 2`, `dp[0] = True`.
- For each number, update `dp` from high `s` to low `s` (so each number is used once).
- Return `dp[target]`.

Complexity: $O(n * \text{target})$ time, $O(\text{target})$ space.

Edge cases to test: `[1, 5, 11, 5]` gives True; `[1, 2, 3, 5]` gives False; odd total `[1, 2]` gives False; one element `[2]` gives False; `[1, 1]` gives True.

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Triangle	Medium	open	watch
2	Delete And Earn	Medium	open	watch
3	Paint House	Medium	open	watch
4	Combination Sum IV	Medium	open	watch
5	Perfect Squares	Medium	open	watch
6	Check if There is a Valid Partition For The Array	Medium	open	watch

Quiz

1. What are the 4 steps of the DP method?
 - A) Sort, search, merge, return
 - B) State, transition, base case, answer
 - C) Push, pop, peek, check
 - D) Input, loop, print, end

2. In House Robber, what is the transition?
- A) $dp[i] = dp[i-1] + nums[i]$
 - B) $dp[i] = \max(dp[i-1], dp[i-2] + nums[i])$
 - C) $dp[i] = \min(dp[i-1], nums[i])$
 - D) $dp[i] = dp[i-2]$
3. In Coin Change, what is $dp[0]$?
- A) 1
 - B) infinity
 - C) 0
 - D) -1
4. What is the final answer in the $O(n^2)$ LIS DP?
- A) $dp[0]$
 - B) $dp[-1]$
 - C) $\max(dp)$
 - D) $\text{sum}(dp)$
5. Why can Climbing Stairs use $O(1)$ space?
- A) Because n is small
 - B) $dp[i]$ only needs $dp[i-1]$ and $dp[i-2]$
 - C) Python saves memory automatically
 - D) It uses recursion
6. DP is useful when subproblems are:
- A) Overlapping
 - B) Independent and unique
 - C) Sorted
7. Climbing Stairs recurrence:
- A) $dp[i] = dp[i-1] + dp[i-2]$
 - B) $dp[i] = 2 * dp[i-1]$
 - C) $dp[i] = i$
8. Top-down DP means:
- A) Recursion + memo
 - B) Loops only
 - C) Greedy

Answer key

1. **B** - State, transition, base case, answer. Defining these four things fully describes any DP solution.
2. **B** - $dp[i] = \max(dp[i-1], dp[i-2] + nums[i])$. Either skip house i , or rob it and add the best from two houses back.
3. **C** - 0. Zero coins are needed to make amount zero.
4. **C** - $\max(dp)$. $dp[i]$ is the LIS ending at i , and the best can end anywhere.
5. **B** - $dp[i]$ only needs $dp[i-1]$ and $dp[i-2]$. Only the last two values are needed, so two variables are enough.
6. **A** - Overlapping.
7. **A** - $dp[i] = dp[i-1] + dp[i-2]$.

More quiz

1. In Decode Ways, why is `dp[0] = 1` ?
 - A. Because "0" is a valid letter
 - B. The empty prefix has exactly one way to decode (do nothing), and it lets the two-digit case add correctly
 - C. It is a random starting value
 - D. Because $A = 1$
2. In Partition Equal Subset Sum, why do you loop `s` from high to low?
 - A. It is faster
 - B. So each number is used at most once in this round
 - C. To sort the numbers
 - D. To allow reusing numbers
3. Which pattern fits "fewest coins to make an amount"?
 - A. Greedy, biggest coin first
 - B. Unbounded knapsack DP with min
 - C. Two pointers
 - D. Backtracking that returns all answers
4. In Maximum Product Subarray, why keep the minimum product too?
 - A. To handle zeros only
 - B. A negative number can turn the minimum into the new maximum
 - C. To save memory
 - D. The minimum is the answer
5. How many centres does a string of length `n` have for expand-around-centre?
 - A. `n`
 - B. `n / 2`
 - C. `2n - 1`
 - D. `n^2`

More quiz: answer key

1. **B** - The empty start gives the base for both one-digit and two-digit steps. For "12", `dp[2] = dp[1] + dp[0] = 1 + 1 = 2`.
2. **B** - Going downward reads `dp[s - x]` values from before this number was added. Going upward would let one number count many times.
3. **B** - Greedy fails for coins like `[1, 3, 4]` and amount 6. DP tries every coin for every amount.
4. **B** - Multiplying the most negative product by a negative number gives the biggest positive product.
5. **C** - There are `n` single-character centres and `n - 1` gaps between characters.

Flashcards

- **Q:** Climbing Stairs is the same pattern as which famous sequence? — **A:** Fibonacci.
- **Q:** Where is the "top" in Min Cost Climbing Stairs? — **A:** Position n , one past the last step.
- **Q:** How does House Robber II break the circle? — **A:** Take the best of robbing houses $0..n-2$ and houses $1..n-1$.
- **Q:** What makes a two-digit chunk valid in Decode Ways? — **A:** Its value is between 10 and 26.
- **Q:** What does `tails[k]` mean in the $O(n \log n)$ LIS? — **A:** The smallest last value of any increasing subsequence of length $k + 1$.
- **Q:** Quick rejection in Partition Equal Subset Sum? — **A:** If the total sum is odd, return False.
- **Q:** What is the Word Break base case? — **A:** `dp[0] = True`, the empty prefix is always breakable.
- **Q:** How do you count palindromic substrings fast? — **A:** Expand around every centre and add 1 for each matching step.
- **Q:** What does Coin Change return when the amount is impossible? — **A:** -1 (the dp value stayed infinity).
- **Q:** A good tester input for Maximum Product Subarray? — **A:** `[-2, 3, -4]`, which needs two negatives to give 24.

DSA topic: 2-D Dynamic Programming

What it is

2-D DP is DP where the state needs two numbers, like (row, col) or (i in string1, j in string2).

Think of a city map with streets in a grid. The number of ways to reach a corner equals ways from the left plus ways from above.

The table is a grid $dp[i][j]$. We fill it row by row so the cells we need are already done.

Same 4-step method: state, transition, base case, answer. Then memo -> table -> space optimisation.

How to recognise it

- Two strings are compared: Longest Common Subsequence, Edit Distance, Interleaving String.
- A grid where you move only right/down: Unique Paths, Minimum Path Sum.
- Choice depends on index AND another value: knapsack (index, capacity), stock (day, holding).
- "Count ways" or "min/max" with two changing inputs.
- A 1-D idea does not capture the full situation.

Pattern 1: Grid paths

$dp[r][c]$ built from the top and left cells.

```
def uniquePaths(m, n):
    dp = [[1] * n for _ in range(m)]    # first row and col = 1
    for r in range(1, m):
        for c in range(1, n):
            dp[r][c] = dp[r-1][c] + dp[r][c-1]
    return dp[m-1][n-1]

def uniquePaths_1row(m, n):
    row = [1] * n                        # space optimised
    for _ in range(1, m):
        for c in range(1, n):
            row[c] += row[c-1]
    return row[-1]
```

Pattern 2: Two strings (LCS / Edit Distance)

$dp[i][j]$ = answer for first i chars of a and first j chars of b. Add an extra row and column for empty strings.

```
def minDistance(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1): dp[i][0] = i # delete all
    for j in range(n + 1): dp[0][j] = j # insert all
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j],      # delete
                                   dp[i][j-1],      # insert
                                   dp[i-1][j-1])      # replace

    return dp[m][n]
```

Pattern 3: Knapsack (index, capacity)

Use for Coin Change II and Target Sum. Loop items outside to count combinations, not orderings.

```
def change(amount, coins):
    dp = [0] * (amount + 1) # 2-D table squeezed into 1 row
    dp[0] = 1
    for c in coins: # item loop outside
        for a in range(c, amount + 1):
            dp[a] += dp[a - c]
    return dp[amount]
```

Worked example: Longest Common Subsequence

Find the length of the longest subsequence common to $a = \text{"abcde"}$ and $b = \text{"ace"}$.

State: $dp[i][j]$ = LCS of $a[:i]$ and $b[:j]$. If $a[i-1] == b[j-1]$, $dp[i][j] = dp[i-1][j-1] + 1$, else $\max(dp[i-1][j], dp[i][j-1])$.

Columns are "", a, c, e. Rows below:

1. Row "" : [0, 0, 0, 0]
2. Row a : [0, 1, 1, 1] (a matches a)
3. Row b : [0, 1, 1, 1] (no match, copy best)
4. Row c : [0, 1, 2, 2] (c matches c: $1 + 1$)
5. Row d : [0, 1, 2, 2]
6. Row e : [0, 1, 2, 3] (e matches e: $2 + 1$)
7. Answer: $dp[5][3] = 3$ ("ace").


```
def longestCommonSubsequence(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

Complexity cheat sheet

- Grid DP ($m \times n$): $O(m * n)$ time, $O(n)$ space with one row.
- Two-string DP: $O(m * n)$ time, $O(\min(m, n))$ space with rolling rows.
- Knapsack: $O(\text{items} * \text{capacity})$ time, $O(\text{capacity})$ space.
- Memoised recursion has the same time, but uses recursion stack space.

Common mistakes

- Creating the table with `[[0] * n] * m`. All rows are the same object.
- Forgetting the extra row/column for the empty string, then indexing `a[i]` instead of `a[i-1]`.
- Swapping loop order in Coin Change II, which counts orderings instead of combinations.
- Space optimising too early and overwriting a value you still need (keep a `prev` variable for the diagonal).
- Wrong base row: Edit Distance needs `dp[i][0] = i`, not 0.

What to say in the interview

"The answer depends on positions in both strings, so my state is `dp[i][j]`."

"If the characters match I take the diagonal; otherwise I take the best of the neighbours."

"I add an extra row and column for empty prefixes so base cases are simple."

"It is $O(m * n)$ time and I can reduce space to one row."

Practice order

- Unique Paths: the simplest 2-D table.
- Longest Common Subsequence: the core two-string template.
- Coin Change II and Target Sum: knapsack style counting.
- Best Time to Buy and Sell Stock with Cooldown: state machine (day, holding).
- Edit Distance, Interleaving String, Distinct Subsequences, then Burst Balloons and Regular Expression Matching last.

Cheat sheet

Idea

State has two parameters: $dp[i][j]$ (two strings, a grid, index + capacity).

Classic shapes

- Grid paths: $dp[r][c] = dp[r-1][c] + dp[r][c-1]$
- Two strings (LCS, edit distance): compare $s[i]$ and $t[j]$
- Knapsack: $dp[i][cap]$

LCS template

```
dp = [[0] * (n + 1) for _ in range(m + 1)]
for i in range(m - 1, -1, -1):
    for j in range(n - 1, -1, -1):
        if a[i] == b[j]:
            dp[i][j] = 1 + dp[i + 1][j + 1]
        else:
            dp[i][j] = max(dp[i + 1][j], dp[i][j + 1])
return dp[0][0]
```

Complexity

$O(m * n)$ time; space can often drop to one row.

Visualise it

- <https://visualgo.net/en/recursion>

Videos

- [Longest Common Subsequence](#) - Abdul Bari (English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Unique Paths (Blind 75)	Medium	open	watch
2	Longest Common Subsequence (Blind 75)	Medium	open	watch
3	Best Time to Buy And Sell Stock With Cooldown	Medium	open	watch
4	Coin Change II	Medium	open	watch

#	Problem	Level	LeetCode	Solution video
5	Target Sum	Medium	open	watch
6	Interleaving String	Medium	open	watch
7	Edit Distance	Medium	open	watch
8	Longest Increasing Path In a Matrix	Hard	open	watch
9	Distinct Subsequences	Hard	open	watch
10	Burst Balloons	Hard	open	watch
11	Regular Expression Matching	Hard	open	watch

Problem hints

Unique Paths (Medium)

Restate: A robot starts at the top-left of an $m \times n$ grid and can move only right or down. Count the different paths to the bottom-right corner.

Hint 1: 2-D grid DP.

Hint 2: You reach a cell only from the cell above or the cell on the left, so the path counts add. The whole first row and first column have exactly 1 path.

State: $dp[r][c]$ = number of paths from the start to cell (r, c) .

Transition: $dp[r][c] = dp[r - 1][c] + dp[r][c - 1]$, with the first row and first column equal to 1.

Hint 3:

- Keep one row of length n filled with 1.
- For each next row, for c from 1 to $n - 1$: $row[c] += row[c - 1]$.
- After $m - 1$ rows, return $row[n - 1]$.
- (Maths shortcut: the answer is $C(m + n - 2, m - 1)$.)

Complexity: $O(m * n)$ time, $O(n)$ space.

Edge cases to test: 1×1 gives 1; 1×5 gives 1; 3×2 gives 3; 3×7 gives 28; $m = n = 100$ (big numbers, fine in Python).

Longest Common Subsequence (Medium)

Restate: Return the length of the longest sequence of characters that appears in both strings in the same order (not necessarily next to each other).

Hint 1: 2-D DP over prefixes of both strings.

Hint 2: Look at the last characters. If they match, they extend the LCS of both shorter prefixes. If not, drop one character from either string and take the better result.

State: $dp[i][j]$ = LCS length of $a[:i]$ and $b[:j]$.

Transition: If $a[i - 1] == b[j - 1]$: $dp[i][j] = dp[i - 1][j - 1] + 1$; else $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$.

Hint 3:

- Make a $(\text{len}(a) + 1) \times (\text{len}(b) + 1)$ table of zeros.
- Fill it row by row with the transition.
- Return `dp[len(a)][len(b)]`.
- To save space, keep only the previous row.

Complexity: $O(m * n)$ time, $O(\min(m, n))$ space with two rows.

Edge cases to test: "abcde", "ace" gives 3; "abc", "def" gives 0; identical strings give their length; one empty string gives 0; "abc", "cba" gives 1.

Best Time to Buy And Sell Stock With Cooldown (Medium)

Restate: Given daily prices, make as many buy-then-sell trades as you like, but after you sell you must rest for one day before buying again. Return the maximum profit.

Hint 1: DP with states (a small state machine), one set of states per day.

Hint 2: Each day you are in one of three states: holding a stock, just sold today, or resting (no stock and free to buy).

State: `hold`, `sold`, `rest` = best profit at the end of day i in each state.

Transition: `hold` = $\max(\text{hold}, \text{rest} - \text{price})$; `sold` = `hold_prev` + `price`; `rest` = $\max(\text{rest}, \text{sold_prev})$.

Hint 3:

- Start: `hold` = -infinity (or `-prices[0]` after day 0), `sold` = 0, `rest` = 0.
- For each price, compute all three new values from the OLD values.
- Buying uses `rest`, not `sold`, which enforces the cooldown.
- Return `max(sold, rest)`.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: [1, 2, 3, 0, 2] gives 3; one day [1] gives 0; falling prices [5, 4, 3] gives 0; [1, 2, 4] gives 3 (one trade); [1, 4, 2, 7] gives 6 (cooldown stops trading on both rises).

Coin Change II (Medium)

Restate: Given coin values (unlimited supply) and an amount, count the number of different combinations that make the amount (order does not matter).

Hint 1: Unbounded knapsack, counting version.

Hint 2: The 2-D idea is "ways using the first i coin types". It shrinks to 1-D if coins are the outer loop and amounts the inner loop (low to high).

State: `dp[i][a]` = ways to make amount a using only the first i coin types.

Transition: `dp[i][a]` = `dp[i - 1][a]` + `dp[i][a - coin_i]` (skip this coin type, or use one more of it).

Hint 3:

- `dp` = [1] + [0] * amount (one way to make 0).
- For each coin, for a from coin to amount: `dp[a] += dp[a - coin]`.
- Return `dp[amount]`.

- Do not swap the loops; that counts orderings instead of combinations.

Complexity: $O(\text{number of coins} * \text{amount})$ time, $O(\text{amount})$ space.

Edge cases to test: amount = 5, [1, 2, 5] gives 4; amount = 3, [2] gives 0; amount = 0 gives 1; amount = 10, [10] gives 1; a coin bigger than the amount.

Target Sum (Medium)

Restate: Put a + or - sign in front of each number so the total equals target. Count how many sign choices work.

Hint 1: DP over (index, running sum), or turn it into a subset-sum count.

Hint 2: Let P be the sum of numbers with +. Then $P - (\text{total} - P) = \text{target}$, so $P = (\text{total} + \text{target}) / 2$. Now count subsets that sum to P . If $\text{total} + \text{target}$ is odd or $|\text{target}| > \text{total}$, the answer is 0.

State: $\text{dp}[i][s]$ = number of ways to reach sum s using the first i numbers (with signs).

Transition: $\text{dp}[i][s] = \text{dp}[i - 1][s - \text{nums}[i - 1]] + \text{dp}[i - 1][s + \text{nums}[i - 1]]$.

Hint 3: (subset-sum version)

- If $\text{abs}(\text{target}) > \text{total}$ or $(\text{total} + \text{target})$ is odd, return 0.
- $P = (\text{total} + \text{target}) // 2$, $\text{dp} = [1] + [0] * P$.
- For each number x , for s from P DOWN to x : $\text{dp}[s] += \text{dp}[s - x]$.
- Return $\text{dp}[P]$.

Complexity: $O(n * \text{total})$ time, $O(\text{total})$ space.

Edge cases to test: [1,1,1,1,1], 3 gives 5; [1], 1 gives 1; [1], 2 gives 0; zeros [0, 0, 1], 1 gives 4 (each 0 can be + or -); a negative target [1, 2], -1 gives 1.

Interleaving String (Medium)

Restate: Say whether s_3 can be built by mixing all characters of s_1 and s_2 while keeping each string's own order.

Hint 1: 2-D DP over how many characters you used from s_1 and from s_2 .

Hint 2: If you used i characters of s_1 and j of s_2 , the next character of s_3 is $s_3[i + j]$. It must come from s_1 or s_2 . First check $\text{len}(s_1) + \text{len}(s_2) == \text{len}(s_3)$.

State: $\text{dp}[i][j]$ = True if $s_3[:i + j]$ can be made from $s_1[:i]$ and $s_2[:j]$.

Transition: $\text{dp}[i][j] = (\text{dp}[i - 1][j] \text{ and } s_1[i - 1] == s_3[i + j - 1]) \text{ or } (\text{dp}[i][j - 1] \text{ and } s_2[j - 1] == s_3[i + j - 1])$, with $\text{dp}[0][0] = \text{True}$.

Hint 3:

- If the lengths do not add up, return False.
- Fill the first row and column (using only s_2 , or only s_1).
- Fill the rest with the transition.
- Return $\text{dp}[\text{len}(s_1)][\text{len}(s_2)]$.

Complexity: $O(m * n)$ time, $O(n)$ space with one row.

Edge cases to test: all three empty gives True; "aabcc", "dbbca", "aadbcbcac" gives True;

"aabcc", "dbbca", "aadbbbacc" gives False; length mismatch "a", "b", "a" gives False; repeated letters where greedy choice fails, like "aa", "ab", "aaba" (True).

Edit Distance (Medium)

Restate: Return the minimum number of single-character inserts, deletes or replaces needed to turn word1 into word2.

Hint 1: 2-D DP over prefixes, like LCS.

Hint 2: If the last characters match, no cost. Otherwise try all three operations and take the cheapest: delete (move up), insert (move left), replace (move diagonally).

State: $dp[i][j]$ = edits to turn $word1[:i]$ into $word2[:j]$.

Transition: If equal: $dp[i][j] = dp[i - 1][j - 1]$; else $dp[i][j] = 1 + \min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])$. Base: $dp[i][0] = i$, $dp[0][j] = j$.

Hint 3:

- Build a $(m + 1) \times (n + 1)$ table.
- Fill the base row and column.
- Fill the rest with the transition.
- Return $dp[m][n]$.

Complexity: $O(m * n)$ time, $O(n)$ space with two rows.

Edge cases to test: "horse", "ros" gives 3; "intention", "execution" gives 5; "", "abc" gives 3; equal strings give 0; "a", "b" gives 1.

Longest Increasing Path In a Matrix (Hard)

Restate: Return the length of the longest path in a grid where each step goes to a neighbour (up, down, left, right) with a strictly larger value.

Hint 1: DFS with memoisation (top-down DP).

Hint 2: Because values must strictly increase, a path can never come back to a cell. So there are no cycles and you do NOT need a visited set. Each cell's answer is fixed and can be cached.

State: $memo[r][c]$ = length of the longest increasing path that starts at (r, c) .

Transition: $memo[r][c] = 1 + \max(memo[nr][nc])$ over neighbours with a larger value (or 1 if none).

Hint 3:

- Make a memo grid of zeros.
- $dfs(r, c)$: if memo is set, return it.
- Try 4 neighbours; recurse only into larger values.
- Store and return $1 + best$.
- Answer = max of $dfs(r, c)$ over all cells.

Complexity: $O(R * C)$ time and space.

Edge cases to test: $[[1]]$ gives 1; all equal values give 1; $[[9,9,4],[6,6,8],[2,1,1]]$ gives 4; a snake-shaped increasing path; a single row $[[1, 2, 3]]$ gives 3.

Distinct Subsequences (Hard)

Restate: Count how many different ways you can delete characters from s to get exactly t .

Hint 1: 2-D DP over positions in s and t .

Hint 2: When $s[i] == t[j]$, you can either use this character of s for $t[j]$, or skip it. When they differ, you must skip it. An empty t can always be made in exactly one way.

State: $dp[i][j]$ = number of ways $s[i:]$ can form $t[j:]$.

Transition: If $s[i] == t[j]$: $dp[i][j] = dp[i + 1][j + 1] + dp[i + 1][j]$; else $dp[i][j] = dp[i + 1][j]$. Base: $dp[i][len(t)] = 1$, $dp[len(s)][j] = 0$ for $j < len(t)$.

Hint 3:

- Build a $(len(s) + 1) \times (len(t) + 1)$ table.
- Set the base cases.
- Fill from the bottom-right toward the top-left.
- Return $dp[0][0]$.

Complexity: $O(m * n)$ time, $O(n)$ space with one row.

Edge cases to test: "rabbbit", "rabbit" gives 3; "babgbag", "bag" gives 5; t longer than s gives 0; t empty gives 1; "aaa", "a" gives 3.

Burst Balloons (Hard)

Restate: Bursting balloon i earns $left * nums[i] * right$, where $left$ and $right$ are its current neighbours (1 past the ends). Return the maximum coins from bursting all balloons.

Hint 1: Interval DP.

Hint 2: Think about the LAST balloon to burst in a range, not the first. When k is last in the open range (l, r) , its neighbours are fixed: $nums[l]$ and $nums[r]$. The left and right parts become independent.

State: $dp[l][r]$ = max coins from bursting all balloons strictly between l and r (after padding $nums$ with 1 at both ends).

Transition: $dp[l][r] = \max(nums[l] * nums[k] * nums[r] + dp[l][k] + dp[k][r])$ for $l < k < r$.

Hint 3:

- Pad: $nums = [1] + nums + [1]$.
- Loop over range length from 2 up to $len(nums) - 1$.
- For each (l, r) , try every k in between with the transition.
- Return $dp[0][len(nums) - 1]$.

Complexity: $O(n^3)$ time, $O(n^2)$ space.

Edge cases to test: $[3, 1, 5, 8]$ gives 167; $[1, 5]$ gives 10; one balloon $[7]$ gives 7; balloons with 0 value $[0, 5]$; all ones $[1, 1, 1]$ gives 3.

Regular Expression Matching (Hard)

Restate: Say whether the whole string s matches pattern p , where $.$ matches any one character and $*$ means "zero or more of the character before it".

Hint 1: 2-D DP (or recursion with memo) over positions in s and p .

Hint 2: Look one character ahead in the pattern. If $p[j + 1]$ is $*$, you have two choices: use zero copies

(skip `p[j]` and `*`), or if the current characters match, use one copy and stay on the same pattern position.

State: `dp[i][j] = True` if `s[i:]` matches `p[j:]`.

Transition: Let `first = i < len(s)` and `p[j] in (s[i], '.')`. If `j + 1 < len(p)` and `p[j + 1] == '*'`: `dp[i][j] = dp[i][j + 2]` or `(first and dp[i + 1][j])`. Else: `dp[i][j] = first and dp[i + 1][j + 1]`.

Hint 3:

- Base: `dp[len(s)][len(p)] = True`.
- Fill from the end backwards (i from `len(s)` down to 0, j from `len(p) - 1` down to 0).
- Note i goes up to `len(s)`: an empty rest of s can still match patterns like `a*`.
- Return `dp[0][0]`.

Complexity: $O(m * n)$ time and space.

Edge cases to test: `"aa"`, `"a"` gives False; `"aa"`, `"a*"` gives True; `"ab"`, `".*"` gives True; `"aab"`, `"c*a*b"` gives True; `"mississippi"`, `"mis*is*p*"` gives False; `"`, `"a*b*"` gives True.

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Unique Paths II	Medium	open	watch
2	Longest Palindromic Subsequence	Medium	open	watch
3	Last Stone Weight II	Medium	open	watch
4	Stone Game	Medium	open	watch
5	Minimum Path Sum	Medium	open	watch
6	Maximal Square	Medium	open	watch

Quiz

1. Why is `[[0] * n] * m` a bug for a DP table?
 - A) It is too slow
 - B) All m rows point to the same list
 - C) It creates a 1-D list
 - D) Python does not allow it
2. In LCS, what do we do when `a[i-1] == b[j-1]`?
 - A) `dp[i][j] = 0`
 - B) `dp[i][j] = dp[i-1][j-1] + 1`
 - C) `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`
 - D) `dp[i][j] = dp[i][j-1]`
3. In Edit Distance, what is `dp[i][0]`?

- A) 0
 - B) i
 - C) 1
 - D) infinity
4. In Coin Change II, why are coins in the outer loop?
- A) It is faster
 - B) So each combination is counted once, not each order
 - C) To avoid negative indexes
 - D) No reason
5. What is the time complexity of Unique Paths on an $m \times n$ grid with DP?
- A) $O(m + n)$
 - B) $O(m * n)$
 - C) $O(2^{(m+n)})$
 - D) $O(\log(m * n))$
6. LCS of strings of length m and n costs:
- A) $O(m + n)$
 - B) $O(m * n)$
 - C) $O(2^n)$
7. Unique Paths in a grid: $dp[r][c] =$
- A) $dp[r-1][c] + dp[r][c-1]$
 - B) $dp[r-1][c-1]$
 - C) $r * c$
8. 2-D DP space can often be reduced to:
- A) One row
 - B) Zero
 - C) A stack

Answer key

1. **B** - All m rows point to the same list. Changing one row changes all rows; use a list comprehension instead.
2. **B** - $dp[i][j] = dp[i-1][j-1] + 1$. A matching character extends the LCS of both shorter prefixes by one.
3. **B** - i . Turning i characters into an empty string needs i deletions.
4. **B** - So each combination is counted once, not each order. With coins outside, coins are added in a fixed order, so orderings are not counted twice.
5. **B** - $O(m * n)$. Each cell is computed once in constant time.
6. **B** - $O(m * n)$.
7. **A** - $dp[r-1][c] + dp[r][c-1]$.
8. **A** - One row.

More quiz

1. In Burst Balloons, why do we choose the LAST balloon to burst in a range?

- A. It is the biggest balloon
 - B. Its neighbours are then fixed (the range ends), so the two sides become independent
 - C. It makes the code shorter
 - D. The first balloon cannot be chosen
2. Which pattern fits "count ways to turn s into t by deleting characters"?
- A. Sliding window
 - B. 2-D DP over positions in s and t
 - C. Greedy
 - D. Heap
3. Target Sum has `nums = [1, 2, 3]` and target 7. What is the answer?
- A. 1
 - B. 3
 - C. 0
 - D. 2
4. In the stock cooldown problem, what enforces the one-day rest after selling?
- A. You buy from the `rest` state, not from the `sold` state
 - B. You sort the prices
 - C. You skip every second day
 - D. You sell only on even days
5. In Longest Increasing Path In a Matrix, why is no visited set needed?
- A. The grid is small
 - B. Strictly increasing steps can never return to a cell, so there are no cycles
 - C. Memo is the visited set
 - D. DFS never repeats

More quiz: answer key

1. **B** - When k is the last balloon in (l, r), it is next to `nums[l]` and `nums[r]` when it bursts. This splits the problem into two smaller ranges.
2. **B** - Each state is "where am I in s and in t", and each step decides to use or skip the current character of s.
3. **C** - The total is 6, which is less than 7, so no sign choice can reach 7.
4. **A** - After a sale you move to `sold`; you must move to `rest` for a day before buying again.
5. **B** - Values strictly grow along a path, so a path is a chain with no loops. Memo only saves time; it is not needed for correctness.

Flashcards

- **Q:** What are the first row and column in Unique Paths? — **A:** All 1, because there is only one straight path to each of them.
- **Q:** What are the three states in stock with cooldown? — **A:** Hold, sold (today), rest (free to buy).
- **Q:** Coin Change II: which loop must be outside? — **A:** The coins loop, so combinations are not counted in different orders.

- **Q:** How does Target Sum become subset sum? — **A:** Count subsets with sum $P = (total + target) / 2$.
- **Q:** First check in Interleaving String? — **A:** $len(s1) + len(s2)$ must equal $len(s3)$.
- **Q:** Edit Distance base case $dp[i][0]$? — **A:** i , because you delete all i characters.
- **Q:** Distinct Subsequences base case for an empty t ? — **A:** 1 way (delete everything).
- **Q:** What padding does Burst Balloons use? — **A:** Add a 1 at both ends of the array.
- **Q:** In regex matching, what are the two choices for x^* ? — **A:** Use zero copies (skip two pattern characters) or use one copy and stay on the same pattern position.
- **Q:** A tester input that breaks a naive regex matcher? — **A:** An empty s with pattern a^*b^* , which must return True.

DSA topic: Greedy

What it is

Greedy means: at each step, take the choice that looks best right now, and never go back.

Think of giving change with Indian coins: always give the biggest coin that fits. For these coins it gives the fewest coins.

Greedy is fast (often $O(n)$), but it is only correct when a local best choice leads to a global best answer.

In interviews, you should give a short reason (or a counter-example check) why greedy works.

How to recognise it

- "Maximum / minimum" with a simple rule and large n (10^5), where DP would be too slow.
- You can scan once and keep a running best (max subarray, jump reach).
- Sorting first makes the choice obvious (meetings, hand of straights).
- "Can you reach the end?", "minimum jumps", "gas station circuit".
- If small examples break your greedy rule, switch to DP.

Pattern 1: Running best (Kadane)

Keep the best sum ending here. If it goes negative, drop it and start again.

```
def maxSubArray(nums):
    best = cur = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)    # extend or restart
        best = max(best, cur)
    return best
```

Pattern 2: Farthest reach

Track the farthest index you can reach. Used in Jump Game I and II.

```
def jump(nums):
    # minimum jumps to last index
    jumps, end, far = 0, 0, 0
    for i in range(len(nums) - 1):
        far = max(far, i + nums[i])
        if i == end:    # must jump now
            jumps += 1
            end = far
    return jumps
```

Pattern 3: Sort then pick

Sort by a key, then make the simple choice. Used in Hand of Straights and many interval problems.

```
from collections import Counter
def isNStraightHand(hand, k):
    count = Counter(hand)
    for x in sorted(count):
        need = count[x]
        if need:
            for v in range(x, x + k):
                if count[v] < need:
                    return False
                count[v] -= need
    return True
```

Worked example: Jump Game

nums[i] is the max jump length from i. Can you reach the last index?

Input: nums = [2, 3, 1, 1, 4]. We go backwards and move the goal left.

1. goal = 4 (last index).
 2. i = 3: 3 + nums[3] = 4 >= goal, so goal = 3.
 3. i = 2: 2 + 1 = 3 >= 3, goal = 2.
 4. i = 1: 1 + 3 = 4 >= 2, goal = 1.
 5. i = 0: 0 + 2 = 2 >= 1, goal = 0.
 6. goal == 0, so return True.
- For [3, 2, 1, 0, 4], goal stays 4 because index 3 has 0, so the answer is False.

```
def canJump(nums):
    goal = len(nums) - 1
    for i in range(len(nums) - 2, -1, -1):
        if i + nums[i] >= goal:
            goal = i
    return goal == 0
```

Complexity cheat sheet

- Kadane, Jump Game, Jump Game II, Gas Station: $O(n)$ time, $O(1)$ space.
- Sort-then-pick: $O(n \log n)$ time.
- Hand of Straights with Counter and sorting: $O(n \log n)$.
- Compare: a DP for the same problem is often $O(n^2)$.

Common mistakes

- Using greedy without checking it on small tricky examples.

- Kadane with best = 0 at the start; it fails when all numbers are negative.
- In Jump Game II, looping to the last index and counting one extra jump.
- Gas Station: forgetting to check that total gas $\geq$ total cost first.
- Not sorting when the greedy rule needs a sorted order.

What to say in the interview

"A greedy choice works here because taking the best local option never blocks a better answer later."

"I tried small examples and could not find a counter-example."

"I scan once and keep a running value, so it is $O(n)$ time and $O(1)$ space."

"If greedy failed, I would fall back to DP."

Practice order

- Maximum Subarray: Kadane is the base pattern.
- Jump Game, then Jump Game II: farthest reach idea.
- Gas Station: running sum with reset.
- Hand of Straights and Merge Triplets to Form Target: sort or filter then pick.
- Partition Labels and Valid Parenthesis String: last-occurrence and range tracking.

Cheat sheet

Idea

Make the locally best choice and never look back. Only correct when you can argue why (exchange argument). Often sort first.

Use it when

- Jump Game: track the farthest reachable index
- Maximum Subarray (Kadane): drop a negative running sum
- Gas Station, Partition Labels

Kadane template

```
best = cur = nums[0]
for x in nums[1:]:
    cur = max(x, cur + x)
    best = max(best, cur)
return best
```

Interview tip

Say: "Greedy works here because ..." - interviewers want the reason, not just code.

Visualise it

- <https://visualgo.net/en/sorting>

Videos

- [Greedy algorithms explained](#) - Tech With Tim (English)
- [Greedy method introduction](#) - Abdul Bari (English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Maximum Subarray (Blind 75)	Medium	open	watch
2	Jump Game (Blind 75)	Medium	open	watch
3	Jump Game II	Medium	open	watch
4	Gas Station	Medium	open	watch
5	Hand of Straights	Medium	open	watch
6	Merge Triplets to Form Target Triplet	Medium	open	watch
7	Partition Labels	Medium	open	watch
8	Valid Parenthesis String	Medium	open	watch

Problem hints

Maximum Subarray (Medium)

Restate: Find the contiguous subarray (at least one number) with the largest sum and return that sum.

Hint 1: Kadane's algorithm: a greedy single pass.

Hint 2: If the running sum so far is negative, it can only hurt what comes next. Drop it and start fresh at the current number.

Hint 3:

- Start `cur = best = nums[0]`.
- For each next `x`: `cur = max(x, cur + x)`.
- `best = max(best, cur)`.
- Return `best`.
- (Follow-up: divide and conquer also works in $O(n \log n)$.)

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: `[-2,1,-3,4,-1,2,1,-5,4]` gives 6; all negative `[-3, -1, -2]` gives -1; one element `[-5]`; `[5, 4, -1, 7, 8]` gives 23 (the whole array); zeros `[0, 0]` gives 0.

Jump Game (Medium)

Restate: Each number is the maximum jump length from that position. Say whether you can reach the last index starting from index 0.

Hint 1: Greedy. Track the farthest index you can reach so far.

Hint 2: If you ever stand on an index beyond the farthest reachable index, you are stuck. (The lesson shows the "move the goal backwards" version; this is the forward version.)

Hint 3:

- `reach = 0`.
- For each index `i`: if `i > reach`, return False.
- `reach = max(reach, i + nums[i])`.
- If `reach >= last index`, return True.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: `[0]` gives True (already at the end); `[2, 3, 1, 1, 4]` gives True; `[3, 2, 1, 0, 4]` gives False; `[2, 0, 0]` gives True; `[0, 1]` gives False.

Jump Game II (Medium)

Restate: Same jumps as Jump Game (the end is always reachable). Return the minimum number of jumps to reach the last index.

Hint 1: Greedy that works like BFS by levels.

Hint 2: All indexes reachable with `j` jumps form a window. The next window ends at the farthest point any index in the current window can reach. Each window is one jump.

Hint 3:

- `jumps = 0`, `curEnd = 0`, `farthest = 0`.
- Loop `i` from 0 to `n - 2` (do not jump from the last index).
- `farthest = max(farthest, i + nums[i])`.
- When `i == curEnd`: `jumps += 1`, `curEnd = farthest`.
- Return `jumps`.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: `[0]` gives 0; `[2, 3, 1, 1, 4]` gives 2; `[1, 2]` gives 1; `[1, 1, 1, 1]` gives 3; a big first jump `[10, 1, 1]` gives 1.

Gas Station (Medium)

Restate: Gas stations are in a circle. Station `i` gives `gas[i]` and driving to the next costs `cost[i]`. Return the start index that lets you go all the way round, or -1 (the answer is unique if it exists).

Hint 1: Greedy single pass with a running tank.

Hint 2: If the tank goes negative after station `i`, then NO station from your current start up to `i` can be the

answer. So jump the start to $i + 1$. If total gas $\geq$ total cost, the last start you pick works.

Hint 3:

- If `sum(gas) < sum(cost)`, return -1.
- `tank = 0, start = 0`.
- For each i : `tank += gas[i] - cost[i]`.
- If `tank < 0`: `start = i + 1, tank = 0`.
- Return `start`.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: `gas = [1,2,3,4,5]`, `cost = [3,4,5,1,2]` gives 3; `gas = [2,3,4]`, `cost = [3,4,3]` gives -1; one station `[5]`, `[4]` gives 0; total gas exactly equal to total cost; the answer is the last index.

Hand of Straights (Medium)

Restate: Say whether the cards can be split into groups of size `groupSize`, where each group is consecutive numbers.

Hint 1: Count cards with a hash map, then build groups from the smallest card up.

Hint 2: The smallest remaining card MUST start a group (nothing smaller can include it). So greedily build `x, x+1, ..., x+groupSize-1` from it.

Hint 3:

- If `len(hand) % groupSize != 0`, return False.
- Count every card.
- Go through distinct cards in sorted order (or use a min-heap).
- For a card x with count $c > 0$, every card from x to $x + \text{groupSize} - 1$ must have count $\geq c$; subtract c from each.
- If any count is too small, return False; else True.

Complexity: $O(n \log n)$ time for sorting, $O(n)$ space.

Edge cases to test: `[1,2,3,6,2,3,4,7,8]`, 3 gives True; `[1,2,3,4,5]`, 4 gives False; `groupSize = 1` always True; duplicates `[1,1,2,2,3,3]`, 3 gives True; a gap `[1,2,4]`, 3 gives False.

Merge Triplets to Form Target Triplet (Medium)

Restate: Merging two triplets takes the max of each position. Say whether you can reach the target triplet by merging some of the given triplets.

Hint 1: Greedy filter.

Hint 2: Any triplet with a value bigger than the target in ANY position is poison: using it would overshoot forever, because max never decreases. Throw those away. Then the merge of all remaining triplets is the best you can do.

Hint 3:

- Keep three flags, one for each position, all False.
- For each triplet: skip it if any value is greater than the target value in that position.

- Otherwise, for each position where the value equals the target, set that flag True.
- Return True if all three flags are True.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: `[[2,5,3],[1,8,4],[1,7,5]], [2,7,5]` gives True; `[[3,4,5],[4,5,6]], [3,2,5]` gives False; one triplet equal to the target gives True; each position matched by a different triplet; a triplet that matches two positions but overshoots the third (must be skipped).

Partition Labels (Medium)

Restate: Split the string into as many parts as possible so that each letter appears in only one part, and return the part sizes.

Hint 1: Greedy with the last index of each letter.

Hint 2: Once a part contains a letter, it must stretch to that letter's last position. Keep extending the end; when your index reaches the end, close the part.

Hint 3:

- Store `last[ch]` = last index of each letter.
- `start = end = 0`.
- For each `i`: `end = max(end, last[s[i]])`.
- If `i == end`: record `end - start + 1`, set `start = i + 1`.

Complexity: $O(n)$ time, $O(1)$ space (26 letters).

Edge cases to test: `"ababcbacadefegdehijhklij"` gives `[9, 7, 8]`; `"eccbbbbbdec"` gives `[10]`; one letter `"a"` gives `[1]`; all different `"abc"` gives `[1, 1, 1]`; all the same `"aaaa"` gives `[4]`.

Valid Parenthesis String (Medium)

Restate: The string has `"(", ")", and ""`, where `""` can be `"(", ")", or empty`. Say whether it can be a valid balanced string.

Hint 1: Greedy with a range of possible open-bracket counts.

Hint 2: Track `lo` and `hi`: the smallest and largest number of open brackets possible so far. `"("` raises both; `)"` lowers both; `"*"` lowers `lo` and raises `hi`. Never let `lo` go below 0.

Hint 3:

- `lo = hi = 0`.
- For each character, update `lo` and `hi` as above.
- If `hi < 0`, return False (too many `)`).
- `lo = max(lo, 0)`.
- At the end, return `lo == 0`.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: `"()"` gives True; `"(*)"` gives True; `"(*)"` gives True; `")("` gives False; `"((*)"` gives False; `"*"` gives True.

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Maximum Sum Circular Subarray	Medium	open	watch
2	Longest Turbulent Array	Medium	open	watch
3	Jump Game VII	Medium	open	watch
4	Minimize Maximum of Array	Medium	open	watch
5	Dota2 Senate	Medium	open	watch
6	Maximum Points You Can Obtain From Cards	Medium	open	watch

Quiz

- In Kadane algorithm, what does $cur = \max(x, cur + x)$ mean?
 - A) Always add x
 - B) Either extend the current subarray or start a new one at x
 - C) Reset to zero
 - D) Take the maximum element
- Why initialise Kadane best with `nums[0]` and not 0?
 - A) It is faster
 - B) If all numbers are negative, the answer is negative
 - C) Python needs it
 - D) To avoid an index error
- In Gas Station, if $\text{sum}(\text{gas}) < \text{sum}(\text{cost})$, what is the answer?
 - A) 0
 - B) -1
 - C) The last index
 - D) Any index
- What is the time complexity of the greedy Jump Game solution?
 - A) $O(n)$
 - B) $O(n \log n)$
 - C) $O(n^2)$
 - D) $O(2^n)$
- What should you do if you find a counter-example to your greedy rule?
 - A) Ignore it
 - B) Switch to another approach like DP
 - C) Sort the input again
 - D) Use a stack
- Kadane's algorithm finds:
 - A) Max subarray sum
 - B) Shortest path
 - C) Sorted order
- A greedy solution must be:
 - A) Recursive

- B) Justified (why local best is global best)
 - C) $O(n^2)$
8. Jump Game greedy tracks:
- A) Farthest reachable index
 - B) Number of zeros
 - C) Sum of array

Answer key

1. **B** - Either extend the current subarray or start a new one at x. If the running sum hurts, starting fresh at x is better.
2. **B** - If all numbers are negative, the answer is negative. Starting at 0 would wrongly return 0 for an all-negative array.
3. **B** - -1. There is not enough total gas to finish the circuit from anywhere.
4. **A** - $O(n)$. We scan the array once and move the goal pointer.
5. **B** - Switch to another approach like DP. One counter-example proves greedy is wrong for that problem.
6. **A** - Max subarray sum.
7. **B** - Justified (why local best is global best).
8. **A** - Farthest reachable index.

More quiz

1. In Gas Station, the tank goes negative after station 4 when you started at station 1. What do you do?
 - A. Try starting at station 2
 - B. Start again from station 5
 - C. Return -1
 - D. Go back to station 0
2. Which pattern fits "each letter must appear in only one part; make as many parts as possible"?
 - A. Binary search
 - B. Greedy with the last index of each letter
 - C. 2-D DP
 - D. Union-Find
3. In Merge Triplets, a triplet is `[3, 1, 9]` and the target is `[3, 5, 7]`. What do you do with it?
 - A. Use it, because it matches position 0
 - B. Skip it, because $9 > 7$ would overshoot
 - C. Use only its first value
 - D. Return False
4. In Valid Parenthesis String, what do `lo` and `hi` mean?
 - A. The first and last index
 - B. The smallest and largest possible number of open brackets so far
 - C. Number of stars and brackets

- D. Left and right pointers
5. In Jump Game II, when do you add one jump?
- A. At every index
 - B. When i reaches the end of the current window
 - C. When nums[i] is 0
 - D. Only at the last index

More quiz: answer key

1. **B** - Any start between 1 and 4 also fails, because it would arrive at station 4 with even less gas. So the next possible start is 5.
2. **B** - A part must reach the last appearance of every letter inside it, so you extend the end greedily and cut as soon as you can.
3. **B** - Merging takes the max, so 9 would stay in position 2 forever and you could never reach 7.
4. **B** - Stars make the count uncertain, so you keep the full range of possible open counts.
5. **B** - Each window is all positions reachable with the same number of jumps, like one BFS level.

Flashcards

- **Q:** Kadane's rule in one line? — **A:** `cur = max(x, cur + x)`; drop the past if it is negative.
- **Q:** Jump Game forward check? — **A:** If index i is greater than the farthest reach so far, return False.
- **Q:** Jump Game II is like which graph algorithm? — **A:** BFS by levels, where each level is one jump.
- **Q:** Quick -1 check in Gas Station? — **A:** If total gas is less than total cost.
- **Q:** First check in Hand of Straights? — **A:** The number of cards must be divisible by groupSize.
- **Q:** Why does the smallest card start a group in Hand of Straights? — **A:** No smaller card exists to put it in the middle of a group.
- **Q:** Which triplets do you throw away in Merge Triplets? — **A:** Any triplet with a value bigger than the target in any position.
- **Q:** When do you close a part in Partition Labels? — **A:** When the current index equals the farthest last index seen in this part.
- **Q:** End condition in Valid Parenthesis String? — **A:** Return True if `lo == 0`.
- **Q:** How do you test that a greedy rule is correct? — **A:** Try small hand-made counter-examples; one failure means you need another approach such as DP.

DSA topic: Intervals

What it is

An interval is a range with a start and an end, like a meeting from 10 to 11.

Interval problems ask: merge overlapping ranges, insert a new range, remove the fewest ranges, or count how many rooms you need.

Think of a calendar. You sort meetings by start time and walk through them in order.

The key trick: sort first. After sorting, you only compare each interval with the last one you kept.

How to recognise it

- Input is a list of pairs [start, end].
- Words like "meetings", "overlap", "merge", "schedule", "rooms", "booking".
- "Minimum number of intervals to remove so the rest do not overlap".
- "Can a person attend all meetings?" or "how many rooms are needed?".
- Queries asking "which smallest interval contains point q".

Pattern 1: Sort by start and merge

Two intervals overlap if $\text{next.start} \leq \text{last.end}$.

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    res = [intervals[0]]
    for s, e in intervals[1:]:
        if s <= res[-1][1]:           # overlap
            res[-1][1] = max(res[-1][1], e)
        else:
            res.append([s, e])
    return res
```

Pattern 2: Sort by end and keep the earliest finish

For "remove the fewest" or "max non-overlapping", always keep the interval that ends first.

```
def eraseOverlapIntervals(intervals):
    intervals.sort(key=lambda x: x[1])
    removed, end = 0, float("-inf")
    for s, e in intervals:
        if s >= end:
            end = e                # keep it
        else:
            removed += 1          # overlap: remove it
    return removed
```

Pattern 3: Min-heap of end times (Meeting Rooms II)

Sort by start. The heap keeps end times of rooms in use. Free a room if it ended before the new start.

```
import heapq
def minMeetingRooms(intervals):
    intervals.sort(key=lambda x: x[0])
    heap = []                # end times
    for s, e in intervals:
        if heap and heap[0] <= s:
            heapq.heappop(heap)    # reuse that room
        heapq.heappush(heap, e)
    return len(heap)
```

Worked example: Insert Interval

intervals are sorted and do not overlap. Insert newInterval and merge if needed.

Input: intervals = [[1,2],[3,5],[6,7],[8,10]], new = [4,8].

1. [1,2]: ends before 4, so add it. res = [[1,2]].
2. [3,5]: overlaps [4,8]. new = [min(3,4), max(5,8)] = [3,8].
3. [6,7]: overlaps [3,8]. new = [3,8].
4. [8,10]: 8 <= 8 overlaps. new = [3,10].
5. No more intervals. Add new. res = [[1,2],[3,10]].

```
def insert(intervals, new):
    res = []
    for i, (s, e) in enumerate(intervals):
        if e < new[0]:
            res.append([s, e])    # fully before
        elif s > new[1]:
            return res + [new] + intervals[i:]    # fully after: done
        else:
            new = [min(s, new[0]), max(e, new[1])]    # overlap: grow new
    res.append(new)
    return res
```

Complexity cheat sheet

- Merge Intervals, Non-overlapping Intervals: $O(n \log n)$ for sorting, $O(n)$ output space.
- Insert Interval (already sorted): $O(n)$.
- Meeting Rooms II with heap: $O(n \log n)$.
- Minimum Interval to Include Each Query: $O((n + q) \log n)$ with sorting and a heap.

Common mistakes

- Forgetting to sort.
- Using $<$ instead of $\leq$ for overlap (does $[1,2]$ and $[2,3]$ overlap? read the problem).
- When merging, setting $end = e$ instead of $\max(end, e)$. A short interval inside a long one breaks it.
- Sorting by start when the problem needs sort by end (Non-overlapping Intervals).
- Changing the input list while looping over it.

What to say in the interview

"I will sort the intervals by start time so overlapping ones are next to each other."

"Then I compare each interval only with the last merged one."

"For the minimum removals I sort by end time, because keeping the earliest finish leaves the most room."

"Sorting costs $O(n \log n)$ and the scan is $O(n)$."

Practice order

- Meeting Rooms: just sort and check neighbours.
- Merge Intervals: the core template.
- Insert Interval: three cases (before, overlap, after).
- Non-overlapping Intervals: sort by end, greedy.
- Meeting Rooms II and Minimum Interval to Include Each Query: add a heap.

Cheat sheet

Idea

Sort by start time, then walk through and merge or count overlaps. Two intervals overlap if `a.start <= b.end` and `b.start <= a.end`.

Merge template


```
intervals.sort()
res = [intervals[0]]
for s, e in intervals[1:]:
    if s <= res[-1][1]:
        res[-1][1] = max(res[-1][1], e)
    else:
        res.append([s, e])
```

Meeting rooms

Min rooms = max overlap: sort starts and ends separately, or use a min-heap of end times.

Complexity

$O(n \log n)$ because of sorting.

Visualise it

- <https://visualgo.net/en/sorting>

Videos

- [Merge Intervals with diagrams](#) - Nikhil Lohia (English)
- [Merge overlapping intervals](#) - take U forward (Hindi + English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Meeting Rooms (Blind 75)	Easy	open	watch
2	Insert Interval (Blind 75)	Medium	open	watch
3	Merge Intervals (Blind 75)	Medium	open	watch
4	Non Overlapping Intervals (Blind 75)	Medium	open	watch
5	Meeting Rooms II (Blind 75)	Medium	open	watch
6	Minimum Interval to Include Each Query	Hard	open	watch

Problem hints

Meeting Rooms (Easy)

Restate: Given meeting time intervals, say whether one person can attend all of them (no two meetings overlap).

Hint 1: Sort the intervals by start time.

Hint 2: After sorting, you only need to compare each meeting with the one just before it. If a meeting starts before the previous one ends, there is a clash.

Hint 3:

- Sort by start.
- For i from 1 to $n - 1$: if `start[i] < end[i - 1]`, return False.
- Return True.
- Meetings that only touch (one ends at 10, next starts at 10) do NOT clash.

Complexity: $O(n \log n)$ time, $O(1)$ extra space (or $O(n)$ depending on the sort).

Edge cases to test: empty list gives True; one meeting gives True; `[[0,30],[5,10],[15,20]]` gives False; touching `[[5,10],[10,15]]` gives True; unsorted input `[[7,10],[2,4]]` gives True.

Insert Interval (Medium)

Restate: Given a sorted list of non-overlapping intervals and one new interval, insert it and merge where needed so the list stays sorted and non-overlapping.

Hint 1: One linear pass in three phases; no sorting needed.

Hint 2: Intervals fully to the left of the new one are copied, intervals that overlap get merged into it, and intervals fully to the right are copied after it.

Hint 3:

- Copy every interval whose end is less than the new start.
- While intervals start at or before the new end, merge: new start = min, new end = max.
- Append the merged new interval.
- Copy all remaining intervals.

Complexity: $O(n)$ time, $O(n)$ space for the output.

Edge cases to test: empty list with `[5,7]` gives `[[5,7]]`; new interval before all others; new interval after all others; `[[1,3],[6,9]]` with `[2,5]` gives `[[1,5],[6,9]]`; new interval covering everything, like `[[1,2],[3,4]]` with `[0,10]`; touching `[[1,5]]` with `[5,7]` gives `[[1,7]]`.

Merge Intervals (Medium)

Restate: Given intervals in any order, merge all overlapping ones and return the result.

Hint 1: Sort by start, then scan once.

Hint 2: After sorting, an interval overlaps the last merged one if its start is `<=` the last end. Then just extend the last end.

Hint 3:

- Sort by start.
- Put the first interval in the output.
- For each next interval: if `start <= out[-1].end`, set `out[-1].end = max(out[-1].end, end)`.
- Otherwise append it as a new interval.

Complexity: $O(n \log n)$ time, $O(n)$ space.

Edge cases to test: one interval; `[[1,3],[2,6],[8,10],[15,18]]` gives `[[1,6],[8,10],`

`[15,18]`]; touching `[[1,4],[4,5]]` gives `[[1,5]]`; nested `[[1,4],[2,3]]` gives `[[1,4]]`; unsorted `[[4,5],[1,4]]` gives `[[1,5]]`.

Non Overlapping Intervals (Medium)

Restate: Return the minimum number of intervals to remove so that the rest do not overlap.

Hint 1: Greedy, sorted by END time (this is the activity selection problem).

Hint 2: Keeping the interval that ends earliest leaves the most room for the rest. So count how many you can KEEP; the answer is n minus that.

Hint 3:

- Sort by end.
- `prevEnd = -infinity`, `kept = 0`.
- For each interval: if `start >= prevEnd`, keep it (`kept += 1`, `prevEnd = end`).
- Return `n - kept`.

Complexity: $O(n \log n)$ time, $O(1)$ extra space.

Edge cases to test: `[[1,2],[2,3],[3,4],[1,3]]` gives 1; `[[1,2],[1,2],[1,2]]` gives 2; touching `[[1,2],[2,3]]` gives 0; one interval gives 0; one long interval covering many short ones, like `[[1,100],[1,2],[3,4]]`, gives 1.

Meeting Rooms II (Medium)

Restate: Given meeting intervals, return the minimum number of rooms needed so that all meetings can happen.

Hint 1: A min-heap of end times works (see the lesson). Another clean way: two sorted arrays and two pointers.

Hint 2: Sort all start times and all end times separately. Walk through starts; each start needs a room, unless some meeting has already ended by then, which frees one. The answer is the peak number of rooms in use.

Hint 3: (two-pointer version)

- `starts = sorted(starts)`, `ends = sorted(ends)`.
- `rooms = best = 0`, `e = 0`.
- For each start s : if $s \geq \text{ends}[e]$, a room is freed ($e += 1$); else `rooms += 1`.
- `best = max(best, rooms)` (or simply return `rooms` at the end, since it never goes down).

Complexity: $O(n \log n)$ time, $O(n)$ space.

Edge cases to test: empty list gives 0; `[[0,30],[5,10],[15,20]]` gives 2; `[[7,10],[2,4]]` gives 1; touching `[[1,5],[5,10]]` gives 1; all meetings at the same time `[[1,5],[1,5],[1,5]]` gives 3.

Minimum Interval to Include Each Query (Hard)

Restate: For each query point q , return the size ($\text{right} - \text{left} + 1$) of the smallest interval that contains q , or -1 if none does.

Hint 1: Offline processing: sort both intervals and queries, then use a min-heap.

Hint 2: Process queries from small to large. Add every interval that has started ($\text{left} \leq q$) to a heap keyed by size. Pop intervals from the top whose right end is less than q ; they can never help again because later queries are even bigger.

Hint 3:

- Sort intervals by left; sort queries but remember their original index.
- For each query q in sorted order: push $(\text{size}, \text{right})$ for every interval with $\text{left} \leq q$.
- Pop from the heap while the top has $\text{right} < q$.
- The answer for q is the top's size, or -1 if the heap is empty.
- Write answers back in the original query order.

Complexity: $O(n \log n + q \log q)$ time, $O(n + q)$ space.

Edge cases to test: $[[1,4], [2,4], [3,6], [4,4]]$ with queries $[2,3,4,5]$ gives $[3,3,1,4]$; a query outside every interval gives -1; duplicate queries $[3, 3]$; a point interval $[5,5]$ with query 5 gives 1; queries given in unsorted order (answers must match the input order).

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Remove Covered Intervals	Medium	open	watch
2	Data Stream as Disjoint Intervals	Hard	open	watch

Quiz

1. After sorting by start, when do two intervals overlap?
 - A) $\text{next.start} \leq \text{last.end}$
 - B) $\text{next.end} \leq \text{last.start}$
 - C) $\text{next.start} > \text{last.end}$
 - D) They never overlap
2. For Non-overlapping Intervals (min removals), how should we sort?
 - A) By start
 - B) By end
 - C) By length
 - D) No sort needed
3. When merging, why use $\max(\text{last_end}, e)$?
 - A) To make the code shorter
 - B) The new interval may be fully inside the last one
 - C) To sort the result
 - D) It is not needed
4. In Meeting Rooms II, what does the heap store?
 - A) Start times
 - B) End times of rooms in use
 - C) Meeting lengths

- D) Room numbers
5. What is the time complexity of Merge Intervals?
- A) $O(n)$
 - B) $O(n \log n)$
 - C) $O(n^2)$
 - D) $O(\log n)$
6. First step in most interval problems:
- A) Sort by start
 - B) Reverse
 - C) Hash
7. Merge Intervals complexity:
- A) $O(n)$
 - B) $O(n \log n)$
 - C) $O(n^2)$
8. Meeting Rooms II (min rooms) can use:
- A) A min-heap of end times
 - B) A trie
 - C) Binary search only

Answer key

1. **A** - $\text{next.start} \leq \text{last.end}$. If the next one starts before the last one ends, they share time.
2. **B** - By end. Keeping the interval that ends first leaves the most room for the rest.
3. **B** - The new interval may be fully inside the last one. If the new interval ends earlier, the merged end must stay the larger one.
4. **B** - End times of rooms in use. The smallest end time tells us if the earliest room is free for the next meeting.
5. **B** - $O(n \log n)$. Sorting dominates; the merge scan is linear.
6. **A** - Sort by start.
7. **B** - $O(n \log n)$.
8. **A** - A min-heap of end times.

More quiz

1. Which sort order fits "remove the fewest intervals so the rest do not overlap"?
 - A. By start
 - B. By end
 - C. By length
 - D. By input order
2. In Meeting Rooms, do `[1, 5]` and `[5, 8]` clash?
 - A. Yes
 - B. No, one ends exactly when the other starts
 - C. Only if they are in different rooms

- D. It depends on the order
3. Why does Insert Interval not need sorting?
- A. The new interval is always first
 - B. The input list is already sorted and non-overlapping
 - C. Sorting is done by Python automatically
 - D. It does need sorting
4. In Minimum Interval to Include Each Query, why is it safe to pop an interval whose right end is less than q?
- A. It is the biggest interval
 - B. Queries are processed in increasing order, so no later query can be inside it
 - C. The heap is full
 - D. It is not safe
5. Which pattern fits "the maximum number of people in a building at the same time, given entry and exit times"?
- A. Binary search
 - B. Sort starts and ends and sweep (same as Meeting Rooms II)
 - C. Backtracking
 - D. Trie

More quiz: answer key

1. **B** - Keeping the interval that finishes first leaves the most space for the others.
2. **B** - The usual rule is that touching intervals do not overlap. Always confirm this with the interviewer, because it changes the `<` vs `<=` check.
3. **B** - Because the list is sorted, one left-to-right pass can find the left part, the overlapping part and the right part.
4. **B** - Queries only get bigger, so an interval that already ended before q will also have ended before every later query.
5. **B** - Peak people at once is the same as peak rooms in use. A sweep over sorted start and end times finds it.

Flashcards

- **Q:** When do two intervals overlap after sorting by start? — **A:** When the next start is less than (or, by problem rules, equal to) the previous end.
- **Q:** What are the three phases of Insert Interval? — **A:** Copy the left part, merge the overlapping part, copy the right part.
- **Q:** In Merge Intervals, why use max for the new end? — **A:** The next interval may be completely inside the current one.
- **Q:** Non Overlapping Intervals answer formula? — **A:** n minus the number of intervals you can keep when sorted by end.
- **Q:** Two ways to solve Meeting Rooms II? — **A:** A min-heap of end times, or two sorted arrays (starts and ends) with two pointers.

- **Q:** What is the size of interval $[3, 6]$ in Minimum Interval to Include Each Query? — **A:** $6 - 3 + 1 = 4$.
- **Q:** What does "offline processing" mean for queries? — **A:** Sort the queries first and answer them in sorted order, then put answers back in original order.
- **Q:** What key does the heap use in Minimum Interval to Include Each Query? — **A:** The interval size, with the right end stored to remove old intervals.
- **Q:** Most important tester question for any interval problem? — **A:** Do touching intervals like $[1, 5]$ and $[5, 8]$ count as overlapping?
- **Q:** Time complexity of most interval problems? — **A:** $O(n \log n)$, because sorting is the main cost.

DSA topic: Math & Geometry

What it is

These problems use simple maths and careful index work: rotating a matrix, spiral order, digits of numbers, powers, and points on a grid.

Think of turning a photo by 90 degrees. Each pixel moves to a new row and column by a fixed rule.

There is usually no fancy algorithm. The challenge is clean indexes and edge cases (negative numbers, overflow, zeros).

How to recognise it

- A matrix must be rotated, spiralled, or zeroed "in place".
- Work with digits: Happy Number, Plus One, Multiply Strings.
- Fast power: Pow(x, n) with large n.
- Points and counting: Detect Squares, lines through points.
- Words like "in place", "O(1) extra space", "without converting to integer".

Pattern 1: Matrix transforms (transpose + reverse)

Rotate 90 degrees clockwise = transpose, then reverse each row.

```
def rotate(matrix):
    n = len(matrix)
    for i in range(n):
        for j in range(i + 1, n):           # transpose: swap across diagonal
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    for row in matrix:
        row.reverse()
```

Pattern 2: Boundaries for spiral walk

Keep top, bottom, left, right. Walk one side, then shrink that boundary.


```
def spiralOrder(m):
    res = []
    top, bot, left, right = 0, len(m) - 1, 0, len(m[0]) - 1
    while top <= bot and left <= right:
        for c in range(left, right + 1): res.append(m[top][c])
        top += 1
        for r in range(top, bot + 1): res.append(m[r][right])
        right -= 1
        if top <= bot:
            for c in range(right, left - 1, -1): res.append(m[bot][c])
            bot -= 1
        if left <= right:
            for r in range(bot, top - 1, -1): res.append(m[r][left])
            left += 1
    return res
```

Pattern 3: Fast power and digit maths

Square the base and halve the power: $O(\log n)$. Use $\% 10$ and $// 10$ to read digits.

```
def myPow(x, n):
    if n < 0:
        x, n = 1 / x, -n
    res = 1
    while n:
        if n & 1:
            res *= x
        x *= x
        n >>= 1
    return res

def digit_square_sum(n):
    total = 0
    while n:
        n, d = divmod(n, 10)
        total += d * d
    return total
```

Worked example: Set Matrix Zeroes

If a cell is 0, set its whole row and column to 0, in place.

Input: $[[1,1,1],[1,0,1],[1,1,1]]$.

1. Scan: zero found at (1,1). Mark rows = {1}, cols = {1}.
2. Row 0: col 1 is in cols, so (0,1) = 0. Row 0 = [1,0,1].
3. Row 1: row 1 is in rows, so whole row = 0. Row 1 = [0,0,0].
4. Row 2: (2,1) = 0. Row 2 = [1,0,1].
5. Result: $[[1,0,1],[0,0,0],[1,0,1]]$.

The $O(1)$ space version stores the marks in the first row and first column, with one extra flag for row 0.

```
def setZeroes(matrix):
    R, C = len(matrix), len(matrix[0])
    rows, cols = set(), set()
    for r in range(R):
        for c in range(C):
            if matrix[r][c] == 0:
                rows.add(r); cols.add(c)
    for r in range(R):
        for c in range(C):
            if r in rows or c in cols:
                matrix[r][c] = 0
```

Complexity cheat sheet

- Rotate Image, Spiral Matrix, Set Matrix Zeroes: $O(R * C)$ time.
- Rotate Image in place: $O(1)$ extra space.
- Pow(x, n): $O(\log n)$ time.
- Happy Number with a seen set: about $O(\log n)$ per step; use Floyd cycle detection for $O(1)$ space.
- Multiply Strings: $O(m * n)$.

Common mistakes

- Transposing with j starting at 0, which swaps every pair twice (no change).
- Spiral: not checking $top \leq bot$ and $left \leq right$ before the bottom and left walks, so items repeat.
- Pow: forgetting negative n.
- Setting zeros while scanning, which spreads zeros too far.
- Using / instead of // for integer division in Python.

What to say in the interview

"To rotate in place, I will transpose the matrix and then reverse each row."

"For the spiral I keep four boundaries and shrink them after each side."

"For power I use fast exponentiation, which is $O(\log n)$."

"Let me check edge cases: one row, one column, negative numbers and zero."

Practice order

- Rotate Image: learn transpose + reverse.
- Spiral Matrix: boundary control.
- Set Matrix Zeroes: first with sets, then the $O(1)$ space version.
- Happy Number, Plus One, Pow(x, n): digit and power maths.
- Multiply Strings and Detect Squares last: more index and counting detail.

Cheat sheet

Idea

Matrix traversal tricks and number tricks. Draw small examples.

Patterns

- Rotate image: transpose, then reverse each row
- Spiral matrix: shrink top/bottom/left/right boundaries
- Set matrix zeroes: use first row/col as markers for $O(1)$ space
- Fast power: square-and-multiply, $O(\log n)$
- Happy number: cycle detection with a set or fast/slow

Visualise it

- <https://visualgo.net/en/sorting>

Videos

- [Spiral traversal of a matrix](#) - take U forward (Hindi + English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Happy Number	Easy	open	watch
2	Plus One	Easy	open	watch
3	Rotate Image (Blind 75)	Medium	open	watch
4	Spiral Matrix (Blind 75)	Medium	open	watch
5	Set Matrix Zeroes (Blind 75)	Medium	open	watch
6	Pow(x, n)	Medium	open	watch
7	Multiply Strings	Medium	open	watch
8	Detect Squares	Medium	open	watch

Problem hints

Happy Number (Easy)

Restate: Replace a number by the sum of the squares of its digits again and again. Say whether it eventually reaches 1 (a "happy" number) instead of looping forever.

Hint 1: This is cycle detection. Use a hash set, or Floyd's slow and fast pointers.

Hint 2: The sequence either reaches 1 or enters a loop that never contains 1. Numbers quickly become small (for example, any 3-digit number goes to at most 243), so a loop is guaranteed if 1 is never reached.

Hint 3:

- Write a helper that returns the sum of squared digits (use `% 10` and `// 10`).
- Keep a set of seen numbers.
- Loop: if `n` is 1, return True; if `n` is in the set, return False.
- Add `n` to the set and replace `n` with the helper result.
- (Floyd version: move slow one step and fast two steps until they meet; happy if they meet at 1.)

Complexity: $O(\log n)$ time per step and a small number of steps; $O(\log n)$ space for the set ($O(1)$ with Floyd).

Edge cases to test: 1 gives True; 19 gives True ($1 + 81 = 82, 68, 100, 1$); 2 gives False; 7 gives True; a large number like 2147483647.

Plus One (Easy)

Restate: A big number is stored as a list of digits (most significant first). Add 1 to it and return the new digit list.

Hint 1: Simulate school addition from the right end.

Hint 2: A 9 becomes 0 and passes a carry left. Any other digit just increases by 1 and you can stop immediately. Only an all-9s number grows by one digit.

Hint 3:

- Loop `i` from the last index down to 0.
- If `digits[i] < 9`: add 1 and return the list.
- Else set `digits[i] = 0` and continue.
- If the loop ends, return `[1] + digits`.

Complexity: $O(n)$ time, $O(1)$ extra space ($O(n)$ only when a new digit is added).

Edge cases to test: [1, 2, 3] gives [1, 2, 4]; [9] gives [1, 0]; [9, 9, 9] gives [1, 0, 0, 0]; [0] gives [1]; [1, 9, 9] gives [2, 0, 0].

Rotate Image (Medium)

Restate: Rotate an $n \times n$ matrix 90 degrees clockwise, in place (no second matrix).

Hint 1: The lesson shows "transpose, then reverse each row". Another way is to rotate ring by ring.

Hint 2: In the ring version, each group of 4 cells moves in a circle: top-left goes to top-right, top-right to bottom-right, bottom-right to bottom-left, bottom-left to top-left. Save one value in a temp variable and shift the other three.

Hint 3: (ring version)

- Use `left = 0`, `right = n - 1`; loop while `left < right` (one ring per loop).
- For each offset `i` from 0 to `right - left - 1`, do a 4-way swap of the cells at that offset on the four sides.

- Then move inward: `left += 1, right -= 1`.

Complexity: $O(n^2)$ time, $O(1)$ extra space.

Edge cases to test: 1×1 `[[5]]` stays the same; 2×2 `[[1,2],[3,4]]` gives `[[3,1],[4,2]]`; 3×3 `[[1,2,3],[4,5,6],[7,8,9]]` gives `[[7,4,1],[8,5,2],[9,6,3]]`; 4×4 (two rings); a matrix with negative numbers or repeated values.

Spiral Matrix (Medium)

Restate: Return all elements of an $m \times n$ matrix in spiral order: right along the top, down the right side, left along the bottom, up the left side, and repeat inward.

Hint 1: Simulation with four boundaries: top, bottom, left, right.

Hint 2: After walking each side, move that boundary inward. The tricky part is non-square matrices: check the boundaries again before walking the bottom row and the left column, or you will repeat elements.

Hint 3:

- `top, bottom, left, right = 0, m - 1, 0, n - 1`.
- While `top <= bottom` and `left <= right`: walk the top row left to right, then `top += 1`.
- Walk the right column top to bottom, then `right -= 1`.
- If `top <= bottom`: walk the bottom row right to left, then `bottom -= 1`.
- If `left <= right`: walk the left column bottom to top, then `left += 1`.

Complexity: $O(m * n)$ time, $O(1)$ extra space (besides the output).

Edge cases to test: 1×1 `[[1]]`; single row `[[1, 2, 3]]`; single column `[[1], [2], [3]]`; 3×4 `[[1,2,3,4],[5,6,7,8],[9,10,11,12]]` gives `[1,2,3,4,8,12,11,10,9,5,6,7]`; check the output length equals $m * n$ (no repeats, no missing).

Set Matrix Zeroes (Medium)

Restate: If a cell is 0, set its whole row and column to 0, in place.

Hint 1: The lesson uses row and column marker sets ($O(m + n)$ space). The follow-up asks for $O(1)$ extra space.

Hint 2: Use the first row and the first column of the matrix itself as the marker arrays. Because they get overwritten, first remember (in two booleans) whether the first row and first column had a zero originally.

Hint 3: ($O(1)$ space version)

- Record `firstRowZero` and `firstColZero`.
- For every other cell ($r \geq 1, c \geq 1$) that is 0: set `matrix[r][0] = 0` and `matrix[0][c] = 0`.
- For every other cell: if `matrix[r][0] == 0` or `matrix[0][c] == 0`, set it to 0.
- Finally, zero the first row if `firstRowZero`, and the first column if `firstColZero`.

Complexity: $O(m * n)$ time, $O(1)$ extra space.

Edge cases to test: no zeros (no change); a zero in the top-left corner `[[0,1],[1,1]]`; a zero only in the first row; a single row `[[1, 0, 3]]`; all zeros.

Pow(x, n) (Medium)

Restate: Compute x raised to the power n , where n can be negative, without using the built-in power function.

Hint 1: Fast power (exponentiation by squaring). See the lesson for the core idea.

Hint 2: For a negative n , compute $1 / x^{(-n)}$. Use the iterative bit version to avoid deep recursion: look at the bits of n ; whenever a bit is 1, multiply the result by the current power of x , and square x each step.

Hint 3:

- If $n < 0$: $x = 1 / x$, $n = -n$.
- $result = 1$.
- While $n > 0$: if $n \& 1$, $result *= x$; then $x *= x$, $n >>= 1$.
- Return $result$.

Complexity: $O(\log n)$ time, $O(1)$ space.

Edge cases to test: $n = 0$ gives 1 (even for $x = 0$ in this problem); $2.0, -2$ gives 0.25; $2.0, 10$ gives 1024.0; $x = 1.0, n = -2147483648$ (in Java/C++ the negation overflows; Python is fine); $x = -2.0, n = 3$ gives -8.0 (sign).

Multiply Strings (Medium)

Restate: Multiply two non-negative integers given as strings and return the product as a string, without converting the whole strings to numbers.

Hint 1: Simulate school (long) multiplication with an array of digits.

Hint 2: The product of $num1[i]$ and $num2[j]$ lands at positions $i + j$ and $i + j + 1$ of a result array of length $m + n$. Add into position $i + j + 1$, then push the carry into $i + j$.

Hint 3:

- Make $res = [0] * (m + n)$.
- Loop i from the right of $num1$, j from the right of $num2$.
- $total = d1 * d2 + res[i + j + 1]$; set $res[i + j + 1] = total \% 10$; add $total // 10$ to $res[i + j]$.
- Skip leading zeros, join to a string.
- If everything is zero, return "0".

Complexity: $O(m * n)$ time, $O(m + n)$ space.

Edge cases to test: "0" x "12345" gives "0" (not "00000"); "2" x "3" gives "6"; "123" x "456" gives "56088"; "99" x "99" gives "9801"; very long inputs (100+ digits).

Detect Squares (Medium)

Restate: Design a class that stores points (duplicates allowed) and, for a query point, counts how many axis-aligned squares with positive area can be formed using the query point plus three stored points.

Hint 1: Hash map from point to count.

Hint 2: Pick each stored point as the DIAGONAL corner of the square. It must satisfy $|px - x| == |py - y|$ and not be 0. Then the other two corners are fixed: (x, py) and (px, y) . Multiply their counts.

Hint 3:

- `add(point)`: increase `count[(x, y)]` (also keep a list of distinct points).

- `count(query)` : loop over every stored distinct point (px, py).
- Skip it if `abs(px - x) != abs(py - y)` or `px == x`.
- Add `count[(px, py)] * count[(x, py)] * count[(px, y)]` to the answer.

Complexity: add is $O(1)$; count is $O(\text{number of distinct points})$ time; $O(n)$ space.

Edge cases to test: a query with no stored points gives 0; the same point added twice doubles the result; a query point equal to a stored point (area 0, must not count); points forming a rectangle that is not a square (gives 0); squares in all four directions from the query point.

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Spiral Matrix II	Medium	open	watch
2	Integer to Roman	Medium	open	watch
3	Robot Bounded In Circle	Medium	open	watch
4	Zigzag Conversion	Medium	open	watch
5	Find Missing Observations	Medium	open	watch
6	Maximum Points on a Line	Hard	open	watch

Quiz

- How do you rotate an $n \times n$ matrix 90 degrees clockwise in place?
 - A) Reverse rows only
 - B) Transpose, then reverse each row
 - C) Sort each row
 - D) Transpose twice
- What is the time complexity of fast power $\text{Pow}(x, n)$?
 - A) $O(n)$
 - B) $O(\log n)$
 - C) $O(n^2)$
 - D) $O(1)$
- In the transpose loop, why does j start at $i + 1$?
 - A) To skip the diagonal and avoid swapping pairs twice
 - B) To save memory
 - C) Because Python lists start at 1
 - D) No reason
- How do you get the last digit of an integer n in Python?
 - A) $n // 10$
 - B) $n \% 10$
 - C) $n / 10$
 - D) $n ** 10$

5. Why not set zeros immediately while scanning in Set Matrix Zeroes?
- A) It is slower
 - B) New zeros would be treated as original zeros and spread wrongly
 - C) Python forbids it
 - D) It uses more memory
6. Rotate an $n \times n$ matrix 90 degrees clockwise in place:
- A) Transpose then reverse rows
 - B) Sort rows
 - C) Reverse columns only
7. $\text{Pow}(x, n)$ with fast exponentiation is:
- A) $O(n)$
 - B) $O(\log n)$
 - C) $O(1)$
8. Spiral order is done by:
- A) Shrinking four boundaries
 - B) Recursion on a heap
 - C) Sorting

Answer key

1. **B** - Transpose, then reverse each row. Transpose swaps rows and columns, and reversing rows completes the clockwise turn.
2. **B** - $O(\log n)$. The power is halved at every step.
3. **A** - To skip the diagonal and avoid swapping pairs twice. Starting at 0 would swap each pair back to its original place.
4. **B** - $n \% 10$. The remainder after dividing by 10 is the last digit.
5. **B** - New zeros would be treated as original zeros and spread wrongly. We must first record original zeros, then apply the changes.
6. **A** - Transpose then reverse rows.
7. **B** - $O(\log n)$.
8. **A** - Shrinking four boundaries.

More quiz

1. In Multiply Strings, where does the product of `num1[i]` and `num2[j]` go first?
 - A. Position `i * j`
 - B. Position `i + j + 1` of the result array
 - C. Position 0
 - D. Position `m + n`
2. Which pattern fits Happy Number?
 - A. Binary search
 - B. Cycle detection (hash set or slow and fast pointers)
 - C. Sorting

- D. Dynamic programming
3. In Spiral Matrix on a single-row matrix `[[1, 2, 3]]`, which check stops you from printing 2 and 1 again?
- A. `left <= right` before the left column
 - B. `top <= bottom` before walking the bottom row
 - C. `m == n`
 - D. No check is needed
4. In the $O(1)$ space Set Matrix Zeroes, why do you save two booleans first?
- A. To count the zeros
 - B. The first row and column are used as markers and get overwritten
 - C. To sort the matrix
 - D. They are not needed
5. In Detect Squares, which stored point do you loop over?
- A. The point directly above the query
 - B. The diagonal corner of the square
 - C. The centre of the square
 - D. Every pair of points

More quiz: answer key

1. **B** - Positions are counted from the left, so digit i and digit j together affect `i + j + 1`, and the carry goes to `i + j`.
2. **B** - The sequence either reaches 1 or repeats a number, so detecting a repeat answers the question.
3. **B** - After the top row is walked, `top` becomes 1, which is greater than `bottom` (0). The check stops the bottom row from being printed again.
4. **B** - Once you write markers into the first row and column, you can no longer tell whether they had an original zero.
5. **B** - The diagonal corner fixes the side length and the other two corners, so one loop is enough.

Flashcards

- **Q:** How do you get the digits of a number without strings? — **A:** Use `n % 10` for the last digit and `n // 10` to remove it.
- **Q:** What is the next number after 19 in Happy Number? — **A:** $1^2 + 9^2 = 82$.
- **Q:** When does Plus One make the list longer? — **A:** Only when every digit is 9.
- **Q:** What is the ring-by-ring way to rotate a matrix? — **A:** Do a 4-way swap of matching cells on the four sides, then move one ring inward.
- **Q:** What are the four boundaries in Spiral Matrix? — **A:** top, bottom, left, right; move each inward after walking that side.
- **Q:** How does Set Matrix Zeroes reach $O(1)$ space? — **A:** It uses the first row and first column as marker arrays.
- **Q:** How does Pow(x, n) handle a negative n? — **A:** Use $1 / x$ as the base and $-n$ as the power.
- **Q:** What is the length of the result array in Multiply Strings? — **A:** `len(num1) + len(num2)`.

- **Q:** What must Multiply Strings return for "0" times anything? — **A:** "0", with no extra leading zeros.
- **Q:** Why is a query point equal to a stored point not a square in Detect Squares? — **A:** The side length would be 0, and the problem needs positive area.

DSA topic: Bit Manipulation

What it is

Computers store numbers in binary: 5 is 101, 6 is 110. Bit manipulation works directly on these 0s and 1s. Think of a row of light switches. Each switch is a bit. You can turn one on, off, or check it.

Main operators: & (and), | (or), ^ (xor), ~ (not), << (shift left), >> (shift right).

These tricks give O(1) extra space and very fast code.

How to recognise it

- "Every element appears twice except one" means XOR.
- "Count the 1 bits", "reverse bits", "power of two".
- "Without using + or -" (Sum of Two Integers).
- "Missing number from 0..n" in O(1) space.
- Subsets of a small set can be stored as a bitmask.

Pattern 1: XOR cancels pairs

$x \oplus x = 0$ and $x \oplus 0 = x$. XOR all values and pairs disappear.

```
def singleNumber(nums):
    res = 0
    for x in nums:
        res ^= x
    return res

def missingNumber(nums):
    res = len(nums)
    for i, x in enumerate(nums):
        res ^= i ^ x          # indexes and values cancel
    return res
```

Pattern 2: Check, set and clear bits

Bit i of x: $(x \gg i) \& 1$. $n \& (n - 1)$ removes the lowest 1 bit.

```
def hammingWeight(n):
    count = 0
    while n:
        n &= n - 1          # drop lowest set bit
        count += 1
    return count

def isPowerOfTwo(n):
    return n > 0 and n & (n - 1) == 0

def reverseBits(n):
    res = 0
    for i in range(32):
        res = (res << 1) | ((n >> i) & 1)
    return res
```

Pattern 3: DP on bits (Counting Bits)

The bits of i equal the bits of $i \gg 1$, plus the last bit.

```
def countBits(n):
    ans = [0] * (n + 1)
    for i in range(1, n + 1):
        ans[i] = ans[i >> 1] + (i & 1)
    return ans
```

Worked example: Sum of Two Integers

Add a and b without $+$ or $-$. XOR gives the sum without carry. AND then shift left gives the carry.

Input: $a = 5$ (101), $b = 3$ (011).

1. $\text{sum} = 101 \oplus 011 = 110$ (6). $\text{carry} = (101 \& 011) \ll 1 = 001 \ll 1 = 010$ (2).
2. $a = 6$, $b = 2$. $\text{sum} = 110 \oplus 010 = 100$ (4). $\text{carry} = (110 \& 010) \ll 1 = 100$ (4).
3. $a = 4$, $b = 4$. $\text{sum} = 100 \oplus 100 = 000$ (0). $\text{carry} = (100 \& 100) \ll 1 = 1000$ (8).
4. $a = 0$, $b = 8$. $\text{sum} = 1000$ (8). $\text{carry} = 0$.
5. $b == 0$, so the answer is 8.

Python integers have no fixed size, so we use a 32-bit mask to handle negative numbers.

```
def getSum(a, b):
    MASK, MAX = 0xFFFFFFFF, 0x7FFFFFFF
    while b & MASK:
        a, b = (a ^ b) & MASK, ((a & b) << 1) & MASK
    return a if a <= MAX else ~(a ^ MASK)  # convert back to negative
```

Complexity cheat sheet

- Single Number, Missing Number: $O(n)$ time, $O(1)$ space.
- Number of 1 Bits: $O(\text{number of set bits})$, at most 32.

- Reverse Bits: $O(32) = O(1)$.
- Counting Bits: $O(n)$.
- Sum of Two Integers: $O(32)$ loop iterations at most.

Common mistakes

- Operator precedence: `-` binds tighter than `&`, and in Java or C `==` binds tighter than `&`. Always write brackets: `(n & (n - 1)) == 0`.
- Forgetting that Python ints are unbounded, so negative numbers never stop in a loop without a mask.
- Using `~x` and expecting a 32-bit result; in Python `~x` is `-x - 1`.
- Shifting the wrong way (`<<` multiplies by 2, `>>` divides by 2).
- Forgetting `n > 0` in the power of two check.

What to say in the interview

"Since every number appears twice except one, XOR of all numbers leaves only the single one."

"`n & (n - 1)` removes the lowest set bit, so the loop runs once per 1 bit."

"Python integers are unbounded, so I will use a 32-bit mask for negative numbers."

"This is $O(n)$ time and $O(1)$ extra space."

Practice order

- Single Number: learn XOR cancelling.
- Number of 1 Bits and Counting Bits: bit checks and DP on bits.
- Reverse Bits and Missing Number: shifting and XOR with indexes.
- Sum of Two Integers: carry with AND and shift, plus masks.
- Reverse Integer last: overflow checks without 64-bit numbers.

Cheat sheet

Idea

Work directly with binary. Know these by heart:

- `x & (x - 1)` clears the lowest set bit (count bits)
- `a ^ a = 0`, `a ^ 0 = a` (Single Number, Missing Number)
- `x >> 1`, `x & 1` read bits one by one

Python note

Python ints are unbounded; for 32-bit problems mask with `0xFFFFFFFF`.

Visualise it

- <https://visualgo.net/en/bitmask>

Videos

- [Binary numbers and bit manipulation \(Python\)](#) - Greg Hogg (English)
- [Introduction to Bit Manipulation](#) - take U forward (Hindi + English)

NeetCode 150 problems for this topic

#	Problem	Level	LeetCode	Solution video
1	Single Number	Easy	open	watch
2	Number of 1 Bits (Blind 75)	Easy	open	watch
3	Counting Bits (Blind 75)	Easy	open	watch
4	Reverse Bits (Blind 75)	Easy	open	watch
5	Missing Number (Blind 75)	Easy	open	watch
6	Sum of Two Integers (Blind 75)	Medium	open	watch
7	Reverse Integer	Medium	open	watch

Problem hints

Single Number (Easy)

Restate: Every number in the list appears twice except one. Find that one, using $O(1)$ extra space.

Hint 1: Use XOR ($\wedge$).

Hint 2: XOR has three useful rules: $a \wedge a = 0$, $a \wedge 0 = a$, and the order does not matter. So when you XOR everything, all the pairs cancel out and only the single number is left.

Hint 3:

- Start `result = 0`.
- For each number: `result ^= number`.
- Return `result`.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: one element `[1]` gives 1; `[2, 2, 1]` gives 1; `[4, 1, 2, 1, 2]` gives 4; negative numbers `[-1, -1, -2]` gives -2; the single number is 0, like `[0, 3, 3]`.

Number of 1 Bits (Easy)

Restate: Given a non-negative integer, return how many 1 bits its binary form has (this count is also called the Hamming weight).

Hint 1: Check bits one by one with `& 1` and right shift, or use the `n & (n - 1)` trick from the lesson.

Hint 2: With the shift method, the last bit is `n & 1`. Shifting right by 1 (`n >>= 1`) moves the next bit into the last place. The trick method instead loops only once per 1 bit.

Hint 3: (shift version)

- `count = 0`.
- While `n > 0`: `count += n & 1`, then `n >>= 1`.
- Return `count`.
- In Java or C++, use an unsigned shift (`>>>` in Java), or a fixed 32-step loop, so negative inputs do not loop forever.

Complexity: $O(32) = O(1)$ time, $O(1)$ space.

Edge cases to test: 0 gives 0; 11 (1011) gives 3; 128 (10000000) gives 1; 2147483647 gives 31; 4294967293 (32-bit 1111...1101) gives 31.

Counting Bits (Easy)

Restate: For every number i from 0 to n , return how many 1 bits i has, as a list.

Hint 1: DP that reuses answers for smaller numbers.

Hint 2: The lesson uses `ans[i >> 1] + (i & 1)`. Another formula: `i & (i - 1)` removes the lowest 1 bit of i , giving a smaller number with exactly one fewer 1 bit.

Hint 3: (lowest-bit version)

- `ans = [0] * (n + 1)`.
- For i from 1 to n : `ans[i] = ans[i & (i - 1)] + 1`.
- Return `ans`.

Complexity: $O(n)$ time, $O(n)$ space for the output ($O(1)$ extra).

Edge cases to test: $n = 0$ gives `[0]`; $n = 2$ gives `[0, 1, 1]`; $n = 5$ gives `[0, 1, 1, 2, 1, 2]`; $n = 8$ (8 has one bit, 7 has three); large n like 100000 (speed).

Reverse Bits (Easy)

Restate: Reverse the order of the 32 bits of an unsigned 32-bit integer and return the new number.

Hint 1: Build the answer bit by bit with shifts.

Hint 2: Always do exactly 32 steps, even if n becomes 0 early. Leading zeros of n become trailing zeros of the answer, and the answer's top bits must be filled correctly.

Hint 3:

- `result = 0`.
- Repeat 32 times: `result = (result << 1) | (n & 1)`, then `n >>= 1`.
- Return `result`.

Complexity: $O(32) = O(1)$ time, $O(1)$ space.

Edge cases to test: 0 gives 0; 1 gives 2147483648 (only the top bit set); 43261596 gives 964176192; all ones 4294967295 stays the same; 2147483648 gives 1.

Missing Number (Easy)

Restate: The list has n distinct numbers from the range 0 to n , so exactly one number is missing. Find it.

Hint 1: XOR, or the sum formula.

Hint 2: XOR all indexes 0.. n with all values. Every present number appears twice (once as index, once as value) and cancels. Only the missing number is left. (Sum method: $n * (n + 1) / 2 - \text{sum}(\text{nums})$.)

Hint 3:

- Start `result = n` (the index that has no matching position).
- For each index i : `result ^= i ^ nums[i]`.
- Return `result`.

Complexity: $O(n)$ time, $O(1)$ space.

Edge cases to test: `[0]` gives 1; `[1]` gives 0; `[3, 0, 1]` gives 2; `[0, 1]` gives 2 (the last number is missing); `[9, 6, 4, 2, 3, 5, 7, 0, 1]` gives 8.

Sum of Two Integers (Medium)

Restate: Add two integers without using `+` or `-`.

Hint 1: XOR adds bits without carry; AND finds where a carry happens. (See the lesson's worked example.)

Hint 2: Repeat: `sum = a ^ b`, `carry = (a & b) << 1`, until carry is 0. In Python, numbers have no fixed size, so you must keep everything inside 32 bits with a mask `0xFFFFFFFF`, and convert back to a negative number at the end if the top bit is set.

Hint 3:

- `mask = 0xFFFFFFFF`.
- While `b != 0`: `a, b = (a ^ b) & mask, ((a & b) << 1) & mask`.
- If `a <= 0x7FFFFFFF`, return `a`.
- Else return `~(a ^ mask)` (turns it back into a negative Python int).

Complexity: $O(32) = O(1)$ time, $O(1)$ space.

Edge cases to test: `1, 2` gives 3; `-1, 1` gives 0 (infinite loop without the mask in Python); `-2, -3` gives -5; `0, 0` gives 0; `-1000, 1000` gives 0.

Reverse Integer (Medium)

Restate: Reverse the digits of a signed 32-bit integer. If the result does not fit in 32 bits, return 0.

Hint 1: Pop digits with `% 10` and push them with `* 10`.

Hint 2: Handle the sign separately (work on the absolute value in Python, because `%` with negative numbers behaves differently from C++ and Java). Check for overflow BEFORE pushing a digit, as if you could not store a 64-bit number.

Hint 3:

- `sign = -1 if x < 0 else 1, x = abs(x), res = 0`.
- While `x > 0`: `digit = x % 10, x //= 10`.
- If `res > (231 - 1) // 10`, or it is equal and the digit is too big, return 0.
- `res = res * 10 + digit`.

- Return `sign * res` (check the negative limit -2^{31} too).

Complexity: $O(\log x)$ time (number of digits), $O(1)$ space.

Edge cases to test: 123 gives 321; -123 gives -321; 120 gives 21 (trailing zero dropped); 0 gives 0; 1534236469 gives 0 (overflow); -2147483648 gives 0.

Bonus practice (after you finish the list above)

#	Problem	Level	LeetCode	Solution video
1	Shuffle the Array	Easy	open	watch
2	Add to Array-Form of Integer	Easy	open	watch
3	Add Binary	Easy	open	watch

Quiz

1. What is $x \wedge x$ for any integer x ?
 - A) x
 - B) 0
 - C) 1
 - D) $2x$
2. What does $n \& (n - 1)$ do?
 - A) Doubles n
 - B) Removes the lowest set bit of n
 - C) Sets all bits
 - D) Reverses the bits
3. Which check correctly tests if n is a power of two?
 - A) $n \% 2 == 0$
 - B) $n > 0$ and $n \& (n - 1) == 0$
 - C) $n \& 1 == 1$
 - D) $n >> 1 == 0$
4. In Counting Bits, `ans[i]` equals?
 - A) `ans[i - 1] + 1`
 - B) `ans[i >> 1] + (i & 1)`
 - C) `i % 2`
 - D) `ans[i // 3]`
5. Why does Sum of Two Integers need a mask in Python?
 - A) Python is slow
 - B) Python ints are unbounded, so negative carries never end
 - C) To print binary
 - D) XOR does not work in Python
6. $a \wedge a$ equals:
 - A) a
 - B) 0

- C) 1
7. $x \& (x - 1)$ does what?
- A) Clears the lowest set bit
 - B) Doubles x
 - C) Sets all bits
8. Single Number (all others appear twice) uses:
- A) XOR of all numbers
 - B) Sorting
 - C) A heap

Answer key

1. **B** - 0. XOR of equal bits is 0, so every bit becomes 0.
2. **B** - Removes the lowest set bit of n . Subtracting 1 flips the lowest 1 and the zeros after it, and AND clears that 1.
3. **B** - $n > 0$ and $n \& (n - 1) == 0$. A power of two has exactly one set bit, and n must be positive.
4. **B** - $\text{ans}[i >> 1] + (i \& 1)$. Shifting right drops the last bit, and $i \& 1$ adds it back.
5. **B** - Python ints are unbounded, so negative carries never end. The mask simulates 32-bit integers so the carry loop stops.
6. **B** - 0.
7. **A** - Clears the lowest set bit.
8. **A** - XOR of all numbers.

More quiz

1. What is $5 \wedge 3$ (XOR)?
 - A. 8
 - B. 6
 - C. 2
 - D. 15
2. Which pattern fits "every number appears three times except one; find it"?
 - A. Plain XOR of all numbers
 - B. Count each of the 32 bit positions modulo 3
 - C. Sorting with binary search only
 - D. A min-heap
3. Why must Reverse Bits always run exactly 32 steps?
 - A. To be slower
 - B. Leading zeros of the input become trailing zeros of the output and must be shifted in
 - C. Because n is always odd
 - D. It does not need to
4. In Missing Number, why does `result` start at n instead of 0?
 - A. To avoid a negative answer
 - B. Indexes go only up to $n - 1$, so n must be added once to complete the range $0..n$

- C. Because n is always missing
 - D. It is a random choice
5. In Python, what goes wrong in Sum of Two Integers without a mask when `a = -1`, `b = 1`?
- A. It returns 2
 - B. The carry keeps moving left forever, because Python ints have no fixed size
 - C. It throws a type error
 - D. Nothing goes wrong

More quiz: answer key

1. **B** - 5 is 101 and 3 is 011. XOR gives 110, which is 6.
2. **B** - Pairs cancel with XOR, but triples do not. Counting each bit position and taking `% 3` leaves only the single number's bits.
3. **B** - If you stop when n becomes 0, the output is shifted too little and the answer is wrong for inputs like 1.
4. **B** - The values cover 0..n minus one number, but indexes cover only 0..n-1. Adding n makes every present number appear exactly twice.
5. **B** - -1 has infinitely many 1 bits in Python, so the carry never reaches zero. The mask keeps everything inside 32 bits.

Flashcards

- **Q:** What are the three key XOR rules? — **A:** `a ^ a = 0`, `a ^ 0 = a`, and order does not matter.
- **Q:** How do you read the last bit of n? — **A:** `n & 1`.
- **Q:** What does `n >> 1` do for a positive n? — **A:** Divides n by 2, dropping the last bit.
- **Q:** Counting Bits formula using the lowest set bit? — **A:** `ans[i] = ans[i & (i - 1)] + 1`.
- **Q:** One line to build reversed bits? — **A:** `result = (result << 1) | (n & 1)`, then `n >>= 1`, 32 times.
- **Q:** Sum formula for Missing Number? — **A:** `n * (n + 1) / 2 - sum(nums)`.
- **Q:** What is the 32-bit mask in hex? — **A:** `0xFFFFFFFF`.
- **Q:** How do you turn a masked 32-bit value back into a negative Python int? — **A:** If it is above `0x7FFFFFFF`, return `~(a ^ 0xFFFFFFFF)`.
- **Q:** Range of a signed 32-bit integer? — **A:** -2^{31} to $2^{31} - 1$, that is -2147483648 to 2147483647.
- **Q:** Best tester inputs for Reverse Integer? — **A:** Trailing zeros (120), negative numbers, 0, and an overflow case like 1534236469.

Testing Your Own Code: The QA Superpower

In this chapter:

- Dry-run your code by hand, line by line, like a debugger
- Choose strong test cases using categories and boundary thinking
- Write quick asserts and small pytest tests for an interview solution
- Talk about testing aloud the way Google interviewers expect from a test engineer
- Apply all of this to two complete worked examples

Time: ~45 minutes | **Level:** Intermediate

Why this is your superpower

Many candidates finish their code, say "I think this works", and stop. Testing your own solution is one of the things coding interviewers evaluate (see chapter dsa-01). For a test-focused role, it matters even more: the interviewer wants to see that you **think like a tester even about your own code**.

You do this every day at work. You look at a login form and immediately think: empty password, very long password, SQL characters, Unicode names, two clicks on submit. This chapter shows you how to bring the same instinct into a 45-minute coding round, where you usually **cannot run the code**.

Think of a railway signal engineer. Before a new track opens, they walk it, check each signal, and run a test train. They do not wait for passengers to find the problems. You will "walk your code" the same way.

Part 1: Dry-running code

A **dry run** means executing the code in your head (or on paper), line by line, tracking every variable. You are the computer.

How to dry run

1. Pick a **small** input: 3-5 elements. Large inputs waste time.
2. Make a small table with one column per variable that changes.
3. Go line by line. Update the table each time a variable changes.
4. At the end, compare the result with the answer you worked out by hand earlier.

Example: this function should return the maximum profit from buying a stock on one day and selling on a later day.

```
def max_profit(prices):
    min_price = float('inf')
    best = 0
    for p in prices:
        min_price = min(min_price, p)    # cheapest buy so far
        best = max(best, p - min_price)  # best sell today
    return best

print(max_profit([7, 1, 5, 3, 6]))    # Output: 5 (buy at 1, sell at 6)
```

Dry run with `[7, 1, 5, 3]`:

p	min_price	p - min_price	best
start	inf	-	0
7	7	0	0
1	1	0	0
5	1	4	4
3	1	2	4

Result 4 (buy at 1, sell at 5). Correct.

Tips for an effective dry run

- **Say values aloud.** "p is 5, min_price stays 1, so profit is 4, best becomes 4." The interviewer can follow and will often help if you slip.
- **Trace what the code does, not what you meant.** The most common failure is reading your intention instead of the actual line. Point at each line in the editor as you go.
- **Watch the boundaries:** the first iteration, the last iteration, and the moment a loop or `while` exits.
- **Check every return path.** If a function has three `return` statements, make sure some test reaches each one.

Part 2: Choosing test cases

Do not test randomly. Use **categories**, exactly like equivalence partitioning at work. An **equivalence class** is a group of inputs that the code should treat the same way, so one test from each group is enough.

The test case checklist

Category	Examples	What it catches
Normal / given example	The example from the problem	Basic logic
Empty input	<code>[]</code> , <code>""</code> , <code>None</code> root	Crashes on <code>nums[0]</code> , wrong default
Single element	<code>[5]</code> , <code>"a"</code> , one node	Loops that assume two items

Category	Examples	What it catches
Two elements	[1, 2], [2, 1]	Pointer and swap bugs
Duplicates	[2, 2, 2], "aaa"	Wrong < vs <=, set misuse
Negatives and zero	[-3, 0, 4]	Assumptions like "values are positive"; falsy 0 bugs
Sorted / reverse sorted	[1, 2, 3], [3, 2, 1]	Worst cases, monotonic logic
All the same answer	No valid answer, every item valid	Missing "not found" return
Max size / large values	n = 10 ⁵ , values near limits	Too slow (O(n ²)), deep recursion
Special characters / Unicode	"A man, a plan", "é", emoji	Case and character class handling

You will not test all categories in every interview. Pick the 4-6 that are most **risky** for this specific problem, and say why.

Boundary thinking

Bugs live at **boundaries**: the edges between one behaviour and another. This is boundary value analysis, which you already know from testing.

In code, boundaries are:

- **Index edges**: index 0, index `len - 1`, and the off-by-one at `len`.
- **Loop exits**: what happens when `left == right` in a `while left < right` loop?
- **Thresholds in the problem**: "at most k" means test sums equal to k, k - 1 and k + 1.
- **Size edges**: 0, 1, 2 items.

Example: in binary search, a classic boundary question is whether the condition is `left < right` or `left <= right`. Test with a one-element list where the target exists and where it does not. That tiny test exposes most binary search bugs.

Part 3: Writing quick tests

In many Google interviews you cannot run code. But writing tests still matters: it shows you know **what** to check. If the environment allows running code, even better.

Quick asserts

`assert` checks that a condition is true. If not, Python raises `AssertionError`. Writing a few asserts after your function is fast and clear.

```
def reverse_words(s):
    return " ".join(reversed(s.split()))

assert reverse_words("the sky is blue") == "blue is sky the"
assert reverse_words("  hello  world ") == "world hello" # extra spaces
assert reverse_words("") == "" # empty
assert reverse_words("one") == "one" # single word
print("all passed")
```

Note how each assert has a short comment naming its category. This tells the interviewer that the tests are deliberate.

A small pytest file

If you are asked "How would you test this properly?", show a parametrised pytest. You know pytest from work, so this is a chance to be confident.

```
import pytest
from solution import reverse_words # your function

@pytest.mark.parametrize("s, expected", [
    ("the sky is blue", "blue is sky the"), # normal
    ("  hello  world ", "world hello"), # extra spaces
    ("", ""), # empty
    ("one", "one"), # single word
    ("a b", "b a"), # two words
])
def test_reverse_words(s, expected):
    assert reverse_words(s) == expected
```

Testing against a brute force

A powerful idea: compare your fast solution with a slow, obviously correct brute force on many random inputs. This is called **randomised differential testing**. You do not usually have time to write it in a 45-minute round, but **mentioning** it shows real testing maturity.

```
import random

def brute_max_profit(prices):
    best = 0
    for i in range(len(prices)):
        for j in range(i + 1, len(prices)):
            best = max(best, prices[j] - prices[i])
    return best

for _ in range(1000):
    prices = [random.randint(0, 20) for _ in range(random.randint(0, 8))]
    assert max_profit(prices) == brute_max_profit(prices), prices
print("1000 random tests passed")
```

Small random sizes (0-8) give many edge cases, including empty lists, quickly.

Part 4: Debugging by hand

When your dry run gives a wrong answer, do not panic and do not rewrite everything. Debug like you would triage a failing test:

1. **Reproduce:** find the smallest input that fails. Shrink `[3, 1, 4, 1, 5]` to `[1, 1]` if that still fails.
2. **Locate:** trace that tiny input and find the first line where a variable gets a value you did not expect.
3. **Explain the cause** aloud: "The loop starts at 1, so the first element is never compared."
4. **Fix the minimum:** change only what is needed.
5. **Re-test:** rerun the failing case and one case that passed before (a mini regression test).

Saying "Let me find the smallest failing input" is a strong signal to an interviewer. It shows a calm, systematic process.

Part 5: Talking about tests aloud

How you **say** it matters, because the interviewer writes feedback from what they hear. Useful phrases:

- "Before I say I am done, let me trace through the example."
- "Now let me test some edge cases. The risky ones here are an empty list, all duplicates, and negative numbers."
- "For the empty case, the loop does not run, so I return 0, which matches what we agreed."
- "I found a bug: for a single element, the right pointer starts out of range. Let me fix that."
- "If I had a test environment, I would also compare this against the brute force on random inputs."
- "For performance, I would test with $n = 10^5$ to confirm it runs in time."

What test-engineering interviewers expect

For SWE-Test, Test Engineer or SDET style roles, interviewers commonly look for more than "it works on the example". Show that you can:

- **Enumerate test categories** systematically, not randomly.
- **Prioritise** by risk: "The highest risk is duplicates, because of the `<=` comparison."
- **Separate** functional tests from performance and robustness tests.
- **Think about invalid input:** "Should I validate that `k` is non-negative, or can I assume valid input?" Ask, and do what the interviewer prefers.
- **Discuss how you would automate** tests: parametrised unit tests, property-based tests, a brute-force oracle.

An **oracle** is the source of truth that tells you the expected output. A brute-force solution is a great oracle. A **property** is a rule that must always hold, such as "the output of a sort is in non-decreasing order and has the same elements as the input". Mentioning these terms naturally shows depth.

Worked example 1: Valid Palindrome

Problem: Given a string `s`, return `True` if it is a palindrome after converting to lowercase and removing all non-alphanumeric characters. Example: "A man, a plan, a canal: Panama" returns `True`.

The solution

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        if not s[left].isalnum():          # skip non-letters/digits
            left += 1
        elif not s[right].isalnum():
            right -= 1
        elif s[left].lower() != s[right].lower():
            return False                  # mismatch found
        else:
            left += 1
            right -= 1
    return True

print(is_palindrome("A man, a plan, a canal: Panama")) # True
```

Dry run with "a,b A"

left	right	s[left]	s[right]	action
0	4	a	A	match, move both
1	3	,	(space)	skip left
2	3	b	(space)	skip right
2	2	-	-	loop ends (left == right)

Returns `True`. "abA" is not a palindrome, but after ignoring case "aba" is. Correct.

Test plan said aloud

"The risky areas are: empty and single-character strings, strings with only punctuation, case differences, digits, and Unicode."

```
assert is_palindrome("") is True          # empty: loop never runs
assert is_palindrome("a") is True         # single char
assert is_palindrome(".,!") is True       # only punctuation -> empty -> True
assert is_palindrome("ab") is False       # two chars, mismatch
assert is_palindrome("0P") is False       # digit vs letter (classic trap)
assert is_palindrome("Aa") is True        # case-insensitive
assert is_palindrome("race a car") is False # given negative example
```

Unicode discussion

Python's `str.isalnum()` and `str.lower()` work on Unicode, so "Été" is treated as letters. Is that what the problem wants? Ask: "Should I treat only ASCII letters and digits as alphanumeric, or any Unicode letter?" Also note that some characters change length when lowercased or have different forms (for example, an accented letter can be one code point or a letter plus a combining accent). A full solution might normalise the string first with `unicodedata.normalize`. You do not need to code this; raising it shows you think about real-world input, like a tester.

Complexity

Time $O(n)$: each pointer moves at most n steps. Space $O(1)$: no copy of the string.

Worked example 2: Merge Intervals

Problem: Given a list of intervals `[start, end]`, merge all overlapping intervals and return the result. Example: `[[1,3],[2,6],[8,10]]` returns `[[1,6],[8,10]]`.

The solution

```
def merge(intervals):
    if not intervals:
        return []
    intervals = sorted(intervals, key=lambda x: x[0]) # sort by start
    merged = [list(intervals[0])]
    for start, end in intervals[1:]:
        last = merged[-1]
        if start <= last[1]: # overlaps (or touches)
            last[1] = max(last[1], end) # extend the last interval
        else:
            merged.append([start, end])
    return merged

print(merge([[1, 3], [2, 6], [8, 10]])) # [[1, 6], [8, 10]]
```

Dry run with `[[8,10],[1,3],[2,6]]` (unsorted on purpose)

After sorting: `[[1,3],[2,6],[8,10]]`. `merged = [[1,3]]`.

current	last	overlap?	merged after
[2,6]	[1,3]	$2 \leq 3$ yes	[[1,6]]
[8,10]	[1,6]	$8 \leq 6$ no	[[1,6],[8,10]]

Correct.

Test plan said aloud

"The interesting boundaries are touching intervals, intervals fully inside others, unsorted input, and empty input."

```
assert merge([]) == [] # empty
assert merge([[1, 4]]) == [[1, 4]] # single
assert merge([[1, 4], [4, 5]]) == [[1, 5]] # touching: boundary
assert merge([[1, 10], [2, 3]]) == [[1, 10]] # fully contained
assert merge([[5, 6], [1, 2]]) == [[1, 2], [5, 6]] # unsorted, no overlap
assert merge([[1, 2], [1, 2]]) == [[1, 2]] # duplicates
assert merge([[-5, -1], [-2, 0]]) == [[-5, 0]] # negatives
```

The "fully contained" test is important. A common bug is writing `last[1] = end` instead of `last[1] = max(last[1], end)`. With `[[1,10],[2,3]]`, that bug gives `[[1,3]]`. This one test catches it, which is why you choose tests by risk.

The "touching" case `[1,4],[4,5]` depends on the rules. Ask: "Do intervals that only touch count as overlapping?" Here we assume yes (`<=`). If not, change to `<`.

Questions a tester would raise

- Does the function modify the caller's list? We used `sorted(...)` and `list(...)` copies, so the input is not changed. Mention this: side effects are a real source of bugs.
- What if an interval has `start > end`? Invalid input: ask whether to validate.
- Large input: sorting is $O(n \log n)$, so 10^5 intervals is fine.

Complexity

Time $O(n \log n)$ for the sort, plus $O(n)$ for the merge pass. Space $O(n)$ for the output and the sorted copy.

Tester's corner

- Everything in this chapter is your daily QA skill set: equivalence classes, boundary values, regression, minimal repro, root cause. You are applying it to your own code instead of someone else's.
- A brute-force solution is a test oracle; comparing fast and slow solutions on random inputs is differential testing.
- Properties such as "output is sorted and has the same elements" lead to property-based testing (for example, with the Hypothesis library in Python).
- Side effects on inputs (mutating the caller's list) are a classic defect; check for them like you check for test pollution.
- Ask about invalid input instead of assuming. It is the coding version of clarifying requirements with a product owner.
- Prioritise tests by risk and say why. Interviewers notice the reasoning, not just the list.

Key takeaways

- Never stop at "I think it works": dry run the example, then test risky edge cases.
- Dry run with a tiny input, a variable table, and values spoken aloud; trace what the code does, not what you meant.
- Use test categories: empty, single, two, duplicates, negatives/zero, sorted/reverse, no answer, max size, Unicode.
- Bugs live at boundaries: index edges, loop exits, thresholds like "at most k", and sizes 0, 1, 2.
- Write quick asserts with category comments; offer pytest parametrisation and brute-force comparison as the "proper" test plan.
- Debug by shrinking to the smallest failing input, finding the first wrong value, fixing minimally, then re-testing.
- For test roles, show systematic, risk-based thinking and mention oracles, properties and automation.

Quiz

1. What is a dry run? A) Running code with no input B) Executing code by hand line by line, tracking variables C) Running only the tests D) Deleting unused code
2. Why should you use a small input for a dry run?
3. True or false: testing the given example is usually enough in an interview.
4. Which test would catch the bug `last[1] = end` in merge intervals? A) `[]` B) `[[1, 4]]` C) `[[1, 10], [2, 3]]` D) `[[5, 6], [1, 2]]`
5. Name three boundary areas in code where bugs often hide.
6. What is a test oracle, and what is a good oracle in a coding interview?
7. Your dry run gives a wrong answer on `[3, 1, 4, 1, 5]`. What would you do first?
8. True or false: `is_palindrome("0P")` should return `False`.
9. Which phrase best shows testing maturity? A) "It works." B) "I'll trust it." C) "The risky cases are empty input and duplicates; let me trace them." D) "Can you run it for me?"
10. The problem does not say whether input can be invalid. What would you do?

Answer key

1. **B** - A dry run executes the code in your head or on paper, tracking every variable.
2. **Speed and clarity** - Small inputs (3-5 elements) are fast to trace and still expose most logic bugs.
3. **False** - The example covers only the normal case; you must also test risky edge cases.
4. **C** - With a contained interval, the buggy line shrinks the end from 10 to 3; the correct `max` keeps 10.
5. **Index edges, loop exits, and thresholds** - Also size edges like 0, 1 and 2 elements.
6. **Source of expected output** - An oracle tells you the correct answer; a slow but simple brute-force solution is a great oracle.
7. **Shrink the input** - Find the smallest failing input, then trace it to the first wrong variable value.
8. **True** - '0' and 'p' are different characters even after lowercasing, so it is not a palindrome.
9. **C** - It names risky categories and commits to checking them, which is systematic, risk-based testing.
10. **Ask the interviewer** - Ask whether to validate or assume valid input, then follow their answer and mention it in your tests.

Flashcards

- **Q:** What is a dry run? — **A:** Executing code by hand, line by line, tracking variable values in a table.
- **Q:** What size input is best for a dry run? — **A:** Small: 3-5 elements.
- **Q:** What are the core edge-case categories? — **A:** Empty, single, two elements, duplicates, negatives/zero, sorted/reverse, no answer, max size, Unicode.
- **Q:** Where do bugs usually hide? — **A:** At boundaries: index edges, loop exits, thresholds, and sizes 0, 1, 2.
- **Q:** What is a test oracle? — **A:** The source of truth for expected output, such as a brute-force solution.
- **Q:** What is randomised differential testing? — **A:** Comparing a fast solution with a brute force on many random inputs.
- **Q:** What is the first step when a dry run fails? — **A:** Shrink to the smallest failing input.
- **Q:** What is a property in property-based testing? — **A:** A rule that must always hold, like "sorted output has the same elements in order".
- **Q:** What bug does the "fully contained interval" test catch? — **A:** Overwriting the end with `end` instead of `max(last_end, end)`.
- **Q:** What should you do when input validity is unclear? — **A:** Ask the interviewer whether to validate or assume valid input.
- **Q:** Why add comments to each assert? — **A:** To show each test targets a deliberate category or risk.

Expert Extras: Beyond NeetCode 150

In this chapter:

- Learn advanced tools that sometimes appear in strong interview loops
- Know when to use each tool, with working Python code and its complexity
- Cover monotonic deque, union-find, topological sort variants, Fenwick and segment trees, KMP and Rabin-Karp, bitmask DP, Dijkstra variants and 0-1 BFS, LRU cache, reservoir sampling and randomised algorithms

Time: ~90 minutes (study over several days) | **Level:** Expert

How to use this chapter

The NeetCode 150 list covers most interview patterns. This chapter is the "extra 10%". Study it only **after** you are comfortable with the core patterns (around week 11 of the plan). Do not try to memorise every line. For each tool, learn three things:

1. **The signal:** which words in a problem should make you think of it.
2. **The core idea** in one sentence.
3. **The complexity**, so you can explain why it beats the simple approach.

Then type each code block from memory two or three times over the next weeks.

1. Monotonic deque

When to use: "maximum (or minimum) in every window of size k", or DP where you need the best value from the last k positions.

Idea: a **deque** (double-ended queue, from `collections`) lets you add and remove at both ends in $O(1)$. Keep indices in the deque so their values are **decreasing**. The front is always the maximum of the current window. Think of a queue at a bus stop where a tall person arriving makes all shorter people in front of them leave, because they can never be "the tallest" again while the tall person is there.

```

from collections import deque

def max_sliding_window(nums, k):
    dq, out = deque(), []
    # dq holds indices; values decreasing
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] <= x:
            dq.pop()
            # smaller values can never be max again
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()
            # front index left the window
        if i >= k - 1:
            out.append(nums[dq[0]])
    return out

print(max_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3))
# Output: [3, 3, 5, 5, 6, 7]

```

Complexity: $O(n)$ time, because each index is pushed and popped at most once. $O(k)$ space. A heap would give $O(n \log n)$.

2. Union-find (disjoint set union)

When to use: "connected components", "are these two in the same group?", "redundant connection", "number of provinces", accounts merging, and Kruskal's minimum spanning tree.

Idea: every element points to a **parent**. Following parents leads to the group's **root**, which names the group. Two improvements make it very fast:

- **Path compression:** during `find`, point every visited node directly at the root, so later finds are short.
- **Union by rank:** attach the shorter tree under the taller one, so trees stay flat. **Rank** is an upper bound on tree height.

Think of company teams: each employee knows a manager, and following managers leads to the department head. Path compression is like everyone saving the head's phone number after asking once.

```

class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))    # each node is its own root
        self.rank = [0] * n
        self.count = n                  # number of separate groups

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])    # path compression
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False    # already connected (cycle edge)
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra    # make ra the taller tree
        self.parent[rb] = ra    # union by rank
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        self.count -= 1
        return True

uf = UnionFind(5)
uf.union(0, 1); uf.union(3, 4)
print(uf.count, uf.find(0) == uf.find(1))    # Output: 3 True

```

Complexity: with both improvements, each operation is amortised $O(\alpha(n))$, where α is the inverse Ackermann function. It grows so slowly that it is at most 4 or 5 for any realistic n , so you can call it "nearly constant". Space $O(n)$.

3. Topological sort variants

A **topological order** lists the nodes of a **directed acyclic graph** (DAG: arrows, no cycles) so that every arrow goes from earlier to later. Think of getting ready in the morning: socks before shoes, shirt before tie.

When to use: "prerequisites", "build order", "course schedule", "alien dictionary", "can all tasks finish?"

Kahn's algorithm (BFS)

Repeatedly take a node with **in-degree** 0 (no remaining arrows coming in).


```

from collections import deque

def topo_sort(n, edges):
    # edge (a, b): a before b
    graph = [[] for _ in range(n)]
    indeg = [0] * n
    for a, b in edges:
        graph[a].append(b)
        indeg[b] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order if len(order) == n else [] # [] means a cycle exists

print(topo_sort(4, [(0, 1), (0, 2), (1, 3), (2, 3)])) # [0, 1, 2, 3]

```

Variants to know

- **Cycle detection:** if the order has fewer than n nodes, there is a cycle.
- **Lexicographically smallest order:** replace the deque with a min-heap (`heapq`), so you always pick the smallest available node. Time becomes $O(V \log V + E)$.
- **Minimum number of rounds** ("parallel courses", fewest semesters): process the queue **level by level**; the number of levels is the answer.
- **DFS with three colours:** white (unvisited), grey (on the current path), black (done). Meeting a grey node means a cycle. The reverse of the finishing order is a topological order.

```

def has_cycle(n, edges):
    graph = [[] for _ in range(n)]
    for a, b in edges:
        graph[a].append(b)
    state = [0] * n # 0 white, 1 grey, 2 black
    def dfs(u):
        state[u] = 1
        for v in graph[u]:
            if state[v] == 1 or (state[v] == 0 and dfs(v)):
                return True # back edge to grey node = cycle
        state[u] = 2
        return False
    return any(state[i] == 0 and dfs(i) for i in range(n))

print(has_cycle(3, [(0, 1), (1, 2), (2, 0)])) # Output: True

```

Complexity: $O(V + E)$ time and space for both, where V is the number of nodes and E the number of edges.

4. Fenwick tree (binary indexed tree)

When to use: prefix sums **with updates**. For example, "range sum query - mutable", "count of smaller numbers after self", counting inversions.

Idea: a plain prefix-sum array answers range sums in $O(1)$ but needs $O(n)$ to update. A **Fenwick tree** stores partial sums so that both updates and prefix queries take $O(\log n)$. Each index is responsible for a block whose size is its lowest set bit, found with $i \& -i$.

Think of a cricket scoreboard where you keep totals for overs 1-8, 9-12, 13-14 and so on. To get the total up to over 14, you add only a few blocks instead of every ball.

```
class Fenwick:
    def __init__(self, n):
        self.tree = [0] * (n + 1)    # 1-based inside

    def update(self, i, delta):        # add delta at index i (0-based)
        i += 1
        while i < len(self.tree):
            self.tree[i] += delta
            i += i & -i                # move to next responsible block

    def prefix(self, i):               # sum of nums[0..i]
        i += 1
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i                # drop the lowest set bit
        return total

    def range_sum(self, l, r):         # sum of nums[l..r]
        return self.prefix(r) - self.prefix(l - 1)

fw = Fenwick(5)
for i, x in enumerate([2, 1, 5, 3, 4]):
    fw.update(i, x)
print(fw.range_sum(1, 3))            # Output: 9 (1 + 5 + 3)
```

Complexity: $O(\log n)$ per update and query; $O(n \log n)$ to build this way; $O(n)$ space.

5. Segment tree

When to use: range queries with updates where the operation is not just a sum: range minimum, range maximum, GCD, or "count of something in a range". It is more flexible than a Fenwick tree.

Idea: a binary tree where each leaf holds one array element and each internal node holds the answer (for example, the sum) for the range covered by its children. The root covers the whole array. A query for any range combines $O(\log n)$ nodes. An update changes one leaf and the $O(\log n)$ nodes above it.

Picture a cricket tournament bracket. Each match node stores the winner of its sub-bracket. If one team's score changes, you update only the path from that team to the final.

The compact **bottom-up** version stores the tree in an array of size $2n$. Leaves are at positions n to $2n-1$, and node i has children $2i$ and $2i+1$.

```

class SegmentTree:
    def __init__(self, nums):
        self.n = len(nums)
        self.tree = [0] * self.n + list(nums)      # Leaves at n..2n-1
        for i in range(self.n - 1, 0, -1):
            self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1]

    def update(self, i, val):                       # set nums[i] = val
        i += self.n
        self.tree[i] = val
        while i > 1:
            i //= 2
            self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1]

    def query(self, l, r):                         # sum of nums[l..r]
        res, l, r = 0, l + self.n, r + self.n + 1
        while l < r:
            if l & 1:
                res += self.tree[l]; l += 1
            if r & 1:
                r -= 1; res += self.tree[r]
            l //= 2; r //= 2
        return res

st = SegmentTree([2, 1, 5, 3, 4])
st.update(2, 10)
print(st.query(1, 3))                            # Output: 14 (1 + 10 + 3)

```

To make a range-minimum tree, replace `+` with `min` and start `res` at `float('inf')`. **Complexity:** $O(n)$ build, $O(\log n)$ per query and update, $O(n)$ space. Range *updates* (add 5 to every element in a range) need an advanced trick called **lazy propagation**; know the name and the idea (delay updates to children until needed).

6. String matching: KMP and Rabin-Karp

When to use: "find a pattern in a text" (`strStr`), "repeated substring pattern", "shortest palindrome", "longest duplicate substring". Python's `text.find(p)` works in practice, but interviewers may ask how to guarantee linear time.

KMP

KMP (Knuth-Morris-Pratt) never moves backwards in the text. It precomputes the **LPS array** (longest proper prefix that is also a suffix) for the pattern. On a mismatch, LPS tells you how much of the pattern still matches, so you skip ahead instead of restarting.

```

def build_lps(p):
    lps, length = [0] * len(p), 0
    for i in range(1, len(p)):
        while length and p[i] != p[length]:
            length = lps[length - 1] # fall back to shorter border
        if p[i] == p[length]:
            length += 1
        lps[i] = length
    return lps

def kmp_search(text, p):
    if not p:
        return 0
    lps, j = build_lps(p), 0
    for i, ch in enumerate(text):
        while j and ch != p[j]:
            j = lps[j - 1]
        if ch == p[j]:
            j += 1
        if j == len(p):
            return i - len(p) + 1 # start index of first match
    return -1

print(build_lps("ababaca")) # [0, 0, 1, 2, 3, 0, 1]
print(kmp_search("abxabcabcab", "abcaby")) # Output: 6

```

Complexity: $O(n + m)$ time for text length n and pattern length m ; $O(m)$ space.

Rabin-Karp (rolling hash)

Idea: compute a numeric **hash** of the pattern and of each window of the text. A **rolling hash** updates the window hash in $O(1)$ when the window slides by one character: remove the leftmost character's contribution, multiply, add the new character. Only when hashes match do you compare the actual strings, because different strings can share a hash (a **collision**).

```

def rabin_karp(text, p):
    n, m = len(text), len(p)
    if m == 0:
        return 0
    if m > n:
        return -1
    base, mod = 256, 1_000_000_007
    high = pow(base, m - 1, mod) # weight of the leftmost char
    hp = ht = 0
    for i in range(m):
        hp = (hp * base + ord(p[i])) % mod
        ht = (ht * base + ord(text[i])) % mod
    for i in range(n - m + 1):
        if hp == ht and text[i:i + m] == p: # verify on hash match
            return i
        if i < n - m: # roll the window
            ht = ((ht - ord(text[i]) * high) * base + ord(text[i + m])) % mod
    return -1

print(rabin_karp("abxabcabcab", "abcaby")) # Output: 6

```

Complexity: $O(n + m)$ expected; $O(n \cdot m)$ worst case if many collisions. Rolling hashes shine when comparing many substrings, for example binary search on length plus hashing for "longest duplicate substring".

7. Bitmask DP

When to use: n is small (around 20 or less) and the state is "which items have been used or visited". Examples: travelling salesman, "shortest path visiting all nodes", assigning tasks to workers, "partition into k equal-sum subsets".

Idea: represent a set of items as the bits of an integer, called a **mask**. Bit i is 1 if item i is in the set. `mask | (1 << i)` adds item i ; `(mask >> i) & 1` checks it. Then do DP over masks.

Think of a delivery partner with a checklist of up to 15 addresses. Each tick pattern on the checklist is one mask.

```
def tsp(dist):
    """Shortest tour starting and ending at city 0, visiting all cities."""
    n, INF = len(dist), float('inf')
    dp = [[INF] * n for _ in range(1 << n)] # dp[mask][u]: at u, visited mask
    dp[1][0] = 0
    for mask in range(1 << n):
        for u in range(n):
            if dp[mask][u] == INF:
                continue
            for v in range(n):
                if (mask >> v) & 1:
                    continue # v already visited
                nxt = mask | (1 << v)
                dp[nxt][v] = min(dp[nxt][v], dp[mask][u] + dist[u][v])
    full = (1 << n) - 1
    return min(dp[full][u] + dist[u][0] for u in range(n))

d = [[0, 10, 15, 20], [10, 0, 35, 25], [15, 35, 0, 30], [20, 25, 30, 0]]
print(tsp(d)) # Output: 80
```

Complexity: $O(2^n \cdot n^2)$ time and $O(2^n \cdot n)$ space. That is fine for n up to about 15-20 and impossible beyond, which is exactly the constraint signal from chapter dsa-03.

8. Dijkstra variants and 0-1 BFS

When to use Dijkstra: shortest path from one source in a graph with **non-negative** edge weights. Signals: "minimum cost", "minimum time", "network delay", weighted grid.

Idea: always expand the closest unfinished node, using a min-heap. Think of Google Maps style routing: from your location, you keep extending the cheapest known route first.

```

import heapq

def dijkstra(n, edges, src):
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        graph[u].append((v, w))
    dist = [float('inf')] * n
    dist[src] = 0
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue # stale entry, skip
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return dist

print(dijkstra(3, [(0, 1, 4), (0, 2, 1), (2, 1, 2)], 0)) # [0, 3, 1]

```

Complexity: $O((V + E) \log V)$ time, $O(V + E)$ space.

Variants to recognise

- **Minimax path** ("path with minimum effort", "swim in rising water"): the cost of a path is its **largest** edge. Replace `d + w` with `max(d, w)`. Dijkstra still works.
- **State expansion**: when the cost depends on extra information (stops used, keys collected, fuel left), make the node a tuple like `(city, stops)` and run Dijkstra or BFS on the bigger state graph. "Cheapest flights within k stops" is often solved with Bellman-Ford limited to $k+1$ rounds or BFS by levels.
- **Negative edges**: Dijkstra is wrong. Use Bellman-Ford ($O(V \cdot E)$).
- **All edges weight 1**: plain BFS is enough.

0-1 BFS

When to use: edge weights are only 0 or 1. Examples: "minimum obstacles to remove in a grid", "minimum cost to make a valid path in a grid".

Idea: use a deque. A 0-weight edge does not increase distance, so push that neighbour to the **front**; a 1-weight edge goes to the **back**. The deque stays sorted by distance without a heap.

```

from collections import deque

def zero_one_bfs(n, graph, src):          # graph[u] = [(v, w)], w in {0, 1}
    dist = [float('inf')] * n
    dist[src] = 0
    dq = deque([src])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return dist

g = [[(1, 1), (2, 0)], [], [(1, 0)]]
print(zero_one_bfs(3, g, 0))             # Output: [0, 0, 0]

```

Complexity: $O(V + E)$ time, which beats Dijkstra's log factor.

9. LRU cache design

When to use: "Design an LRU cache with $O(1)$ get and put". It is a very common design-plus-code question. **LRU** means Least Recently Used: when the cache is full, remove the item that was used longest ago. Think of a small shelf at a kirana shop: items customers asked for recently stay on the shelf; the item nobody asked for in the longest time goes back to the godown.

Version A: OrderedDict

`collections.OrderedDict` remembers insertion order and can move a key to the end in $O(1)$.

```

from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.data = OrderedDict()          # oldest first, newest last

    def get(self, key):
        if key not in self.data:
            return -1
        self.data.move_to_end(key)         # mark as recently used
        return self.data[key]

    def put(self, key, value):
        if key in self.data:
            self.data.move_to_end(key)
        self.data[key] = value
        if len(self.data) > self.cap:
            self.data.popitem(last=False)   # evict least recently used

```

Some interviewers accept this; many then say "Now do it without OrderedDict." Be ready.

Version B: dict plus doubly linked list

A **doubly linked list** has nodes with both `prev` and `next` pointers, so you can remove any node in $O(1)$ if you hold a reference to it. The dict maps key to node. Two **sentinel** (dummy) nodes, `head` and `tail`, remove special cases for empty lists.

```
class Node:
    def __init__(self, key=0, val=0):
        self.key, self.val = key, val
        self.prev = self.next = None

class LRU:
    def __init__(self, capacity):
        self.cap, self.map = capacity, {}
        self.head, self.tail = Node(), Node()  # sentinels
        self.head.next, self.tail.prev = self.tail, self.head

    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev

    def _add_front(self, node):  # front = most recent
        node.prev, node.next = self.head, self.head.next
        self.head.next.prev = node
        self.head.next = node
```

```
# (continuation of class LRU)
def get(self, key):
    if key not in self.map:
        return -1
    node = self.map[key]
    self._remove(node)
    self._add_front(node)
    return node.val

def put(self, key, value):
    if key in self.map:
        self._remove(self.map[key])
    node = Node(key, value)
    self.map[key] = node
    self._add_front(node)
    if len(self.map) > self.cap:
        lru = self.tail.prev  # least recently used
        self._remove(lru)
        del self.map[lru.key]  # why Node stores key
```

Complexity: $O(1)$ for `get` and `put`; $O(\text{capacity})$ space. Test it with capacity 1, updating an existing key, and getting a key just before eviction.

10. Reservoir sampling

When to use: "pick a random element (or k elements) from a stream" or "from a linked list of unknown length", with each element equally likely, using $O(k)$ memory.

Idea: keep the first k items. For item number i (0-based, $i \geq k$), pick a random integer j from 0 to i . If $j < k$, replace slot j with the new item. Each item ends up kept with probability $k / (\text{number of items seen})$.

Think of choosing one lucky customer from a queue whose length you do not know, while only remembering one name at a time.

```
import random

def reservoir_sample(stream, k):
    res = []
    for i, x in enumerate(stream):
        if i < k:
            res.append(x)
        else:
            j = random.randint(0, i)    # inclusive on both ends
            if j < k:
                res[j] = x              # replace with probability k/(i+1)
    return res

print(len(reservoir_sample(range(1000), 5)))    # Output: 5
```

Complexity: $O(n)$ time for n items, $O(k)$ space.

11. Randomised algorithm basics

A **randomised algorithm** uses random choices to get good **expected** performance, protecting against bad inputs. Two words to know:

- **Las Vegas algorithm:** always correct; only the running time is random (example: quickselect).
- **Monte Carlo algorithm:** fixed running time, but may be wrong with a small probability (example: some primality tests).

Quickselect

When to use: "k-th largest / smallest element" in expected $O(n)$, faster than sorting.

```
import random

def quickselect(nums, k):
    # k-th smallest, k is 1-based
    # random pivot avoids bad cases
    pivot = random.choice(nums)
    lows = [x for x in nums if x < pivot]
    highs = [x for x in nums if x > pivot]
    equal = len(nums) - len(lows) - len(highs)
    if k <= len(lows):
        return quickselect(lows, k)
    if k <= len(lows) + equal:
        return pivot
    return quickselect(highs, k - len(lows) - equal)

print(quickselect([3, 2, 1, 5, 6, 4], 2))    # Output: 2
```

Complexity: expected $O(n)$ time, worst case $O(n^2)$ (very unlikely with random pivots). This simple version uses $O(n)$ extra space; an in-place partition version uses $O(1)$ extra.

Fisher-Yates shuffle

When to use: "shuffle an array" so every order is equally likely.

```
def shuffle(a):  
    for i in range(len(a) - 1, 0, -1):  
        j = random.randint(0, i)      # pick from the unshuffled part  
        a[i], a[j] = a[j], a[i]  
    return a
```

$O(n)$ time, $O(1)$ extra space. A common wrong version picks `j` from the whole array every time, which makes some orders more likely than others. Python's `random.shuffle` implements a correct shuffle.

Summary table

Tool	Signal	Time
Monotonic deque	Max/min of every window	$O(n)$
Union-find	Dynamic connectivity, groups	$\sim O(1)$ amortised per op
Topological sort	Prerequisites, build order	$O(V + E)$
Fenwick tree	Prefix sums with updates	$O(\log n)$ per op
Segment tree	Range min/max/sum with updates	$O(\log n)$ per op
KMP	Guaranteed linear pattern search	$O(n + m)$
Rabin-Karp	Many substring comparisons	$O(n + m)$ expected
Bitmask DP	$n \leq \sim 20$, subsets of items as state	$O(2^n \cdot n^2)$ typical
Dijkstra	Weighted shortest path, weights ≥ 0	$O((V + E) \log V)$
0-1 BFS	Weights only 0 or 1	$O(V + E)$
LRU cache	$O(1)$ get/put with eviction	$O(1)$ per op
Reservoir sampling	Random pick from a stream	$O(n)$, $O(k)$ space
Quickselect	k-th element	Expected $O(n)$

Tester's corner

- Random algorithms are hard to test: fix the seed (`random.seed(42)`) for repeatable unit tests, and use statistical tests (run many trials, check frequencies) for fairness.
- The wrong Fisher-Yates version is a great example of a bug that passes every functional test but fails a distribution test.

- LRU caches need state-sequence tests: a series of `put` and `get` calls with a check after each step, including capacity 1 and updates of existing keys.
- Union-find, Fenwick and segment trees are perfect for differential testing against a brute force on random inputs.
- Rabin-Karp shows why you must verify after a hash match: collisions are rare but real. The same idea applies to checksums in test data validation.
- Topological sort appears in real test infrastructure: ordering test setup steps, and build systems that run tasks in dependency order (the Bazel documentation describes builds as a graph of dependencies).

Key takeaways

- Study these tools only after the core patterns; learn the signal, the idea and the complexity for each.
- Monotonic deque gives $O(n)$ window max/min; union-find with path compression and union by rank is nearly $O(1)$ per operation.
- Kahn's algorithm gives a topological order and detects cycles; use a heap for the smallest order and levels for minimum rounds.
- Fenwick trees handle prefix sums with updates; segment trees handle general range queries, both in $O(\log n)$.
- KMP guarantees $O(n + m)$ matching; Rabin-Karp uses a rolling hash and must verify on matches.
- Bitmask DP fits n up to about 20; Dijkstra needs non-negative weights; 0-1 BFS handles 0/1 weights in $O(V + E)$.
- Know LRU cache both with `OrderedDict` and with dict plus a doubly linked list; know reservoir sampling and `quickselect`.

Quiz

1. Which tool gives the maximum of every window of size k in $O(n)$? A) Heap B) Monotonic deque C) Fenwick tree D) Trie
2. What two optimisations make union-find nearly constant time per operation?
3. True or false: Dijkstra's algorithm works correctly with negative edge weights.
4. Kahn's algorithm returns fewer than n nodes. What does that mean?
5. Which structure supports both point updates and range minimum queries in $O(\log n)$? A) Prefix sum array B) Segment tree C) Hash map D) Stack
6. Why must Rabin-Karp compare the actual strings when hashes match?
7. A problem has $n = 16$ cities and asks for the shortest route visiting all. Which technique fits?
8. In 0-1 BFS, where do you push a neighbour reached by a 0-weight edge?
9. Your LRU cache uses a doubly linked list. Why does each node store its key as well as its value?
10. You need to test a shuffle function. What would you do to check it is fair?

Answer key

1. **B** - A monotonic deque keeps candidates in decreasing order, so each index is added and removed once.
2. **Path compression and union by rank** - Together they give amortised $O(\alpha(n))$ per operation.
3. **False** - Dijkstra assumes non-negative weights; use Bellman-Ford when negative edges exist.
4. **A cycle exists** - Nodes in a cycle never reach in-degree 0, so they are never added to the order.
5. **B** - A segment tree answers range queries and handles point updates in $O(\log n)$.
6. **Collisions** - Different strings can have the same hash, so a hash match must be verified.
7. **Bitmask DP** - With n around 16, $O(2^n \cdot n^2)$ is feasible and the state is the set of visited cities.
8. **The front** - A 0-weight edge does not increase distance, so it goes to the front to keep the deque in distance order.
9. **To delete from the dict on eviction** - When removing the tail node, you need its key to delete the dict entry in $O(1)$.
10. **Statistical test** - Run many shuffles of a small array, count how often each order appears, and check the counts are roughly equal.

Flashcards

- **Q:** What does a monotonic deque store for sliding window maximum? — **A:** Indices whose values are in decreasing order; the front is the window max.
- **Q:** What is path compression? — **A:** During find, pointing each visited node directly at the root.
- **Q:** What is union by rank? — **A:** Attaching the shorter tree under the taller one to keep trees flat.
- **Q:** How does Kahn's algorithm detect a cycle? — **A:** The output order has fewer than n nodes.
- **Q:** What does $i \& -i$ give in a Fenwick tree? — **A:** The lowest set bit of i , which is the size of the block index i covers.
- **Q:** What is the LPS array in KMP? — **A:** For each prefix, the length of the longest proper prefix that is also a suffix.
- **Q:** What is a rolling hash? — **A:** A hash updated in $O(1)$ as a window slides by one character.
- **Q:** When is bitmask DP feasible? — **A:** When n is small, about 20 or less, and the state is a subset of items.
- **Q:** What is the time complexity of Dijkstra with a heap? — **A:** $O((V + E) \log V)$.
- **Q:** When do you use 0-1 BFS? — **A:** When every edge weight is 0 or 1; it runs in $O(V + E)$.
- **Q:** How does reservoir sampling decide to keep item i (0-based, $i \geq k$)? — **A:** Pick j in $0..i$ at random; if $j < k$, replace slot j .
- **Q:** What is the difference between Las Vegas and Monte Carlo algorithms? — **A:** Las Vegas is always correct with random time; Monte Carlo has fixed time but may be wrong with small probability.

The Mock Interview Playbook

In this chapter:

- Run a 45-minute coding round minute by minute, with English phrases for each step
- Recover when you are stuck and take hints gracefully
- Handle nerves before and during the interview
- Follow a final two-week revision plan and a day-of-interview checklist
- Score your own mock interviews with a clear rubric

Time: ~40 minutes | **Level:** Intermediate

Why a playbook

In cricket, a good team does not invent its plan on match day. It has a plan for the powerplay, the middle overs and the death overs, and players have practised each phase in the nets. A coding interview is the same. You already know the framework from chapter dsa-03. This chapter turns it into a **script** for a real 45-minute round, so that under pressure your mouth and hands know what to do.

Practise this playbook in every Sunday test and every day of week 12 (December 21-27, 2026).

The 45-minute round, minute by minute

The timings below are a guide. Interviewers differ; some spend longer on introductions or add a follow-up question. The order matters more than the exact minutes.

Minutes	Phase	Your goal
0-3	Introductions	Calm, friendly start
3-8	Understand and clarify	Restate, examples, 3-5 questions
8-15	Approaches	Brute force, then optimised idea, agree on plan
15-30	Code	Clean, narrated code
30-38	Test and fix	Dry run, edge cases, fix bugs
38-41	Complexity and follow-ups	State Big-O, discuss improvements
41-45	Your questions	Ask 1-2 genuine questions

Minutes 0-3: Introductions

Keep your introduction to 30-45 seconds. Do not tell your whole career story.

"Hi, I'm Sahil. I'm an SDET with about five and a half years of experience, mostly in test automation with Playwright, Selenium and Python, and API testing. Recently I've worked on testing blockchain applications. I'm excited to be here."

If the interviewer introduces themselves, listen and say thank you. A small smile helps you and them relax.

Minutes 3-8: Understand and clarify

Read the problem fully. Do not start talking about solutions in the first ten seconds.

Phrases to use:

- "Let me restate the problem to make sure I understand. We get ..., and we need to return ..."
- "Let me try this small example by hand. For [2, 7, 11, 15] and target 9, the answer is indices 0 and 1."
- "A few clarifying questions. Can the input be empty? Can there be duplicates? Can values be negative?"
- "How large can the input be? That will help me choose the approach."
- "If there are multiple valid answers, can I return any of them?"
- "Can I assume the input is valid, or should I handle invalid input?"

Write the agreed answers as short comments at the top of the document. This becomes your contract and your test list later.

```
# Input: List of ints, may be empty, may have negatives and duplicates
# Output: indices of two numbers summing to target, or [] if none
# n up to 10^5 -> aim for O(n) or O(n log n)
```

Minutes 8-15: Approaches

First state the brute force, then improve it. Then agree on the plan before coding.

- "The straightforward approach is to check every pair. That is $O(n^2)$ time and $O(1)$ space."
- "We can do better. For each number, I need to know quickly whether its partner exists. A hash map gives $O(1)$ lookups, so the whole thing becomes $O(n)$ time and $O(n)$ space."
- "Another option is sorting with two pointers, which is $O(n \log n)$, but it loses the original indices, so I prefer the hash map."
- "Here's my plan in comments. Does this approach sound good to you before I start coding?"

The last question is very important. If the interviewer has a concern, it is much cheaper to hear it now than after you have written 25 lines.

Minutes 15-30: Code

Code steadily. Talk, but less than before. Explain decisions, not every keystroke.

- "I'll use a dictionary called `seen` that maps a value to its index."

- "I'm checking for the partner before inserting the current number, so I don't pair a number with itself."
- "I'll write a small helper for the neighbour check to keep the main loop clean."
- "I'm not sure about the exact name of this method; I'll write it as `popleft` from `collections.deque`, which I believe is correct."

Good habits while coding:

- Use clear names: `left`, `right`, `seen`, `count`, not `a`, `b`, `x2`.
- Keep functions short. Use a helper if logic repeats.
- If you notice a bug while coding, say so and fix it: "Wait, this should be `<=`, because the window can end at the last index."
- Leave a small `# TODO: handle empty input` if you want to come back, and say it aloud so you do not forget.

Minutes 30-38: Test and fix

Never say "Done" right after the last line. Say:

- "Now let me test it. First I'll trace the example."
- "i is 0, num is 2, need is 7, not in seen, so seen becomes {2: 0}. i is 1, num is 7, need is 2, which is in seen, so I return [0, 1]. Correct."
- "Now edge cases: an empty list returns [] because the loop never runs. Duplicates like [3, 3] with target 6: at i = 1, need is 3, seen has 3 at index 0, so [0, 1]. Correct."
- "I see a bug here: for a single-element input, this line reads index 1, which is out of range. Let me fix it."

Chapter dsa-30 covers this phase in detail. As a test engineer, this is where you can clearly beat other candidates.

Minutes 38-41: Complexity and follow-ups

- "Time complexity is $O(n)$, since each element is visited once and dictionary operations are $O(1)$ on average."
- "Space complexity is $O(n)$ for the dictionary in the worst case."
- "If memory were very limited, I could sort and use two pointers for $O(1)$ extra space, but then I'd need to keep track of original indices."
- "If the data came as a stream, I'd keep the dictionary and check each new number as it arrives."

If the interviewer asks a follow-up, treat it like a mini-problem: clarify, propose, and code only if asked.

Minutes 41-45: Your questions

Always have two questions ready. Good questions are genuine and specific:

- "What does a typical week look like for a test engineer on your team?"
- "How does your team decide what to test with unit tests versus integration or end-to-end tests?"

- "What is one challenge your team is working on in test infrastructure right now?"
- "What do you enjoy most about working at Google?"

Avoid questions about your performance in this interview, and avoid salary questions here; those are for the recruiter.

How to recover when stuck

Getting stuck is normal. Interviewers often care more about **how you behave when stuck** than about whether you get stuck. Think of a delivery rider whose route is blocked: a good rider does not freeze; they try a side road and keep the customer informed.

A recovery checklist

When you have had no progress for 2-3 minutes, go through these out loud:

1. **Go back to the example.** "Let me solve a slightly bigger example by hand and watch what I do." Your hands often find the algorithm before your head does.
2. **Simplify the problem.** "What if the array were sorted? What if there were only positive numbers? What if k were 1?" Solve the easier version, then extend it.
3. **Run through the pattern list.** "Could a hash map help? Two pointers? Sorting first? A heap? Binary search on the answer? BFS?" Use the cheat sheet from chapter dsa-03.
4. **Think about what is repeated** in the brute force and how to store it.
5. **Work backwards** from the output: "What would I need to know at the last step to produce the answer?"
6. **Fall back to brute force.** "I'll code the brute force first so we have a working solution, then optimise." A working $O(n^2)$ solution is far better than an unfinished $O(n)$ one.

Phrases that keep the interviewer with you:

- "I'm a bit stuck on how to avoid recomputing this sum. Let me think aloud."
- "One idea is ..., but I see a problem with it: ..."
- "Let me try a different angle."

Never go silent for more than about 30 seconds. If you need to think quietly, say so: "Can I take thirty seconds to think about this?"

How to take hints

Interviewers give hints to help you, and taking a hint well is a positive signal. Taking it badly (ignoring it, or arguing) is a negative one.

- **Listen fully.** Do not interrupt the hint.
- **Repeat it in your words:** "So you're suggesting I think about what the sorted order gives me?"
- **Connect it to the problem:** "If it's sorted, then duplicates are next to each other, so I can skip them with a pointer. That removes the need for a set."

- **Thank them briefly and move on:** "That helps, thank you." Do not apologise many times.
- **If you do not understand the hint,** ask: "Could you say a bit more about what you mean by that?"

A hint is like a fielder's signal from the boundary in cricket. Use it quickly and keep playing.

Handling nerves

Being nervous is normal and even useful; it means you care. The goal is not zero nerves but steady nerves.

Before the interview

- **Practise under realistic conditions.** Week 12 mocks with a timer, a plain editor, and speaking aloud make the real thing feel familiar.
- **Sleep well** the night before. A tired brain makes more off-by-one errors than a nervous one.
- **Prepare your setup** the day before: laptop charged, stable internet, quiet room, water, a notebook and pen.
- **Warm up** one hour before with one easy problem you have solved before. Do not try new hard problems that day.

During the interview

- **Breathe slowly.** Breathe in for 4 counts, hold for 4, out for 6. Do it while reading the problem.
- **Slow down your speech.** Nervous people speak fast. Speaking slowly sounds confident and gives you thinking time.
- **Write things down.** Comments and examples in the document reduce the load on your memory.
- **Treat it as working together.** The interviewer is a future colleague solving a problem with you, not a judge.
- **One bad moment is not a bad interview.** If you make a mistake, fix it calmly and continue. Interviewers see the whole round.

Final two weeks revision plan

Use this for the two weeks before your interview. If your interview is at the end of the 12-week plan, this matches weeks 11 and 12.

Days	Focus	Daily work
14-12 days before	Pattern review 1	Arrays, hashing, two pointers, sliding window, stack: re-code 2 old problems per pattern from memory
11-9 days before	Pattern review 2	Binary search, linked lists, trees, tries, heaps
8-6 days before	Pattern review 3	Graphs, backtracking, DP, greedy, intervals
5-3 days before	Full mocks	One 45-minute mock per day with a partner or recorded; score with the rubric

Days	Focus	Daily work
2 days before	Weak spots	Read the mistakes log; redo the 5 problems you failed most often
1 day before	Light review	Read the cheat sheets and flashcards; one easy warm-up problem; rest early
Interview day	Warm-up only	One easy familiar problem, then stop

Rules for these two weeks:

- **No new topics** in the last 5 days. Deepen what you know.
- **Mix in system design and test design** for 30-45 minutes daily, using Volumes 2 and 3.
- **Practise your leadership and Googlyness stories** aloud twice a week.

Day-of-interview checklist

Night before

- ☐ Laptop charged, charger near you
- ☐ Interview link, time and time zone confirmed (India is UTC+5:30)
- ☐ Camera and microphone tested
- ☐ Quiet room arranged; family informed
- ☐ Clothes ready; water bottle ready

One hour before

- ☐ One easy warm-up problem in a plain editor
- ☐ Read the framework steps and your top 3 mistakes from the log
- ☐ Close extra apps and notifications; silence your phone

During each round

- ☐ Restate the problem and work an example
- ☐ Ask 3-5 clarifying questions
- ☐ State brute force with complexity, then the better approach
- ☐ Get agreement on the plan before coding
- ☐ Narrate while coding
- ☐ Dry run and test edge cases before saying "done"
- ☐ State time and space complexity
- ☐ Ask your questions at the end

After each round

- ☐ Take two minutes to breathe and drink water
- ☐ Do not replay mistakes; the next round is a fresh start

How to self-score a mock

After each mock, score yourself (or ask your mock partner to score you) using this rubric. Each area gets 1 to 4 points. Watch your recording if you have one; memory is too kind.

Area	1 - Weak	2 - Mixed	3 - Good	4 - Strong
Understanding and clarifying	Started coding immediately	Asked one question	Restated and asked key questions	Restated, worked an example, asked all questions that matter
Problem solving	No working idea	Brute force only	Found optimal idea with help	Found optimal idea mostly alone, explained why
Coding	Did not finish or major errors	Finished with several bugs	Clean code, minor bugs	Clean, readable, near-runnable code
Testing	No testing	Traced the example only	Example plus some edge cases	Systematic, risk-based cases; found and fixed own bugs
Complexity	Not stated or wrong	Stated without reasons	Correct with reasons	Correct, with trade-offs and alternatives
Communication	Long silences	Talked sometimes	Clear most of the time	Clear, structured, took hints well

Total out of 24:

- **20-24:** Interview-ready for this difficulty. Increase difficulty.
- **15-19:** Close. Focus on your lowest area for the next week.
- **Below 15:** Go back to pattern practice for the weak topic before the next mock.

Write the score, date, problem and lowest area in a tracker. Over weeks 11 and 12, you want to see the total rise and the lowest area change. If the same area stays lowest for three mocks, make it your only focus for two days.

Tester's corner

- A mock interview is a rehearsal environment, like staging before production. Find your failures there.
- The rubric is a test report with six test areas. Track scores over time like pass rates over builds.
- Your mistakes log plus the "lowest area" column is a defect trend analysis: fix the most frequent root cause first.
- The day-of checklist is a pre-release checklist. Checklists prevent simple, avoidable failures under stress.
- In the testing phase of a round, use the vocabulary you know: boundary values, equivalence classes, negative tests, regression after a fix.
- Questions about how the team decides between unit, integration and end-to-end tests show genuine test-engineering interest.

Key takeaways

- Follow the 45-minute flow: introduce, clarify, approaches, code, test, complexity, your questions.
- Always state the brute force, then the better approach, and get agreement before coding.
- When stuck: return to the example, simplify, scan the patterns, and fall back to a working brute force. Never go silent for long.
- Take hints by listening, repeating them in your words, connecting them to the problem, and moving on.
- Manage nerves with realistic practice, sleep, slow breathing and slow speech.
- In the last two weeks: review patterns, run daily mocks, fix weak spots, and learn no new topics in the final five days.
- Score every mock out of 24 across six areas and focus on your lowest area.

Quiz

1. What should you do in the first minutes after reading the problem? A) Start coding B) Restate it, work an example and ask clarifying questions C) State the complexity D) Ask about salary
2. Why should you ask "Does this approach sound good?" before coding?
3. True or false: if you cannot find the optimal solution, it is better to code nothing than to code a brute force.
4. You have made no progress for three minutes. Name two things from the recovery checklist you would try.
5. The interviewer gives a hint you do not understand. What would you do?
6. Which is a good question to ask the interviewer at the end? A) "Did I pass?" B) "What is the salary?" C) "How does your team balance unit and end-to-end tests?" D) "Can I skip the next round?"
7. True or false: you should learn new advanced topics in the last five days before the interview.
8. How many areas are in the self-scoring rubric, and what is the maximum total?
9. You scored 2 in Testing in three mocks in a row. What would you do?
10. Which breathing pattern does this chapter suggest for nerves? A) In 1, out 1 B) In 4, hold 4, out 6 C) Hold for 30 seconds D) No breathing technique

Answer key

1. **B** - Restating, working an example and clarifying prevents solving the wrong problem.
2. **Early correction** - The interviewer can flag problems before you spend 15 minutes coding the wrong idea.
3. **False** - A working brute force is much better than an unfinished optimal solution; you can optimise afterwards.
4. **Any two** - Return to the example, simplify the problem, scan the pattern list, look for repeated work, work backwards, or code the brute force.
5. **Ask politely** - Say "Could you say a bit more about what you mean?", then repeat it in your words and connect it to the problem.
6. **C** - It is genuine and specific to the role; performance and salary questions are not for this moment.
7. **False** - The last five days are for deepening known patterns, mocks and rest, not new topics.

8. **Six areas, 24 points** - Understanding, problem solving, coding, testing, complexity and communication, each scored 1-4.
9. **Focus on it** - Make testing your only focus for two days: practise dry runs and edge-case categories from chapter dsa-30, then re-test in a mock.
10. **B** - Breathe in for 4 counts, hold for 4, out for 6.

Flashcards

- **Q:** What are the phases of a 45-minute coding round? — **A:** Introductions, clarify, approaches, code, test, complexity and follow-ups, your questions.
- **Q:** How long should your self-introduction be? — **A:** About 30-45 seconds.
- **Q:** What should you say before starting to code? — **A:** "Here's my plan. Does this approach sound good to you?"
- **Q:** What is the first move when stuck? — **A:** Go back to the example and solve a slightly bigger one by hand.
- **Q:** What is the fallback when no optimal idea comes? — **A:** Code the brute force first, then optimise.
- **Q:** How do you take a hint well? — **A:** Listen fully, repeat it in your words, connect it to the problem, thank them and move on.
- **Q:** What is the longest you should stay silent? — **A:** About 30 seconds; ask for thinking time if you need more.
- **Q:** What do you do in the last five days before the interview? — **A:** No new topics: mocks, weak spots from the mistakes log, light review and rest.
- **Q:** What are the six rubric areas? — **A:** Understanding, problem solving, coding, testing, complexity, communication.
- **Q:** What total score means interview-ready? — **A:** 20-24 out of 24.
- **Q:** What breathing pattern helps with nerves? — **A:** In for 4, hold for 4, out for 6.